

F28M35x Peripheral Driver Library

USER'S GUIDE



Copyright

Copyright © 2016 Texas Instruments Incorporated. All rights reserved. ControlSUITE is a registered trademark of Texas Instruments. Other names and brands may be claimed as the property of others.

 Please be aware that an important notice concerning availability, standard warranty, and use in critical applications of Texas Instruments semiconductor products and disclaimers thereto appears at the end of this document.

Texas Instruments
12203 Southwest Freeway
Houston, TX 77477
<http://www.ti.com/c2000>



Revision Information

This is version 208 of this document, last updated on Tue Sep 6 09:31:54 CDT 2016.

Table of Contents

Copyright	2
Revision Information	2
1 Introduction	5
2 Programming Model	7
2.1 Introduction	7
2.2 Direct Register Access Model	7
2.3 Software Driver Model	8
2.4 Combining The Models	9
3 Controller Area Network (CAN)	11
3.1 Introduction	11
3.2 API Functions	11
3.3 CAN Message Objects	35
3.4 Programming Examples	36
4 DCSM	39
4.1 Introduction	39
4.2 API Functions	39
4.3 Programming Example	46
5 Ethernet Controller	49
5.1 Introduction	49
5.2 API Functions	49
5.3 Programming Example	62
6 External Peripheral Interface (EPI)	65
6.1 Introduction	65
6.2 API Functions	65
7 Flash	83
7.1 Introduction	83
7.2 API Functions	83
8 GPIO	87
8.1 Introduction	87
8.2 API Functions	87
8.3 Programming Example	105
9 Inter-Integrated Circuit (I2C)	107
9.1 Introduction	107
9.2 API Functions	108
9.3 Programming Example	122
10 Inter-processor Communication (IPC)	123
10.1 Introduction	123
10.2 M3 IPC API Drivers	124
10.3 M3 IPC-Lite API Drivers	145
10.4 M3 IPC Utility Drivers	157
10.5 C28 IPC API Drivers	160
10.6 C28 IPC-Lite API Drivers	181
10.7 C28 IPC Utility Drivers	193
11 Interrupt Controller (NVIC)	197
11.1 Introduction	197

11.2	API Functions	198
11.3	Programming Example	205
12	Memory Protection Unit (MPU)	207
12.1	Introduction	207
12.2	API Functions	207
12.3	Programming Example	214
13	Synchronous Serial Interface (SSI)	217
13.1	Introduction	217
13.2	API Functions	217
13.3	Programming Example	226
14	System Control	229
14.1	Introduction	229
14.2	API Functions	230
14.3	Programming Example	250
15	System Tick (SysTick)	251
15.1	Introduction	251
15.2	API Functions	251
15.3	Programming Example	255
16	Timer	257
16.1	Introduction	257
16.2	API Functions	257
16.3	Programming Example	270
17	UART	273
17.1	Introduction	273
17.2	API Functions	273
17.3	Programming Example	293
18	uCRC Peripheral	295
18.1	Introduction	295
18.2	API Functions	295
18.3	Programming Example	297
19	uDMA Controller	299
19.1	Introduction	299
19.2	API Functions	300
19.3	Programming Example	322
20	USB Controller	323
20.1	Introduction	323
20.2	Using USB with the uDMA Controller	323
20.3	API Functions	327
20.4	Programming Example	364
21	Watchdog Timer	365
21.1	Introduction	365
21.2	API Functions	366
21.3	Programming Example	374
	IMPORTANT NOTICE	376

1 Introduction

The Texas Instruments® ControlSUITE® Peripheral Driver Library is a set of drivers for accessing the Master Subsystem peripherals found on the Concerto family of C2000 microcontrollers. While they are not drivers in the pure operating system sense (that is, they do not have a common interface and do not connect into a global device driver infrastructure), they do provide a mechanism that makes it easy to use the device's peripherals.

The capabilities and organization of the drivers are governed by the following design goals:

- They are written entirely in C except where absolutely not possible.
- They demonstrate how to use the peripheral in its common mode of operation.
- They are easy to understand.
- They are reasonably efficient in terms of memory and processor usage.
- They are as self-contained as possible.
- Where possible, computations that can be performed at compile time are done there instead of at run time.

Some consequences of these design goals are:

- The drivers are not necessarily as efficient as they could be (from a code size and/or execution speed point of view). While the most efficient piece of code for operating a peripheral would be written in assembly and custom tailored to the specific requirements of the application, further size optimizations of the drivers would make them more difficult to understand.
- The drivers do not support the full capabilities of the hardware. Some of the peripherals provide complex capabilities which cannot be utilized by the drivers in this library, though the existing code can be used as a reference upon which to add support for the additional capabilities.
- The APIs have a means of removing all error checking code. Because the error checking is usually only useful during initial program development, it can be removed to improve code size and speed.

For many applications, the drivers can be used as is. But in some cases, the drivers will have to be enhanced or rewritten in order to meet the functionality, memory, or processing requirements of the application. If so, the existing driver can be used as a reference on how to operate the peripheral.

Source Code Overview

The following is an overview of the organization of the peripheral driver library source code.

<code>EULA.txt</code>	The full text of the End User License Agreement that covers the use of this software package.
<code>driverlib/</code>	This directory contains the source code for the drivers.
<code>hw_*.h</code>	Header files, one per peripheral, that describe all the registers and the bit fields within those registers for each peripheral. These header files are used by the drivers to directly access a peripheral, and can be used by application code to bypass the peripheral driver library API.

inc/

This directory holds the part specific header files used for the direct register access programming model.

2 Programming Model

Introduction	7
Direct Register Access Model	7
Software Driver Model	8
Combining The Models	9

2.1 Introduction

The peripheral driver library provides support for two programming models: the direct register access model and the software driver model. Each model can be used independently or combined, based on the needs of the application or the programming environment desired by the developer.

Each programming model has advantages and disadvantages. Use of the direct register access model generally results in smaller and more efficient code than using the software driver model. However, the direct register access model requires detailed knowledge of the operation of each register and bit field, as well as their interactions and any sequencing required for proper operation of the peripheral; the developer is insulated from these details by the software driver model, generally requiring less time to develop applications.

2.2 Direct Register Access Model

In the direct register access model, the peripherals are programmed by the application by writing values directly into the peripheral's registers. A set of macros is provided that simplifies this process. These macros are stored in several header files contained in the `inc` directory. By including the header files `inc/hw_types.h` and `inc/hw_memmap.h`, macros are available for accessing all registers on the Concerto devices. Individual bitfield access can easily be added by simply including the `inc/hw_peripheral.h` header file for the desired peripheral.

The defines used by the direct register access model follow a naming convention that makes it easier to know how to use a particular macro. The rules are as follows:

- Values that end in `_R` are used to access the value of a register. For example, `SSI0_CR0_R` is used to access the `CR0` register in the `SSI0` module.
- Values that end in `_M` represent the mask for a multi-bit field in a register. If the value placed in the multi-bit field is a number, there is a macro with the same base name but ending with `_S` (for example, `SSI_CR0_SCR_M` and `SSI_CR0_SCR_S`). If the value placed into the multi-bit field is an enumeration, then there are a set of macros with the same base name but ending with identifiers for the various enumeration values (for example, the `SSI_CR0_FRF_M` macro defines the bit field, and the `SSI_CR0_FRF_NMW`, `SSI_CR0_FRF_TI`, and `SSI_CR0_FRF_MOTO` macros provide the enumerations for the bit field).
- Values that end in `_S` represent the number of bits to shift a value in order to align it with a multi-bit field. These values match the macro with the same base name but ending with `_M`.
- All other macros represent the value of a bit field.

- All register name macros start with the module name and instance number (for example, `SSI0` for the first SSI module) and are followed by the name of the register as it appears in the data sheet (for example, the `CR0` register in the data sheet results in `SSI0_CR0_R`).
- All register bit fields start with the module name, followed by the register name, and then followed by the bit field name as it appears in the data sheet. For example, the `SCR` bit field in the `CR0` register in the `SSI` module will be identified by `SSI_CR0_SCR`. . . . In the case where the bit field is a single bit, there will be nothing further (for example, `SSI_CR0_SPH` is a single bit in the `CR0` register). If the bit field is more than a single bit, there will be a mask value (`_M`) and either a shift (`_S`) if the bit field contains a number or a set of enumerations if not.

Given these definitions, the `CR0` register can be programmed as follows:

```
SSI0_CR0_R = ((5 << SSI_CR0_SCR_S) | SSI_CR0_SPH | SSI_CR0_SPO |  
             SSI_CR0_FRF_MOTO | SSI_CR0_DSS_8);
```

Alternatively, the following has the same effect (although it is not as easy to understand):

```
SSI0_CR0_R = 0x000005c7;
```

Extracting the value of the `SCR` field from the `CR0` register is as follows:

```
ulValue = (SSI0_CR0_R & SSI_CR0_SCR_M) >> SSI0_CR0_SCR_S;
```

The GPIO modules have many registers that do not have bit field definitions. For these registers, the register bits represent the individual GPIO pins; so bit zero in these registers corresponds to the **Px0** pin on the part (where **x** is replaced by a GPIO module letter), bit one corresponds to the **Px1** pin, and so on.

2.3 Software Driver Model

In the software driver model, the API provided by the peripheral driver library is used by applications to control the peripherals. Because these drivers provide complete control of the peripherals in their normal mode of operation, it is possible to write an entire application without direct access to the hardware. This method provides for rapid development of the application without requiring detailed knowledge of how to program the peripherals.

Corresponding to the direct register access model example, the following call also programs the `CR0` register in the `SSI` module (though the register name is hidden by the API):

```
SSISConfigSetExpClk(SSI0_BASE, 50000000, SSI_FRF_MOTO_MODE_3,  
                   SSI_MODE_MASTER, 1000000, 8);
```

The resulting value in the `CR0` register might not be exactly the same because [SSISConfigSetExpClk\(\)](#) may compute a different value for the `SCR` bit field than what was used in the direct register access model example.

The drivers in the peripheral driver library are described in the remaining chapters in this document. They combine to form the software driver model.

2.4 Combining The Models

The direct register access model and software driver model can be used together in a single application, allowing the most appropriate model to be applied as needed to any particular situation within the application. For example, the software driver model can be used to configure the peripherals (because this is not performance critical) and the direct register access model can be used for operation of the peripheral (which may be more performance critical). Or, the software driver model can be used for peripherals that are not performance critical (such as a UART used for data logging) and the direct register access model for performance critical peripherals.

3 Controller Area Network (CAN)

Introduction	11
API Functions	11
CAN Message Objects	35
Programming Example	36

3.1 Introduction

The Controller Area Network (CAN) APIs provide a set of functions for accessing the Concerto CAN modules. Functions are provided to configure the CAN controllers, configure message objects, and manage CAN interrupts.

The Concerto CAN module provides hardware processing of the CAN data link layer. It can be configured with message filters and preloaded message data so that it can autonomously send and receive messages on the bus, and notify the application accordingly. It automatically handles generation and checking of CRCs, error processing, and retransmission of CAN messages.

The message objects are stored in the CAN controller and provide the main interface for the CAN module on the CAN bus. There are 32 message objects that can each be programmed to handle a separate message ID, or can be chained together for a sequence of frames with the same ID. The message identifier filters provide masking that can be programmed to match any or all of the message ID bits, and frame types.

This driver is contained in `driverlib/can.c`, with `driverlib/can.h` containing the API definitions for use by applications.

3.2 API Functions

Data Structures

- [tCANBitClkParms](#)
- [tCANMsgObject](#)

Defines

- [CAN_CLK_M3](#)
- [CAN_CLK_X1](#)
- [CAN_CLK_XCLKIN](#)
- [CAN_INT_ERROR](#)
- [CAN_INT_IE0](#)
- [CAN_INT_IE1](#)
- [CAN_INT_STATUS](#)
- [CAN_STATUS_BUS_OFF](#)
- [CAN_STATUS_EPASS](#)
- [CAN_STATUS_EWARN](#)

- [CAN_STATUS_LEC_ACK](#)
- [CAN_STATUS_LEC_BIT0](#)
- [CAN_STATUS_LEC_BIT1](#)
- [CAN_STATUS_LEC_CRC](#)
- [CAN_STATUS_LEC_FORM](#)
- [CAN_STATUS_LEC_MSK](#)
- [CAN_STATUS_LEC_NONE](#)
- [CAN_STATUS_LEC_STUFF](#)
- [CAN_STATUS_PDA](#)
- [CAN_STATUS_PERR](#)
- [CAN_STATUS_RXOK](#)
- [CAN_STATUS_TXOK](#)
- [CAN_STATUS_WAKE_UP](#)
- [MSG_OBJ_DATA_LOST](#)
- [MSG_OBJ_EXTENDED_ID](#)
- [MSG_OBJ_FIFO](#)
- [MSG_OBJ_NEW_DATA](#)
- [MSG_OBJ_NO_FLAGS](#)
- [MSG_OBJ_REMOTE_FRAME](#)
- [MSG_OBJ_RX_INT_ENABLE](#)
- [MSG_OBJ_STATUS_MASK](#)
- [MSG_OBJ_TX_INT_ENABLE](#)
- [MSG_OBJ_USE_DIR_FILTER](#)
- [MSG_OBJ_USE_EXT_FILTER](#)
- [MSG_OBJ_USE_ID_FILTER](#)

Enumerations

- [tCANIntStsReg](#)
- [tCANStsReg](#)
- [tMsgObjType](#)

Functions

- unsigned long [CANBitRateSet](#) (unsigned long ulBase, unsigned long ulSourceClock, unsigned long ulBitRate)
- void [CANBitTimingGet](#) (unsigned long ulBase, [tCANBitClkParms](#) *pClkParms)
- void [CANBitTimingSet](#) (unsigned long ulBase, [tCANBitClkParms](#) *pClkParms)
- void [CANClkSourceSelect](#) (unsigned long ulBase, unsigned char ucSource)
- void [CANDisable](#) (unsigned long ulBase)
- void [CANEnable](#) (unsigned long ulBase)
- tBoolean [CANErrCntrGet](#) (unsigned long ulBase, unsigned long *pulRxCnt, unsigned long *pulTxCount)
- void [CANInit](#) (unsigned long ulBase)
- void [CANIntClear](#) (unsigned long ulBase, unsigned long ulIntClr)

- void [CANIntDisable](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [CANIntEnable](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [CANIntRegister](#) (unsigned long ulBase, unsigned char ucIntNumber, void (*pfnHandler)(void))
- unsigned long [CANIntStatus](#) (unsigned long ulBase, [tCANIntStsReg](#) eIntStsReg)
- void [CANIntUnregister](#) (unsigned long ulBase, unsigned char ucIntNumber)
- void [CANMessageClear](#) (unsigned long ulBase, unsigned long ulObjID)
- void [CANMessageGet](#) (unsigned long ulBase, unsigned long ulObjID, [tCANMsgObject](#) *pMsgObject, tBoolean bClrPendingInt)
- void [CANMessageSet](#) (unsigned long ulBase, unsigned long ulObjID, [tCANMsgObject](#) *pMsgObject, [tMsgObjType](#) eMsgType)
- tBoolean [CANRetryGet](#) (unsigned long ulBase)
- void [CANRetrySet](#) (unsigned long ulBase, tBoolean bAutoRetry)
- unsigned long [CANStatusGet](#) (unsigned long ulBase, [tCANStsReg](#) eStatusReg)

3.2.1 Detailed Description

The CAN APIs provide all of the functions needed by the application to implement an interrupt-driven CAN stack. These functions may be used to control any of the available CAN ports on a Concerto microcontroller, and can be used with one port without causing conflicts with the other port.

The CAN module is disabled by default, so the the [CANInit\(\)](#) function must be called before any other CAN functions are called. This call initializes the message objects to a safe state prior to enabling the controller on the CAN bus. Also, the bit timing values must be programmed prior to enabling the CAN controller. The [CANBitTimingSet\(\)](#) function should be called with the appropriate bit timing values for the CAN bus. Once these two functions have been called, a CAN controller can be enabled using the [CANEnable\(\)](#), and later disabled using [CANDisable\(\)](#) if needed. Calling [CANDisable\(\)](#) does not reinitialize a CAN controller, so it can be used to temporarily remove a CAN controller from the bus.

The CAN controller is highly configurable and contains 32 message objects that can be programmed to automatically transmit and receive CAN messages under certain conditions. Message objects allow the application to perform some actions automatically without interaction from the microcontroller. Some examples of these actions are the following:

- Send a data frame immediately
- Send a data frame when a matching remote frame is seen on the CAN bus
- Receive a specific data frame
- Receive data frames that match a certain identifier pattern

To configure message objects to perform any of these actions, the application must first set up one of the 32 message objects using [CANMessageSet\(\)](#). This function must be used to configure a message object to send data, or to configure a message object to receive data. Each message object can be configured to generate interrupts on transmission or reception of CAN messages.

When data is received from the CAN bus, the application can use the [CANMessageGet\(\)](#) function to read the received message. This function can also be used to read a message object that is already configured in order to populate a message structure prior to making changes to the configuration

of a message object. Reading the message object using this function will also clear any pending interrupt on the message object.

Once a message object has been configured using [CANMessageSet\(\)](#), it has allocated the message object and will continue to perform its programmed function unless it is released with a call to [CANMessageClear\(\)](#). The application is not required to clear out a message object before setting it with a new configuration, because each time [CANMessageSet\(\)](#) is called, it will overwrite any previously programmed configuration.

The 32 message objects are identical except for priority. The lowest numbered message objects have the highest priority. Priority affects operation in two ways. First, if multiple actions are ready at the same time, the one with the highest priority message object will occur first. And second, when multiple message objects have interrupts pending, the highest priority will be presented first when reading the interrupt status. It is up to the application to manage the 32 message objects as a resource, and determine the best method for allocating and releasing them.

The CAN controller can generate interrupts on several conditions:

- When any message object transmits a message
- When any message object receives a message
- On warning conditions such as an error counter reaching a limit or occurrence of various bus errors
- On controller error conditions such as entering the bus-off state

An interrupt handler must be installed in order to process CAN interrupts. If dynamic interrupt configuration is desired, the [CANIntRegister\(\)](#) can be used to register the interrupt handler. This will place the vector in a RAM-based vector table. However, if the application uses a pre-loaded vector table in flash, then the CAN controller handler should be entered in the appropriate slot in the vector table. In this case, [CANIntRegister\(\)](#) is not needed, but the interrupt will need to be enabled on the host processor master interrupt controller using the [IntEnable\(\)](#) function. The CAN module interrupts are enabled using the [CANIntEnable\(\)](#) function. They can be disabled by using the [CANIntDisable\(\)](#) function.

Once CAN interrupts are enabled, the handler will be invoked whenever a CAN interrupt is triggered. The handler can determine which condition caused the interrupt by using the [CANIntStatus\(\)](#) function. Multiple conditions can be pending when an interrupt occurs, so the handler must be designed to process all pending interrupt conditions before exiting. Each interrupt condition must be cleared before exiting the handler. There are two ways to do this. The [CANIntClear\(\)](#) function will clear a specific interrupt condition without further action required by the handler. However, the handler can also clear the condition by performing certain actions. If the interrupt is a status interrupt, the interrupt can be cleared by reading the status register with [CANStatusGet\(\)](#). If the interrupt is caused by one of the message objects, then it can be cleared by reading the message object using [CANMessageGet\(\)](#).

There are several status registers that can be used to help the application manage the controller. The status registers are read using the [CANStatusGet\(\)](#) function. There is a controller status register that provides general status information such as error or warning conditions. There are also several status registers that provide information about all of the message objects at once using a 32-bit bit map of the status, with one bit representing each message object. These status registers can be used to determine:

- Which message objects have unprocessed received data
- Which message objects have pending transmission requests
- Which message objects are allocated for use

3.2.2 Data Structure Documentation

3.2.2.1 tCANBitClkParms

Definition:

```
typedef struct
{
    unsigned int uSyncPropPhase1Seg;
    unsigned int uPhase2Seg;
    unsigned int uSJW;
    unsigned int uQuantumPrescaler;
}
tCANBitClkParms
```

Members:

uSyncPropPhase1Seg This value holds the sum of the Synchronization, Propagation, and Phase Buffer 1 segments, measured in time quanta. The valid values for this setting range from 2 to 16.

uPhase2Seg This value holds the Phase Buffer 2 segment in time quanta. The valid values for this setting range from 1 to 8.

uSJW This value holds the Resynchronization Jump Width in time quanta. The valid values for this setting range from 1 to 4.

uQuantumPrescaler This value holds the CAN_CLK divider used to determine time quanta. The valid values for this setting range from 1 to 1023.

Description:

This structure is used for encapsulating the values associated with setting up the bit timing for a CAN controller. The structure is used when calling the CANGetBitTiming and CANSetBitTiming functions.

3.2.2.2 tCANMsgObject

Definition:

```
typedef struct
{
    unsigned long ulMsgID;
    unsigned long ulMsgIDMask;
    unsigned long ulFlags;
    unsigned long ulMsgLen;
    unsigned char *pucMsgData;
}
tCANMsgObject
```

Members:

ulMsgID The CAN message identifier used for 11 or 29 bit identifiers.

ulMsgIDMask The message identifier mask used when identifier filtering is enabled.

ulFlags This value holds various status flags and settings specified by tCANObjFlags.

ulMsgLen This value is the number of bytes of data in the message object.

pucMsgData This is a pointer to the message object's data.

Description:

The structure used for encapsulating all the items associated with a CAN message object in the CAN controller.

3.2.3 Define Documentation

3.2.3.1 CAN_CLK_M3

Definition:

```
#define CAN_CLK_M3
```

Description:

This flag is used to clock the CAN controller with the M3/Master Subsystem clock.

3.2.3.2 CAN_CLK_X1

Definition:

```
#define CAN_CLK_X1
```

Description:

This flag is used to clock the CAN controller with the X1/X2 oscillator clock.

3.2.3.3 CAN_CLK_XCLKIN

Definition:

```
#define CAN_CLK_XCLKIN
```

Description:

This flag is used to clock the CAN controller with the clock from XClockIn

3.2.3.4 CAN_INT_ERROR

Definition:

```
#define CAN_INT_ERROR
```

Description:

This flag is used to allow a CAN controller to generate error interrupts.

3.2.3.5 CAN_INT_IE0

Definition:

```
#define CAN_INT_IE0
```

Description:

This flag is used to allow a CAN controller to generate interrupts on interrupt line 0

3.2.3.6 CAN_INT_IE1

Definition:

```
#define CAN_INT_IE1
```

Description:

This flag is used to allow a CAN controller to generate interrupts on interrupt line 1

3.2.3.7 CAN_INT_STATUS

Definition:

```
#define CAN_INT_STATUS
```

Description:

This flag is used to allow a CAN controller to generate status interrupts.

3.2.3.8 CAN_STATUS_BUS_OFF

Definition:

```
#define CAN_STATUS_BUS_OFF
```

Description:

CAN controller has entered a Bus Off state.

3.2.3.9 CAN_STATUS_EPASS

Definition:

```
#define CAN_STATUS_EPASS
```

Description:

CAN controller error level has reached error passive level.

3.2.3.10 CAN_STATUS_EWARN

Definition:

```
#define CAN_STATUS_EWARN
```

Description:

CAN controller error level has reached warning level.

3.2.3.11 CAN_STATUS_LEC_ACK

Definition:

```
#define CAN_STATUS_LEC_ACK
```

Description:

An acknowledge error has occurred.

3.2.3.12 CAN_STATUS_LEC_BIT0

Definition:

```
#define CAN_STATUS_LEC_BIT0
```

Description:

The bus remained a bit level of 0 for longer than is allowed.

3.2.3.13 CAN_STATUS_LEC_BIT1

Definition:

```
#define CAN_STATUS_LEC_BIT1
```

Description:

The bus remained a bit level of 1 for longer than is allowed.

3.2.3.14 CAN_STATUS_LEC_CRC

Definition:

```
#define CAN_STATUS_LEC_CRC
```

Description:

A CRC error has occurred.

3.2.3.15 CAN_STATUS_LEC_FORM

Definition:

```
#define CAN_STATUS_LEC_FORM
```

Description:

A formatting error has occurred.

3.2.3.16 CAN_STATUS_LEC_MSK

Definition:

```
#define CAN_STATUS_LEC_MSK
```

Description:

This is the mask for the last error code field.

3.2.3.17 CAN_STATUS_LEC_NONE

Definition:

```
#define CAN_STATUS_LEC_NONE
```

Description:

There was no error.

3.2.3.18 CAN_STATUS_LEC_STUFF

Definition:

```
#define CAN_STATUS_LEC_STUFF
```

Description:

A bit stuffing error has occurred.

3.2.3.19 CAN_STATUS_PDA

Definition:

```
#define CAN_STATUS_PDA
```

Description:

CAN controller is in local power down mode.

3.2.3.20 CAN_STATUS_PERR

Definition:

```
#define CAN_STATUS_PERR
```

Description:

CAN controller has detected a parity error.

3.2.3.21 CAN_STATUS_RXOK

Definition:

```
#define CAN_STATUS_RXOK
```

Description:

A message was received successfully since the last read of this status.

3.2.3.22 CAN_STATUS_TXOK

Definition:

```
#define CAN_STATUS_TXOK
```

Description:

A message was transmitted successfully since the last read of this status.

3.2.3.23 CAN_STATUS_WAKE_UP

Definition:

```
#define CAN_STATUS_WAKE_UP
```

Description:

CAN controller has initiated a system wakeup.

3.2.3.24 MSG_OBJ_DATA_LOST

Definition:

```
#define MSG_OBJ_DATA_LOST
```

Description:

This indicates that data was lost since this message object was last read.

3.2.3.25 MSG_OBJ_EXTENDED_ID

Definition:

```
#define MSG_OBJ_EXTENDED_ID
```

Description:

This indicates that a message object will use or is using an extended identifier.

3.2.3.26 MSG_OBJ_FIFO

Definition:

```
#define MSG_OBJ_FIFO
```

Description:

This indicates that this message object is part of a FIFO structure and not the final message object in a FIFO.

3.2.3.27 MSG_OBJ_NEW_DATA

Definition:

```
#define MSG_OBJ_NEW_DATA
```

Description:

This indicates that new data was available in the message object.

3.2.3.28 MSG_OBJ_NO_FLAGS

Definition:

```
#define MSG_OBJ_NO_FLAGS
```

Description:

This indicates that a message object has no flags set.

3.2.3.29 MSG_OBJ_REMOTE_FRAME

Definition:

```
#define MSG_OBJ_REMOTE_FRAME
```

Description:

This indicates that a message object is a remote frame.

3.2.3.30 MSG_OBJ_RX_INT_ENABLE

Definition:

```
#define MSG_OBJ_RX_INT_ENABLE
```

Description:

This indicates that receive interrupts should be enabled, or are enabled.

3.2.3.31 MSG_OBJ_STATUS_MASK

Definition:

```
#define MSG_OBJ_STATUS_MASK
```

Description:

This define is used with the flag values to allow checking only status flags and not configuration flags.

3.2.3.32 MSG_OBJ_TX_INT_ENABLE

Definition:

```
#define MSG_OBJ_TX_INT_ENABLE
```

Description:

This definition is used with the [tCANMsgObject](#) ulFlags value and indicates that transmit interrupts should be enabled, or are enabled.

3.2.3.33 MSG_OBJ_USE_DIR_FILTER

Definition:

```
#define MSG_OBJ_USE_DIR_FILTER
```

Description:

This indicates that a message object will use or is using filtering based on the direction of the transfer. If the direction filtering is used, then ID filtering must also be enabled.

3.2.3.34 MSG_OBJ_USE_EXT_FILTER

Definition:

```
#define MSG_OBJ_USE_EXT_FILTER
```

Description:

This indicates that a message object will use or is using message identifier filtering based on the extended identifier. If the extended identifier filtering is used, then ID filtering must also be enabled.

3.2.3.35 MSG_OBJ_USE_ID_FILTER

Definition:

```
#define MSG_OBJ_USE_ID_FILTER
```

Description:

This indicates that a message object will use or is using filtering based on the object's message identifier.

3.2.4 Enumeration Documentation

3.2.4.1 tCANIntStsReg

Description:

This data type is used to identify the interrupt status register. This is used when calling the [CANIntStatus\(\)](#) function.

Enumerators:

CAN_INT_STS_CAUSE Read the CAN interrupt status information.

CAN_INT_STS_OBJECT Read a message object's interrupt status.

3.2.4.2 tCANStsReg

Description:

This data type is used to identify which of several status registers to read when calling the [CANStatusGet\(\)](#) function.

Enumerators:

CAN_STS_CONTROL Read the full CAN controller status.

CAN_STS_TXREQUEST Read the full 32-bit mask of message objects with a transmit request set.

CAN_STS_NEWDAT Read the full 32-bit mask of message objects with new data available.

CAN_STS_MSGVAL Read the full 32-bit mask of message objects that are enabled.

3.2.4.3 tMsgObjType

Description:

This definition is used to determine the type of message object that will be set up via a call to the [CANMessageSet\(\)](#) API.

Enumerators:

MSG_OBJ_TYPE_TX Transmit message object.

MSG_OBJ_TYPE_TX_REMOTE Transmit remote request message object.

MSG_OBJ_TYPE_RX Receive message object.

MSG_OBJ_TYPE_RX_REMOTE Receive remote request message object.

MSG_OBJ_TYPE_RXTX_REMOTE Remote frame receive remote, with auto-transmit message object.

3.2.5 Function Documentation

3.2.5.1 CANBitRateSet

This function is used to set the CAN bit timing values to a nominal setting based on a desired bit rate.

Prototype:

```
unsigned long  
CANBitRateSet(unsigned long ulBase,  
              unsigned long ulSourceClock,  
              unsigned long ulBitRate)
```

Parameters:

ulBase is the base address of the CAN controller.

ulSourceClock is the clock frequency for the CAN peripheral in Hz.

ulBitRate is the desired bit rate.

Description:

This function will set the CAN bit timing for the bit rate passed in the *ulBitRate* parameter based on the *ulSourceClock* parameter. The CAN bit clock is calculated to be an average timing value that should work for most systems. If tighter timing requirements are needed, then the [CANBitTimingSet\(\)](#) function is available for full customization of all of the CAN bit timing values. Since not all bit rates can be matched exactly, the bit rate is set to the value closest to the desired bit rate without being higher than the *ulBitRate* value.

Returns:

This function returns the bit rate that the CAN controller was configured to use or it returns 0 to indicate that the bit rate was not changed because the requested bit rate was not valid.

3.2.5.2 CANBitTimingGet

Reads the current settings for the CAN controller bit timing.

Prototype:

```
void  
CANBitTimingGet(unsigned long ulBase,  
                tCANBitClkParms *pClkParms)
```

Parameters:

ulBase is the base address of the CAN controller.

pClkParms is a pointer to a structure to hold the timing parameters.

Description:

This function reads the current configuration of the CAN controller bit clock timing, and stores the resulting information in the structure supplied by the caller. Refer to [CANBitTimingSet\(\)](#) for the meaning of the values that are returned in the structure pointed to by *pClkParms*.

This function replaces the original CANGetBitTiming() API and performs the same actions. A macro is provided in `can.h` to map the original API to this API.

Returns:

None.

3.2.5.3 CANBitTimingSet

Configures the CAN controller bit timing.

Prototype:

```
void  
CANBitTimingSet(unsigned long ulBase,  
                 tCANBitClkParms *pClkParms)
```

Parameters:

ulBase is the base address of the CAN controller.

pClkParms points to the structure with the clock parameters.

Description:

Configures the various timing parameters for the CAN bus bit timing: Propagation segment, Phase Buffer 1 segment, Phase Buffer 2 segment, and the Synchronization Jump Width. The values for Propagation and Phase Buffer 1 segments are derived from the combination *pClkParms->uSyncPropPhase1Seg* parameter. Phase Buffer 2 is determined from the *pClkParms->uPhase2Seg* parameter. These two parameters, along with *pClkParms->uSJW* are based in units of bit time quanta. The actual quantum time is determined by the *pClkParms->uQuantumPrescaler* value, which specifies the divisor for the CAN module clock.

The total bit time, in quanta, will be the sum of the two Seg parameters, as follows:

$$\text{bit_time_q} = \text{uSyncPropPhase1Seg} + \text{uPhase2Seg} + 1$$

Note that the Sync_Seg is always one quantum in duration, and will be added to derive the correct duration of Prop_Seg and Phase1_Seg.

The equation to determine the actual bit rate is as follows:

$$\text{CAN Clock} / ((\text{uSyncPropPhase1Seg} + \text{uPhase2Seg} + 1) * (\text{uQuantumPrescaler}))$$

This means that with *uSyncPropPhase1Seg* = 4, *uPhase2Seg* = 1, *uQuantumPrescaler* = 2 and an 8 MHz CAN clock, that the bit rate will be (8 MHz) / ((5 + 2 + 1) * 2) or 500 Kbit/sec.

Returns:

None.

3.2.5.4 CANClkSourceSelect

Select CAN peripheral clock source

Prototype:

```
void  
CANClkSourceSelect(unsigned long ulBase,  
                   unsigned char ucSource)
```

Parameters:

ulBase is the base address of the CAN controller to disable.

ucSource is the clock source to select for the given CAN peripheral:

- 0 - M3/Master Subsystem Clock (default at reset)
- 1 - Oscillator Clock (direct from X1/X2)
- 2 - XClockIn

Description:

Selects the desired clock source for use with a given CAN peripheral.

Returns:

None.

3.2.5.5 CANDisable

Disables the CAN controller.

Prototype:

```
void  
CANDisable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the CAN controller to disable.

Description:

Disables the CAN controller for message processing. When disabled, the controller will no longer automatically process data on the CAN bus. The controller can be restarted by calling [CANEnable\(\)](#). The state of the CAN controller and the message objects in the controller are left as they were before this call was made.

Returns:

None.

3.2.5.6 CANEnable

Enables the CAN controller.

Prototype:

```
void  
CANEnable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the CAN controller to enable.

Description:

Enables the CAN controller for message processing. Once enabled, the controller will automatically transmit any pending frames, and process any received frames. The controller can be stopped by calling [CANDisable\(\)](#). Prior to calling [CANEnable\(\)](#), [CANInit\(\)](#) should have been called to initialize the controller and the CAN bus clock should be configured by calling [CANBitTimingSet\(\)](#).

Returns:

None.

3.2.5.7 CANErrCntrGet

Reads the CAN controller error counter register.

Prototype:

```
tBoolean  
CANErrCntrGet (unsigned long ulBase,  
               unsigned long *pulRxCount,  
               unsigned long *pulTxCount)
```

Parameters:

ulBase is the base address of the CAN controller.

pulRxCount is a pointer to storage for the receive error counter.

pulTxCount is a pointer to storage for the transmit error counter.

Description:

Reads the error counter register and returns the transmit and receive error counts to the caller along with a flag indicating if the controller receive counter has reached the error passive limit. The values of the receive and transmit error counters are returned through the pointers provided as parameters.

After this call, **pulRxCount* will hold the current receive error count and **pulTxCount* will hold the current transmit error count.

Returns:

Returns **true** if the receive error count has reached the error passive limit, and **false** if the error count is below the error passive limit.

3.2.5.8 CANInit

Initializes the CAN controller after reset.

Prototype:

```
void  
CANInit (unsigned long ulBase)
```

Parameters:

ulBase is the base address of the CAN controller.

Description:

After reset, the CAN controller is left in the disabled state. However, the memory used for message objects contains undefined values and must be cleared prior to enabling the CAN controller the first time. This prevents unwanted transmission or reception of data before the message objects are configured. This function must be called before enabling the controller the first time.

Returns:

None.

3.2.5.9 CANIntClear

Clears a CAN interrupt source.

Prototype:

```
void  
CANIntClear(unsigned long ulBase,  
            unsigned long ulIntClr)
```

Parameters:

ulBase is the base address of the CAN controller.
ullIntClr is a value indicating which interrupt source to clear.

Description:

This function can be used to clear a specific interrupt source. The *ullIntClr* parameter should be one of the following values:

- **CAN_INT_INTID_STATUS** - Clears a status interrupt.
- 1-32 - Clears the specified message object interrupt

It is not necessary to use this function to clear an interrupt. This should only be used if the application wants to clear an interrupt source without taking the normal interrupt action.

Normally, the status interrupt is cleared by reading the controller status using [CANStatusGet\(\)](#). A specific message object interrupt is normally cleared by reading the message object using [CANMessageGet\(\)](#).

Note:

Since there is a write buffer in the Cortex-M3 processor, it may take several clock cycles before the interrupt source is actually cleared. Therefore, it is recommended that the interrupt source be cleared early in the interrupt handler (as opposed to the very last action) to avoid returning from the interrupt handler before the interrupt source is actually cleared. Failure to do so may result in the interrupt handler being immediately reentered (since NVIC still sees the interrupt source asserted).

Returns:

None.

3.2.5.10 CANIntDisable

Disables individual CAN controller interrupt sources.

Prototype:

```
void  
CANIntDisable(unsigned long ulBase,  
              unsigned long ulIntFlags)
```

Parameters:

ulBase is the base address of the CAN controller.
ullIntFlags is the bit mask of the interrupt sources to be disabled.

Description:

Disables the specified CAN controller interrupt sources. Only enabled interrupt sources can cause a processor interrupt.

The *ullIntFlags* parameter has the same definition as in the [CANIntEnable\(\)](#) function.

Returns:

None.

3.2.5.11 CANIntEnable

Enables individual CAN controller interrupt sources.

Prototype:

```
void  
CANIntEnable(unsigned long ulBase,  
              unsigned long ulIntFlags)
```

Parameters:

ulBase is the base address of the CAN controller.

ulIntFlags is the bit mask of the interrupt sources to be enabled.

Description:

Enables specific interrupt sources of the CAN controller. Only enabled sources will cause a processor interrupt.

The *ulIntFlags* parameter is the logical OR of any of the following:

- **CAN_INT_ERROR** - a controller error condition has occurred
- **CAN_INT_STATUS** - a message transfer has completed, or a bus error has been detected
- **CAN_INT_IE0** - allow CAN controller to generate interrupts on interrupt line 0
- **CAN_INT_IE1** - allow CAN controller to generate interrupts on interrupt line 1

In order to generate status or error interrupts, **CAN_INT_IE0** must be enabled. Further, for any particular transaction from a message object to generate an interrupt, that message object must have interrupts enabled (see [CANMessageSet\(\)](#)). **CAN_INT_ERROR** will generate an interrupt if the controller enters the “bus off” condition, or if the error counters reach a limit. **CAN_INT_STATUS** will generate an interrupt under quite a few status conditions and may provide more interrupts than the application needs to handle. When an interrupt occurs, use [CANIntStatus\(\)](#) to determine the cause.

Returns:

None.

3.2.5.12 CANIntRegister

Registers an interrupt handler for the CAN controller.

Prototype:

```
void  
CANIntRegister(unsigned long ulBase,  
               unsigned char ucIntNumber,  
               void (*pfnHandler)(void))
```

Parameters:

ulBase is the base address of the CAN controller.

ucIntNumber is the interrupt line to register (0 or 1).

pfnHandler is a pointer to the function to be called when the enabled CAN interrupts occur.

Description:

This function registers the interrupt handler in the interrupt vector table, and enables CAN interrupts on the interrupt controller; specific CAN interrupt sources must be enabled using

[CANIntEnable\(\)](#). The interrupt handler being registered must clear the source of the interrupt using [CANIntClear\(\)](#).

If the application is using a static interrupt vector table stored in flash, then it is not necessary to register the interrupt handler this way. Instead, [IntEnable\(\)](#) should be used to enable CAN interrupts on the interrupt controller.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

3.2.5.13 CANIntStatus

Returns the current CAN controller interrupt status.

Prototype:

```
unsigned long  
CANIntStatus(unsigned long ulBase,  
              tCANIntStsReg eIntStsReg)
```

Parameters:

ulBase is the base address of the CAN controller.

eIntStsReg indicates which interrupt status register to read

Description:

Returns the value of one of two interrupt status registers. The interrupt status register read is determined by the *eIntStsReg* parameter, which can have one of the following values:

- **CAN_INT_STS_CAUSE** - indicates the cause of the interrupt
- **CAN_INT_STS_OBJECT** - indicates pending interrupts of all message objects

CAN_INT_STS_CAUSE returns the value of the controller interrupt register and indicates the cause of the interrupt. It will be a value of **CAN_INT_INT0ID_STATUS** if the cause is a status interrupt. In this case, the status register should be read with the [CANStatusGet\(\)](#) function. Calling this function to read the status will also clear the status interrupt. If the value of the interrupt register is in the range 1-32, then this indicates the number of the highest priority message object that has an interrupt pending. The message object interrupt can be cleared by using the [CANIntClear\(\)](#) function, or by reading the message using [CANMessageGet\(\)](#) in the case of a received message. The interrupt handler can read the interrupt status again to make sure all pending interrupts are cleared before returning from the interrupt.

CAN_INT_STS_OBJECT returns a bit mask indicating which message objects have pending interrupts. This can be used to discover all of the pending interrupts at once, as opposed to repeatedly reading the interrupt register by using **CAN_INT_STS_CAUSE**.

Returns:

Returns the value of one of the interrupt status registers.

3.2.5.14 CANIntUnregister

Unregisters an interrupt handler for the CAN controller.

Prototype:

```
void  
CANIntUnregister(unsigned long ulBase,  
                 unsigned char ucIntNumber)
```

Parameters:

ulBase is the base address of the controller.

ucIntNumber is the interrupt line to un-register (0 or 1).

Description:

This function unregisters the previously registered interrupt handler and disables the interrupt on the interrupt controller.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

3.2.5.15 CANMessageClear

Clears a message object so that it is no longer used.

Prototype:

```
void  
CANMessageClear(unsigned long ulBase,  
                unsigned long ulObjID)
```

Parameters:

ulBase is the base address of the CAN controller.

ulObjID is the message object number to disable (1-32).

Description:

This function frees the specified message object from use. Once a message object has been "cleared," it will no longer automatically send or receive messages, or generate interrupts.

Returns:

None.

3.2.5.16 CANMessageGet

Reads a CAN message from one of the message object buffers.

Prototype:

```
void  
CANMessageGet(unsigned long ulBase,
```

```
unsigned long ulObjID,  
tCANMsgObject *pMsgObject,  
tBoolean bClrPendingInt)
```

Parameters:

ulBase is the base address of the CAN controller.

ulObjID is the object number to read (1-32).

pMsgObject points to a structure containing message object fields.

bClrPendingInt indicates whether an associated interrupt should be cleared.

Description:

This function is used to read the contents of one of the 32 message objects in the CAN controller, and return it to the caller. The data returned is stored in the fields of the caller-supplied structure pointed to by *pMsgObject*. The data consists of all of the parts of a CAN message, plus some control and status information.

Normally this is used to read a message object that has received and stored a CAN message with a certain identifier. However, this could also be used to read the contents of a message object in order to load the fields of the structure in case only part of the structure needs to be changed from a previous setting.

When using `CANMessageGet`, all of the same fields of the structure are populated in the same way as when the `CANMessageSet()` function is used, with the following exceptions:

pMsgObject->*ulFlags*:

- **MSG_OBJ_NEW_DATA** indicates if this is new data since the last time it was read
- **MSG_OBJ_DATA_LOST** indicates that at least one message was received on this message object, and not read by the host before being overwritten.

Returns:

None.

3.2.5.17 CANMessageSet

Configures a message object in the CAN controller.

Prototype:

```
void  
CANMessageSet(unsigned long ulBase,  
              unsigned long ulObjID,  
              tCANMsgObject *pMsgObject,  
              tMsgObjType eMsgType)
```

Parameters:

ulBase is the base address of the CAN controller.

ulObjID is the object number to configure (1-32).

pMsgObject is a pointer to a structure containing message object settings.

eMsgType indicates the type of message for this object.

Description:

This function is used to configure any one of the 32 message objects in the CAN controller. A message object can be configured as any type of CAN message object as well as several

options for automatic transmission and reception. This call also allows the message object to be configured to generate interrupts on completion of message receipt or transmission. The message object can also be configured with a filter/mask so that actions are only taken when a message that meets certain parameters is seen on the CAN bus.

The *eMsgType* parameter must be one of the following values:

- **MSG_OBJ_TYPE_TX** - CAN transmit message object.
- **MSG_OBJ_TYPE_TX_REMOTE** - CAN transmit remote request message object.
- **MSG_OBJ_TYPE_RX** - CAN receive message object.
- **MSG_OBJ_TYPE_RX_REMOTE** - CAN receive remote request message object.
- **MSG_OBJ_TYPE_RXTX_REMOTE** - CAN remote frame receive remote, then transmit message object.

The message object pointed to by *pMsgObject* must be populated by the caller, as follows:

- *ulMsgID* - contains the message ID, either 11 or 29 bits.
- *ulMsgIDMask* - mask of bits from *ulMsgID* that must match if identifier filtering is enabled.
- *ulFlags*
 - Set **MSG_OBJ_TX_INT_ENABLE** flag to enable interrupt on transmission.
 - Set **MSG_OBJ_RX_INT_ENABLE** flag to enable interrupt on receipt.
 - Set **MSG_OBJ_USE_ID_FILTER** flag to enable filtering based on the identifier mask specified by *ulMsgIDMask*.
- *ulMsgLen* - the number of bytes in the message data. This should be non-zero even for a remote frame; it should match the expected bytes of the data responding data frame.
- *pucMsgData* - points to a buffer containing up to 8 bytes of data for a data frame.

Example: To send a data frame or remote frame(in response to a remote request), take the following steps:

1. Set *eMsgType* to **MSG_OBJ_TYPE_TX**.
2. Set *pMsgObject->ulMsgID* to the message ID.
3. Set *pMsgObject->ulFlags*. Make sure to set **MSG_OBJ_TX_INT_ENABLE** to allow an interrupt to be generated when the message is sent.
4. Set *pMsgObject->ulMsgLen* to the number of bytes in the data frame.
5. Set *pMsgObject->pucMsgData* to point to an array containing the bytes to send in the message.
6. Call this function with *ulObjID* set to one of the 32 object buffers.

Example: To receive a specific data frame, take the following steps:

1. Set *eMsgObjType* to **MSG_OBJ_TYPE_RX**.
2. Set *pMsgObject->ulMsgID* to the full message ID, or a partial mask to use partial ID matching.
3. Set *pMsgObject->ulMsgIDMask* bits that should be used for masking during comparison.
4. Set *pMsgObject->ulFlags* as follows:
 - Set **MSG_OBJ_RX_INT_ENABLE** flag to be interrupted when the data frame is received.
 - Set **MSG_OBJ_USE_ID_FILTER** flag to enable identifier based filtering.
5. Set *pMsgObject->ulMsgLen* to the number of bytes in the expected data frame.
6. The buffer pointed to by *pMsgObject->pucMsgData* and *pMsgObject->ulMsgLen* are not used by this call as no data is present at the time of the call.

7. Call this function with *ulObjID* set to one of the 32 object buffers.

If you specify a message object buffer that already contains a message definition, it will be overwritten.

Returns:

None.

3.2.5.18 CANRetryGet

Returns the current setting for automatic retransmission.

Prototype:

```
tBoolean  
CANRetryGet(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the CAN controller.

Description:

Reads the current setting for the automatic retransmission in the CAN controller and returns it to the caller.

Returns:

Returns **true** if automatic retransmission is enabled, **false** otherwise.

3.2.5.19 CANRetrySet

Sets the CAN controller automatic retransmission behavior.

Prototype:

```
void  
CANRetrySet(unsigned long ulBase,  
             tBoolean bAutoRetry)
```

Parameters:

ulBase is the base address of the CAN controller.

bAutoRetry enables automatic retransmission.

Description:

Enables or disables automatic retransmission of messages with detected errors. If *bAutoRetry* is **true**, then automatic retransmission is enabled, otherwise it is disabled.

Returns:

None.

3.2.5.20 CANStatusGet

Reads one of the controller status registers.

Prototype:

```
unsigned long  
CANStatusGet(unsigned long ulBase,  
              tCANStsReg eStatusReg)
```

Parameters:

ulBase is the base address of the CAN controller.

eStatusReg is the status register to read.

Description:

Reads a status register of the CAN controller and returns it to the caller. The different status registers are:

- **CAN_STS_CONTROL** - the main controller status
- **CAN_STS_TXREQUEST** - bit mask of objects pending transmission
- **CAN_STS_NEWDAT** - bit mask of objects with new data
- **CAN_STS_MSGVAL** - bit mask of objects with valid configuration

When reading the main controller status register, a pending status interrupt will be cleared. This should be used in the interrupt handler for the CAN controller if the cause is a status interrupt. The controller status register fields are as follows:

- **CAN_STATUS_PDA** - controller in local power down mode
- **CAN_STATUS_WAKE_UP** - controller initiated system wake up
- **CAN_STATUS_PERR** - parity error detected
- **CAN_STATUS_BUS_OFF** - controller is in bus-off condition
- **CAN_STATUS_EWARN** - an error counter has reached a limit of at least 96
- **CAN_STATUS_EPASS** - CAN controller is in the error passive state
- **CAN_STATUS_RXOK** - a message was received successfully (independent of any message filtering).
- **CAN_STATUS_TXOK** - a message was successfully transmitted
- **CAN_STATUS_LEC_NONE** - no error
- **CAN_STATUS_LEC_STUFF** - stuffing error detected
- **CAN_STATUS_LEC_FORM** - a format error occurred in the fixed format part of a message
- **CAN_STATUS_LEC_ACK** - a transmitted message was not acknowledged
- **CAN_STATUS_LEC_BIT1** - dominant level detected when trying to send in recessive mode
- **CAN_STATUS_LEC_BIT0** - recessive level detected when trying to send in dominant mode
- **CAN_STATUS_LEC_CRC** - CRC error in received message

The remaining status registers are 32-bit bit maps to the message objects. They can be used to quickly obtain information about the status of all the message objects without needing to query each one. They contain the following information:

- **CAN_STS_TXREQUEST** - if a message object's TxRequest bit is set, that means that a transmission is pending on that object. The application can use this to determine which objects are still waiting to send a message.
- **CAN_STS_NEWDAT** - if a message object's NewDat bit is set, that means that a new message has been received in that object, and has not yet been picked up by the host application

- **CAN_STS_MSGVAL** - if a message object's MsgVal bit is set, that means it has a valid configuration programmed. The host application can use this to determine which message objects are empty/unused.

Returns:

Returns the value of the status register.

3.3 CAN Message Objects

This section will explain how to configure the CAN message objects in various modes using the [CANMessageSet\(\)](#) and [CANMessageGet\(\)](#) APIs. The configuration of a message object is determined by two parameters that are passed into the [CANMessageSet\(\)](#) API. These are the [tCANMsgObject](#) structure and the [tMsgObjType](#) type field. It is important to note that the ulObjID parameter is the index of one of the 32 message objects that are available and is not the message object's identifier.

Message objects can be defined as one of five types based on the needs of the application. They are defined in the [tMsgObjType](#) enumeration and can only be one of those values. The simplest of the message object types are MSG_OBJ_TYPE_TX and MSG_OBJ_TYPE_RX which are used to send or receive messages for a given message identifier or a range of identifiers. The message type MSG_OBJ_TYPE_TX_REMOTE is used to transmit a remote request for data from another CAN node on the network. These message objects do not transmit any data but once they send the request will automatically turn into receive message object and wait for data from a remote CAN device. The message type MSG_OBJ_TYPE_RX_REMOTE is the receiving end of a remote request, and receive remote requests for data and generate an interrupt to let the application know when to supply and transmit data back to the CAN controller that issued the remote request for data. The message type MSG_OBJ_TYPE_RXTX_REMOTE is similar to the MSG_OBJ_TYPE_RX_REMOTE except that it automatically responds with data that the application placed in the message object.

The remaining information used to configure a CAN message object is contained in the [tCANMsgObject](#) structure which is used when calling [CANMessageSet\(\)](#) or will be filled by data read from the message object when calling [CANMessageGet\(\)](#). The CAN message identifier is simply stored into the ulMsgID member of the [tCANMsgObject](#) structure and is the 11 or 20 bit CAN identifier for this message object. The ulMsgIDMask is the mask is used in combination with the ulMsgID value to determine a match when the MSG_OBJ_USE_ID_FILTER flag is set for a message object. The ulMsgIDMask is ignored if MSG_OBJ_USE_ID_FILTER flag is not set. The last of the configuration parameters are specified in the ulFlags which are defined as a combination of the MSG_OBJ_* values. The MSG_OBJ_TX_INT_ENABLE and MSG_OBJ_RX_INT_ENABLE flags will enable transmit complete or receive data interrupts. If the CAN network is only using extended(20 bit) identifiers then the MSG_OBJ_EXTENDED_ID flag should be specified. The [CANMessageSet\(\)](#) function will force this flag set if the identifier is greater than an 11 bit identifier can hold. The MSG_OBJ_USE_ID_FILTER is used to enable filtering based on the message identifiers as messages are seen by the CAN controller. The combination of ulMsgID and ulMsgIDMask will determine if a message is accepted for a given message object. In some cases it may be necessary to add a filter based on the direction of the message, so in these cases the MSG_OBJ_USE_DIR_FILTER is used to only accept the direction specified in the message type. Another additional filter flag is MSG_OBJ_USE_EXT_FILTER which will filter on only extended identifiers. In a mixed 11 bit and 20 bit identifier system, this will prevent an 11 bit identifier being confused with a 20 bit identifier of the same value. It is not necessary to specify this if there are only extended identifiers being used in the system. To determine if the incoming message identifier matches a given message object,

the incoming message identifier is ANDed with ulMsgIDMask and compared with ulMsgID. The "C" logic would be the following:

```
if((IncomingID & ulMsgIDMask) == ulMsgID)
{
    // Accept the message.
}
else
{
    // Ignore the message.
}
```

The last of the flags to affect [CANMessageSet\(\)](#) is the MSG_OBJ_FIFO flag. This flag is used when combining multiple message objects in a FIFO. This is useful when an application needs to receive more than the 8 bytes of data that can be received by a single CAN message object. It can also be used to reduce the likelihood of causing an overrun of data on a single message object that may be receiving data faster than the application can handle when using a single message object. If multiple message objects are going to be used in a FIFO they must be read in sequential order based on the message object number and have the exact same message identifiers and filtering values. All but the last of the message objects in a FIFO should have the MSG_OBJ_FIFO and the last message object in the FIFO should not have the MSG_OBJ_FIFO flag set to specify that is the last entry in the FIFO. See the CAN FIFO configuration example in the Programming Examples section of this document.

The remaining flags are all used when calling [CANMessageGet\(\)](#) when reading data or checking the status of a message object. If the MSG_OBJ_NEW_DATA flag is set in the [tCANMsgObject](#) ulFlags variable then the data returned was new and not stale data from a previous call to [CANMessageGet\(\)](#). If the MSG_OBJ_DATA_LOST flag is set then data was lost since this message object was last read with [CANMessageGet\(\)](#). The MSG_OBJ_REMOTE_FRAME flag will be set if the message object was configured as a remote message object and a remote request was received.

When sending or receiving data, the last two variables define the size and a pointer to the data used by [CANMessageGet\(\)](#) and [CANMessageSet\(\)](#). The ulMsgLen variable in [tCANMsgObject](#) specifies the number of bytes to send when calling [CANMessageSet\(\)](#) and the number of bytes to read when calling [CANMessageGet\(\)](#). The pucMsgData variable in [tCANMsgObject](#) is the pointer to the data to send ulMsgLen bytes, or the pointer to the buffer to read ulMsgLen bytes into.

3.4 Programming Examples

This example code will send out data from CAN controller 0 to be received by CAN controller 1. In order to actually receive the data, an external cable must be connected between the two ports. In this example, both controllers are configured for 1 Mbit operation.

```
tCANBitClkParms CANBitClk;
tCANMsgObject sMsgObjectRx;
tCANMsgObject sMsgObjectTx;
unsigned char ucBufferIn[8];
unsigned char ucBufferOut[8];

// Reset the state of all the message objects and the state of the CAN
```

```

// module to a known state.
CANInit(CAN0_BASE);
CANInit(CAN1_BASE);

// Configure the controller for 1 Mbit operation.
CANSetBitTiming(CAN1_BASE, &CANBitClk);

// Take the CAN0 device out of INIT state.
CANEnable(CAN0_BASE);
CANEnable(CAN1_BASE);

// Configure a receive object.
sMsgObjectRx.ulMsgID = (0x400);
sMsgObjectRx.ulMsgIDMask = 0x7f8;
sMsgObjectRx.ulFlags = MSG_OBJ_USE_ID_FILTER | MSG_OBJ_FIFO;

// The first three message objects have the MSG_OBJ_FIFO set to
// indicate that they are part of a FIFO.
CANMessageSet(CAN0_BASE, 1, &sMsgObjectRx, MSG_OBJ_TYPE_RX);
CANMessageSet(CAN0_BASE, 2, &sMsgObjectRx, MSG_OBJ_TYPE_RX);
CANMessageSet(CAN0_BASE, 3, &sMsgObjectRx, MSG_OBJ_TYPE_RX);

// Last message object does not have the MSG_OBJ_FIFO set to
// indicate that this is the last message.
sMsgObjectRx.ulFlags = MSG_OBJ_USE_ID_FILTER;
CANMessageSet(CAN0_BASE, 4, &sMsgObjectRx, MSG_OBJ_TYPE_RX);

// Configure and start transmit of message object.
sMsgObjectTx.ulMsgID = 0x400;
sMsgObjectTx.ulFlags = 0;
sMsgObjectTx.ulMsgLen = 8;
sMsgObjectTx.pucMsgData = ucBufferOut;
CANMessageSet(CAN0_BASE, 2, &sMsgObjectTx, MSG_OBJ_TYPE_TX);

// Wait for new data to become available.
while((CANStatusGet(CAN1_BASE, CAN_STS_NEWDAT) & 1) == 0)
{
    // Read the message out of the message object.
    CANMessageGet(CAN1_BASE, 1, &sMsgObjectRx, true);
}

// Process new data in sMsgObjectRx.pucMsgData.
...

```

This example code will configure a set of CAN message objects in FIFO mode, using CAN controller 0.

```

tCANBitClkParms CANBitClk;
tCANMsgObject sMsgObjectRx;
unsigned char ucBufferIn[8];
unsigned char ucBufferOut[8];

```

```
// Reset the state of all the message objects and the state of
// the CAN module to a known state.
CANInit(CAN0_BASE);

// Configure the controller for 1 Mbit operation.
CANBitRateSet(CAN0_BASE, 8000000, 1000000);

// Take the CAN0 device out of INIT state.
CANEnable(CAN0_BASE);

// Configure a receive object this CAN FIFO to receive message
// objects with message ID 0x400-0x407.
sMsgObjectRx.ulMsgID = (0x400);
sMsgObjectRx.ulMsgIDMask = 0x7f8;
sMsgObjectRx.ulFlags = MSG_OBJ_USE_ID_FILTER | MSG_OBJ_FIFO;

// The first three message objects have the MSG_OBJ_FIFO set
// to indicate that they are part of a FIFO.
CANMessageSet(CAN0_BASE, 1, &sMsgObjectRx, MSG_OBJ_TYPE_RX);
CANMessageSet(CAN0_BASE, 2, &sMsgObjectRx, MSG_OBJ_TYPE_RX);
CANMessageSet(CAN0_BASE, 3, &sMsgObjectRx, MSG_OBJ_TYPE_RX);

// Last message object does not have the MSG_OBJ_FIFO set to
// indicate that this is the last message.
sMsgObjectRx.ulFlags = MSG_OBJ_USE_ID_FILTER;
CANMessageSet(CAN0_BASE, 4, &sMsgObjectRx, MSG_OBJ_TYPE_RX);

...
```

4 DCSM

Introduction	39
API Functions	39
Programming Example	46

4.1 Introduction

The DCSM (dual core security module) API provides a set of functions and data structs for unlocking and getting the status of the M3 DCSM.

This driver is contained in `driverlib/dcs.c`, with `driverlib/dcs.h` containing the API definitions and data structs for use by applications.

4.2 API Functions

Functions

- unsigned long [DCSMGetFlashSectorZone](#) (unsigned long sector)
- unsigned long [DCSMGetRAMZone](#) (unsigned long ramsection)
- unsigned long [DCSMGetZone1EXEstatus](#) (unsigned long sector)
- unsigned long [DCSMGetZone2EXEstatus](#) (unsigned long sector)
- void [DCSMGrabFlashSectors](#) (void)
- void [DCSMGrabRAMBlocks](#) (void)
- void [DCSMReadZone1EXEONLY](#) (void)
- void [DCSMReadZone2EXEONLY](#) (void)
- void [DCSMSecureZone1](#) (void)
- void [DCSMSecureZone2](#) (void)
- void [DCSMUnlockZone1CSM](#) (CSMPSWDKEY *psCMDKey)
- void [DCSMUnlockZone1ECSL](#) (ECSLPSWDKEY *psECSLKey)
- void [DCSMUnlockZone2CSM](#) (CSMPSWDKEY *psCMDKey)
- void [DCSMUnlockZone2ECSL](#) (ECSLPSWDKEY *psECSLKey)
- unsigned long [DCSMZone1CSMSecurityStatus](#) (void)
- unsigned long [DCSMZone1ECSLSecurityStatus](#) (void)
- unsigned long [DCSMZone2CSMSecurityStatus](#) (void)
- unsigned long [DCSMZone2ECSLSecurityStatus](#) (void)

4.2.1 Detailed Description

The DCSM API functions can be grouped into five broad groups.

The [DCSMUnlockZone1CSM\(\)](#), [DCSMUnlockZone1ECSL\(\)](#), [DCSMUnlockZone2CSM\(\)](#) and [DCSMUnlockZone2ECSL\(\)](#) functions enable the user to unlock the device. The functions accept pointers to a CSMPSWDKEY or ECSLPSWDKEY struct as their arguments.

The [DCSMSecureZone1\(\)](#) and [DCSMSecureZone2\(\)](#) functions are used to secure or lock a zone once a password is programmed into the DCSM password lock flash locations.

The [DCSMGetRAMZone\(\)](#) and [DCSMGetFlashSectorZone\(\)](#) functions return the zone a given RAM section or flash sector belongs to.

The [DCSMZone1CSMSecurityStatus\(\)](#), [DCSMZone2CSMSecurityStatus\(\)](#), [DCSMZone1ECSLSecurityStatus\(\)](#) and [DCSMZone2ECSLSecurityStatus\(\)](#) functions return the security status of a zone.

The [DCSMGetZone1EXEstatus\(\)](#) and [DCSMGetZone2EXEstatus\(\)](#) return the EXE_ONLY status of a zone.

The functions [DCSMGrabRAMBlocks\(\)](#), [DCSMReadZone1EXEONLY\(\)](#), [DCSMGrabFlashSectors\(\)](#) and [DCSMReadZone2EXEONLY\(\)](#) are used to transfer specific flash location values to registers by reading the flash locations. This helps in getting the status of a DCSM security zone.

4.2.2 Function Documentation

4.2.2.1 DCSMGetFlashSectorZone

Returns the security zone a flash sector belongs to

Prototype:

```
unsigned long  
DCSMGetFlashSectorZone(unsigned long sector)
```

Parameters:

sector is the flash sector value. Valid ranges are 1 for sector 1 through 12 for sector 12.

Description:

This function returns the security zone a flash sector belongs to.

Returns:

Returns 0 if the sector is inaccessible. 1 if the sector belongs to zone 1, 2 if the sector belongs to zone 2 and 3 if the sector doesn't belong to any zone (or if the sector is unsecure)

4.2.2.2 DCSMGetRAMZone

Returns the security zone a RAM section belongs to

Prototype:

```
unsigned long  
DCSMGetRAMZone(unsigned long ramsection)
```

Parameters:

ramsection is the RAM section. Valid values are 0 for C0 RAM and 1 for C1 RAM.

Description:

This function returns the security zone a RAM section belongs to.

Returns:

Returns 0 if the section is inaccessible , 1 if the section belongs to zone 1, 2 if the section belongs to zone 2 and 3 if the section doesn't belong to any zone (or if the section is unsecure)

4.2.2.3 DCSMGetZone1EXEstatus

Returns the EXE-ONLY status of zone 1 for a flash sector

Prototype:

```
unsigned long  
DCSMGetZone1EXEstatus(unsigned long sector)
```

Parameters:

sector is the flash sector value. Valid ranges are 0 for sector 0 through 12 for sector 12.

Description:

This function takes in a valid sector value and returns the status of EXE ONLY security protection for the sector.

Returns:

Returns 0 if the sector is not EXE-ONLY protected , 1 if the sector is EXE-ONLY protected.

4.2.2.4 DCSMGetZone2EXEstatus

Returns the EXE-ONLY status of zone 2 for a flash sector

Prototype:

```
unsigned long  
DCSMGetZone2EXEstatus(unsigned long sector)
```

Parameters:

sector is the flash sector value. Valid ranges are 1 for sector 1 through 13 for sector 13.

Description:

This function takes in a valid sector value and returns the status of EXE ONLY security protection for the sector.

Returns:

Returns 0 if the sector is not EXE-ONLY protected , 1 if the sector is EXE-ONLY protected.

4.2.2.5 DCSMGrabFlashSectors

Allows access to flash sectors by reading zone select locations.

Prototype:

```
void  
DCSMGrabFlashSectors(void)
```

Description:

This function reads zone 1 and zone 2 GRABSECTCR registers, allowing access to flash memory sectors.

Returns:

None

4.2.2.6 DCSMGrabRAMBlocks

Allows access to RAM blocks by reading zone select locations.

Prototype:

```
void  
DCSMGrabRAMBlocks(void)
```

Description:

This function reads zone 1 and zone 2 GRABRAMR flash locations , allowing access to RAM memory blocks.

Returns:

None

4.2.2.7 DCSMReadZone1EXEONLY

Loads M3-Z1-EXEONLY register

Prototype:

```
void  
DCSMReadZone1EXEONLY(void)
```

Description:

This function reads M3_Z1_EXEONLY flash location transferring the contents of this location to M3_Z1_EXEONLY register

Returns:

None

4.2.2.8 DCSMReadZone2EXEONLY

Loads M3-Z2-EXEONLY register

Prototype:

```
void  
DCSMReadZone2EXEONLY(void)
```

Description:

This function reads M3_Z2_EXEONLY flash location transferring the contents of this location to M3_Z2_EXEONLY register

Returns:

None

4.2.2.9 DCSMSecureZone1

Secures zone 1 by setting the FORCESEC bit of M3_Z1_CR register

Prototype:

```
void  
DCSMSecureZone1(void)
```

Description:

This function locks and enables reloading of CSM and ECSL lock registers by setting the FORCESEC bit (bit 15 of M3_Z1_CR register).

Returns:

None

4.2.2.10 DCSMSecureZone2

Secures zone 2 by setting the FORCESEC bit of M3_Z2_CR register

Prototype:

```
void  
DCSMSecureZone2(void)
```

Description:

This function locks and enables reloading of CSM and ECSL lock registers by setting the FORCESEC bit (bit 15 of M3_Z2_CR register).

Returns:

None

4.2.2.11 DCSMUnlockZone1CSM

Unlocks Zone 1 CSM.

Prototype:

```
void  
DCSMUnlockZone1CSM(CSMPSWDKEY *psCMDKey)
```

Parameters:

psCMDKey is a pointer to the CSMPSWDKEY struct that has the CSM password for zone 1.

Description:

This function unlocks the CSM password. It first reads the four password locations in the flash. If any of the password values is different from 0xFFFFFFFF, it unlocks the device by writing the provided passwords into CSM Key registers

Returns:

None.

4.2.2.12 DCSMUnlockZone1ECSL

Unlocks Zone 1 ECSL.

Prototype:

```
void  
DCSMUnlockZone1ECSL (ECSLPSWDKEY *psECSLKey)
```

Parameters:

psECSLKey is a pointer to the ESCLPSWDKEY struct that has the ECSL password for zone 1.

Description:

This function unlocks the ECSL password. It first reads the two password locations in the flash. If any of the password values is different from 0xFFFFFFFF, it unlocks the device by writing the provided passwords into ECSL Key registers

Returns:

None.

4.2.2.13 DCSMUnlockZone2CSM

Unlocks Zone 2 CSM.

Prototype:

```
void  
DCSMUnlockZone2CSM (CSMPSWDKEY *psCMDKey)
```

Parameters:

psCMDKey is a pointer to the CSMPSWDKEY that has the CSM password for zone 2.

Description:

This function unlocks the CSM password. It first reads the four password locations in the flash. If any of the password values is different from 0xFFFFFFFF, it unlocks the device by writing the provided passwords into CSM Key registers

Returns:

None.

4.2.2.14 DCSMUnlockZone2ECSL

Unlocks Zone 2 ECSL.

Prototype:

```
void  
DCSMUnlockZone2ECSL (ECSLPSWDKEY *psECSLKey)
```

Parameters:

psECSLKey is a pointer to the ESCLPSWDKEY struct that has the ECSL password for zone 2.

Description:

This function unlocks the ECSL password. It first reads the two password locations in the flash. If any of the password values is different from 0xFFFFFFFF, it unlocks the device by writing the provided passwords into ECSL Key registers

Returns:

None.

4.2.2.15 DCSMZone1CSMSecurityStatus

Returns the CSM security status of zone 1

Prototype:

```
unsigned long  
DCSMZone1CSMSecurityStatus(void)
```

Description:

This function returns the security status of zone 1 CSM

Returns:

Returns 0 if zone is not CSM secure and 1 if zone is secure.

4.2.2.16 DCSMZone1ECSLSecurityStatus

Returns the ECSL security status of zone 1

Prototype:

```
unsigned long  
DCSMZone1ECSLSecurityStatus(void)
```

Description:

This function returns the security status of zone 1 ECSL

Returns:

Returns 0 if zone 2 is not ECSL secure and 1 if zone is ECSL secure

4.2.2.17 DCSMZone2CSMSecurityStatus

Returns the CSM security status of zone 2

Prototype:

```
unsigned long  
DCSMZone2CSMSecurityStatus(void)
```

Description:

This function returns the security status of zone 2 CSM

Returns:

Returns 0 if zone is not CSM secure, 1 if zone is secure and 2 if both CSM-ALLZERO and CSM-ALLONE bits are set(invalid combination).

4.2.2.18 DCSMZone2ECSLSecurityStatus

Returns the ECSL security status of zone 2

Prototype:

```
unsigned long  
DCSMZone2ECSLSecurityStatus(void)
```

Description:

This function returns the security status of zone 2 ECSL

Returns:

Returns 0 if zone 2 is not ECSL secure, 1 if zone is ECSL secure and 2 if both ECSL-ALLZERO and ECSL-ALLONE bits are set (invalid combination).

4.3 Programming Example

The following example shows how to unlock a code that has both zone1 and zone2 CSM locked with 0xFFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFE and both zone1 and zone2 ECSL locked with 0xFFFFFFFF FFFFFFFE

```
#include "dcsm.h"  
:  
// some code  
:  
// declare a structure to hold the password and a pointer  
// to the structure to pass to the API functions  
CSMPSWDKEY testZ1CMDKEY, testZ2CMDKEY, *ptrCMDKey;  
ECSLPSWDKEY testZ1ECSLKEY, testZ2ECSLKEY, *ptrECSLKEY;  
  
// Fill the structures with the correct key passwords  
testZ1CMDKEY.csmkey0 = 0xFFFFFFFFFE;  
testZ1CMDKEY.csmkey1 = 0xFFFFFFFFFF;  
testZ1CMDKEY.csmkey2 = 0xFFFFFFFFFF;  
testZ1CMDKEY.csmkey3 = 0xFFFFFFFFFF;  
  
testZ1ECSLKEY.ecslkey0 = 0xFFFFFFFFFE;  
testZ1ECSLKEY.ecslkey1 = 0xFFFFFFFFFF;  
  
testZ2CMDKEY.csmkey0 = 0xFFFFFFFFFE;  
testZ2CMDKEY.csmkey1 = 0xFFFFFFFFFF;  
testZ2CMDKEY.csmkey2 = 0xFFFFFFFFFF;  
testZ2CMDKEY.csmkey3 = 0xFFFFFFFFFF;  
  
testZ2ECSLKEY.ecslkey0 = 0xFFFFFFFFFE;  
testZ2ECSLKEY.ecslkey1 = 0xFFFFFFFFFF;  
  
ptrCMDKey = &testZ1CMDKEY;  
DCSMUnlockZone1CSM(ptrCMDKey);
```

```
ptrCMDKey = &testZ2CMDKEY;  
DCSMUnlockZone2CSM(ptrCMDKey);  
  
ptrECSLKEY = &testZ1ECSLKEY;  
DCSMUnlockZone1ECSL(ptrECSLKEY);  
  
ptrECSLKEY = &testZ2ECSLKEY;  
DCSMUnlockZone2ECSL(ptrECSLKEY);
```


5 Ethernet Controller

Introduction	49
API Functions	49
Programming Example	62

5.1 Introduction

The Concerto Ethernet controller consists of a fully integrated media access controller (MAC) and requires an external physical interface (PHY) . The Ethernet controller conforms to IEEE 802.3 specifications and fully supports 10BASE-T and 100BASE-TX standards.

The Ethernet API provides the set of functions required to implement an interrupt-driven Ethernet driver for this Ethernet controller. Functions are provided to configure and control the MAC, to access the register set on the PHY, to transmit and receive Ethernet packets, and to configure and control the interrupts that are available.

This driver is contained in `driverlib/ethernet.c`, with `driverlib/ethernet.h` containing the API definitions for use by applications.

5.2 API Functions

Functions

- unsigned long [EthernetConfigGet](#) (unsigned long ulBase)
- void [EthernetConfigSet](#) (unsigned long ulBase, unsigned long ulConfig)
- void [EthernetDisable](#) (unsigned long ulBase)
- void [EthernetEnable](#) (unsigned long ulBase)
- void [EthernetInitExpClk](#) (unsigned long ulBase, unsigned long ulEthClk)
- void [EthernetIntClear](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [EthernetIntDisable](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [EthernetIntEnable](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [EthernetIntRegister](#) (unsigned long ulBase, void (*pfnHandler)(void))
- unsigned long [EthernetIntStatus](#) (unsigned long ulBase, tBoolean bMasked)
- void [EthernetIntUnregister](#) (unsigned long ulBase)
- void [EthernetMACAddrGet](#) (unsigned long ulBase, unsigned char *pucMACAddr)
- void [EthernetMACAddrSet](#) (unsigned long ulBase, unsigned char *pucMACAddr)
- tBoolean [EthernetPacketAvail](#) (unsigned long ulBase)
- long [EthernetPacketGet](#) (unsigned long ulBase, unsigned char *pucBuf, long lBufLen)
- long [EthernetPacketGetNonBlocking](#) (unsigned long ulBase, unsigned char *pucBuf, long lBufLen)
- long [EthernetPacketPut](#) (unsigned long ulBase, unsigned char *pucBuf, long lBufLen)
- long [EthernetPacketPutNonBlocking](#) (unsigned long ulBase, unsigned char *pucBuf, long lBufLen)
- void [EthernetPHYPowerOff](#) (unsigned long ulBase)

- void [EthernetPHYPowerOn](#) (unsigned long ulBase)
- unsigned long [EthernetPHYRead](#) (unsigned long ulBase, unsigned char ucRegAddr)
- void [EthernetPHYWrite](#) (unsigned long ulBase, unsigned char ucRegAddr, unsigned long ulData)
- tBoolean [EthernetSpaceAvail](#) (unsigned long ulBase)

5.2.1 Detailed Description

For any application, the [EthernetInitExpClk\(\)](#) function must be called first to prepare the Ethernet controller for operation. This function will configure the Ethernet controller options that are based on system parameters, such as the system clock speed.

Once initialized, access to the PHY is available via the [EthernetPHYRead\(\)](#) and [EthernetPHYWrite\(\)](#) functions. By default, the PHY will auto-negotiate the line speed and duplex modes. For most applications, this will be sufficient. If a special configuration is required, the PHY read and write functions can be used to reconfigure the PHY to the desired mode of operation.

The MAC must also be configured using the [EthernetConfigSet\(\)](#) function. The parameters for this function will allow the configuration of options such as Promiscuous Mode, Multicast Reception, Transmit Data Length Padding, and so on. The [EthernetConfigGet\(\)](#) function can be used to query the current configuration of the Ethernet MAC.

The MAC address, used for incoming packet filtering, must also be programmed using the [EthernetMACAddrSet\(\)](#) function. The current value can be queried using the [EthernetMACAddrGet\(\)](#) function.

When configuration has been completed, the Ethernet controller can be enabled using the [EthernetEnable\(\)](#) function. When getting ready to terminate operations on the Ethernet controller, the [EthernetDisable\(\)](#) function may be called.

After the Ethernet controller has been enabled, Ethernet frames can be transmitted and received using the [EthernetPacketPut\(\)](#) and [EthernetPacketGet\(\)](#) functions. Care must be taken when using these functions, as they are blocking functions, and will not return until data is available (for RX) or buffer space is available (for TX). The [EthernetSpaceAvail\(\)](#) and [EthernetPacketAvail\(\)](#) functions can be called to determine if there is room for a TX packet or if there is an RX packet available prior to calling these blocking functions. Alternatively, the [EthernetPacketGetNonBlocking\(\)](#) and [EthernetPacketPutNonBlocking\(\)](#) functions will return immediately if a packet cannot be processed. Otherwise, the packet will be processed normally.

When developing a mapping layer for a TCP/IP stack, you may wish to use the interrupt capability of the Ethernet controller. The [EthernetIntRegister\(\)](#) and [EthernetIntUnregister\(\)](#) functions are used to register an ISR with the system and to enable or disable the Ethernet controller's interrupt signal. The [EthernetIntEnable\(\)](#) and [EthernetIntDisable\(\)](#) functions are used to manipulate the individual interrupt sources available in the Ethernet controller (for example, RX Error, TX Complete). The [EthernetIntStatus\(\)](#) and [EthernetIntClear\(\)](#) functions would be used to query the active interrupts to determine which process to service, and to clear the indicated interrupts prior to returning from the registered ISR.

5.2.2 Function Documentation

5.2.2.1 EthernetConfigGet

Gets the current configuration of the Ethernet controller.

Prototype:

```
unsigned long  
EthernetConfigGet(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the controller.

Description:

This function will query the control registers of the Ethernet controller and return a bit-mapped configuration value.

See also:

The description of the [EthernetConfigSet\(\)](#) function provides detailed information for the bit-mapped configuration values that will be returned.

Returns:

Returns the bit-mapped Ethernet controller configuration value.

5.2.2.2 EthernetConfigSet

Sets the configuration of the Ethernet controller.

Prototype:

```
void  
EthernetConfigSet(unsigned long ulBase,  
                  unsigned long ulConfig)
```

Parameters:

ulBase is the base address of the controller.

ulConfig is the configuration for the controller.

Description:

After the [EthernetInitExpClk\(\)](#) function has been called, this API function can be used to configure the various features of the Ethernet controller.

The Ethernet controller provides three control registers that are used to configure the controller's operation. The transmit control register provides settings to enable full duplex operation, to auto-generate the frame check sequence, and to pad the transmit packets to the minimum length as required by the IEEE standard. The receive control register provides settings to enable reception of packets with bad frame check sequence values and to enable multi-cast or promiscuous modes. The timestamp control register provides settings that enable support logic in the controller that allow the use of the General Purpose Timer 3 to capture timestamps for the transmitted and received packets.

The *ulConfig* parameter is the logical OR of the following values:

- **ETH_CFG_TS_TSEN** - Enable TX and RX interrupt status as CCP timer inputs

- **ETH_CFG_RX_BADCRCDIS** - Disable reception of packets with a bad CRC
- **ETH_CFG_RX_PRMTSEN** - Enable promiscuous mode reception (all packets)
- **ETH_CFG_RX_AMULEN** - Enable reception of multicast packets
- **ETH_CFG_TX_DPLXEN** - Enable full duplex transmit mode
- **ETH_CFG_TX_CRCEN** - Enable transmit with auto CRC generation
- **ETH_CFG_TX_PADEN** - Enable padding of transmit data to minimum size

These bit-mapped values are programmed into the transmit, receive, and/or timestamp control register.

Returns:

None.

5.2.2.3 EthernetDisable

Disables the Ethernet controller.

Prototype:

```
void  
EthernetDisable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the controller.

Description:

When terminating operations on the Ethernet interface, this function should be called. This function will disable the transmitter and receiver, and will clear out the receive FIFO.

Returns:

None.

5.2.2.4 EthernetEnable

Enables the Ethernet controller for normal operation.

Prototype:

```
void  
EthernetEnable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the controller.

Description:

Once the Ethernet controller has been configured using the [EthernetConfigSet\(\)](#) function and the MAC address has been programmed using the [EthernetMACAddrSet\(\)](#) function, this API function can be called to enable the controller for normal operation.

This function will enable the controller's transmitter and receiver, and will reset the receive FIFO.

Returns:

None.

5.2.2.5 EthernetInitExpClk

Initializes the Ethernet controller for operation.

Prototype:

```
void  
EthernetInitExpClk(unsigned long ulBase,  
                  unsigned long ulEthClk)
```

Parameters:

ulBase is the base address of the controller.

ulEthClk is the rate of the clock supplied to the Ethernet module.

Description:

This function will prepare the Ethernet controller for first time use in a given hardware/software configuration. This function should be called before any other Ethernet API functions are called.

The peripheral clock will be the same as the processor clock. This will be the value returned by [SysCtlClockGet\(\)](#), or it can be explicitly hard-coded if it is constant and known (to save the code/execution overhead of a call to [SysCtlClockGet\(\)](#)).

This function replaces the original EthernetInit() API and performs the same actions. A macro is provided in `ethernet.h` to map the original API to this API.

Note:

If the device configuration is changed (for example, the system clock is reprogrammed to a different speed), then the Ethernet controller must be disabled by calling the [EthernetDisable\(\)](#) function and the controller must be reinitialized by calling the [EthernetInitExpClk\(\)](#) function again. After the controller has been reinitialized, the controller should be reconfigured using the appropriate Ethernet API calls.

Returns:

None.

5.2.2.6 EthernetIntClear

Clears Ethernet interrupt sources.

Prototype:

```
void  
EthernetIntClear(unsigned long ulBase,  
                unsigned long ulIntFlags)
```

Parameters:

ulBase is the base address of the controller.

ulIntFlags is a bit mask of the interrupt sources to be cleared.

Description:

The specified Ethernet interrupt sources are cleared so that they no longer assert. This must be done in the interrupt handler to keep it from being called again immediately upon exit.

The *ulIntFlags* parameter has the same definition as the *ulIntFlags* parameter to [EthernetIntEnable\(\)](#).

Note:

Because there is a write buffer in the Cortex-M3 processor, it may take several clock cycles before the interrupt source is actually cleared. Therefore, it is recommended that the interrupt source be cleared early in the interrupt handler (as opposed to the very last action) to avoid returning from the interrupt handler before the interrupt source is actually cleared. Failure to do so may result in the interrupt handler being immediately reentered (because the interrupt controller still sees the interrupt source asserted).

Returns:

None.

5.2.2.7 EthernetIntDisable

Disables individual Ethernet interrupt sources.

Prototype:

```
void  
EthernetIntDisable(unsigned long ulBase,  
                   unsigned long ulIntFlags)
```

Parameters:

ulBase is the base address of the controller.

ulIntFlags is the bit mask of the interrupt sources to be disabled.

Description:

Disables the indicated Ethernet interrupt sources. Only the sources that are enabled can be reflected to the processor interrupt; disabled sources have no effect on the processor.

The *ulIntFlags* parameter has the same definition as the *ulIntFlags* parameter to [EthernetIntEnable\(\)](#).

Returns:

None.

5.2.2.8 EthernetIntEnable

Enables individual Ethernet interrupt sources.

Prototype:

```
void  
EthernetIntEnable(unsigned long ulBase,  
                  unsigned long ulIntFlags)
```

Parameters:

ulBase is the base address of the controller.

ulIntFlags is the bit mask of the interrupt sources to be enabled.

Description:

Enables the indicated Ethernet interrupt sources. Only the sources that are enabled can be reflected to the processor interrupt; disabled sources have no effect on the processor.

The *ulIntFlags* parameter is the logical OR of any of the following:

- **ETH_INT_PHY** - An interrupt from the PHY has occurred. The integrated PHY supports a number of interrupt conditions. The PHY register, PHY_MR17, must be read to determine which PHY interrupt has occurred. This register can be read using the [EthernetPHYRead\(\)](#) API function.
- **ETH_INT_MDIO** - This interrupt indicates that a transaction on the management interface has completed successfully.
- **ETH_INT_RXER** - This interrupt indicates that an error has occurred during reception of a frame. This error can indicate a length mismatch, a CRC failure, or an error indication from the PHY.
- **ETH_INT_RXOF** - This interrupt indicates that a frame has been received that exceeds the available space in the RX FIFO.
- **ETH_INT_TX** - This interrupt indicates that the packet stored in the TX FIFO has been successfully transmitted.
- **ETH_INT_TXER** - This interrupt indicates that an error has occurred during the transmission of a packet. This error can be either a retry failure during the back-off process, or an invalid length stored in the TX FIFO.
- **ETH_INT_RX** - This interrupt indicates that one (or more) packets are available in the RX FIFO for processing.

Returns:

None.

5.2.2.9 EthernetIntRegister

Registers an interrupt handler for an Ethernet interrupt.

Prototype:

```
void  
EthernetIntRegister(unsigned long ulBase,  
                    void (*pfnHandler)(void))
```

Parameters:

ulBase is the base address of the controller.

pfnHandler is a pointer to the function to be called when the enabled Ethernet interrupts occur.

Description:

This function sets the handler to be called when the Ethernet interrupt occurs. This will enable the global interrupt in the interrupt controller; specific Ethernet interrupts must be enabled via [EthernetIntEnable\(\)](#). It is the interrupt handler's responsibility to clear the interrupt source.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

5.2.2.10 EthernetIntStatus

Gets the current Ethernet interrupt status.

Prototype:

```
unsigned long  
EthernetIntStatus(unsigned long ulBase,  
                  tBoolean bMasked)
```

Parameters:

ulBase is the base address of the controller.

bMasked is false if the raw interrupt status is required and true if the masked interrupt status is required.

Description:

This returns the interrupt status for the Ethernet controller. Either the raw interrupt status or the status of interrupts that are allowed to reflect to the processor can be returned.

Returns:

Returns the current interrupt status, enumerated as a bit field of values described in [EthernetIntEnable\(\)](#).

5.2.2.11 EthernetIntUnregister

Unregisters an interrupt handler for an Ethernet interrupt.

Prototype:

```
void  
EthernetIntUnregister(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the controller.

Description:

This function unregisters the interrupt handler. This will disable the global interrupt in the interrupt controller so that the interrupt handler no longer is called.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

5.2.2.12 EthernetMACAddrGet

Gets the MAC address of the Ethernet controller.

Prototype:

```
void  
EthernetMACAddrGet(unsigned long ulBase,  
                   unsigned char *pucMACAddr)
```

Parameters:

ulBase is the base address of the controller.

pucMACAddr is the pointer to the location in which to store the array of MAC-48 address octets.

Description:

This function will read the currently programmed MAC address into the *pucMACAddr* buffer.

See also:

Refer to [EthernetMACAddrSet\(\)](#) API description for more details about the MAC address format.

Returns:

None.

5.2.2.13 EthernetMACAddrSet

Sets the MAC address of the Ethernet controller.

Prototype:

```
void  
EthernetMACAddrSet(unsigned long ulBase,  
                   unsigned char *pucMACAddr)
```

Parameters:

ulBase is the base address of the controller.

pucMACAddr is the pointer to the array of MAC-48 address octets.

Description:

This function will program the IEEE-defined MAC-48 address specified in *pucMACAddr* into the Ethernet controller. This address is used by the Ethernet controller for hardware-level filtering of incoming Ethernet packets (when promiscuous mode is not enabled).

The MAC-48 address is defined as 6 octets, illustrated by the following example address. The numbers are shown in hexadecimal format.

AC-DE-48-00-00-80

In this representation, the first three octets (AC-DE-48) are the Organizationally Unique Identifier (OUI). This is a number assigned by the IEEE to an organization that requests a block of MAC addresses. The last three octets (00-00-80) are a 24-bit number managed by the OUI owner to uniquely identify a piece of hardware within that organization that is to be connected to the Ethernet.

In this representation, the octets are transmitted from left to right, with the “AC” octet being transmitted first and the “80” octet being transmitted last. Within an octet, the bits are transmitted LSB to MSB. For this address, the first bit to be transmitted would be “0”, the LSB of “AC”, and the last bit to be transmitted would be “1”, the MSB of “80”.

Returns:

None.

5.2.2.14 EthernetPacketAvail

Check for packet available from the Ethernet controller.

Prototype:

```
tBoolean  
EthernetPacketAvail(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the controller.

Description:

The Ethernet controller provides a register that contains the number of packets available in the receive FIFO. When the last bytes of a packet are successfully received (that is, the frame check sequence bytes), the packet count is incremented. Once the packet has been fully read (including the frame check sequence bytes) from the FIFO, the packet count will be decremented.

Returns:

Returns **true** if there are one or more packets available in the receive FIFO, including the current packet being read, and **false** otherwise.

5.2.2.15 EthernetPacketGet

Waits for a packet from the Ethernet controller.

Prototype:

```
long
EthernetPacketGet(unsigned long ulBase,
                  unsigned char *pucBuf,
                  long lBufLen)
```

Parameters:

ulBase is the base address of the controller.

pucBuf is the pointer to the packet buffer.

lBufLen is the maximum number of bytes to be read into the buffer.

Description:

This function reads a packet from the receive FIFO of the controller and places it into *pucBuf*. The function will wait until a packet is available in the FIFO. Then the function will read the entire packet from the receive FIFO. If there are more bytes in the packet than will fit into *pucBuf* (as specified by *lBufLen*), the function will return the negated length of the packet and the buffer will contain *lBufLen* bytes of the packet. Otherwise, the function will return the length of the packet that was read and *pucBuf* will contain the entire packet (excluding the frame check sequence bytes).

Note:

This function is blocking and will not return until a packet arrives.

Returns:

Returns the negated packet length **-n** if the packet is too large for *pucBuf*, and returns the packet length **n** otherwise.

5.2.2.16 EthernetPacketGetNonBlocking

Receives a packet from the Ethernet controller.

Prototype:

```
long
EthernetPacketGetNonBlocking(unsigned long ulBase,
                             unsigned char *pucBuf,
                             long lBufLen)
```

Parameters:

ulBase is the base address of the controller.

pucBuf is the pointer to the packet buffer.

lBufLen is the maximum number of bytes to be read into the buffer.

Description:

This function reads a packet from the receive FIFO of the controller and places it into *pucBuf*. If no packet is available the function will return immediately. Otherwise, the function will read the entire packet from the receive FIFO. If there are more bytes in the packet than will fit into *pucBuf* (as specified by *lBufLen*), the function will return the negated length of the packet and the buffer will contain *lBufLen* bytes of the packet. Otherwise, the function will return the length of the packet that was read and *pucBuf* will contain the entire packet (excluding the frame check sequence bytes).

This function replaces the original EthernetPacketNonBlockingGet() API and performs the same actions. A macro is provided in `ethernet.h` to map the original API to this API.

Note:

This function will return immediately if no packet is available.

Returns:

Returns **0** if no packet is available, the negated packet length **-n** if the packet is too large for *pucBuf*, and the packet length **n** otherwise.

5.2.2.17 EthernetPacketPut

Waits to send a packet from the Ethernet controller.

Prototype:

```
long
EthernetPacketPut(unsigned long ulBase,
                  unsigned char *pucBuf,
                  long lBufLen)
```

Parameters:

ulBase is the base address of the controller.

pucBuf is the pointer to the packet buffer.

lBufLen is number of bytes in the packet to be transmitted.

Description:

This function writes *lBufLen* bytes of the packet contained in *pucBuf* into the transmit FIFO of the controller and then activates the transmitter for this packet. This function will wait until the transmit FIFO is empty. Once space is available, the function will return once *lBufLen* bytes of the packet have been placed into the FIFO and the transmitter has been started. The function will not wait for the transmission to complete. The function will return the negated *lBufLen* if the length is larger than the space available in the transmit FIFO.

Note:

This function blocks and will wait until space is available for the transmit packet before returning.

Returns:

Returns the negated packet length **-IBufLen** if the packet is too large for FIFO, and the packet length **IBufLen** otherwise.

5.2.2.18 EthernetPacketPutNonBlocking

Sends a packet to the Ethernet controller.

Prototype:

```
long  
EthernetPacketPutNonBlocking(unsigned long ulBase,  
                             unsigned char *pucBuf,  
                             long lBufLen)
```

Parameters:

ulBase is the base address of the controller.

pucBuf is the pointer to the packet buffer.

IBufLen is number of bytes in the packet to be transmitted.

Description:

This function writes *IBufLen* bytes of the packet contained in *pucBuf* into the transmit FIFO of the controller and then activates the transmitter for this packet. If no space is available in the FIFO, the function will return immediately. If space is available, the function will return once *IBufLen* bytes of the packet have been placed into the FIFO and the transmitter has been started. The function will not wait for the transmission to complete. The function will return the negated *IBufLen* if the length is larger than the space available in the transmit FIFO.

This function replaces the original EthernetPacketNonBlockingPut() API and performs the same actions. A macro is provided in `ethernet.h` to map the original API to this API.

Note:

This function does not block and will return immediately if no space is available for the transmit packet.

Returns:

Returns **0** if no space is available in the transmit FIFO, the negated packet length **-IBufLen** if the packet is too large for FIFO, and the packet length **IBufLen** otherwise.

5.2.2.19 EthernetPHYPowerOff

Powers off the Ethernet PHY.

Prototype:

```
void  
EthernetPHYPowerOff(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the controller.

Description:

This function will power off the Ethernet PHY, reducing the current consumption of the device. While in the powered off state, the Ethernet controller will be unable to connect to the Ethernet.

Returns:

None.

5.2.2.20 EthernetPHYPowerOn

Powers on the Ethernet PHY.

Prototype:

```
void  
EthernetPHYPowerOn(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the controller.

Description:

This function will power on the Ethernet PHY, enabling it return to normal operation. By default, the PHY is powered on, so this function only needs to be called if [EthernetPHYPowerOff\(\)](#) has previously been called.

Returns:

None.

5.2.2.21 EthernetPHYRead

Reads from a PHY register.

Prototype:

```
unsigned long  
EthernetPHYRead(unsigned long ulBase,  
                unsigned char ucRegAddr)
```

Parameters:

ulBase is the base address of the controller.

ucRegAddr is the address of the PHY register to be accessed.

Description:

This function will return the contents of the PHY register specified by *ucRegAddr*.

Returns:

Returns the 16-bit value read from the PHY.

5.2.2.22 EthernetPHYWrite

Writes to the PHY register.

Prototype:

```
void  
EthernetPHYWrite(unsigned long ulBase,  
                 unsigned char ucRegAddr,  
                 unsigned long ulData)
```

Parameters:

ulBase is the base address of the controller.

ucRegAddr is the address of the PHY register to be accessed.

ulData is the data to be written to the PHY register.

Description:

This function will write the *ulData* to the PHY register specified by *ucRegAddr*.

Returns:

None.

5.2.2.23 EthernetSpaceAvail

Checks for packet space available in the Ethernet controller.

Prototype:

```
tBoolean  
EthernetSpaceAvail(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the controller.

Description:

The Ethernet controller's transmit FIFO is designed to support a single packet at a time. After the packet has been written into the FIFO, the transmit request bit must be set to enable the transmission of the packet. Only after the packet has been transmitted can a new packet be written into the FIFO. This function will simply check to see if a packet is in progress. If so, there is no space available in the transmit FIFO.

Returns:

Returns **true** if a space is available in the transmit FIFO, and **false** otherwise.

5.3 Programming Example

The following example shows how to use the this API to initialize the Ethernet controller to transmit and receive packets.

```
unsigned char pucMACAddress[6];  
unsigned char pucMyRxPacket[];  
unsigned char pucMyTxPacket[];  
unsigned long ulMyTxPacketLength;  
  
// Initialize the Ethernet controller for operation  
EthernetInitExpClk(ETH_BASE, SysCtlClockGet());
```

```
// Configure the Ethernet controller for normal operation
// Enable TX Duplex Mode
// Enable TX Padding
EthernetConfigSet(ETH_BASE, (ETH_CFG_TX_DPLXEN | ETH_CFG_TX_PADEN));

// Program the MAC Address (01-23-45-67-89-AB)
pucMACAddress[0] = 0x01;
pucMACAddress[1] = 0x23;
pucMACAddress[2] = 0x45;
pucMACAddress[3] = 0x67;
pucMACAddress[4] = 0x89;
pucMACAddress[5] = 0xAB;
EthernetMACAddrSet(ETH_BASE, pucMACAddress);

// Enable the Ethernet controller
EthernetEnable(ETH_BASE);

// Send a packet.
// (assume that the packet has been filled in appropriately
// elsewhere in the code).
EthernetPacketPut(ETH_BASE, pucMyTxPacket, ulMyTxPacketLength);

// Wait for a packet to come in.
EthernetPacketGet(ETH_BASE, pucMyRxPacket, sizeof(pucMyRxPacket));
```


6 External Peripheral Interface (EPI)

Introduction	65
API Functions	65

6.1 Introduction

The EPI API provides functions to use the EPI module available in the Concerto microcontroller. The EPI module provides a physical interface for external peripherals and memories. The EPI can be configured to support several types of external interfaces and different sized address and data buses.

Some features of the EPI module are:

- configurable interface modes including SDRAM, HostBus, and simple read/write protocols
- configurable address and data sizes
- configurable bus cycle timing
- blocking and non-blocking reads and writes
- FIFO for streaming reads
- interrupt and uDMA support

This driver is contained in `driverlib/epi.c`, with `driverlib/epi.h` containing the API definitions for use by applications.

6.2 API Functions

Functions

- void [EPIAddressMapSet](#) (unsigned long ulBase, unsigned long ulMap)
- void [EPIConfigGPModeSet](#) (unsigned long ulBase, unsigned long ulConfig, unsigned long ulFrameCount, unsigned long ulMaxWait)
- void [EPIConfigHB16Set](#) (unsigned long ulBase, unsigned long ulConfig, unsigned long ulMaxWait)
- void [EPIConfigHB8Set](#) (unsigned long ulBase, unsigned long ulConfig, unsigned long ulMaxWait)
- void [EPIConfigSDRAMSet](#) (unsigned long ulBase, unsigned long ulConfig, unsigned long ulRefresh)
- void [EPIDividerSet](#) (unsigned long ulBase, unsigned long ulDivider)
- void [EPIFIFOConfig](#) (unsigned long ulBase, unsigned long ulConfig)
- void [EPIIntDisable](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [EPIIntEnable](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [EPIIntErrorClear](#) (unsigned long ulBase, unsigned long ulErrFlags)
- unsigned long [EPIIntErrorStatus](#) (unsigned long ulBase)

- void [EPIIntRegister](#) (unsigned long ulBase, void (*pfnHandler)(void))
- unsigned long [EPIIntStatus](#) (unsigned long ulBase, tBoolean bMasked)
- void [EPIIntUnregister](#) (unsigned long ulBase)
- void [EPIModeSet](#) (unsigned long ulBase, unsigned long ulMode)
- unsigned long [EPINonBlockingReadAvail](#) (unsigned long ulBase)
- void [EPINonBlockingReadConfigure](#) (unsigned long ulBase, unsigned long ulChannel, unsigned long ulDataSize, unsigned long ulAddress)
- unsigned long [EPINonBlockingReadCount](#) (unsigned long ulBase, unsigned long ulChannel)
- unsigned long [EPINonBlockingReadGet16](#) (unsigned long ulBase, unsigned long ulCount, unsigned short *pusBuf)
- unsigned long [EPINonBlockingReadGet32](#) (unsigned long ulBase, unsigned long ulCount, unsigned long *pulBuf)
- unsigned long [EPINonBlockingReadGet8](#) (unsigned long ulBase, unsigned long ulCount, unsigned char *pucBuf)
- void [EPINonBlockingReadStart](#) (unsigned long ulBase, unsigned long ulChannel, unsigned long ulCount)
- void [EPINonBlockingReadStop](#) (unsigned long ulBase, unsigned long ulChannel)
- unsigned long [EPIWriteFIFOCountGet](#) (unsigned long ulBase)

6.2.1 Detailed Description

The function [EPIModeSet\(\)](#) is used to select the interface mode. The clock divider is set with the [EPIDividerSet\(\)](#) function which will determine the speed of the external bus. The external device is mapped into the processor memory or peripheral space using the [EPIAddressMapSet\(\)](#) function.

Once the mode is selected, the interface is configured with one of the configuration functions. If SDRAM mode was chosen, the the function [EPIConfigSDRAMSet\(\)](#) is used to configure the SDRAM interface. If a non-moded interface was selected, then the function [EPIConfigNoModeSet\(\)](#) should be used.

After the mode has been selected and configured, then the device can be accessed by reading and writing to the memory or peripheral address space that was programmed with [EPIAddressMapSet\(\)](#).

There are more sophisticated ways to use the read/write interface. When an application is writing to the mapped memory or peripheral space, the writes will stall the processor until the write to the external interface is completed. However, the EPI contains an internal transaction FIFO and can buffer up to 4 pending writes without stalling the processor. Prior to writing, the application can test to see if the EPI can take more write operations without stalling the processor by using the function [EPINonBlockingWriteCount\(\)](#) which will return the number of non-blocking writes that can be made.

For efficient reads from the external device, the EPI contains a programmable read FIFO. This can be used to set a starting address and a count, and the FIFO will perform sequential reads from the device and store the values in the FIFO. The application can then periodically drain the FIFO either by polling, or by interrupts, or by using the uDMA controller. A non-blocking read is configured by using the function [EPINonBlockingReadConfigure\(\)](#). The read operation is started with [EPINonBlockingReadStart\(\)](#) and can be stopped by calling [EPINonBlockingReadStop\(\)](#). The function [EPINonBlockingReadCount\(\)](#) can be used to determine the number of items remaining to be read, while the function [EPINonBlockingReadAvail\(\)](#) returns the number of items in the FIFO that can be read immediately without stalling. There are 3 functions available for reading data

from the FIFO and into a buffer provided by the application. These functions are [EPINonBlockingReadGet32\(\)](#), [EPINonBlockingReadGet16\(\)](#), [EPINonBlockingReadGet8\(\)](#), to read the data from the FIFO as 32-bit, 16-bit, or 8-bit data items.

The read FIFO and write transaction FIFO can be configured with the function [EPIFIFOConfig\(\)](#). This function is used to set the FIFO trigger levels, and to enable error interrupts to be generated when a read or write is stalled.

Interrupts are enabled or disabled with the functions [EPIIntEnable\(\)](#) and [EPIIntDisable\(\)](#). The interrupt status can be read by calling [EPIIntStatus\(\)](#). If there is an error interrupt pending, the cause of the error can be determined with the function [EPIIntErrorStatus\(\)](#). The error can then be cleared with [EPIIntErrorClear\(\)](#).

If dynamic interrupt registration is being used by the application, then an EPI interrupt handler can be registered by calling [EPIIntRegister\(\)](#). This will load the interrupt handler's address into the vector table. The handler can be removed with [EPIIntUnregister\(\)](#).

6.2.2 Function Documentation

6.2.2.1 EPIAddressMapSet

Configures the address map for the external interface.

Prototype:

```
void  
EPIAddressMapSet(unsigned long ulBase,  
                 unsigned long ulMap)
```

Parameters:

ulBase is the EPI module base address.
ulMap is the address mapping configuration.

Description:

This function is used to configure the address mapping for the external interface. This determines the base address of the external memory or device within the processor peripheral and/or memory space.

The parameter *ulMap* is the logical OR of the following:

- **EPI_ADDR_CODE_SIZE_256B**, **EPI_ADDR_CODE_SIZE_64KB**,
EPI_ADDR_CODE_SIZE_16MB, or **EPI_ADDR_CODE_SIZE_512MB** to choose a
code address space of 256 bytes, 64 Kbytes, 16 Mbytes or 512 Mbytes
- **EPI_ADDR_CODE_BASE_NONE**, **EPI_ADDR_CODE_BASE_1** to choose the base ad-
dress of the code space as none or 0x10000000
- **EPI_ADDR_PER_SIZE_256B**, **EPI_ADDR_PER_SIZE_64KB**,
EPI_ADDR_PER_SIZE_16MB, or **EPI_ADDR_PER_SIZE_512MB** to choose a pe-
ripheral address space of 256 bytes, 64 Kbytes, 16 Mbytes or 512 Mbytes
- **EPI_ADDR_PER_BASE_NONE**, **EPI_ADDR_PER_BASE_A**, or
EPI_ADDR_PER_BASE_C to choose the base address of the peripheral space as
none, 0xA0000000, or 0xC0000000
- **EPI_ADDR_RAM_SIZE_256B**, **EPI_ADDR_RAM_SIZE_64KB**,
EPI_ADDR_RAM_SIZE_16MB, or **EPI_ADDR_RAM_SIZE_512MB** to choose a RAM
address space of 256 bytes, 64 Kbytes, 16 Mbytes or 512 Mbytes

- **EPI_ADDR_RAM_BASE_NONE**, **EPI_ADDR_RAM_BASE_6**, or **EPI_ADDR_RAM_BASE_8** to choose the base address of the RAM space as none, 0x60000000, or 0x80000000

Returns:

None.

6.2.2.2 EPIConfigGPMoDeSet

Configures the interface for general-purpose mode operation.

Prototype:

```
void  
EPIConfigGPMoDeSet(unsigned long ulBase,  
                    unsigned long ulConfig,  
                    unsigned long ulFrameCount,  
                    unsigned long ulMaxWait)
```

Parameters:

ulBase is the EPI module base address.

ulConfig is the interface configuration.

ulFrameCount is the frame size in clocks, if the frame signal is used (0-15).

ulMaxWait is the maximum number of external clocks to wait when the external clock enable is holding off the transaction (0-255).

Description:

This function is used to configure the interface when used in general-purpose operation as chosen with the function [EPIMoDeSet\(\)](#). The parameter *ulConfig* is the logical OR of any of the following:

- **EPI_GPMODE_CLKPIN** - interface clock is output on a pin
- **EPI_GPMODE_CLKGATE** - clock is stopped when there is no transaction, otherwise it is free-running
- **EPI_GPMODE_RDYEN** - the external peripheral drives an iRDY signal into pin EPI0S27. If absent, the peripheral is assumed to be ready at all times. This flag may only be used with a free-running clock (**EPI_GPMODE_CLKGATE** is absent).
- **EPI_GPMODE_FRAMEPIN** - framing signal is emitted on a pin
- **EPI_GPMODE_FRAME50** - framing signal is 50/50 duty cycle, otherwise it is a pulse
- **EPI_GPMODE_READWRITE** - read and write strobes are emitted on pins
- **EPI_GPMODE_WRITE2CYCLE** - a two cycle write is used, otherwise a single-cycle write is used
- **EPI_GPMODE_READ2CYCLE** - a two cycle read is used, otherwise a single-cycle read is used
- **EPI_GPMODE_ASIZE_NONE**, **EPI_GPMODE_ASIZE_4**, **EPI_GPMODE_ASIZE_12**, or **EPI_GPMODE_ASIZE_20** to choose no address bus, or an address bus size of 4, 12, or 20 bits
- **EPI_GPMODE_DSIZE_8**, **EPI_GPMODE_DSIZE_16**, **EPI_GPMODE_DSIZE_24**, or **EPI_GPMODE_DSIZE_32** to select a data bus size of 8, 16, 24, or 32 bits
- **EPI_GPMODE_WORD_ACCESS** - use Word Access mode to route bytes to the correct byte lanes allowing data to be stored in the upper bits of the word when necessary.

The parameter *ulFrameCount* is the number of clocks used to form the framing signal, if the framing signal is used. The behavior depends on whether the frame signal is a pulse or a 50/50 duty cycle. This value is not used if the framing signal is not enabled with the option **EPI_GPMODE_FRAMEPIN**.

The parameter *ulMaxWait* is used if the external clock enable is turned on with the **EPI_GPMODE_CLKENA** option is used. In the case that external clock enable is used, this parameter determines the maximum number of clocks to wait when the external clock enable signal is holding off a transaction. A value of 0 means to wait forever. If a non-zero value is used and exceeded, an interrupt will occur and the transaction aborted.

Returns:

None.

6.2.2.3 EPIConfigHB16Set

Configures the interface for Host-bus 16 operation.

Prototype:

```
void  
EPIConfigHB16Set(unsigned long ulBase,  
                 unsigned long ulConfig,  
                 unsigned long ulMaxWait)
```

Parameters:

ulBase is the EPI module base address.

ulConfig is the interface configuration.

ulMaxWait is the maximum number of external clocks to wait if a FIFO ready signal is holding off the transaction.

Description:

This function is used to configure the interface when used in Host-bus 16 operation as chosen with the function [EPIModeSet\(\)](#). The parameter *ulConfig* is the logical OR of any of the following:

- one of **EPI_HB16_MODE_ADMUX**, **EPI_HB16_MODE_ADDEMUX**, **EPI_HB16_MODE_SRAM**, or **EPI_HB16_MODE_FIFO** to select the HB16 mode
- **EPI_HB16_USE_TXEMPTY** - enable TXEMPTY signal with FIFO
- **EPI_HB16_USE_RXFULL** - enable RXFULL signal with FIFO
- **EPI_HB16_WRHIGH** - use active high write strobe, otherwise it is active low
- **EPI_HB16_RDHIGH** - use active high read strobe, otherwise it is active low
- one of **EPI_HB16_WRWAIT_0**, **EPI_HB16_WRWAIT_1**, **EPI_HB16_WRWAIT_2**, or **EPI_HB16_WRWAIT_3** to select the number of write wait states (default is 0 wait states)
- one of **EPI_HB16_RDWAIT_0**, **EPI_HB16_RDWAIT_1**, **EPI_HB16_RDWAIT_2**, or **EPI_HB16_RDWAIT_3** to select the number of read wait states (default is 0 wait states)
- **EPI_HB16_WORD_ACCESS** - use Word Access mode to route bytes to the correct byte lanes allowing data to be stored in bits [31:8]. If absent, all data transfers use bits [7:0].
- **EPI_HB16_BSEL** - enables byte selects. In this mode, two EPI signals operate as byte selects allowing 8-bit transfers. If this flag is not specified, data must be read and written using only 16-bit transfers.

- **EPI_HB16_CSBAUD_DUAL** - use different baud rates when accessing devices on each CSn. CS0n uses the baud rate specified by the lower 16 bits of the divider passed to [EPIDividerSet\(\)](#) and CS1n uses the divider passed in the upper 16 bits. If this option is absent, both chip selects use the baud rate resulting from the divider in the lower 16 bits of the parameter passed to [EPIDividerSet\(\)](#).
- one of **EPI_HB16_CSCFG_CS**, **EPI_HB16_CSCFG_ALE**, **EPI_HB16_CSCFG_DUAL_CS** or **EPI_HB16_CSCFG_ALE_DUAL_CS**. **EPI_HB16_CSCFG_CS** sets EPI30 to operate as a Chip Select (CSn) signal. **EPI_HB16_CSCFG_ALE** sets EPI30 to operate as an address latch (ALE). **EPI_HB16_CSCFG_DUAL_CS** sets EPI30 to operate as CS0n and EPI27 as CS1n with the asserted chip select determined from the most significant address bit for the respective external address map. **EPI_HB16_CSCFG_ALE_DUAL_CS** sets EPI30 as an address latch (ALE), EPI27 as CS0n and EPI26 as CS1n with the asserted chip select determined from the most significant address bit for the respective external address map.

The parameter *ulMaxWait* is used if the FIFO mode is chosen. If a FIFO is used along with RXFULL or TXEMPTY ready signals, then this parameter determines the maximum number of clocks to wait when the transaction is being held off by the FIFO using one of these ready signals. A value of 0 means to wait forever.

Returns:

None.

6.2.2.4 EPIConfigHB8Set

Configures the interface for Host-bus 8 operation.

Prototype:

```
void  
EPIConfigHB8Set(unsigned long ulBase,  
                 unsigned long ulConfig,  
                 unsigned long ulMaxWait)
```

Parameters:

ulBase is the EPI module base address.

ulConfig is the interface configuration.

ulMaxWait is the maximum number of external clocks to wait if a FIFO ready signal is holding off the transaction.

Description:

This function is used to configure the interface when used in Host-bus 8 operation as chosen with the function [EPIModeSet\(\)](#). The parameter *ulConfig* is the logical OR of any of the following:

- one of **EPI_HB8_MODE_ADMUX**, **EPI_HB8_MODE_ADDEMUX**, **EPI_HB8_MODE_SRAM**, or **EPI_HB8_MODE_FIFO** to select the HB8 mode
- **EPI_HB8_USE_TXEMPTY** - enable TXEMPTY signal with FIFO
- **EPI_HB8_USE_RXFULL** - enable RXFULL signal with FIFO
- **EPI_HB8_WRHIGH** - use active high write strobe, otherwise it is active low
- **EPI_HB8_RDHIGH** - use active high read strobe, otherwise it is active low
- one of **EPI_HB8_WRWAIT_0**, **EPI_HB8_WRWAIT_1**, **EPI_HB8_WRWAIT_2**, or **EPI_HB8_WRWAIT_3** to select the number of write wait states (default is 0 wait states)

- one of **EPI_HB8_RDWAIT_0**, **EPI_HB8_RDWAIT_1**, **EPI_HB8_RDWAIT_2**, or **EPI_HB8_RDWAIT_3** to select the number of read wait states (default is 0 wait states)
- **EPI_HB8_WORD_ACCESS** - use Word Access mode to route bytes to the correct byte lanes allowing data to be stored in bits [31:8]. If absent, all data transfers use bits [7:0].
- **EPI_HB8_CSBAUD_DUAL** - use different baud rates when accessing devices on each CSn. CS0n uses the baud rate specified by the lower 16 bits of the divider passed to [EPIDividerSet\(\)](#) and CS1n uses the divider passed in the upper 16 bits. If this option is absent, both chip selects use the baud rate resulting from the divider in the lower 16 bits of the parameter passed to [EPIDividerSet\(\)](#).
- one of **EPI_HB8_CSCFG_CS**, **EPI_HB8_CSCFG_ALE**, **EPI_HB8_CSCFG_DUAL_CS** or **EPI_HB8_CSCFG_ALE_DUAL_CS**. **EPI_HB8_CSCFG_CS** sets EPI30 to operate as a Chip Select (CSn) signal. **EPI_HB8_CSCFG_ALE** sets EPI30 to operate as an address latch (ALE). **EPI_HB8_CSCFG_DUAL_CS** sets EPI30 to operate as CS0n and EPI27 as CS1n with the asserted chip select determined from the most significant address bit for the respective external address map. **EPI_HB8_CSCFG_ALE_DUAL_CS** sets EPI30 as an address latch (ALE), EPI27 as CS0n and EPI26 as CS1n with the asserted chip select determined from the most significant address bit for the respective external address map.

The parameter *ulMaxWait* is used if the FIFO mode is chosen. If a FIFO is used along with RXFULL or TXEMPTY ready signals, then this parameter determines the maximum number of clocks to wait when the transaction is being held off by the FIFO using one of these ready signals. A value of 0 means to wait forever.

Returns:

None.

6.2.2.5 EPIConfigSDRAMSet

Configures the SDRAM mode of operation.

Prototype:

```
void  
EPIConfigSDRAMSet(unsigned long ulBase,  
                  unsigned long ulConfig,  
                  unsigned long ulRefresh)
```

Parameters:

ulBase is the EPI module base address.

ulConfig is the SDRAM interface configuration.

ulRefresh is the refresh count in core clocks (0-2047).

Description:

This function is used to configure the SDRAM interface, when the SDRAM mode is chosen with the function [EPIModeSet\(\)](#). The parameter *ulConfig* is the logical OR of several sets of choices:

The processor core frequency must be specified with one of the following:

- **EPI_SDRAM_CORE_FREQ_0_15** - core clock is 0 MHz < clk <= 15 MHz
- **EPI_SDRAM_CORE_FREQ_15_30** - core clock is 15 MHz < clk <= 30 MHz
- **EPI_SDRAM_CORE_FREQ_30_50** - core clock is 30 MHz < clk <= 50 MHz

- **EPI_SDRAM_CORE_FREQ_50_100** - core clock is 50 MHz < clk <= 100 MHz

The low power mode is specified with one of the following:

- **EPI_SDRAM_LOW_POWER** - enter low power, self-refresh state
- **EPI_SDRAM_FULL_POWER** - normal operating state

The SDRAM device size is specified with one of the following:

- **EPI_SDRAM_SIZE_64MBIT** - 64 Mbit device (8 MB)
- **EPI_SDRAM_SIZE_128MBIT** - 128 Mbit device (16 MB)
- **EPI_SDRAM_SIZE_256MBIT** - 256 Mbit device (32 MB)
- **EPI_SDRAM_SIZE_512MBIT** - 512 Mbit device (64 MB)

The parameter *ulRefresh* sets the refresh counter in units of core clock ticks. It is an 11-bit value with a range of 0 - 2047 counts.

Returns:

None.

6.2.2.6 EPIDividerSet

Sets the clock divider for the EPI module.

Prototype:

```
void
EPIDividerSet(unsigned long ulBase,
               unsigned long ulDivider)
```

Parameters:

ulBase is the EPI module base address.

ulDivider is the value of the clock divider to be applied to the external interface (0-65535).

Description:

This functions sets the clock divider(s) that will be used to determine the clock rate of the external interface. The *ulDivider* value is used to derive the EPI clock rate from the system clock based upon the following formula.

$$\text{EPIClock} = (\text{Divider} == 0) ? \text{SysClk} : (\text{SysClk} / (((\text{Divider} / 2) + 1) * 2))$$

For example, a divider value of 1 results in an EPI clock rate of half the system clock, value of 2 or 3 yield one quarter of the system clock and a value of 4 results in one sixth of the system clock rate.

In cases where a dual chip select mode is in use and different clock rates are required for each chip select, the *ulDivider* parameter must contain two dividers. The lower 16 bits define the divider to be used with CS0n and the upper 16 bits define the divider for CS1n.

Returns:

None.

6.2.2.7 EPIFIFOConfig

Configures the read FIFO.

Prototype:

```
void  
EPIFIFOConfig(unsigned long ulBase,  
              unsigned long ulConfig)
```

Parameters:

ulBase is the EPI module base address.

ulConfig is the FIFO configuration.

Description:

This function configures the FIFO trigger levels and error generation. The parameter *ulConfig* is the logical OR of the following:

- **EPI_FIFO_CONFIG_WTFULLERR** - enables an error interrupt when a write is attempted and the write FIFO is full
- **EPI_FIFO_CONFIG_RSTALLERR** - enables an error interrupt when a read is stalled due to an interleaved write or other reason
- **EPI_FIFO_CONFIG_TX_EMPTY**, **EPI_FIFO_CONFIG_TX_1_4**, **EPI_FIFO_CONFIG_TX_1_2**, or **EPI_FIFO_CONFIG_TX_3_4** to set the TX FIFO trigger level to empty, 1/4, 1/2, or 3/4 level
- **EPI_FIFO_CONFIG_RX_1_8**, **EPI_FIFO_CONFIG_RX_1_4**, **EPI_FIFO_CONFIG_RX_1_2**, **EPI_FIFO_CONFIG_RX_3_4**, **EPI_FIFO_CONFIG_RX_7_8**, or **EPI_FIFO_CONFIG_RX_FULL** to set the RX FIFO trigger level to 1/8, 1/4, 1/2, 3/4, 7/8 or full level

Returns:

None.

6.2.2.8 EPIIntDisable

Disables EPI interrupt sources.

Prototype:

```
void  
EPIIntDisable(unsigned long ulBase,  
              unsigned long ulIntFlags)
```

Parameters:

ulBase is the EPI module base address.

ulIntFlags is a bit mask of the interrupt sources to be disabled.

Description:

This function disables the specified EPI sources for interrupt generation. The *ulIntFlags* parameter can be the logical OR of any of the following values: **EPI_INT_UDMA_TXREQ**, **EPI_INT_UDMA_RXREQ**, **EPI_INT_RXREQ**, **EPI_INT_TXREQ**, or **I2S_INT_ERR**.

Returns:

Returns None.

6.2.2.9 EPIIntEnable

Enables EPI interrupt sources.

Prototype:

```
void  
EPIIntEnable(unsigned long ulBase,  
              unsigned long ulIntFlags)
```

Parameters:

ulBase is the EPI module base address.

ulIntFlags is a bit mask of the interrupt sources to be enabled.

Description:

This function enables the specified EPI sources to generate interrupts. The *ulIntFlags* parameter can be the logical OR of any of the following values:

- **EPI_INT_UDMA_TXREQ** - allows uDMA write to trigger an interrupt
- **EPI_INT_UDMA_RXREQ** - allows uDMA read to trigger an interrupt
- **EPI_INT_TXREQ** - transmit FIFO is below the trigger level
- **EPI_INT_RXREQ** - read FIFO is above the trigger level
- **EPI_INT_ERR** - an error condition occurred

Returns:

Returns None.

6.2.2.10 EPIIntErrorClear

Clears pending EPI error sources.

Prototype:

```
void  
EPIIntErrorClear(unsigned long ulBase,  
                  unsigned long ulErrFlags)
```

Parameters:

ulBase is the EPI module base address.

ulErrFlags is a bit mask of the error sources to be cleared.

Description:

This function clears the specified pending EPI errors. The *ulErrFlags* parameter can be the logical OR of any of the following values: **EPI_INT_CLR_DMAWRIC**, **EPI_INT_CLR_DMARDIC**, **EPI_INT_ERR_WTFULL**, **EPI_INT_ERR_RSTALL**, or **EPI_INT_ERR_TIMEOUT**.

Returns:

Returns None.

6.2.2.11 EPIIntErrorStatus

Gets the EPI error interrupt status.

Prototype:

```
unsigned long  
EPIIntErrorStatus(unsigned long ulBase)
```

Parameters:

ulBase is the EPI module base address.

Description:

This function returns the error status of the EPI. If the return value of the function [EPIIntStatus\(\)](#) has the flag **EPI_INT_ERR** set, then this function can be used to determine the cause of the error.

This function returns a bit mask of error flags, which can be the logical OR of any of the following:

- **EPI_INT_CLR_DMAWRIC** - occurs when a write uDMA has completed
- **EPI_INT_CLR_DMARDIC** - occurs when a read uDMA has completed
- **EPI_INT_ERR_WTFULL** - occurs when a write stalled when the transaction FIFO was full
- **EPI_INT_ERR_RSTALL** - occurs when a read stalled
- **EPI_INT_ERR_TIMEOUT** - occurs when the external clock enable held off a transaction longer than the configured maximum wait time

Returns:

Returns the interrupt error flags as the logical OR of any of the following: **EPI_INT_ERR_WTFULL**, **EPI_INT_ERR_RSTALL**, or **EPI_INT_ERR_TIMEOUT**.

6.2.2.12 EPIIntRegister

Registers an interrupt handler for the EPI module.

Prototype:

```
void  
EPIIntRegister(unsigned long ulBase,  
               void (*pfnHandler)(void))
```

Parameters:

ulBase is the EPI module base address.

pfnHandler is a pointer to the function to be called when the interrupt is activated.

Description:

This sets and enables the handler to be called when the EPI module generates an interrupt. Specific EPI interrupts must still be enabled with the [EPIIntEnable\(\)](#) function.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

6.2.2.13 EPIIntStatus

Gets the EPI interrupt status.

Prototype:

```
unsigned long  
EPIIntStatus(unsigned long ulBase,  
              tBoolean bMasked)
```

Parameters:

ulBase is the EPI module base address.

bMasked is set **true** to get the masked interrupt status, or **false** to get the raw interrupt status.

Description:

This function returns the EPI interrupt status. It can return either the raw or masked interrupt status.

Returns:

Returns the masked or raw EPI interrupt status, as a bit field of any of the following values: **EPI_INT_UDMA_TXREQ**, **EPI_INT_UDMA_RXREQ**, **EPI_INT_TXREQ**, **EPI_INT_RXREQ**, or **EPI_INT_ERR**

6.2.2.14 EPIIntUnregister

Unregisters an interrupt handler for the EPI module.

Prototype:

```
void  
EPIIntUnregister(unsigned long ulBase)
```

Parameters:

ulBase is the EPI module base address.

Description:

This function will disable and clear the handler to be called when the EPI interrupt occurs.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

6.2.2.15 EPIModeSet

Sets the usage mode of the EPI module.

Prototype:

```
void  
EPIModeSet(unsigned long ulBase,  
            unsigned long ulMode)
```

Parameters:

ulBase is the EPI module base address.
ulMode is the usage mode of the EPI module.

Description:

This functions sets the operating mode of the EPI module. The parameter *ulMode* must be one of the following:

- **EPI_MODE_GENERAL** - use for general-purpose mode operation
- **EPI_MODE_SDRAM** - use with SDRAM device
- **EPI_MODE_HB8** - use with host-bus 8-bit interface
- **EPI_MODE_HB16** - use with host-bus 16-bit interface
- **EPI_MODE_DISABLE** - disable the EPI module

Selection of any of the above modes will enable the EPI module, except for **EPI_MODE_DISABLE** which should be used to disable the module.

Returns:

None.

6.2.2.16 EPINonBlockingReadAvail

Get the count of items available in the read FIFO.

Prototype:

```
unsigned long  
EPINonBlockingReadAvail(unsigned long ulBase)
```

Parameters:

ulBase is the EPI module base address.

Description:

This function gets the number of items that are available to read in the read FIFO. The read FIFO is filled by a non-blocking read transaction which is configured by the functions [EPINonBlockingReadConfigure\(\)](#) and [EPINonBlockingReadStart\(\)](#).

Returns:

The number of items available to read in the read FIFO.

6.2.2.17 EPINonBlockingReadConfigure

Configures a non-blocking read transaction.

Prototype:

```
void  
EPINonBlockingReadConfigure(unsigned long ulBase,  
                             unsigned long ulChannel,  
                             unsigned long ulDataSize,  
                             unsigned long ulAddress)
```

Parameters:

ulBase is the EPI module base address.

ulChannel is the read channel (0 or 1).

ulDataSize is the size of the data items to read.

ulAddress is the starting address to read.

Description:

This function is used to configure a non-blocking read channel for a transaction. Two channels are available which can be used in a ping-pong method for continuous reading. It is not necessary to use both channels to perform a non-blocking read.

The parameter *ulDataSize* is one of **EPI_NBCONFIG_SIZE_8**, **EPI_NBCONFIG_SIZE_16**, or **EPI_NBCONFIG_SIZE_32** for 8-bit, 16-bit, or 32-bit sized data transfers.

The parameter *ulAddress* is the starting address for the read, relative to the external device. The start of the device is address 0.

Once configured, the non-blocking read is started by calling [EPINonBlockingReadStart\(\)](#). If the addresses to be read from the device are in a sequence, it is not necessary to call this function multiple times. Until it is changed, the EPI module will remember the last address that was used for a non-blocking read (per channel).

Returns:

None.

6.2.2.18 EPINonBlockingReadCount

Get the count remaining for a non-blocking transaction.

Prototype:

```
unsigned long
EPINonBlockingReadCount(unsigned long ulBase,
                        unsigned long ulChannel)
```

Parameters:

ulBase is the EPI module base address.

ulChannel is the read channel (0 or 1).

Description:

This function gets the remaining count of items for a non-blocking read transaction.

Returns:

The number of items remaining in the non-blocking read transaction.

6.2.2.19 EPINonBlockingReadGet16

Read available data from the read FIFO, as 16-bit data items.

Prototype:

```
unsigned long
EPINonBlockingReadGet16(unsigned long ulBase,
                        unsigned long ulCount,
                        unsigned short *pusBuf)
```

Parameters:

ulBase is the EPI module base address.

ulCount is the maximum count of items to read.

pulBuf is the caller supplied buffer where the read data should be stored.

Description:

This function reads 16-bit data items from the read FIFO and stores the values in a caller supplied buffer. The function will read and store data from the FIFO until there is no more data in the FIFO or the maximum count is reached as specified in the parameter *ulCount*. The actual count of items will be returned.

Returns:

The number of items read from the FIFO.

6.2.2.20 EPINonBlockingReadGet32

Read available data from the read FIFO, as 32-bit data items.

Prototype:

```
unsigned long  
EPINonBlockingReadGet32(unsigned long ulBase,  
                        unsigned long ulCount,  
                        unsigned long *pulBuf)
```

Parameters:

ulBase is the EPI module base address.

ulCount is the maximum count of items to read.

pulBuf is the caller supplied buffer where the read data should be stored.

Description:

This function reads 32-bit data items from the read FIFO and stores the values in a caller supplied buffer. The function will read and store data from the FIFO until there is no more data in the FIFO or the maximum count is reached as specified in the parameter *ulCount*. The actual count of items will be returned.

Returns:

The number of items read from the FIFO.

6.2.2.21 EPINonBlockingReadGet8

Read available data from the read FIFO, as 8-bit data items.

Prototype:

```
unsigned long  
EPINonBlockingReadGet8(unsigned long ulBase,  
                      unsigned long ulCount,  
                      unsigned char *pucBuf)
```

Parameters:

ulBase is the EPI module base address.

ulCount is the maximum count of items to read.

pucBuf is the caller supplied buffer where the read data should be stored.

Description:

This function reads 8-bit data items from the read FIFO and stores the values in a caller supplied buffer. The function will read and store data from the FIFO until there is no more data in the FIFO or the maximum count is reached as specified in the parameter *ulCount*. The actual count of items will be returned.

Returns:

The number of items read from the FIFO.

6.2.2.22 EPINonBlockingReadStart

Starts a non-blocking read transaction.

Prototype:

```
void  
EPINonBlockingReadStart(unsigned long ulBase,  
                        unsigned long ulChannel,  
                        unsigned long ulCount)
```

Parameters:

ulBase is the EPI module base address.

ulChannel is the read channel (0 or 1).

ulCount is the number of items to read (1-4095).

Description:

This function starts a non-blocking read that was previously configured with the function [EPINonBlockingReadConfigure\(\)](#). Once this function is called, the EPI module will begin reading data from the external device into the read FIFO. The EPI will stop reading when the FIFO fills up and resume reading when the application drains the FIFO, until the total specified count of data items has been read.

Once a read transaction is completed and the FIFO drained, another transaction can be started from the next address by calling this function again.

Returns:

None.

6.2.2.23 EPINonBlockingReadStop

Stops a non-blocking read transaction.

Prototype:

```
void  
EPINonBlockingReadStop(unsigned long ulBase,  
                       unsigned long ulChannel)
```

Parameters:

ulBase is the EPI module base address.

ulChannel is the read channel (0 or 1).

Description:

This function cancels a non-blocking read transaction that is already in progress.

Returns:

None.

6.2.2.24 EPIWriteFIFOCountGet

Reads the number of empty slots in the write transaction FIFO.

Prototype:

```
unsigned long  
EPIWriteFIFOCountGet(unsigned long ulBase)
```

Parameters:

ulBase is the EPI module base address.

Description:

This function returns the number of slots available in the transaction FIFO. It can be used in a polling method to avoid attempting a write that would stall.

Returns:

The number of empty slots in the transaction FIFO.

7 Flash

Introduction	83
API Functions	83

7.1 Introduction

The flash functions are currently limited for the Concerto devices, but additional supported is being added each release.

Currently the flash driver only provides an initialization function to setup flash wait states, cache, and prefetch mechanisms.

CAUTION: The flash function(s) should only be run from RAM. Please copy the function to RAM using the memcpy function in that can be found in `utils/memcopy.c`.

This driver is contained in `driverlib/flash.c`, with `driverlib/flash.h` containing the API definitions for use by applications.

7.2 API Functions

Functions

- void `FlashGainPump` (void)
- void `FlashInit` ()
- void `FlashLeavePump` (void)
- void `FlashSetup` (unsigned char ucFlashWaitState)

7.2.1 Detailed Description

Currently the Flash API only contains the function `InitFlash()`.

7.2.2 Function Documentation

7.2.2.1 FlashGainPump

Master subsystem requests control of the Flash pump.

Prototype:

```
void  
FlashGainPump(void)
```

Description:

This function allows master subsystem to take control of the Flash pump.

CAUTION: This function MUST be executed out of RAM. Executing it out of OTP/Flash will yield unpredictable results.

Returns:
None

7.2.2.2 FlashInit

Initialize Flash waitstates and caches

Prototype:
void
FlashInit()

Description:
This function initializes the Flash Control registers

CAUTION: This function MUST be executed out of RAM. Executing it out of OTP/Flash will yield unpredictable results.

Returns:
None

7.2.2.3 FlashLeavePump

Master subsystem releases control of the Flash pump.

Prototype:
void
FlashLeavePump(void)

Description:
This function allows master subsystem to release control of the Flash pump.

CAUTION: This function MUST be executed out of RAM. Executing it out of OTP/Flash will yield unpredictable results.

Returns:
None

7.2.2.4 FlashSetup

Setup flash waitstates and caches

Prototype:
void
FlashSetup(unsigned char ucFlashWaitState)

Parameters:
ucFlashWaitState This function sets up the Flash Control registers

Description:

CAUTION: This function MUST be executed out of RAM. Executing it out of OTP/Flash will yield unpredictable results.

CAUTION: The flash wait state variable must be a valid value given the current M3 CPU frequency! Setting a value too low will yield unpredictable results.

Returns:

None

8 GPIO

Introduction	87
API Functions	87
Programming Example	105

8.1 Introduction

The GPIO module provides control for up to eight independent GPIO pins (the actual number present depend upon the GPIO port and part number). Each pin has the following capabilities:

- Can be configured as an input or an output. On reset, they default to being an input.
- In input mode, can generate interrupts on high level, low level, rising edge, falling edge, or both edges.
- Optional weak pull-up resistors. On reset, they default to disabled.
- Optional open-drain operation. On reset, they default to standard push/pull operation.
- Can be configured to be a GPIO or a peripheral pin. On reset, they default to being GPIOs.

Most of the GPIO functions can operate on more than one GPIO pin (within a single module) at a time. The *ucPins* parameter to these functions is used to specify the pins that are affected; the GPIO pins whose corresponding bits in this parameter that are set will be affected (where pin 0 is in bit 0, pin 1 in bit 1, and so on). For example, if *ucPins* is 0x09, then pins 0 and 3 will be affected by the function.

This is most useful for the [GPiOPinRead\(\)](#) and [GPiOPinWrite\(\)](#) functions; a read will return only the value of the requested pins (with the other pin values masked out) and a write will affect the requested pins simultaneously (that is, the state of multiple GPIO pins can be changed at the same time). This data masking for the GPIO pin state occurs in the hardware; a single read or write is issued to the hardware, which interprets some of the address bits as an indication of the GPIO pins to operate upon (and therefore the ones to not affect). See the part data sheet for details of the GPIO data register address-based bit masking.

For functions that have a *ucPin* (singular) parameter, only a single pin is affected by the function. In this case, this value specifies the pin number (that is, 0 through 7).

This driver is contained in `driverlib/gpio.c`, with `driverlib/gpio.h` containing the API definitions for use by applications.

8.2 API Functions

Functions

- unsigned long [GPiODirModeGet](#) (unsigned long ulPort, unsigned char ucPin)
- void [GPiODirModeSet](#) (unsigned long ulPort, unsigned char ucPins, unsigned long ulPinIO)
- unsigned long [GPiOIntTypeGet](#) (unsigned long ulPort, unsigned char ucPin)
- void [GPiOIntTypeSet](#) (unsigned long ulPort, unsigned char ucPins, unsigned long ulIntType)

- void [GPIOPadConfigGet](#) (unsigned long ulPort, unsigned char ucPin, unsigned long *pulPinType)
- void [GPIOPadConfigSet](#) (unsigned long ulPort, unsigned char ucPins, unsigned long ulPinType)
- void [GPIOPinConfigure](#) (unsigned long ulPinConfig)
- void [GPIOPinConfigureCoreSelect](#) (unsigned long ulBase, unsigned char ucPins, unsigned char ucCore)
- void [GPIOPinIntClear](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinIntDisable](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinIntEnable](#) (unsigned long ulPort, unsigned char ucPins)
- long [GPIOPinIntStatus](#) (unsigned long ulPort, tBoolean bMasked)
- void [GPIOPinLock](#) (unsigned long ulPort, unsigned char ucPins)
- unsigned long [GPIOPinLockStatus](#) (unsigned long ulPort, unsigned char ucPins)
- long [GPIOPinRead](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinTypeCAN](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinTypeEPI](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinTypeGPIOInput](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinTypeGPIOOutput](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinTypeGPIOOutputHigh](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinTypeGPIOOutputLow](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinTypeGPIOOutputOD](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinTypeI2C](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinTypeSSI](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinTypeTimer](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinTypeUART](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinTypeUSBAnalog](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinTypeUSBDigital](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinUnlock](#) (unsigned long ulPort, unsigned char ucPins)
- void [GPIOPinWrite](#) (unsigned long ulPort, unsigned char ucPins, unsigned char ucVal)
- void [GPIOPortIntRegister](#) (unsigned long ulPort, void (*pfnIntHandler)(void))
- void [GPIOPortIntUnregister](#) (unsigned long ulPort)

8.2.1 Detailed Description

The GPIO API is broken into three groups of functions: those that deal with configuring the GPIO pins, those that deal with interrupts, and those that access the pin value.

The GPIO pins are configured with [GPIOPinConfigureCoreSelect\(\)](#), [GPIODirModeSet\(\)](#), and [GPIOPadConfigSet\(\)](#). The configuration can be read back with [GPIODirModeGet\(\)](#) and [GPIOPadConfigGet\(\)](#). There are also convenience functions for configuring the pin in the required or recommended configuration for a particular peripheral; these are [GPIOPinTypeCAN\(\)](#), [GPIOPinTypeComparator\(\)](#), [GPIOPinTypeGPIOInput\(\)](#), [GPIOPinTypeGPIOOutput\(\)](#), [GPIOPinTypeGPIOOutputOD\(\)](#), [GPIOPinTypeI2C\(\)](#), [GPIOPinTypePWM\(\)](#), [GPIOPinTypeQEI\(\)](#), [GPIOPinTypeSSI\(\)](#), [GPIOPinTypeTimer\(\)](#), and [GPIOPinTypeUART\(\)](#).

The GPIO interrupts are handled with [GPIOIntTypeSet\(\)](#), [GPIOIntTypeGet\(\)](#), [GPIOPinIntEnable\(\)](#), [GPIOPinIntDisable\(\)](#), [GPIOPinIntStatus\(\)](#), [GPIOPinIntClear\(\)](#), [GPIOPortIntRegister\(\)](#), and [GPIOPortIntUnregister\(\)](#).

The GPIO pin state is accessed with [GPIOPinRead\(\)](#) and [GPIOPinWrite\(\)](#).

8.2.2 Function Documentation

8.2.2.1 GPIODirModeGet

Gets the direction and mode of a single pin.

Prototype:

```
unsigned long
GPIODirModeGet(unsigned long ulPort,
                unsigned char ucPin)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPin is the pin number.

Description:

This function gets the direction and control mode for a specified pin on the selected GPIO port. The pin can be configured as either an input or output under software control, or it can be under hardware control. The type of control and direction are returned as an enumerated data type.

Returns:

Returns one of the enumerated data types described for [GPIODirModeSet\(\)](#).

8.2.2.2 GPIODirModeSet

Sets the direction and mode of the specified pin(s).

Prototype:

```
void
GPIODirModeSet(unsigned long ulPort,
                unsigned char ucPins,
                unsigned long ulPinIO)
```

Parameters:

ulPort is the base address of the GPIO port

ucPins is the bit-packed representation of the pin(s).

ulPinIO is the pin direction and/or mode.

Description:

This function will set the specified pin(s) on the selected GPIO port as either an input or output under software control, or it will set the pin to be under hardware control.

The parameter *ulPinIO* is an enumerated data type that can be one of the following values:

- **GPIO_DIR_MODE_IN**
- **GPIO_DIR_MODE_OUT**
- **GPIO_DIR_MODE_HW**

where **GPIO_DIR_MODE_IN** specifies that the pin will be programmed as a software controlled input, **GPIO_DIR_MODE_OUT** specifies that the pin will be programmed as a software controlled output, and **GPIO_DIR_MODE_HW** specifies that the pin will be placed under hardware control.

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Note:

[GPIOPadConfigSet\(\)](#) must also be used to configure the corresponding pad(s) in order for them to propagate the signal to/from the GPIO.

Returns:

None.

8.2.2.3 GPIOIntTypeGet

Gets the interrupt type for a pin.

Prototype:

```
unsigned long
GPIOIntTypeGet(unsigned long ulPort,
                unsigned char ucPin)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPin is the pin number.

Description:

This function gets the interrupt type for a specified pin on the selected GPIO port. The pin can be configured as a falling edge, rising edge, or both edge detected interrupt, or it can be configured as a low level or high level detected interrupt. The type of interrupt detection mechanism is returned as an enumerated data type.

Returns:

Returns one of the enumerated data types described for [GPIOIntTypeSet\(\)](#).

8.2.2.4 GPIOIntTypeSet

Sets the interrupt type for the specified pin(s).

Prototype:

```
void
GPIOIntTypeSet(unsigned long ulPort,
                unsigned char ucPins,
                unsigned long ulIntType)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

ullIntType specifies the type of interrupt trigger mechanism.

Description:

This function sets up the various interrupt trigger mechanisms for the specified pin(s) on the selected GPIO port.

The parameter *uIntType* is an enumerated data type that can be one of the following values:

- **GPIO_FALLING_EDGE**
- **GPIO_RISING_EDGE**
- **GPIO_BOTH_EDGES**
- **GPIO_LOW_LEVEL**
- **GPIO_HIGH_LEVEL**

where the different values describe the interrupt detection mechanism (edge or level) and the particular triggering event (falling, rising, or both edges for edge detect, low or high for level detect).

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Note:

In order to avoid any spurious interrupts, the user must ensure that the GPIO inputs remain stable for the duration of this function.

Returns:

None.

8.2.2.5 GPIOPadConfigGet

Gets the pad configuration for a pin.

Prototype:

```
void  
GPIOPadConfigGet(unsigned long ulPort,  
                 unsigned char ucPin,  
                 unsigned long *pulPinType)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPin is the pin number.

pulPinType is a pointer to storage for the output drive type.

Description:

This function gets the pad configuration for a specified pin on the selected GPIO port. The values returned in *pulStrength* and *pulPinType* correspond to the values used in [GPIOPadConfigSet\(\)](#). This function also works for pin(s) configured as input pin(s); however, the only meaningful data returned is whether the pin is terminated with a pull-up or down resistor.

Returns:

None

8.2.2.6 GPIOPadConfigSet

Sets the pad configuration for the specified pin(s).

Prototype:

```
void  
GPIOPadConfigSet(unsigned long ulPort,  
                  unsigned char ucPins,  
                  unsigned long ulPinType)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

ulPinType specifies the pin type.

Description:

This function sets the type for the specified pin(s) on the selected GPIO port. For pin(s) configured as input ports, the pad is configured as requested, but the only real effect on the input is the configuration of the pull-up termination.

The parameter *ulPinType* can be one of the following values:

- **GPIO_PIN_TYPE_STD**
- **GPIO_PIN_TYPE_STD_WPU**
- **GPIO_PIN_TYPE_OD**
- **GPIO_PIN_TYPE_OD_WPU**
- **GPIO_PIN_TYPE_ANALOG**

where **GPIO_PIN_TYPE_STD*** specifies a push-pull pin, **GPIO_PIN_TYPE_OD*** specifies an open-drain pin, and ***_WPU** specifies a weak pull-up.

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

None.

8.2.2.7 GPIOPinConfigure

Configures the alternate function of a GPIO pin.

Prototype:

```
void  
GPIOPinConfigure(unsigned long ulPinConfig)
```

Parameters:

ulPinConfig is the pin configuration value, specified as only one of the **GPIO_PIN_?** values.

Description:

This function configures the pin mux that selects the peripheral function associated with a particular GPIO pin. Only one peripheral function at a time can be associated with a GPIO pin, and each peripheral function should only be associated with a single GPIO pin at a time (despite the fact that many of them can be associated with more than one GPIO pin).

Returns:

None.

8.2.2.8 GPIOPinConfigureCoreSelect

Configures the core to which GPIO pin(s) is(are) mapped to.

Prototype:

```
void  
GPIOPinConfigureCoreSelect(unsigned long ulBase,  
                           unsigned char ucPins,  
                           unsigned char ucCore)
```

Parameters:

ulBase Base address of the GPIO port.

ucPins Mask corresponding to the pins to change.

ucCore Core to give control to (GPIO_PIN_C_CORE_SELECT or GPIO_PIN_M_CORE_SELECT).

Description:

This function configures the core select register for the GPIO pin(s). All the GPIO are assigned to master core on reset, calling this function with GPIO_PIN_C_CORE_SELECT param will configure the GPIO pin to Control system.

Returns:

None.

8.2.2.9 GPIOPinIntClear

Clears the interrupt for the specified pin(s).

Prototype:

```
void  
GPIOPinIntClear(unsigned long ulPort,  
                unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

Clears the interrupt for the specified pin(s).

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Note:

Because there is a write buffer in the Cortex-M3 processor, it may take several clock cycles before the interrupt source is actually cleared. Therefore, it is recommended that the interrupt source be cleared early in the interrupt handler (as opposed to the very last action) to avoid

returning from the interrupt handler before the interrupt source is actually cleared. Failure to do so may result in the interrupt handler being immediately reentered (because the interrupt controller still sees the interrupt source asserted).

Returns:

None.

8.2.2.10 GPIOPinIntDisable

Disables interrupts for the specified pin(s).

Prototype:

```
void  
GPIOPinIntDisable(unsigned long ulPort,  
                  unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

Masks the interrupt for the specified pin(s).

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

None.

8.2.2.11 GPIOPinIntEnable

Enables interrupts for the specified pin(s).

Prototype:

```
void  
GPIOPinIntEnable(unsigned long ulPort,  
                 unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

Unmasks the interrupt for the specified pin(s).

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

None.

8.2.2.12 GPIOPinIntStatus

Gets interrupt status for the specified GPIO port.

Prototype:

```
long  
GPIOPinIntStatus(unsigned long ulPort,  
                  tBoolean bMasked)
```

Parameters:

ulPort is the base address of the GPIO port.

bMasked specifies whether masked or raw interrupt status is returned.

Description:

If *bMasked* is set as **true**, then the masked interrupt status is returned; otherwise, the raw interrupt status will be returned.

Returns:

Returns a bit-packed byte, where each bit that is set identifies an active masked or raw interrupt, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on. Bits 31:8 should be ignored.

8.2.2.13 GPIOPinLock

Locks the specified pin(s).

Prototype:

```
void  
GPIOPinLock(unsigned long ulPort,  
             unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

This function locks the specified pin(s). Locking a pin prevents writes to the respective bits in the GPIO_O_AFSEL, GPIO_O_PUR, and GPIO_O_DEN registers.

Returns:

None.

8.2.2.14 GPIOPinLockStatus

Gets the lock status for the specified pin(s).

Prototype:

```
unsigned long  
GPIOPinLockStatus(unsigned long ulPort,  
                   unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

This function checks the lock status for the specified pin(s).

Returns:

A bit-packed representation of the lock status for the specified pin(s). A 1 in the pin's bit position means the pin is locked, likewise a 0 means the pin is unlocked.

8.2.2.15 GPIOPinRead

Reads the values present of the specified pin(s).

Prototype:

```
long
GPIOPinRead(unsigned long ulPort,
             unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

The values at the specified pin(s) are read, as specified by *ucPins*. Values are returned for both input and output pin(s), and the value for pin(s) that are not specified by *ucPins* are set to 0.

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

Returns a bit-packed byte providing the state of the specified pin, where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on. Any bit that is not specified by *ucPins* is returned as a 0. Bits 31:8 should be ignored.

8.2.2.16 GPIOPinTypeCAN

Configures pin(s) for use as a CAN device.

Prototype:

```
void
GPIOPinTypeCAN(unsigned long ulPort,
                unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

The CAN pins must be properly configured for the CAN peripherals to function correctly. This function provides a typical configuration for those pin(s); other configurations may work as well depending upon the board setup (for example, using the on-chip pull-ups).

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Note:

This cannot be used to turn any pin into a CAN pin; it only configures a CAN pin for proper operation.

Returns:

None.

8.2.2.17 GPIOPinTypeEPI

Configures pin(s) for use by the external peripheral interface.

Prototype:

```
void  
GPIOPinTypeEPI(unsigned long ulPort,  
                unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

The external peripheral interface pins must be properly configured for the external peripheral interface to function correctly. This function provides a typical configuration for those pin(s); other configurations may work as well depending upon the board setup (for example, using the on-chip pull-ups).

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Note:

This cannot be used to turn any pin into an external peripheral interface pin; it only configures an external peripheral interface pin for proper operation.

Returns:

None.

8.2.2.18 GPIOPinTypeGPIOInput

Configures pin(s) for use as GPIO inputs.

Prototype:

```
void
GPIOPinTypeGPIOInput(unsigned long ulPort,
                      unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

The GPIO pins must be properly configured in order to function correctly as GPIO inputs. This function provides the proper configuration for those pin(s).

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

None.

8.2.2.19 GPIOPinTypeGPIOOutput

Configures pin(s) for use as GPIO outputs.

Prototype:

```
void
GPIOPinTypeGPIOOutput(unsigned long ulPort,
                      unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

The GPIO pins must be properly configured in order to function correctly as GPIO outputs. This function provides the proper configuration for those pin(s).

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

None.

8.2.2.20 GPIOPinTypeGPIOOutputHigh

Configures pin(s) for use as GPIO outputs initialized in high state.

Prototype:

```
void
GPIOPinTypeGPIOOutputHigh(unsigned long ulPort,
                          unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

The GPIO pins must be properly configured in order to function correctly as GPIO outputs. This function provides the proper configuration for those pin(s) and sets the initialized output as 1, or high.

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

None.

8.2.2.21 GPIOPinTypeGPIOOutputLow

Configures pin(s) for use as GPIO outputs initialized in low state.

Prototype:

```
void
GPIOPinTypeGPIOOutputLow(unsigned long ulPort,
                          unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

The GPIO pins must be properly configured in order to function correctly as GPIO outputs. This function provides the proper configuration for those pin(s) and sets the initialized output as 0, or low.

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

None.

8.2.2.22 GPIOPinTypeGPIOOutputOD

Configures pin(s) for use as GPIO open drain outputs.

Prototype:

```
void
GPIOPinTypeGPIOOutputOD(unsigned long ulPort,
                        unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

The GPIO pins must be properly configured in order to function correctly as GPIO outputs; this is especially true of Fury-class devices where the digital input enable is turned off by default. This function provides the proper configuration for those pin(s).

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

None.

8.2.2.23 GPIOPinTypeI2C

Configures pin(s) for use by the I2C peripheral.

Prototype:

```
void  
GPIOPinTypeI2C(unsigned long ulPort,  
               unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

The I2C pins must be properly configured for the I2C peripheral to function correctly. This function provides the proper configuration for those pin(s).

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Note:

This cannot be used to turn any pin into an I2C pin; it only configures an I2C pin for proper operation.

Returns:

None.

8.2.2.24 GPIOPinTypeSSI

Configures pin(s) for use by the SSI peripheral.

Prototype:

```
void  
GPIOPinTypeSSI(unsigned long ulPort,  
               unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

The SSI pins must be properly configured for the SSI peripheral to function correctly. This function provides a typical configuration for those pin(s); other configurations may work as well depending upon the board setup (for example, using the on-chip pull-ups).

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Note:

This cannot be used to turn any pin into a SSI pin; it only configures a SSI pin for proper operation.

Returns:

None.

8.2.2.25 GPIOPinTypeTimer

Configures pin(s) for use by the Timer peripheral.

Prototype:

```
void  
GPIOPinTypeTimer(unsigned long ulPort,  
                  unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

The CCP pins must be properly configured for the timer peripheral to function correctly. This function provides a typical configuration for those pin(s); other configurations may work as well depending upon the board setup (for example, using the on-chip pull-ups).

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Note:

This cannot be used to turn any pin into a timer pin; it only configures a timer pin for proper operation.

Returns:

None.

8.2.2.26 GPIOPinTypeUART

Configures pin(s) for use by the UART peripheral.

Prototype:

```
void
GPIOPinTypeUART(unsigned long ulPort,
                 unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

The UART pins must be properly configured for the UART peripheral to function correctly. This function provides a typical configuration for those pin(s); other configurations may work as well depending upon the board setup (for example, using the on-chip pull-ups).

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Note:

This cannot be used to turn any pin into a UART pin; it only configures a UART pin for proper operation.

Returns:

None.

8.2.2.27 GPIOPinTypeUSBAnalog

Configures pin(s) for use by the USB peripheral.

Prototype:

```
void
GPIOPinTypeUSBAnalog(unsigned long ulPort,
                     unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

Some USB analog pins must be properly configured for the USB peripheral to function correctly. This function provides the proper configuration for any USB pin(s). This can also be used to configure the EPEN and PFAULT pins so that they are no longer used by the USB controller.

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Note:

This cannot be used to turn any pin into a USB pin; it only configures a USB pin for proper operation.

Returns:

None.

8.2.2.28 GPIOPinTypeUSBDigital

Configures pin(s) for use by the USB peripheral.

Prototype:

```
void  
GPIOPinTypeUSBDigital(unsigned long ulPort,  
                       unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

Some USB digital pins must be properly configured for the USB peripheral to function correctly. This function provides a typical configuration for the digital USB pin(s); other configurations may work as well depending upon the board setup (for example, using the on-chip pull-ups).

This function should only be used with EPEN and PFAULT pins as all other USB pins are analog in nature or are not used in devices without OTG functionality.

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Note:

This cannot be used to turn any pin into a USB pin; it only configures a USB pin for proper operation.

Returns:

None.

8.2.2.29 GPIOPinUnlock

Unlocks the specified pin(s).

Prototype:

```
void  
GPIOPinUnlock(unsigned long ulPort,  
              unsigned char ucPins)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

Description:

This function unlocks the specified pin(s). Unlocking a pin allows writes to the respective bits in the GPIO_O_AFSEL, GPIO_O_PUR, and GPIO_O_DEN registers.

Returns:

None.

8.2.2.30 GPIOPinWrite

Writes a value to the specified pin(s).

Prototype:

```
void  
GPIOPinWrite(unsigned long ulPort,  
             unsigned char ucPins,  
             unsigned char ucVal)
```

Parameters:

ulPort is the base address of the GPIO port.

ucPins is the bit-packed representation of the pin(s).

ucVal is the value to write to the pin(s).

Description:

Writes the corresponding bit values to the output pin(s) specified by *ucPins*. Writing to a pin configured as an input pin has no effect.

The pin(s) are specified using a bit-packed byte, where each bit that is set identifies the pin to be accessed, and where bit 0 of the byte represents GPIO port pin 0, bit 1 represents GPIO port pin 1, and so on.

Returns:

None.

8.2.2.31 GPIOPortIntRegister

Registers an interrupt handler for a GPIO port.

Prototype:

```
void  
GPIOPortIntRegister(unsigned long ulPort,  
                   void (*pfnIntHandler)(void))
```

Parameters:

ulPort is the base address of the GPIO port.

pfnIntHandler is a pointer to the GPIO port interrupt handling function.

Description:

This function will ensure that the interrupt handler specified by *pfnIntHandler* is called when an interrupt is detected from the selected GPIO port. This function will also enable the corresponding GPIO interrupt in the interrupt controller; individual pin interrupts and interrupt sources must be enabled with [GPIOPinIntEnable\(\)](#).

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

8.2.2.32 GPIOPortIntUnregister

Removes an interrupt handler for a GPIO port.

Prototype:

```
void  
GPIOPortIntUnregister(unsigned long ulPort)
```

Parameters:

ulPort is the base address of the GPIO port.

Description:

This function will unregister the interrupt handler for the specified GPIO port. This function will also disable the corresponding GPIO port interrupt in the interrupt controller; individual GPIO interrupts and interrupt sources must be disabled with [GPIOPinIntDisable\(\)](#).

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

8.3 Programming Example

The following example shows how to use the GPIO API to initialize the GPIO, enable interrupts, read data from pins, and write data to pins.

```
int iVal;  
  
// Register the port-level interrupt handler. This handler is the  
// first level interrupt handler for all the pin interrupts.  
GPIOPortIntRegister(GPIO_PORTA_BASE, PortAIntHandler);  
  
// Initialize the GPIO pin configuration.  
// Set pins 2, 4, and 5 as input, SW controlled.  
GPIOPinTypeGPIOInput(GPIO_PORTA_BASE,  
                      GPIO_PIN_2 | GPIO_PIN_4 | GPIO_PIN_5);  
  
// Set pins 0 and 3 as output, SW controlled.  
GPIOPinTypeGPIOOutput(GPIO_PORTA_BASE, GPIO_PIN_0 | GPIO_PIN_3);  
  
// Make pins 2 and 4 rising edge triggered interrupts.  
GPIOIntTypeSet(GPIO_PORTA_BASE, GPIO_PIN_2 |  
                GPIO_PIN_4, GPIO_RISING_EDGE);  
  
// Make pin 5 high level triggered interrupts.  
GPIOIntTypeSet(GPIO_PORTA_BASE, GPIO_PIN_5, GPIO_HIGH_LEVEL);  
  
// Read some pins.  
iVal = GPIOPinRead(GPIO_PORTA_BASE, (GPIO_PIN_0 | GPIO_PIN_2 |  
                                     GPIO_PIN_3 | GPIO_PIN_4 |
```

```
GPIO_PIN_5));  
  
// Write some pins. Even though pins 2, 4, and 5 are  
// specified, those pins are unaffected by this write  
// since they are configured as inputs. At the end of  
// this write, pin 0 will be a 0, and pin 3 will be a 1.  
GPIOPinWrite(GPIO_PORTA_BASE, (GPIO_PIN_0 | GPIO_PIN_2 |  
                                GPIO_PIN_3 | GPIO_PIN_4 |  
                                GPIO_PIN_5), 0xF8);  
  
// Enable the pin interrupts.  
GPIOPinIntEnable(GPIO_PORTA_BASE, GPIO_PIN_2 | GPIO_PIN_4 |  
                    GPIO_PIN_5);
```

9 Inter-Integrated Circuit (I2C)

Introduction	107
API Functions	108
Programming Example	122

9.1 Introduction

The Inter-Integrated Circuit (I2C) API provides a set of functions for using the Concerto I2C master and slave modules. Functions are provided to initialize the I2C modules, to send and receive data, obtain status, and to manage interrupts for the I2C modules.

The I2C master and slave modules provide the ability to communicate to other IC devices over an I2C bus. The I2C bus is specified to support devices that can both transmit and receive (write and read) data. Also, devices on the I2C bus can be designated as either a master or a slave. The Concerto I2C modules support both sending and receiving data as either a master or a slave, and also support the simultaneous operation as both a master and a slave. Finally, the Concerto I2C modules can operate at two speeds: Standard (100 kb/s) and Fast (400 kb/s).

Both the master and slave I2C modules can generate interrupts. The I2C master module will generate interrupts when a transmit or receive operation is completed (or aborted due to an error). The I2C slave module will generate interrupts when data has been sent or requested by a master.

9.1.1 Master Operations

When using this API to drive the I2C master module, the user must first initialize the I2C master module with a call to [I2CMasterInitExpClk\(\)](#). That function will set the bus speed and enable the master module.

The user may transmit or receive data after the successful initialization of the I2C master module. Data is transferred by first setting the slave address using [I2CMasterSlaveAddrSet\(\)](#). That function is also used to define whether the transfer is a send (a write to the slave from the master) or a receive (a read from the slave by the master). Then, if connected to an I2C bus that has multiple masters, the Concerto I2C master must first call [I2CMasterBusBusy\(\)](#) before attempting to initiate the desired transaction. After determining that the bus is not busy, if trying to send data, the user must call the [I2CMasterDataPut\(\)](#) function. The transaction can then be initiated on the bus by calling the [I2CMasterControl\(\)](#) function with any of the following commands:

- **I2C_MASTER_CMD_SINGLE_SEND**
- **I2C_MASTER_CMD_SINGLE_RECEIVE**
- **I2C_MASTER_CMD_BURST_SEND_START**
- **I2C_MASTER_CMD_BURST_RECEIVE_START**

Any of those commands will result in the master arbitrating for the bus, driving the start sequence onto the bus, and sending the slave address and direction bit across the bus. The remainder of the transaction can then be driven using either a polling or interrupt-driven method.

For the single send and receive cases, the polling method will involve looping on the return from [I2CMasterBusy\(\)](#). Once that function indicates that the I2C master is no longer busy, the bus transaction has been completed and can be checked for errors using [I2CMasterErr\(\)](#). If there are no

errors, then the data has been sent or is ready to be read using [I2CMasterDataGet\(\)](#). For the burst send and receive cases, the polling method also involves calling the [I2CMasterControl\(\)](#) function for each byte transmitted or received (using either the `I2C_MASTER_CMD_BURST_SEND_CONT` or `I2C_MASTER_CMD_BURST_RECEIVE_CONT` commands), and for the last byte sent or received (using either the `I2C_MASTER_CMD_BURST_SEND_FINISH` or `I2C_MASTER_CMD_BURST_RECEIVE_FINISH` commands). If any error is detected during the burst transfer, the [I2CMasterControl\(\)](#) function should be called using the appropriate stop command (`I2C_MASTER_CMD_BURST_SEND_ERROR_STOP` or `I2C_MASTER_CMD_BURST_RECEIVE_ERROR_STOP`).

For the interrupt-driven transaction, the user must register an interrupt handler for the I2C devices and enable the I2C master interrupt; the interrupt will occur when the master is no longer busy.

9.1.2 Slave Operations

When using this API to drive the I2C slave module, the user must first initialize the I2C slave module with a call to [I2CSlaveInit\(\)](#). This will enable the I2C slave module and initialize the slave's own address. After the initialization is complete, the user may poll the slave status using [I2CSlaveStatus\(\)](#) to determine if a master requested a send or receive operation. Depending on the type of operation requested, the user can call [I2CSlaveDataPut\(\)](#) or [I2CSlaveDataGet\(\)](#) to complete the transaction. Alternatively, the I2C slave can handle transactions using an interrupt handler registered with [I2CIntRegister\(\)](#), and by enabling the I2C slave interrupt.

This driver is contained in `driverlib/i2c.c`, with `driverlib/i2c.h` containing the API definitions for use by applications.

9.2 API Functions

Functions

- void [I2CIntRegister](#) (unsigned long ulBase, void (*pfnHandler)(void))
- void [I2CIntUnregister](#) (unsigned long ulBase)
- tBoolean [I2CMasterBusBusy](#) (unsigned long ulBase)
- tBoolean [I2CMasterBusy](#) (unsigned long ulBase)
- void [I2CMasterControl](#) (unsigned long ulBase, unsigned long ulCmd)
- unsigned long [I2CMasterDataGet](#) (unsigned long ulBase)
- void [I2CMasterDataPut](#) (unsigned long ulBase, unsigned char ucData)
- void [I2CMasterDisable](#) (unsigned long ulBase)
- void [I2CMasterEnable](#) (unsigned long ulBase)
- unsigned long [I2CMasterErr](#) (unsigned long ulBase)
- void [I2CMasterInitExpClk](#) (unsigned long ulBase, unsigned long ullI2CCLK, tBoolean bFast)
- void [I2CMasterIntClear](#) (unsigned long ulBase)
- void [I2CMasterIntDisable](#) (unsigned long ulBase)
- void [I2CMasterIntEnable](#) (unsigned long ulBase)
- tBoolean [I2CMasterIntStatus](#) (unsigned long ulBase, tBoolean bMasked)
- void [I2CMasterSlaveAddrSet](#) (unsigned long ulBase, unsigned char ucSlaveAddr, tBoolean bReceive)

- unsigned long [I2CSlaveDataGet](#) (unsigned long ulBase)
- void [I2CSlaveDataPut](#) (unsigned long ulBase, unsigned char ucData)
- void [I2CSlaveDisable](#) (unsigned long ulBase)
- void [I2CSlaveEnable](#) (unsigned long ulBase)
- void [I2CSlaveInit](#) (unsigned long ulBase, unsigned char ucSlaveAddr)
- void [I2CSlaveIntClear](#) (unsigned long ulBase)
- void [I2CSlaveIntClearEx](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [I2CSlaveIntDisable](#) (unsigned long ulBase)
- void [I2CSlaveIntDisableEx](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [I2CSlaveIntEnable](#) (unsigned long ulBase)
- void [I2CSlaveIntEnableEx](#) (unsigned long ulBase, unsigned long ulIntFlags)
- tBoolean [I2CSlaveIntStatus](#) (unsigned long ulBase, tBoolean bMasked)
- unsigned long [I2CSlaveIntStatusEx](#) (unsigned long ulBase, tBoolean bMasked)
- unsigned long [I2CSlaveStatus](#) (unsigned long ulBase)

9.2.1 Detailed Description

The I2C API is broken into three groups of functions: those that deal with interrupts, those that handle status and initialization, and those that deal with sending and receiving data.

The I2C master and slave interrupts are handled by the [I2CIntRegister\(\)](#), [I2CIntUnregister\(\)](#), [I2CMasterIntEnable\(\)](#), [I2CMasterIntDisable\(\)](#), [I2CMasterIntClear\(\)](#), [I2CMasterIntStatus\(\)](#), [I2CSlaveIntEnable\(\)](#), [I2CSlaveIntDisable\(\)](#), [I2CSlaveIntClear\(\)](#), [I2CSlaveIntStatus\(\)](#), [I2CSlaveIntEnableEx\(\)](#), [I2CSlaveIntDisableEx\(\)](#), [I2CSlaveIntClearEx\(\)](#), and [I2CSlaveIntStatusEx\(\)](#) functions.

Status and initialization functions for the I2C modules are [I2CMasterInitExpClk\(\)](#), [I2CMasterEnable\(\)](#), [I2CMasterDisable\(\)](#), [I2CMasterBusBusy\(\)](#), [I2CMasterBusy\(\)](#), [I2CMasterErr\(\)](#), [I2CSlaveInit\(\)](#), [I2CSlaveEnable\(\)](#), [I2CSlaveDisable\(\)](#), and [I2CSlaveStatus\(\)](#).

Sending and receiving data from the I2C modules are handled by the [I2CMasterSlaveAddrSet\(\)](#), [I2CMasterControl\(\)](#), [I2CMasterDataGet\(\)](#), [I2CMasterDataPut\(\)](#), [I2CSlaveDataGet\(\)](#), and [I2CSlaveDataPut\(\)](#) functions.

The [I2CMasterInit\(\)](#) API from previous versions of the peripheral driver library has been replaced by the [I2CMasterInitExpClk\(\)](#) API. A macro has been provided in `i2c.h` to map the old API to the new API, allowing existing applications to link and run with the new API. It is recommended that new applications utilize the new API in favor of the old one.

9.2.2 Function Documentation

9.2.2.1 I2CIntRegister

Registers an interrupt handler for the I2C module.

Prototype:

```
void
I2CIntRegister(unsigned long ulBase,
               void (*pfnHandler)(void))
```

Parameters:

ulBase is the base address of the I2C Master module.

pfnHandler is a pointer to the function to be called when the I2C interrupt occurs.

Description:

This sets the handler to be called when an I2C interrupt occurs. This will enable the global interrupt in the interrupt controller; specific I2C interrupts must be enabled via [I2CMasterIntEnable\(\)](#) and [I2CSlaveIntEnable\(\)](#). If necessary, it is the interrupt handler's responsibility to clear the interrupt source via [I2CMasterIntClear\(\)](#) and [I2CSlaveIntClear\(\)](#).

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

9.2.2.2 I2CIntUnregister

Unregisters an interrupt handler for the I2C module.

Prototype:

```
void  
I2CIntUnregister(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Master module.

Description:

This function will clear the handler to be called when an I2C interrupt occurs. This will also mask off the interrupt in the interrupt controller so that the interrupt handler no longer is called.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

9.2.2.3 I2CMasterBusBusy

Indicates whether or not the I2C bus is busy.

Prototype:

```
tBoolean  
I2CMasterBusBusy(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Master module.

Description:

This function returns an indication of whether or not the I2C bus is busy. This function can be used in a multi-master environment to determine if another master is currently using the bus.

Returns:

Returns **true** if the I2C bus is busy; otherwise, returns **false**.

9.2.2.4 I2CMasterBusy

Indicates whether or not the I2C Master is busy.

Prototype:

```
tBoolean  
I2CMasterBusy(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Master module.

Description:

This function returns an indication of whether or not the I2C Master is busy transmitting or receiving data.

Returns:

Returns **true** if the I2C Master is busy; otherwise, returns **false**.

9.2.2.5 I2CMasterControl

Controls the state of the I2C Master module.

Prototype:

```
void  
I2CMasterControl(unsigned long ulBase,  
                 unsigned long ulCmd)
```

Parameters:

ulBase is the base address of the I2C Master module.

ulCmd command to be issued to the I2C Master module

Description:

This function is used to control the state of the Master module send and receive operations. The *ucCmd* parameter can be one of the following values:

- I2C_MASTER_CMD_SINGLE_SEND
- I2C_MASTER_CMD_SINGLE_RECEIVE
- I2C_MASTER_CMD_BURST_SEND_START
- I2C_MASTER_CMD_BURST_SEND_CONT
- I2C_MASTER_CMD_BURST_SEND_FINISH
- I2C_MASTER_CMD_BURST_SEND_ERROR_STOP
- I2C_MASTER_CMD_BURST_RECEIVE_START
- I2C_MASTER_CMD_BURST_RECEIVE_CONT
- I2C_MASTER_CMD_BURST_RECEIVE_FINISH
- I2C_MASTER_CMD_BURST_RECEIVE_ERROR_STOP

Returns:

None.

9.2.2.6 I2CMasterDataGet

Receives a byte that has been sent to the I2C Master.

Prototype:

```
unsigned long  
I2CMasterDataGet(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Master module.

Description:

This function reads a byte of data from the I2C Master Data Register.

Returns:

Returns the byte received from by the I2C Master, cast as an unsigned long.

9.2.2.7 I2CMasterDataPut

Transmits a byte from the I2C Master.

Prototype:

```
void  
I2CMasterDataPut(unsigned long ulBase,  
                 unsigned char ucData)
```

Parameters:

ulBase is the base address of the I2C Master module.

ucData data to be transmitted from the I2C Master

Description:

This function will place the supplied data into I2C Master Data Register.

Returns:

None.

9.2.2.8 I2CMasterDisable

Disables the I2C master block.

Prototype:

```
void  
I2CMasterDisable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Master module.

Description:

This will disable operation of the I2C master block.

Returns:

None.

9.2.2.9 I2CMasterEnable

Enables the I2C Master block.

Prototype:

```
void  
I2CMasterEnable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Master module.

Description:

This will enable operation of the I2C Master block.

Returns:

None.

9.2.2.10 I2CMasterErr

Gets the error status of the I2C Master module.

Prototype:

```
unsigned long  
I2CMasterErr(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Master module.

Description:

This function is used to obtain the error status of the Master module send and receive operations.

Returns:

Returns the error status, as one of **I2C_MASTER_ERR_NONE**, **I2C_MASTER_ERR_ADDR_ACK**, **I2C_MASTER_ERR_DATA_ACK**, or **I2C_MASTER_ERR_ARB_LOST**.

9.2.2.11 I2CMasterInitExpClk

Initializes the I2C Master block.

Prototype:

```
void  
I2CMasterInitExpClk(unsigned long ulBase,  
                    unsigned long ulI2CClk,  
                    tBoolean bFast)
```

Parameters:

ulBase is the base address of the I2C Master module.

ulI2CClk is the rate of the clock supplied to the I2C module.

bFast set up for fast data transfers

Description:

This function initializes operation of the I2C Master block. Upon successful initialization of the I2C block, this function will have set the bus speed for the master, and will have enabled the I2C Master block.

If the parameter *bFast* is **true**, then the master block will be set up to transfer data at 400 kbps; otherwise, it will be set up to transfer data at 100 kbps.

The peripheral clock will be the same as the processor clock. This will be the value returned by [SysCtlClockGet\(\)](#), or it can be explicitly hard coded if it is constant and known (to save the code/execution overhead of a call to [SysCtlClockGet\(\)](#)).

This function replaces the original I2CMasterInit() API and performs the same actions. A macro is provided in `i2c.h` to map the original API to this API.

Returns:

None.

9.2.2.12 I2CMasterIntClear

Clears I2C Master interrupt sources.

Prototype:

```
void  
I2CMasterIntClear(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Master module.

Description:

The I2C Master interrupt source is cleared, so that it no longer asserts. This must be done in the interrupt handler to keep it from being called again immediately upon exit.

Note:

Because there is a write buffer in the Cortex-M3 processor, it may take several clock cycles before the interrupt source is actually cleared. Therefore, it is recommended that the interrupt source be cleared early in the interrupt handler (as opposed to the very last action) to avoid returning from the interrupt handler before the interrupt source is actually cleared. Failure to do so may result in the interrupt handler being immediately reentered (because the interrupt controller still sees the interrupt source asserted).

Returns:

None.

9.2.2.13 I2CMasterIntDisable

Disables the I2C Master interrupt.

Prototype:

```
void  
I2CMasterIntDisable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Master module.

Description:

Disables the I2C Master interrupt source.

Returns:

None.

9.2.2.14 I2CMasterIntEnable

Enables the I2C Master interrupt.

Prototype:

```
void  
I2CMasterIntEnable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Master module.

Description:

Enables the I2C Master interrupt source.

Returns:

None.

9.2.2.15 I2CMasterIntStatus

Gets the current I2C Master interrupt status.

Prototype:

```
tBoolean  
I2CMasterIntStatus(unsigned long ulBase,  
                   tBoolean bMasked)
```

Parameters:

ulBase is the base address of the I2C Master module.

bMasked is false if the raw interrupt status is requested and true if the masked interrupt status is requested.

Description:

This returns the interrupt status for the I2C Master module. Either the raw interrupt status or the status of interrupts that are allowed to reflect to the processor can be returned.

Returns:

The current interrupt status, returned as **true** if active or **false** if not active.

9.2.2.16 I2CMasterSlaveAddrSet

Sets the address that the I2C Master will place on the bus.

Prototype:

```
void  
I2CMasterSlaveAddrSet(unsigned long ulBase,  
                      unsigned char ucSlaveAddr,  
                      tBoolean bReceive)
```

Parameters:

ulBase is the base address of the I2C Master module.

ucSlaveAddr 7-bit slave address

bReceive flag indicating the type of communication with the slave

Description:

This function will set the address that the I2C Master will place on the bus when initiating a transaction. When the *bReceive* parameter is set to **true**, the address will indicate that the I2C Master is initiating a read from the slave; otherwise the address will indicate that the I2C Master is initiating a write to the slave.

Returns:

None.

9.2.2.17 I2CSlaveDataGet

Receives a byte that has been sent to the I2C Slave.

Prototype:

```
unsigned long  
I2CSlaveDataGet(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Slave module.

Description:

This function reads a byte of data from the I2C Slave Data Register.

Returns:

Returns the byte received from by the I2C Slave, cast as an unsigned long.

9.2.2.18 I2CSlaveDataPut

Transmits a byte from the I2C Slave.

Prototype:

```
void  
I2CSlaveDataPut(unsigned long ulBase,  
                unsigned char ucData)
```

Parameters:

ulBase is the base address of the I2C Slave module.
ucData data to be transmitted from the I2C Slave

Description:

This function will place the supplied data into I2C Slave Data Register.

Returns:

None.

9.2.2.19 I2CSlaveDisable

Disables the I2C slave block.

Prototype:

```
void  
I2CSlaveDisable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Slave module.

Description:

This will disable operation of the I2C slave block.

Returns:

None.

9.2.2.20 I2CSlaveEnable

Enables the I2C Slave block.

Prototype:

```
void  
I2CSlaveEnable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Slave module.

Description:

This will enable operation of the I2C Slave block.

Returns:

None.

9.2.2.21 I2CSlaveInit

Initializes the I2C Slave block.

Prototype:

```
void  
I2CSlaveInit(unsigned long ulBase,  
              unsigned char ucSlaveAddr)
```

Parameters:

ulBase is the base address of the I2C Slave module.

ucSlaveAddr 7-bit slave address

Description:

This function initializes operation of the I2C Slave block. Upon successful initialization of the I2C blocks, this function will have set the slave address and have enabled the I2C Slave block.

The parameter *ucSlaveAddr* is the value that will be compared against the slave address sent by an I2C master.

Returns:

None.

9.2.2.22 I2CSlaveIntClear

Clears I2C Slave interrupt sources.

Prototype:

```
void  
I2CSlaveIntClear(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Slave module.

Description:

The I2C Slave interrupt source is cleared, so that it no longer asserts. This must be done in the interrupt handler to keep it from being called again immediately upon exit.

Note:

Because there is a write buffer in the Cortex-M3 processor, it may take several clock cycles before the interrupt source is actually cleared. Therefore, it is recommended that the interrupt source be cleared early in the interrupt handler (as opposed to the very last action) to avoid returning from the interrupt handler before the interrupt source is actually cleared. Failure to do so may result in the interrupt handler being immediately reentered (because the interrupt controller still sees the interrupt source asserted).

Returns:

None.

9.2.2.23 I2CSlaveIntClearEx

Clears I2C Slave interrupt sources.

Prototype:

```
void  
I2CSlaveIntClearEx(unsigned long ulBase,  
                   unsigned long ulIntFlags)
```

Parameters:

ulBase is the base address of the I2C Slave module.

ulIntFlags is a bit mask of the interrupt sources to be cleared.

Description:

The specified I2C Slave interrupt sources are cleared, so that they no longer assert. This must be done in the interrupt handler to keep it from being called again immediately upon exit.

The *ulIntFlags* parameter has the same definition as the *ulIntFlags* parameter to [I2CSlaveIntEnableEx\(\)](#).

Note:

Because there is a write buffer in the Cortex-M3 processor, it may take several clock cycles before the interrupt source is actually cleared. Therefore, it is recommended that the interrupt source be cleared early in the interrupt handler (as opposed to the very last action) to avoid returning from the interrupt handler before the interrupt source is actually cleared. Failure to do so may result in the interrupt handler being immediately reentered (because the interrupt controller still sees the interrupt source asserted).

Returns:

None.

9.2.2.24 I2CSlaveIntDisable

Disables the I2C Slave interrupt.

Prototype:

```
void  
I2CSlaveIntDisable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Slave module.

Description:

Disables the I2C Slave interrupt source.

Returns:

None.

9.2.2.25 I2CSlaveIntDisableEx

Disables individual I2C Slave interrupt sources.

Prototype:

```
void  
I2CSlaveIntDisableEx(unsigned long ulBase,  
                     unsigned long ulIntFlags)
```

Parameters:

ulBase is the base address of the I2C Slave module.

ulIntFlags is the bit mask of the interrupt sources to be disabled.

Description:

Disables the indicated I2C Slave interrupt sources. Only the sources that are enabled can be reflected to the processor interrupt; disabled sources have no effect on the processor.

The *ullntFlags* parameter has the same definition as the *ullntFlags* parameter to [I2CSlaveIntEnableEx\(\)](#).

Returns:

None.

9.2.2.26 I2CSlaveIntEnable

Enables the I2C Slave interrupt.

Prototype:

```
void  
I2CSlaveIntEnable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Slave module.

Description:

Enables the I2C Slave interrupt source.

Returns:

None.

9.2.2.27 I2CSlaveIntEnableEx

Enables individual I2C Slave interrupt sources.

Prototype:

```
void  
I2CSlaveIntEnableEx(unsigned long ulBase,  
                    unsigned long ulIntFlags)
```

Parameters:

ulBase is the base address of the I2C Slave module.

ullntFlags is the bit mask of the interrupt sources to be enabled.

Description:

Enables the indicated I2C Slave interrupt sources. Only the sources that are enabled can be reflected to the processor interrupt; disabled sources have no effect on the processor.

The *ullntFlags* parameter is the logical OR of any of the following:

- **I2C_SLAVE_INT_STOP** - Stop condition detected interrupt
- **I2C_SLAVE_INT_START** - Start condition detected interrupt
- **I2C_SLAVE_INT_DATA** - Data interrupt

Returns:

None.

9.2.2.28 I2CSlaveIntStatus

Gets the current I2C Slave interrupt status.

Prototype:

```
tBoolean  
I2CSlaveIntStatus(unsigned long ulBase,  
                  tBoolean bMasked)
```

Parameters:

ulBase is the base address of the I2C Slave module.

bMasked is false if the raw interrupt status is requested and true if the masked interrupt status is requested.

Description:

This returns the interrupt status for the I2C Slave module. Either the raw interrupt status or the status of interrupts that are allowed to reflect to the processor can be returned.

Returns:

The current interrupt status, returned as **true** if active or **false** if not active.

9.2.2.29 I2CSlaveIntStatusEx

Gets the current I2C Slave interrupt status.

Prototype:

```
unsigned long  
I2CSlaveIntStatusEx(unsigned long ulBase,  
                    tBoolean bMasked)
```

Parameters:

ulBase is the base address of the I2C Slave module.

bMasked is false if the raw interrupt status is requested and true if the masked interrupt status is requested.

Description:

This returns the interrupt status for the I2C Slave module. Either the raw interrupt status or the status of interrupts that are allowed to reflect to the processor can be returned.

Returns:

Returns the current interrupt status, enumerated as a bit field of values described in [I2CSlaveIntEnableEx\(\)](#).

9.2.2.30 I2CSlaveStatus

Gets the I2C Slave module status

Prototype:

```
unsigned long  
I2CSlaveStatus(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the I2C Slave module.

Description:

This function will return the action requested from a master, if any. Possible values are:

- **I2C_SLAVE_ACT_NONE**
- **I2C_SLAVE_ACT_RREQ**
- **I2C_SLAVE_ACT_TREQ**
- **I2C_SLAVE_ACT_RREQ_FBR**

Returns:

Returns **I2C_SLAVE_ACT_NONE** to indicate that no action has been requested of the I2C Slave module, **I2C_SLAVE_ACT_RREQ** to indicate that an I2C master has sent data to the I2C Slave module, **I2C_SLAVE_ACT_TREQ** to indicate that an I2C master has requested that the I2C Slave module send data, and **I2C_SLAVE_ACT_RREQ_FBR** to indicate that an I2C master has sent data to the I2C slave and the first byte following the slave's own address has been received.

9.3 Programming Example

The following example shows how to use the I2C API to send data as a master.

```
// Initialize Master and Slave
I2CMasterInitExpClk(I2C_MASTER_BASE, SysCtlClockGet(), true);

// Specify slave address
I2CMasterSlaveAddrSet(I2C_MASTER_BASE, 0x3B, false);

// Place the character to be sent in the data register
I2CMasterDataPut(I2C_MASTER_BASE, 'Q');

// Initiate send of character from Master to Slave
I2CMasterControl(I2C_MASTER_BASE, I2C_MASTER_CMD_SINGLE_SEND);

// Delay until transmission completes
while(I2CMasterBusBusy(I2C_MASTER_BASE))
{
}
```

10 Inter-processor Communication (IPC)

Introduction	123
M3 IPC API Drivers	124
M3 IPC-Lite API Drivers	145
M3 IPC Utility Drivers	157
C28 IPC API Drivers	160
C28 IPC-Lite API Drivers	181
C28 IPC Utility Drivers	193

10.1 Introduction

The inter-processor communication (IPC) API provides a set of functions for using the IPC to communicate between the M3 and C28 processors.

Functions and structures are provided for sending and receiving basic IPC commands via 2 solutions:

1. The main IPC drivers communicate through Put and Get ring buffers in the CtoM and MtoC MSG RAM's, and allow the user to queue up commands and use multiple interrupt service routines (ISR's) for IPC communications.
2. The IPC-Lite drivers communicate via the IPC registers only, and require no additional memory. They are limited to usage with a single IPC ISR, and commands cannot be queued one after another.

Because the IPC facilitates communication between the M3 and C28 processors, there are both M3 IPC API drivers and C28 IPC API drivers. The API documentation for both processors' drivers is located in controlSUITE under the `/device_support/F28M35x/(VERSION)/doc/` directory.

The M3 driver is contained in `MWare/driverlib/ipc.c`, `MWare/driverlib/ipc_lite.c` and `MWare/driverlib/ipc_util.c` with `MWare/driverlib/ipc.h` containing the API definitions for use by applications.

The C28 driver is contained in `F28M35x_common/source/F28M35x_ipc_common/source/F28M35x_ipc.c`, `F28M35x_common/source/F28M35x_ipc_common/source/F28M35x_ipc_lite.c`, and `F28M35x_common/source/F28M35x_ipc_common/source/F28M35x_ipc_util.c` with `F28M35x_common/source/F28M35x_ipc_common/include/F28M35x_ipc_drivers.h` containing the API definitions for use by applications.

Note:

CTOM IPC INT1 and IPC Flag 32 are used by the C28 boot ROM to report system status errors to the M3 master system during C28 boot time. If the M3 application code uses CTOM IPC INT1, the application must either handle the CTOM Boot ROM status commands in the ISR (Recommended to ensure C28 has booted properly), or ignore the commands by setting the CTOMIPACK bits for IPC_FLAG1 and IPC_FLAG32.

For examples of how to use both the IPC and IPC-Lite drivers, see the examples in controlSUITE for your device.

10.2 M3 IPC API Drivers

Functions

- void [IPCCtoMBlockRead](#) ([tlpcMessage](#) *psMessage)
- void [IPCCtoMBlockWrite](#) ([tlpcMessage](#) *psMessage)
- void [IPCCtoMBlockWrite_Protected](#) ([tlpcMessage](#) *psMessage)
- void [IPCCtoMClearBits](#) ([tlpcMessage](#) *psMessage)
- void [IPCCtoMClearBits_Protected](#) ([tlpcMessage](#) *psMessage)
- void [IPCCtoMDataRead](#) (volatile [tlpcController](#) *psController, [tlpcMessage](#) *psMessage, unsigned short bBlock)
- void [IPCCtoMDataRead_Protected](#) (volatile [tlpcController](#) *psController, [tlpcMessage](#) *psMessage, unsigned short bBlock)
- void [IPCCtoMDataWrite](#) ([tlpcMessage](#) *psMessage)
- void [IPCCtoMDataWrite_Protected](#) ([tlpcMessage](#) *psMessage)
- void [IPCCtoMFunctionCall](#) ([tlpcMessage](#) *psMessage)
- void [IPCCtoMSetBits](#) ([tlpcMessage](#) *psMessage)
- void [IPCCtoMSetBits_Protected](#) ([tlpcMessage](#) *psMessage)
- unsigned short [lpcGet](#) (volatile [tlpcController](#) *psController, [tlpcMessage](#) *psMessage, unsigned short bBlock)
- void [IPCMInitialize](#) (volatile [tlpcController](#) *psController, unsigned short usMlpcInterrupt, unsigned short usClpcInterrupt)
- unsigned short [IPCMtoCBlockRead](#) (volatile [tlpcController](#) *psController, unsigned long ulAddress, unsigned long ulShareAddress, unsigned short usLength, unsigned short bBlock, unsigned long ulResponseFlag)
- unsigned short [IPCMtoCBlockWrite](#) (volatile [tlpcController](#) *psController, unsigned long ulAddress, unsigned long ulShareAddress, unsigned short usLength, unsigned short usWordLength, unsigned short bBlock)
- unsigned short [IPCMtoCBlockWrite_Protected](#) (volatile [tlpcController](#) *psController, unsigned long ulAddress, unsigned long ulShareAddress, unsigned short usLength, unsigned short usWordLength, unsigned short bBlock)
- unsigned short [IPCMtoCClearBits](#) (volatile [tlpcController](#) *psController, unsigned long ulAddress, unsigned long ulMask, unsigned short usLength, unsigned short bBlock)
- unsigned short [IPCMtoCClearBits_Protected](#) (volatile [tlpcController](#) *psController, unsigned long ulAddress, unsigned long ulMask, unsigned short usLength, unsigned short bBlock)
- unsigned short [IPCMtoCDataRead](#) (volatile [tlpcController](#) *psController, unsigned long ulAddress, void *pvData, unsigned short usLength, unsigned short bBlock, unsigned long ulResponseFlag)
- unsigned short [IPCMtoCDataRead_Protected](#) (volatile [tlpcController](#) *psController, unsigned long ulAddress, void *pvData, unsigned short usLength, unsigned short bBlock, unsigned long ulResponseFlag)
- unsigned short [IPCMtoCDataWrite](#) (volatile [tlpcController](#) *psController, unsigned long ulAddress, unsigned long ulData, unsigned short usLength, unsigned short bBlock, unsigned long ulResponseFlag)
- unsigned short [IPCMtoCDataWrite_Protected](#) (volatile [tlpcController](#) *psController, unsigned long ulAddress, unsigned long ulData, unsigned short usLength, unsigned short bBlock, unsigned long ulResponseFlag)
- unsigned short [IPCMtoCFunctionCall](#) (volatile [tlpcController](#) *psController, unsigned long ulAddress, unsigned long ulParam, unsigned short bBlock)

- unsigned short [IPC_MtoC_SendMessage](#) (volatile [tlpcController](#) *psController, unsigned long ulCommand, unsigned long ulAddress, unsigned long ulDataW1, unsigned long ulDataW2, unsigned short bBlock)
- unsigned short [IPC_MtoC_SetBits](#) (volatile [tlpcController](#) *psController, unsigned long ulAddress, unsigned long ulMask, unsigned short usLength, unsigned short bBlock)
- unsigned short [IPC_MtoC_SetBits_Protected](#) (volatile [tlpcController](#) *psController, unsigned long ulAddress, unsigned long ulMask, unsigned short usLength, unsigned short bBlock)
- unsigned short [lpcPut](#) (volatile [tlpcController](#) *psController, [tlpcMessage](#) *psMessage, unsigned short bBlock)

10.2.1 Detailed Description

The main IPC drivers utilize circular buffers (1 Put buffer and 1 Get buffer for each IPC interrupt used) stored in the CtoM and MtoC message RAM's to send messages between processors. [tlpcController](#) data structures store the data for a single Put and Get buffer pair.

Note:

Although the main IPC drivers have the benefit of allowing queued commands for sequential processing, they require additional memory (message RAM's), code changes (command linker file changes and data structure variable definitions), and setup to begin using. For a simpler IPC driver solution which can be used instantly without any additional code changes, see the **IPC-Lite** API drivers.

The figure below shows how the IPC drivers are implemented in the dual core system. Note that buffers and indexes in purple store the MtoC M3 Put buffer/ C28 Get buffer and read/write indexes, while buffers and indexes in yellow store the CtoM C28 Put buffer/ M3 Get buffer and read/write indexes. Additionally, each MtoC circular buffer is tied to an MtoC IPC interrupt, and each CtoM circular buffer is tied to a CtoM IPC interrupt.

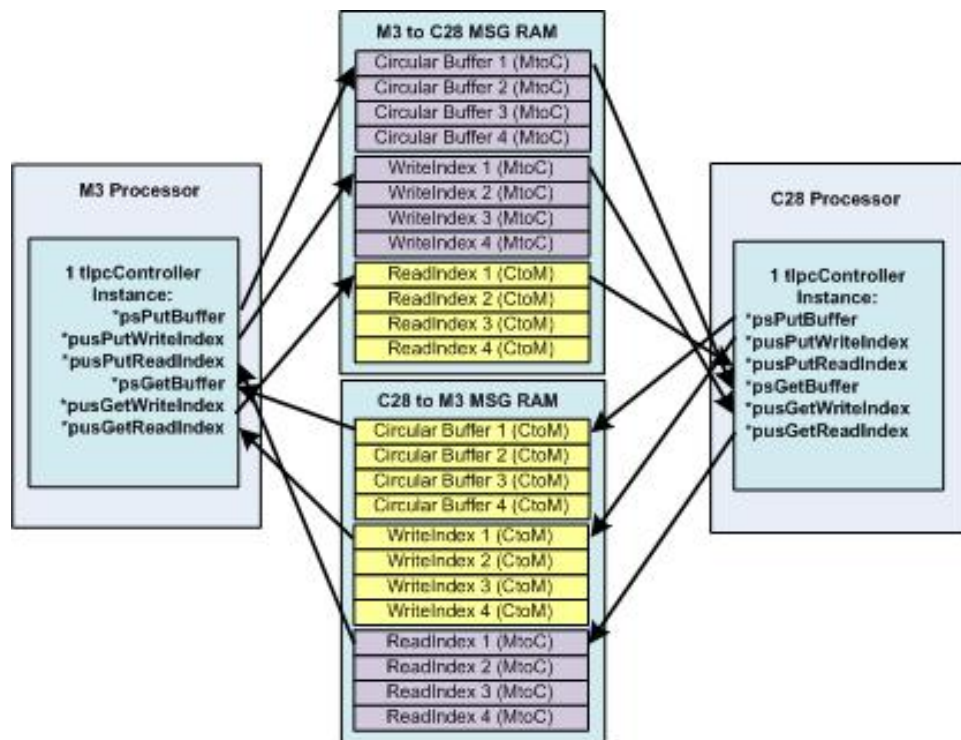


Figure 10.1: Main IPC Drivers Infrastructure Implementation

When one processor (X) wishes to perform an operation on the other processor (Y):

1. Processor X writes messages into its Put Buffer (equivalent to Processor Y's Get Buffer), increments the PutWriteIndex, and sets Processor Y's XtoY IPC interrupt flag.
2. Processor Y sees IPC interrupt flag and subsequently reads the messages from the Get Buffer and increments the GetReadIndex.
3. Processor Y then proceeds to process the commands according to the message contents and then acknowledges XtoY IPC interrupt flag.
4. Processor X can also read and process messages that Processor Y writes to its Put Buffer (which is Processor X's Get Buffer).

Messages are written to and read from each circular buffer in a FIFO fashion as shown below. N is the maximum number of messages that can be stored in a circular buffer and must be an even number. By default, N= 4 in the main IPC drivers. There can be N-1 Puts into the circular buffer until the receiving processor's application code must perform a Get.

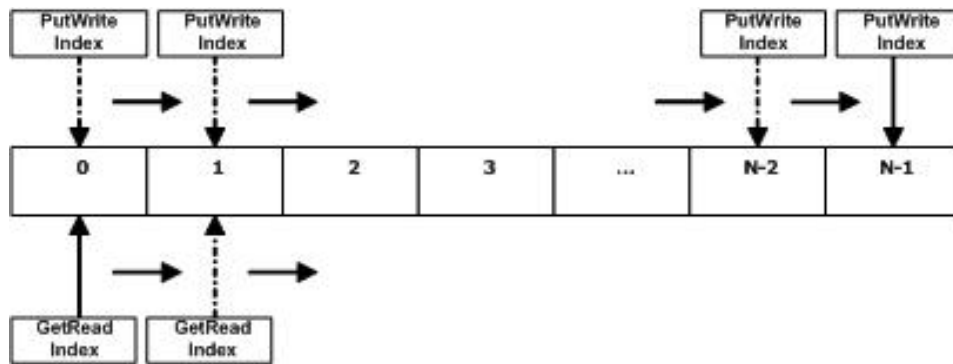


Figure 10.2: Writing to and Reading from Circular Buffer via Indexes

10.2.2 Usage and Examples

The IPC API driver functions will Put commands into the PutBuffer and Get commands from the GetBuffer in a manner that is transparent to the application software. The application code is responsible for initializing the IPC controllers, calling the API driver functions for those commands it wishes to execute, decoding received commands, and acknowledging received IPC interrupt flags.

There are a number of steps which must be taken in the application project in order to use the main IPC API driver functions.

1. Add the `lpc.c` and `lpc_util.c` files which include the `lpc.h` file to the application project. For M3 projects, make sure `MWare/driverlib/lpc.c` and `/driverlib/lpc_util.c` are added as source files in your project, both of which include `/driverlib/lpc.h`. For C28 projects, make sure `/F28M35x_common/source/F28M35x_lpc.c` and `F28M35x_common/source/F28M35x_lpc_util.c` are added as source files in your project, both of which include `/F28M35x_common/include/F28M35x_lpc.h`. The `lpc_util.c` file includes utility functions that can be used by application code and that are used by both the main IPC driver functions and the IPC-lite driver functions.
2. Add IPC buffer and index section definitions to application project's `.cmd` command linker file. Memory regions for the CtoM and MtoC message RAM's should be defined. Additionally, there should be 1 section reserved for the PUTBUFFER, PUTWRITEIDX, and GETREADIDX, and 1 section reserved for the GETBUFFER, GETWRIDEIDX, PUTREADIDX which are mapped to the beginning of each message RAM block by grouping in the `.cmd` file.

The code below is a sample of what a C28 project's `.CMD` linker file should look like when using the main IPC API drivers:

```
MEMORY
{
    PAGE 0 :
        CTOMRAM      : origin = 0x03F800,  length = 0x000400
        MTOGRAM      : origin = 0x03FC00,  length = 0x000400
}

SECTIONS
{
```

```
GROUP : > CTOMRAM, PAGE = 1
{
    PUTBUFFER
    PUTWRITEIDX
    GETREADIDX
}

GROUP : > MTOGRAM, PAGE = 1
{
    GETBUFFER : TYPE = DSECT
    GETWRITEIDX : TYPE = DSECT
    PUTREADIDX : TYPE = DSECT
}

}
```

The code below is a sample of what an M3 project's .CMD linker file should look like when using the main IPC API drivers:

```
MEMORY
{
    CTOMRAM (RX) : origin = 0x2007F000, length = 0x00000800
    MTOGRAM (RWX): origin = 0x2007F800, length = 0x00000800
}

SECTIONS
{
    GROUP : > MTOGRAM
    {
        PUTBUFFER
        PUTWRITEIDX
        GETREADIDX
    }

    GROUP : > CTOMRAM
    {
        GETBUFFER : TYPE = DSECT
        GETWRITEIDX : TYPE = DSECT
        PUTREADIDX : TYPE = DSECT
    }
}
```

1. Application source code must define and initialize at least 1 volatile global [tlpcController](#) instance on both the M3 and the C28.

The IPC controller data structure ([tlpcController](#)) locally stores the pointers to the Put/Get buffers and their read/write indexes. Application code must define at least 1 instance of the [tlpcController](#) data structure on the M3 and a corresponding data structure on the C28 for a single M3 IPC interrupt and C28 IPC interrupt communication pair (i.e. M3 will send messages to C28 and inform C28 via a C28 IPC interrupt flag, and C28 will respond with a message sent to an M3 IPC interrupt flag). A code example of the definition and initialization of the [tlpcController](#) instance for M3 is shown below (C28 calls [IPCCInitialize\(\)](#) instead of [IPCMIInitialize\(\)](#)):

```
// *****
```



```

// At least 1 volatile global tIpcController instance is required
// when using IPC API Drivers.
//*****
volatile tIpcController g_sIpcController1;
volatile tIpcController g_sIpcController2;

void
main(void)
{
    // Initialize IPC Controllers
    IPCMInitialize (&g_sIpcController1, IPC_INT1, IPC_INT1);
    IPCMInitialize (&g_sIpcController2, IPC_INT2, IPC_INT2);
}

```

After these initialization steps, the M3 and C28 application code can set up an IPC interrupt service routine (ISR) corresponding to each [tlpcController](#) instance that was defined, and then proceed to call the IPC command functions using the respective controllers. Generally when an IPC command function is called for the sending processor, the responding function for the receiving processor has the same name. For instance, the M3 application code might call the [IPCMtoCDataWrite\(\)](#) function to write data to memory accessible only by the C28. The C28 will decode the command received in the Get Buffer message, and upon seeing an IPC_DATA_WRITE command, it will call its own [IPCMtoCDataWrite\(\)](#) function to process the command.

For IPC commands that must occur sequentially, the application code must use a single [tlpcController](#) instance when calling the sequential IPC functions. This ensures that a single IPC interrupt processes the messages in the order they were received by the Get Buffer. If there is another set of IPC commands that occurs independently from the first set of IPC commands, these can be called using a different [tlpcController](#) instance.

Note:

CTOM IPC INT1 and IPC Flag 32 are used by the C28 boot ROM to report system status errors to the M3 master system during C28 boot time. If the M3 application code uses CTOM IPC INT1, the application must either handle the CTOM Boot ROM status commands in the ISR (Recommended to ensure C28 has booted properly), or ignore the commands by setting the CTOMIPACK bits for IPC_FLAG1 and IPC_FLAG32.

For examples demonstrating many of the basic IPC functions, see the IPC examples in the control-SUITE device_support package for your device.

10.2.3 Customization

There are a number of settings which can be customized for different application needs when using the main IPC API drivers in the headerfile (ipc.h/ (Device)_ipc.h). Any change to the C28 header file must also be made to the M3 header file and vice versa:

1. **IPC_BUFFER_SIZE:** This is the definition for the maximum number of [tlpcMessage](#) messages in the circular buffer. By default this value is 4, but it can be any even number as long as there is enough memory in the message RAM block for the buffer.
2. **NUM_IPC_INTERRUPTS:** This is the number of IPC interrupts for which an IPC driver API circular buffer is reserved (the number must be the same on both the C28 and M3, because a single IPC controller needs information for 1 M3 IPC interrupt and 1 C28 IPC interrupt). This

number is set to 4 by default. The value can be decreased to save message RAM memory if, for instance, one of the other interrupt channels is used for IPC-Lite function processing instead.

3. **tlpcMessage**: This is the data structure of the messages transmitted by the IPC API driver functions. Although the data structure is defined to expect a command, address, and two 32-bit data words for additional information as defined by the IPC API driver functions, the structure can also be used merely to transmit 4 generic 32-bit data words between processors, if desired. Application code will then have to process the Get accordingly in the IPC ISR.

10.2.4 Function Documentation

10.2.4.1 IPCCtoMBlockRead

Reads a block of data from an M3 master system address and stores into shared RAM

Prototype:

```
void  
IPCCtoMBlockRead(tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the C28 control system.

Description:

This function will respond to the C28 control system request to read a block of data from the M3 master system, by reading the data and placing that data into the shared RAM location specified in *psMessage*.

Returns:

None.

10.2.4.2 IPCCtoMBlockWrite

Writes a block of data to an M3 master system address from shared RAM

Prototype:

```
void  
IPCCtoMBlockWrite(tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the C28 control system.

Description:

This function will write a block of data to an address on the M3 master system. The data is first written by the C28 to shared RAM. This function reads the shared RAM location, word size (16- or 32-bit), and block size from *psMessage* and writes the block to the M3 address specified in *psMessage*.

Note:

Writes to registers accessible via the APB bus must be 32-bits wide.

Returns:

None.

10.2.4.3 IPCCToMBlockWrite_Protected

Writes a block of data to an M3 master system write-protected address from shared RAM

Prototype:

```
void  
IPCCToMBlockWrite_Protected(tIpCMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the C28 control system.

Description:

This function will write a block of data to a write-protected group of addresses on the M3 master system. The data is first written by the C28 to shared RAM. This function reads the shared RAM location, word size (16- or 32-bit), and block size from *psMessage* and writes the block to the M3 address specified in *psMessage*.

Note:

Writes to registers accessible via the APB bus must be 32-bits wide.

Returns:

None.

10.2.4.4 IPCCToMClearBits

Clears the designated bits in a 32-bit data word at an M3 master system address

Prototype:

```
void  
IPCCToMClearBits(tIpCMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the C28 control system.

Description:

This function will allow the C28 control system to clear the bits in a 16/32-bit word on the M3 master system via an M3 address and mask passed in via the *psMessage*.

Returns:

None.

10.2.4.5 IPCCToMClearBits_Protected

Clears the designated bits in a write-protected 16/32-bit data word at an M3 master system address

Prototype:

```
void  
IPCctoMClearBits_Protected(tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the C28 control system.

Description:

This function will allow the C28 control system to clear the bits in a 16/32-bit write-protected word on the M3 master system via an M3 address and mask passed in via the *psMessage*.

Returns:

None.

10.2.4.6 IPCctoMDataRead

Responds to 16/32-bit data word read command from C28 control system

Prototype:

```
void  
IPCctoMDataRead(volatile tIpcController *psController,  
                tIpcMessage *psMessage,  
                unsigned short bBlock)
```

Parameters:

psController specifies the address of a *tIpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

psMessage specifies the pointer to the message received from the C28 control system.

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the C28 control system to read a 16/32-bit data word at the M3 address specified in /e *psMessage*, and send a Write command with the read data back to the C28 control system. It will also send the Response Flag used to track the read back to the C28. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

None.

10.2.4.7 IPCctoMDataRead_Protected

Responds to 16/32-bit data word read command from C28 control system to read into a write-protected word on the C28 system.

Prototype:

```
void  
IPCctoMDataRead_Protected(volatile tIpcController *psController,  
                           tIpcMessage *psMessage,  
                           unsigned short bBlock)
```

Parameters:

psController specifies the address of a *tlpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

psMessage specifies the pointer to the message received from the C28 control system.

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the C28 control system to read a 16/32-bit data word at the M3 address specified in /e *psMessage*, and send a Write Protected command with the read data back to the C28 control system at a write protected address. It will also send the Response Flag used to track the read back to the C28. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

None.

10.2.4.8 IPCctoMDataWrite

Responds to 16/32-bit data word write command from C28 control system

Prototype:

```
void  
IPCctoMDataWrite(tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the C28 control system.

Description:

This function will allow the M3 master system to write a 16/32-bit word received from the C28 control system to the address indicated in *psMessage*. In the event that the IPC_DATA_WRITE command was received as a result of an IPC_DATA_READ command, this function will also clear the IPC response flag tracking the read so other threads in the M3 system will be aware that the data is ready.

Returns:

None.

10.2.4.9 IPCctoMDataWrite_Protected

Responds to 16/32-bit write-protected data word write command from C28 control system

Prototype:

```
void  
IPCctoMDataWrite_Protected(tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the C28 control system.

Description:

This function will allow the M3 master system to write a 16/32-bit word received from the C28 control system to the write-protected address indicated in **psMessage*. In the event that the IPC_DATA_WRITE_PROTECTED command was received as a result of an IPC_DATA_READ_PROTECTED command, this function will also clear the IPC response flag tracking the read so other threads in the M3 system will be aware that the data is ready.

Returns:

None.

10.2.4.10 IPCctoMFunctionCall

Calls an M3 function with a single optional parameter.

Prototype:

```
void  
IPCctoMFunctionCall (tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the C28 control system.

Description:

This function will allow the C28 control system to call an M3 function with a a single optional parameter. There is no return value from the executed function.

Returns:

None.

10.2.4.11 IPCctoMSetBits

Sets the designated bits in a 16/32-bit data word at an M3 master system address

Prototype:

```
void  
IPCctoMSetBits (tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the C28 control system.

Description:

This function will allow the C28 control system to set the bits in a 16/32-bit word on the M3 master system via an M3 address and mask passed in via the *psMessage*.

Returns:

None.

10.2.4.12 IPCCtoMSetBits_Protected

Sets the designated bits in a 16/32-bit write-protected data word at an M3 master system address

Prototype:

```
void  
IPCCtoMSetBits_Protected(tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the C28 control system.

Description:

This function will allow the C28 control system to set the bits in a write-protected 16/32-bit word on the M3 master system via an M3 address and mask passed in via the *psMessage*.

Returns:

None

10.2.4.13 IpcGet

Reads a message from the GetBuffer.

Prototype:

```
unsigned short  
IpcGet(volatile tIpcController *psController,  
       tIpcMessage *psMessage,  
       unsigned short bBlock)
```

Parameters:

psController specifies the address of a *tlpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

psMessage specifies the address of the *tlpcMessage* instance where the message from Get-Buffer should be written to.

bBlock specifies whether to allow function to block until GetBuffer has a message (1= wait until message available, 0 = exit with STATUS_FAIL if no message).

Description:

This function checks if there is a message in the GetBuffer. If so, it get the message in the GetBuffer pointed to by the ReadIndex and writes it to the address pointed to by *psMessage*. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

1 if GetBuffer is empty. 0 if Get occurs successfully.

10.2.4.14 IPCMInitialize

Initializes M3 master system IPC Driver controller

Prototype:

```
void  
IPCMInitialize(volatile tIpcController *psController,  
               unsigned short usMipcInterrupt,  
               unsigned short usCipcInterrupt)
```

Parameters:

psController specifies the address of a [tIpcController](#) instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

usMipcInterrupt specifies the M3 IPC interrupt number used by psController.

usCipcInterrupt specifies the C28 IPC interrupt number used by psController.

Description:

This function initializes the IPC driver controller with circular buffer and index addresses for an IPC interrupt pair (M3-C28). The *usCipcInterrupt* and *usMipcInterrupt* parameters can be 1 of the following values: **IPC_INT1**, **IPC_INT2**, **IPC_INT3**, **IPC_INT4**.

Note:

If an interrupt is currently in use by an [tIpcController](#) instance, that particular interrupt should not be tied to a second [tIpcController](#) instance.

For a particular usMipcInterrupt - usCipcInterrupt pair, there must be an instance of [tIpcController](#) defined and initialized on both the M3 and the C28 systems.

Returns:

None.

10.2.4.15 IPCMtoCBlockRead

Sends the command to read a block of data from an C28 control system address

Prototype:

```
unsigned short  
IPCMtoCBlockRead(volatile tIpcController *psController,  
                 unsigned long ulAddress,  
                 unsigned long ulShareAddress,  
                 unsigned short usLength,  
                 unsigned short bBlock,  
                 unsigned long ulResponseFlag)
```

Parameters:

psController specifies the address of a [tIpcController](#) instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the C28 memory block starting address to read from.

ulShareAddress specifies the M3 shared memory address the read block will read to.

usLength designates the block size in 16-bit words.

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

ulResponseFlag indicates the MtoC Flag number mask used to report when the read block data is available starting at /e ulShareAddress. (*ulResponseFlag* MUST use IPC flags 17-32, and not 1-16)

Description:

This function will allow the M3 master system to send a read block command to the C28 master system and set a ResponseFlag to track the status of the read. The C28 master system will process the read and place the data in shared memory at the location specified in the *ulShareAddress* parameter and then clear the ResponseFlag, indicating that the block is ready. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**. The *ulResponseFlag* parameter can be any single one of the flags **IPC_FLAG17 - IPC_FLAG32** or **NO_FLAG**.

Returns:

status of command (0=success, 1=error - PutBuffer was full, command was not sent)

10.2.4.16 IPCMtoCBlockWrite

Writes a block of data to a C28 control system address

Prototype:

```
unsigned short
IPCMtoCBlockWrite(volatile tIpcController *psController,
                  unsigned long ulAddress,
                  unsigned long ulShareAddress,
                  unsigned short usLength,
                  unsigned short usWordLength,
                  unsigned short bBlock)
```

Parameters:

psController specifies the address of a *tIpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the C28 memory block starting address to write to.

ulShareAddress specifies the M3 shared memory address where data to write from resides.

usLength designates the block size in 16- or 32-bit words (depends on *usWordLength*).

usWordLength designates the word size (16-bits = 1 or 32-bits = 2).

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the M3 master system to write a block of data to the C28 control system starting from the location specified by the *ulAddress* parameter. Prior to calling this function, the M3 master system code should place the data to write in shared memory starting at *ulShareAddress*. The *usWordLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**. The *ulResponseFlag* parameter can be any single one of the flags **IPC_FLAG17 - IPC_FLAG32** or **NO_FLAG**.

Note:

If the Sx SARAM blocks are used to pass the RAM block between the processors, the MSXMSEL register must be configured for M3 master access prior to the M3 writing the block to *ulShareAddress* in order to give the M3 write access to the Sx memory block(s) used for passing data.

Returns:

status of command (0=success, 1=error - PutBuffer was full, command

10.2.4.17 IPCMtoCBlockWrite_Protected

Writes a block of data to a write-protected C28 control system address

Prototype:

```
unsigned short  
IPCMtoCBlockWrite_Protected(volatile tIpcController *psController,  
                             unsigned long ulAddress,  
                             unsigned long ulShareAddress,  
                             unsigned short usLength,  
                             unsigned short usWordLength,  
                             unsigned short bBlock)
```

Parameters:

psController specifies the address of a **tIpcController** instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the write-protected C28 memory block starting address to write to.

ulShareAddress specifies the M3 shared memory address where data to write from resides.

usLength designates the block size in 16- or 32-bit words (depends on **usWordLength**).

usWordLength designates the word size (16-bits = 1 or 32-bits = 2).

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the M3 master system to write a block of data to a write-protected region on the C28 control system starting from the location specified by the **ulAddress** parameter. Prior to calling this function, the M3 master system code should place the data to write in shared memory starting at /e **ulShareAddress**. The **usWordLength** parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The **bBlock** parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**. The **ulResponseFlag** parameter can be any single one of the flags **IPC_FLAG17** - **IPC_FLAG32** or **NO_FLAG**.

Note:

If the Sx SARAM blocks are used to pass the RAM block between the processors, the MSXM-SEL register must be configured for M3 master access prior to the M3 writing the block to /e **ulShareAddress** in order to give the M3 write access to the Sx memory block(s) used for passing data.

Returns:

status of command (0=success, 1=error - PutBuffer was full, command was not sent)

10.2.4.18 IPCMtoCClearBits

Clears the designated bits in a 16-bit data word at a C28 control system address

Prototype:

```
unsigned short  
IPCMtoCClearBits(volatile tIpcController *psController,  
                 unsigned long ulAddress,  
                 unsigned long ulMask,
```

```
unsigned short usLength,
unsigned short bBlock)
```

Parameters:

psController specifies the address of a [tlpcController](#) instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the C28 address to write to

ulMask specifies the 16/32-bit mask for bits which should be cleared at *ulAddress*. 16-bit masks should fill the lower 16-bits (upper 16-bits will be all 0x0000).

usLength specifies the length of the bit mask (1=16-bits, 2=32-bits)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the M3 master system to clear bits specified by the *ulMask* variable in a 16/32-bit word on the C28 control system. The *usLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

status of command (0=success, 1=error - PutBuffer was full, command was not sent)

10.2.4.19 IPCMtoCClearBits_Protected

Clears the designated bits in a 16-bit write-protected data word at a C28 control system address

Prototype:

```
unsigned short
IPCMtoCClearBits_Protected(volatile tlpcController *psController,
                           unsigned long ulAddress,
                           unsigned long ulMask,
                           unsigned short usLength,
                           unsigned short bBlock)
```

Parameters:

psController specifies the address of a [tlpcController](#) instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the C28 address to write to

ulMask specifies the 16/32-bit mask for bits which should be cleared at *ulAddress*. 16-bit masks should fill the lower 16-bits (upper 16-bits will be all 0x0000).

usLength specifies the length of the bit mask (1=16-bits, 2=32-bits)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the M3 master system to set bits specified by the *ulMask* variable in a write-protected 16/32-bit word on the C28 control system. The *usLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

status of command (0=success, 1=error - PutBuffer was full, command was not sent)

10.2.4.20 IPCMtoCDataRead

Sends the command to read either a 16- or 32-bit data word from a C28 control system address

Prototype:

```
unsigned short
IPCMtoCDataRead(volatile tIpcController *psController,
                 unsigned long ulAddress,
                 void *pvData,
                 unsigned short usLength,
                 unsigned short bBlock,
                 unsigned long ulResponseFlag)
```

Parameters:

psController specifies the address of a *tIpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the C28 address to read from

pvData is a pointer to the 16/32-bit variable where read data will be stored.

usLength designates 16- or 32-bit read (1 = 16-bit, 2 = 32-bit)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

ulResponseFlag indicates the MtoC Flag number mask used to report when the read data is available at pvData. (*ulResponseFlag* MUST use IPC flags 17-32, and not 1-16).

Description:

This function will allow the M3 master system to send a 16/32-bit data read command to the C28 control system and set a ResponseFlag to track the status of the read. The C28 control system will respond with a DataWrite command which will place the data in the M3 address pointed to by *pvData*. When the M3 receives the DataWrite command and writes the read data into **pvData* location, it will clear the ResponseFlag, indicating to the rest of the system that the data is ready. The *usLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**. The *ulResponseFlag* parameter can be any single one of the flags **IPC_FLAG17** - **IPC_FLAG32** or **NO_FLAG**.

Returns:

status of command (0=success, 1=error - PutBuffer was full, command was not sent)

10.2.4.21 IPCMtoCDataRead_Protected

Sends the command to read either a 16- or 32-bit data word from a C28 control system address into a write-protected M3 address.

Prototype:

```
unsigned short
IPCMtoCDataRead_Protected(volatile tIpcController *psController,
                           unsigned long ulAddress,
                           void *pvData,
                           unsigned short usLength,
                           unsigned short bBlock,
                           unsigned long ulResponseFlag)
```

Parameters:

psController specifies the address of a [tlpcController](#) instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the C28 address to read from

pvData is a pointer to the 16/32-bit write-protected variable where read data will be stored.

usLength designates 16- or 32-bit read (1 = 16-bit, 2 = 32-bit)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

ulResponseFlag indicates the MtoC Flag number mask used to report when the read data is available at pvData. (*ulResponseFlag* MUST use IPC flags 17-32, and not 1-16)

Description:

This function will allow the M3 master system to send a 16/32-bit data read command to the C28 control system and set a ResponseFlag to track the status of the read. The C28 control system will respond with a DataWrite command which will place the data in the M3 address pointed to by *pvData*. When the M3 receives the DataWrite command and writes the read data into **pvData* location, it will clear the ResponseFlag, indicating to the rest of the system that the data is ready. The *usLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**. The *ulResponseFlag* parameter can be any single one of the flags **IPC_FLAG17** - **IPC_FLAG32** or **NO_FLAG**.

Returns:

status of command (0=success, 1=error - PutBuffer was full, command was not sent)

10.2.4.22 IPCMtoCDataWrite

Writes a 16/32-bit data word to a C28 control system address

Prototype:

```
unsigned short
IPCMtoCDataWrite(volatile tIpcController *psController,
                 unsigned long ulAddress,
                 unsigned long ulData,
                 unsigned short usLength,
                 unsigned short bBlock,
                 unsigned long ulResponseFlag)
```

Parameters:

psController specifies the address of a [tlpcController](#) instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the C28 address to write to

ulData specifies the 16/32-bit word which will be written. For 16-bit words, only the lower 16-bits of ulData will be considered by the master system.

usLength is the length of the word to write (1 = 16-bits, 2 = 32-bits)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

ulResponseFlag is used to pass the *ulResponseFlag* back to the C28 only when this function is called in response to [IPCctoMDataRead\(\)](#). Otherwise, set to 0.

Description:

This function will allow the M3 master system to write a 16/32-bit word via the *ulData* variable to an address on the C28 master system. The *usLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**. The *ulResponseFlag* parameter can be any single one of the flags **IPC_FLAG17** - **IPC_FLAG32** or **NO_FLAG**.

Returns:

status of command (0=success, 1=error)

10.2.4.23 IPCMtoCDataWrite_Protected

Writes a 16/32-bit data word to a write-protected C28 control system address

Prototype:

```
unsigned short
IPCMtoCDataWrite_Protected(volatile tIpcController *psController,
                           unsigned long ulAddress,
                           unsigned long ulData,
                           unsigned short usLength,
                           unsigned short bBlock,
                           unsigned long ulResponseFlag)
```

Parameters:

psController specifies the address of a [tIpcController](#) instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the write-protected C28 address to write to

ulData specifies the 16/32-bit word which will be written. For 16-bit words, only the lower 16-bits of ulData will be considered by the master system.

usLength is the length of the word to write (1 = 16-bits, 2 = 32-bits)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

ulResponseFlag is used to pass the *ulResponseFlag* back to the C28 only when this function is called in response to [IPCctoMDataRead\(\)](#). Otherwise, set to 0.

Description:

This function will allow the M3 master system to write a 16/32-bit word via the *ulData* variable to a write-protected address on the C28 control system. The *usLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**. The *ulResponseFlag* parameter can be any single one of the flags **IPC_FLAG17** - **IPC_FLAG32** or **NO_FLAG**.

Returns:

status of command (0=success, 1=error)

10.2.4.24 IPCMtoCFunctionCall

Calls a C28 function with 1 optional parameter.

Prototype:

```
unsigned short
IPCMtoCFunctionCall (volatile tIpcController *psController,
                     unsigned long ulAddress,
                     unsigned long ulParam,
                     unsigned short bBlock)
```

Parameters:

psController specifies the address of a [tIpcController](#) instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the C28 function address

ulParam specifies the 32-bit optional parameter value. If not used, this can be a dummy value.

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the M3 control system to call a function on the C28. The *ulParam* variable is a single optional 32-bit parameter to pass to the function. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

status of command (**STATUS_PASS** =success, **STATUS_FAIL** = error because PutBuffer was full, command was not sent)

10.2.4.25 IPCMtoCSendMessage

Sends generic message to C28 control system

Prototype:

```
unsigned short
IPCMtoCSendMessage (volatile tIpcController *psController,
                    unsigned long ulCommand,
                    unsigned long ulAddress,
                    unsigned long ulDataW1,
                    unsigned long ulDataW2,
                    unsigned short bBlock)
```

Parameters:

psController specifies the address of a [tIpcController](#) instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulCommand specifies 32-bit command word to insert into message.

ulAddress specifies 32-bit address word to insert into message.

ulDataW1 specifies 1st 32-bit data word to insert into message.

ulDataW2 specifies 2nd 32-bit data word to insert into message.

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the M3 master system to send a generic message to the C28 control system. Note that the user should create a corresponding function on the C28 side to interpret/use the message or fill parameters in such a way that the existing IPC drivers can

recognize the command and properly process the message. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

status of command (0=success, 1=error - PutBuffer was full, command was not sent)

10.2.4.26 IPCMtoCSetBits

Sets the designated bits in a 16-bit data word at an C28 control system address

Prototype:

```
unsigned short
IPCMtoCSetBits (volatile tlpcController *psController,
                unsigned long ulAddress,
                unsigned long ulMask,
                unsigned short usLength,
                unsigned short bBlock)
```

Parameters:

psController specifies the address of a *tlpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the C28 address to write to

ulMask specifies the 16/32-bit mask for bits which should be set at *ulAddress*. 16-bit masks should fill the lower 16-bits (upper 16-bits will be all 0x0000).

usLength specifies the length of the bit mask (1=16-bits, 2=32-bits)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot). This function will allow the M3 master system to set bits specified by the *ulMask* variable in a 16/32-bit word on the C28 control system. The *usLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

status of command (0=success, 1=error - PutBuffer was full, command was not sent)

10.2.4.27 unsigned short IPCMtoCSetBits_Protected (volatile *tlpcController* * *psController*, unsigned long *ulAddress*, unsigned long *ulMask*, unsigned short *usLength*, unsigned short *bBlock*)

Sets the designated bits in a 16-bit write-protected data word at a C28 control system address

Parameters:

psController specifies the address of a *tlpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the C28 address to write to

ulMask specifies the 16/32-bit mask for bits which should be set at *ulAddress*. 16-bit masks should fill the lower 16-bits (upper 16-bits will be all 0x0000).

usLength specifies the length of the bit mask (1=16-bits, 2=32-bits)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the M3 master system to set bits specified by the *ulMask* variable in a write-protected 16/32-bit word on the C28 control system. The *usLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

status of command (0=success, 1=error - PutBuffer was full, command was not sent)

10.2.4.28 IpcPut

Writes a message into the PutBuffer.

Prototype:

```
unsigned short
IpcPut(volatile tIpcController *psController,
       tIpcMessage *psMessage,
       unsigned short bBlock)
```

Parameters:

psController specifies the address of a *tIpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

psMessage specifies the address of the *tIpcMessage* instance to be written to PutBuffer.

bBlock specifies whether to allow function to block until PutBuffer has a free slot (1= wait until free spot available, 0 = exit with STATUS_FAIL if no free slot).

Description:

This function checks if there is a free slot in the PutBuffer. If so, it puts the message pointed to by *psMessage* into the free slot and increments the WriteIndex. Then it sets the appropriate IPC interrupt flag specified by *psController->usPutFlag*. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

1 if PutBuffer is full. 0 if Put occurs successfully.

10.3 M3 IPC-Lite API Drivers

Functions

- void *IPCLiteCtoMClearBits* (unsigned long ulFlag, unsigned long ulStatusFlag)
- void *IPCLiteCtoMClearBits_Protected* (unsigned long ulFlag, unsigned long ulStatusFlag)
- void *IPCLiteCtoMDataRead* (unsigned long ulFlag, unsigned long ulStatusFlag)
- void *IPCLiteCtoMDataWrite* (unsigned long ulFlag, unsigned long ulStatusFlag)
- void *IPCLiteCtoMDataWrite_Protected* (unsigned long ulFlag, unsigned long ulStatusFlag)
- void *IPCLiteCtoMFunctionCall* (unsigned long ulFlag, unsigned long ulStatusFlag)

- void [IPCLiteCtoMSetBits](#) (unsigned long ulFlag, unsigned long ulStatusFlag)
- void [IPCLiteCtoMSetBits_Protected](#) (unsigned long ulFlag, unsigned long ulStatusFlag)
- unsigned short [IPCLiteMtoCBootBranch](#) (unsigned long ulAddress)
- unsigned short [IPCLiteMtoCClearBits](#) (unsigned long ulFlag, unsigned long ulAddress, unsigned long ulMask, unsigned short usLength, unsigned long ulStatusFlag)
- unsigned short [IPCLiteMtoCClearBits_Protected](#) (unsigned long ulFlag, unsigned long ulAddress, unsigned long ulMask, unsigned short usLength, unsigned long ulStatusFlag)
- unsigned short [IPCLiteMtoCDataRead](#) (unsigned long ulFlag, unsigned long ulAddress, unsigned short usLength, unsigned long ulStatusFlag)
- unsigned short [IPCLiteMtoCDataWrite](#) (unsigned long ulFlag, unsigned long ulAddress, unsigned long ulData, unsigned short usLength, unsigned long ulStatusFlag)
- unsigned short [IPCLiteMtoCDataWrite_Protected](#) (unsigned long ulFlag, unsigned long ulAddress, unsigned long ulData, unsigned short usLength, unsigned long ulStatusFlag)
- unsigned short [IPCLiteMtoCFunctionCall](#) (unsigned long ulFlag, unsigned long ulAddress, unsigned long ulParam, unsigned long ulStatusFlag)
- unsigned short [IPCLiteMtoCGetResult](#) (void *pvData, unsigned short usLength, unsigned long ulStatusFlag)
- unsigned short [IPCLiteMtoCSetBits](#) (unsigned long ulFlag, unsigned long ulAddress, unsigned long ulMask, unsigned short usLength, unsigned long ulStatusFlag)
- unsigned short [IPCLiteMtoCSetBits_Protected](#) (unsigned long ulFlag, unsigned long ulAddress, unsigned long ulMask, unsigned short usLength, unsigned long ulStatusFlag)

10.3.1 Detailed Description

The IPC-Lite drivers utilize the IPC registers to send command messages one at a time via a single IPC interrupt. No additional shared memory/message RAM communication is required. Only one command can be sent and processed while the IPC interrupt flag is set. Once the IPC interrupt flag is acknowledged (cleared), another command can be sent. Only a single ISR can be used with the IPC-Lite drivers at a time.

10.3.2 Usage and Examples

The IPC-Lite drivers operate as follows in a 2 processor device where X and Y are the arbitrary processor identifiers:

1. Processor X's application code calls IPC API driver function (i.e. [IPCLiteXtoYDataWrite\(\)](#)). If the IPC interrupt flag is still busy processing another command, this function will return `STATUS_FAIL` and not send any command.
2. Processor X's IPC API Driver Function inserts values into `XTOYIPCCOM`, `XTOYIPCADDR`, `XTOYIPCDATAW`, `XTOYIPCDATAR` registers as needed by IPC command.
3. Processor X's IPC API function then sets XtoY IPC INT flag to trigger an interrupt on processor Y.
4. Processor Y's XtoY IPC INT ISR is triggered. In the ISR, processor Y's application code reads the `XTOYIPCCOM` register and decodes the command to determine which IPC API driver function to call (i.e. [IPCLiteXtoYDataWrite\(\)](#)).

5. The IPC API driver function processes the command, writes any data to return to processor X in the XTOYDATAR register, acknowledges the IPC interrupt flag, and the interrupt ISR is exited.
6. When processor X's application code needs the return data from processor Y for that function, it calls the `IPCLiteXtoYGetResult()` function in a loop and waits until the function returns `STATUS_PASS`. At this point, the value sent from processor Y can be read by processor X.
7. Only now can Processor X send another command to Processor Y.

There is only one step the application project must take in order to use the main IPC API driver functions.

- Add the `lpc_lite.c` and `lpc_util.c` files which include the `lpc.h` file to the application project. For M3 projects, make sure `/Mware/driverlib/lpc_lite.c` and `/driverlib/lpc_util.c` are added as source files in your project, both of which include `/driverlib/lpc.h`. For C28 projects, make sure `/F28M35x_common/source/F28M35x_lpc_lite.c` and `/F28M35x_common/source/F28M35x_lpc_util.c` are added as source files in your project, both of which include `/F28M35x_common/include/F28M35x_lpc.h`. The `lpc_util.c` file includes utility functions that can be used by application code and that are used by both the main IPC driver functions and the IPC-lite driver functions.

The M3 and C28 application code can immediately start calling the IPC command functions. Generally when an IPC command function is called for the sending processor, the responding function for the receiving processor has the same name. For instance, the M3 application code might call the `IPCMtoCDataRead()` function to write data to memory accessible only by the C28. The C28 will decode the command in the `MTOCIPCCOM` register when the interrupt is flagged, and upon seeing an `IPC_DATA_READ` command, it will call its own `IPCMtoCDataRead()` function to process the command.

For examples demonstrating many of the basic IPC-Lite functions, see the IPC-Lite examples in the `controlSUITE` device_support package for your device.

10.3.3 Function Documentation

10.3.3.1 IPCLiteCtoMClearBits

Clears the designated bits in a 16/32-bit data word at an M3 master system address

Prototype:

```
void
IPCLiteCtoMClearBits(unsigned long ulFlag,
                     unsigned long ulStatusFlag)
```

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the master system.

Description:

This function will allow the C28 control system to clear bits specified by a mask variable in a 16/32-bit word on the M3 master system, and then read back the word into the `CTOMIPCDATAR` register. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** -

IPC_FLAG32, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1 - IPC_FLAG32** and **NO_FLAG**.

10.3.3.2 void IPCLiteCtoMClearBits_Protected (unsigned long *ulFlag*, unsigned long *ulStatusFlag*)

Clears the designated bits in a 16/32-bit data word at an M3 master system write-protected address

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the master system.

This function will allow the C28 control system to clear bits specified by a mask variable in a 16/32-bit word on the M3 master system, and then read back the word into the CTOMIPCDATAR register. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1 - IPC_FLAG32**, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1 - IPC_FLAG32** and **NO_FLAG**.

10.3.3.3 IPCLiteCtoMDataRead

Reads either a 16- or 32-bit data word from an M3 master system address

Prototype:

```
void  
IPCLiteCtoMDataRead(unsigned long ulFlag,  
                     unsigned long ulStatusFlag)
```

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the master system.

Description:

This function will allow the C28 control system to read 16/32-bit data from the M3 master system. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1 - IPC_FLAG32**, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1 - IPC_FLAG32** and **NO_FLAG**.

10.3.3.4 void IPCLiteCtoMDataWrite (unsigned long *ulFlag*, unsigned long *ulStatusFlag*)

Writes a 16/32-bit data word to an M3 master system address

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the master system.

This function will allow the C28 control system to write a 16/32-bit word to an address on the M3 master system. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**.

10.3.3.5 IPCLiteCtoMDataWrite_Protected

Writes a 16/32-bit data word to a write-protected M3 master system address

Prototype:

```
void  
IPCLiteCtoMDataWrite_Protected(unsigned long ulFlag,  
                                unsigned long ulStatusFlag)
```

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the master system.

Description:

This function will allow the C28 control system to write a 16/32-bit word to an address on the M3 master system. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**.

10.3.3.6 void IPCLiteCtoMFunctionCall (unsigned long *ulFlag*, unsigned long *ulStatusFlag*)

Calls an M3 function with a single optional parameter and return value.

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the control system.

This function will allow the C28 control system to call an M3 function with a single optional parameter and places an optional return value in the CTOMIPCDATAR register. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**.

10.3.3.7 IPCLiteCtoMSetBits

Sets the designated bits in a 16/32-bit data word at an M3 master system address

Prototype:

```
void  
IPCLiteCtoMSetBits(unsigned long ulFlag,  
                    unsigned long ulStatusFlag)
```

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the master system.

Description:

This function will allow the C28 control system to set bits specified by a mask variable in a 16/32-bit word on the M3 master system, and then read back the word into the CTOMIPCDATAR register. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**.

10.3.3.8 void IPCLiteCtoMSetBits_Protected (unsigned long *ulFlag*, unsigned long *ulStatusFlag*)

Sets the designated bits in a 16-bit data word at an M3 master system write-protected address

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the master system.

This function will allow the C28 control system to set bits specified by a mask variable in a write-protected 16/32-bit word on the M3 master system, and then read back the word into the CTOMIPCDATAR register. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**.

10.3.3.9 IPCLiteMtoCBootBranch

Requests C28 boot ROM to branch to a specific address.

Prototype:

```
unsigned short  
IPCLiteMtoCBootBranch(unsigned long ulAddress)
```

Parameters:

ulAddress specifies the C28 address the boot ROM should branch to.

Description:

This function will allow the M3 master system to request the C28 to exit the boot ROM and subsequently branch to *ulAddress*. **IPC_FLAG1** and **IPC_FLAG32** are used for communication with the C28 boot ROM during the C28 boot process.

Note:

This function should only be called during the C28 boot process before the C28 has started running application code.

Returns:

status of command (0=success, 1=error)

10.3.3.10 IPCLiteMtoCClearBits

Sets the designated bits in a 16/32-bit data word at an C28 control system address

Prototype:

```
unsigned short
IPCLiteMtoCClearBits(unsigned long ulFlag,
                     unsigned long ulAddress,
                     unsigned long ulMask,
                     unsigned short usLength,
                     unsigned long ulStatusFlag)
```

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulAddress specifies the C28 address to write to.

ulMask specifies the 16/32-bit mask for bits which should be set at C28 ulAddress. (For 16-bit mask, only the lower 16-bits of ulMask are considered.

usLength specifies the length of the **ulMask** (1 = 16-bit, 2 = 32-bit).

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

Description:

This function will allow the M3 master system to set bits specified by the **usMask** variable in a 16/32-bit word on the C28 control system. The data word at /e ulAddress after the set bits command is then read into the MTOCIPCDATAR register. After calling this function, a call to [IPCLiteMtoCGetResult\(\)](#) will read the data value in the MTOCIPCDATAR register into a 16/32-bit variable in the M3 application. The **usLength** parameter accepts the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The **ulFlag** parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**. The **ulStatusFlag** parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.3.3.11 IPCLiteMtoCClearBits_Protected

Clears the designated bits in a 16/32-bit write-protected data word at an C28 control system address

Prototype:

```
unsigned short
IPCLiteMtoCClearBits_Protected(unsigned long ulFlag,
                               unsigned long ulAddress,
                               unsigned long ulMask,
                               unsigned short usLength,
                               unsigned long ulStatusFlag)
```

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulAddress specifies the C28 write-protected address to write to.

ulMask specifies the 16/32-bit mask for bits which should be cleared at C28 ulAddress. (For 16-bit mask, only the lower 16-bits of ulMask are considered.

usLength specifies the length of the **ulMask** (1 = 16-bit, 2 = 32-bit).

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

Description:

This function will allow the M3 master system to clear bits specified by the **usMask** variable in a write-protected 16/32-bit word on the C28 control system. The data word at /e ulAddress after the clear bits command is then read into the MTOCIPCDATAR register. After calling this function, a call to [IPCLiteMtoCGetResult\(\)](#) will read the data value in the MTOCIPCDATAR register into a 16/32-bit variable in the M3 application. The **usLength** parameter accepts the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The **ulFlag** parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**. The **ulStatusFlag** parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.3.3.12 IPCLiteMtoCDataRead

Reads either a 16- or 32-bit data word from an C28 control system address

Prototype:

```
unsigned short
IPCLiteMtoCDataRead(unsigned long ulFlag,
                    unsigned long ulAddress,
                    unsigned short usLength,
                    unsigned long ulStatusFlag)
```

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulAddress specifies the C28 address to read from

usLength designates 16- or 32-bit read (1 = 16-bit, 2 = 32-bit)

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

Description:

This function will allow the M3 master system to read 16/32-bit data from the C28 control system into the MTOCIPCDATAR register. After calling this function, a call to [IPCLiteMtoCGetResult\(\)](#) will read the data value in the MTOCIPCDATAR register into a 16- or 32-bit variable in the M3 application. The **usLength** parameter accepts the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The **ulFlag** parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**. The **ulStatusFlag** parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.3.3.13 IPCLiteMtoCDataWrite

Writes a 16/32-bit data word to an C28 control system address

Prototype:

```
unsigned short
IPCLiteMtoCDataWrite(unsigned long ulFlag,
                      unsigned long ulAddress,
                      unsigned long ulData,
                      unsigned short usLength,
                      unsigned long ulStatusFlag)
```

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulAddress specifies the C28 address to write to

ulData specifies the 16/32-bit word which will be written. For 16-bit words, only the lower 16-bits of ulData will be considered by the control system.

usLength is the length of the word to write (0 = 16-bits, 1 = 32-bits)

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

Description:

This function will allow the M3 master system to write a 16/32-bit word via the *ulData* variable to an address on the C28 control system. The *usLength* parameter accepts the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**. The *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.3.3.14 IPCLiteMtoCDataWrite_Protected

Writes a 16/32-bit data word to a protected C28 control system address

Prototype:

```
unsigned short
IPCLiteMtoCDataWrite_Protected(unsigned long ulFlag,
                                unsigned long ulAddress,
                                unsigned long ulData,
                                unsigned short usLength,
                                unsigned long ulStatusFlag)
```

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulAddress specifies the C28 address to write to

ulData specifies the 16/32-bit word which will be written. For 16-bit words, only the lower 16-bits of ulData will be considered by the control system.

usLength is the length of the word to write (0 = 16-bits, 1 = 32-bits)

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

Description:

This function will allow the M3 master system to write a 16/32-bit word via the *ulData* variable to a write-protected address on the C28 control system. The *usLength* parameter accepts the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**. The *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.3.3.15 IPCLiteMtoCFunctionCall

Calls a C28 function with 1 optional parameter and an optional return value.

Prototype:

```
unsigned short
IPCLiteMtoCFunctionCall(unsigned long ulFlag,
                        unsigned long ulAddress,
                        unsigned long ulParam,
                        unsigned long ulStatusFlag)
```

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulAddress specifies the C28 function address

ulParam specifies the 32-bit optional parameter value

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

Description:

This function will allow the M3 master system to call a function on the C28. the *ulParam* variable is a single optional 32-bit parameter to pass to the function. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**. The *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.3.3.16 IPCLiteMtoCGetResult

Reads single word data result of MtoC IPC command

Prototype:

```
unsigned short  
IPCLiteMtoCGetResult(void *pvData,  
                     unsigned short usLength,  
                     unsigned long ulStatusFlag)
```

Parameters:

pvData is a pointer to the 16/32-bit variable where the result data will be stored.

usLength designates 16- or 32-bit read.

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system. If a status flag was not used with the command call, set this parameter to 0.

Description:

Allows the caller to read the 16/32-bit data result of non-blocking IPC functions from the MTO-CIPCDATAR register if the status flag is cleared indicating the IPC command was successfully interpreted. If the status flag is not cleared, the command was not recognized, and the function will return STATUS_FAIL. To determine what data is read from a call to this function, see the descriptions of the non-blocking IPC functions. The *usLength* parameter accepts the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *ulStatusFlag* parameter accepts any of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** or **STATUS_FAIL**.

Returns:

status of command (0=success, 1=error)

10.3.3.17 IPCLiteMtoCSetBits

Sets the designated bits in a 16/32-bit data word at an C28 control system.

Prototype:

```
unsigned short  
IPCLiteMtoCSetBits(unsigned long ulFlag,  
                   unsigned long ulAddress,  
                   unsigned long ulMask,  
                   unsigned short usLength,  
                   unsigned long ulStatusFlag)
```

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulAddress specifies the C28 address to write to.

ulMask specifies the 16/32-bit mask for bits which should be set at C28 ulAddress. (For 16-bit mask, only the lower 16-bits of ulMask are considered.

usLength specifies the length of the *ulMask* (1 = 16-bit, 2 = 32-bit).

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

Description:

This function will allow the M3 master system to set bits specified by the *usMask* variable in a 16/32-bit word on the C28 control system. The data word at /e *ulAddress* after the set bits command is then read into the MTOCIPCDATAR register. After calling this function, a call to [IPCLiteMtoCGetResult\(\)](#) will read the data value in the MTOCIPCDATAR register into a 16/32-bit variable in the M3 application. The *usLength* parameter accepts the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**. The *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.3.3.18 IPCLiteMtoCSetBits_Protected

Sets the designated bits in a 16/32-bit write-protected data word at an C28 control system address

Prototype:

```
unsigned short
IPCLiteMtoCSetBits_Protected(unsigned long ulFlag,
                             unsigned long ulAddress,
                             unsigned long ulMask,
                             unsigned short usLength,
                             unsigned long ulStatusFlag)
```

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulAddress specifies the C28 write-protected address to write to.

ulMask specifies the 16/32-bit mask for bits which should be set at C28 *ulAddress*. (For 16-bit mask, only the lower 16-bits of *ulMask* are considered.

usLength specifies the length of the *ulMask* (1 = 16-bit, 2 = 32-bit).

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

Description:

This function will allow the M3 master system to set bits specified by the *usMask* variable in a write-protected 16/32-bit word on the C28 control system. The data word at /e *ulAddress* after the set bits command is then read into the MTOCIPCDATAR register. After calling this function, a call to [IPCLiteMtoCGetResult\(\)](#) will read the data value in the MTOCIPCDATAR register into a 16/32-bit variable in the M3 application. The *usLength* parameter accepts the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**. The *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.4 M3 IPC Utility Drivers

Functions

- void [IPCCtoMFlagAcknowledge](#) (unsigned long ulFlags)
- unsigned short [IPCCtoMFlagBusy](#) (unsigned long ulFlags)
- unsigned short [IPCMtoCBootControlSystem](#) (unsigned long ulBootMode)
- unsigned short [IPCMtoCFlagBusy](#) (unsigned long ulFlags)
- void [IPCMtoCFlagClear](#) (unsigned long ulFlags)
- void [IPCMtoCFlagSet](#) (unsigned long ulFlags)
- unsigned long [IPCMtoCSharedRamConvert](#) (unsigned long ulShareAddress)

10.4.1 Detailed Description

The IPC Utility driver functions provide convenient functions to set/clear/check the status of IPC flags, convert between M3 and C28 MSG/shared RAM addresses, and a function to allow the M3 to run the C28 peripheral loaders while the C28 is in boot mode. These functions can be used in conjunction with either the IPC-Lite or main IPC drivers as long as the `ipc_util.c` (M3) or `F28M35x_ipc_Util.c` (C28) file is added to the project.

10.4.2 Function Documentation

10.4.2.1 IPCCtoMFlagAcknowledge

M3 Acknowledges CtoM IPC Flag.

Prototype:

```
void
IPCCtoMFlagAcknowledge(unsigned long ulFlags)
```

Parameters:

ulFlags specifies the IPC flag mask for flags being acknowledged.

Description:

This function will allow the M3 master system to acknowledge/clear the IPC flag set by the C28 control system. The *ulFlags* parameter can be any of the IPC flag values: **IPC_FLAG1** - **IPC_FLAG32**.

Returns:

None.

10.4.2.2 IPCCtoMFlagBusy

Determines whether the given CtoM IPC flags are busy or not.

Prototype:

```
unsigned short  
IPCtoMFlagBusy(unsigned long ulFlags)
```

Parameters:

ulFlags specifies CtoM IPC Flag number masks to check the status of.

Description:

Allows the caller to determine whether the designated IPC flags are pending. The *ulFlags* parameter can be any of the IPC flag values: **IPC_FLAG1** - **IPC_FLAG32**.

Returns:

Returns **1** if the IPC flags are busy or **0** if designated IPC flags are free.

10.4.2.3 IPCMtoCBootControlSystem

Executes a C28 control system bootloader.

Prototype:

```
unsigned short  
IPCMtoCBootControlSystem(unsigned long ulBootMode)
```

Parameters:

ulBootMode specifies which C28 control system boot mode to execute.

Description:

This function will allow the M3 master system to boot the C28 control system via the following modes: Boot to RAM, Boot to Flash, Boot via SPI, SCI, I2C, or parallel I/O. Unlike other IPCLite driver functions, this function blocks and waits until the control system boot ROM is configured and ready to receive MTOCIPC INT1 interrupts. It then blocks and waits until IPC INT1 and IPC FLAG32 are available in the C28 boot ROM prior to sending the command to execute the selected bootloader. The *ulBootMode* parameter accepts one of the following values: **CBROM_MTOC_BOOTMODE_COMMAND_ILLEGAL**, **CBROM_MTOC_BOOTMODE_BOOT_FROM_RAM**, **CBROM_MTOC_BOOTMODE_BOOT_FROM_FLASH**, **CBROM_MTOC_BOOTMODE_BOOT_FROM_SCI**, **CBROM_MTOC_BOOTMODE_BOOT_FROM_SPI**, **CBROM_MTOC_BOOTMODE_BOOT_FROM_I2C**, **CBROM_MTOC_BOOTMODE_BOOT_FROM_PARALLEL**.

Returns:

0 (success) if command is sent, or 1 (failure) if boot mode is invalid and command was not sent.

10.4.2.4 IPCMtoCFlagBusy

Determines whether the given MtoC IPC flags are busy or not.

Prototype:

```
unsigned short  
IPCMtoCFlagBusy(unsigned long ulFlags)
```

Parameters:

ulFlags specifies MtoC IPC Flag number masks to check the status of.

Description:

Allows the caller to determine whether the designated IPC flags are available for further master to control system communication. If **0** is returned, then all designated tasks have completed and are available. The *ulFlags* parameter can be any of the IPC flag values: **IPC_FLAG1** - **IPC_FLAG32**.

Returns:

Returns **1** if the IPC flags are busy or **0** if designated IPC flags are free.

10.4.2.5 IPCMtoCFlagClear

M3 Clears MtoC IPC Flag

Prototype:

```
void  
IPCMtoCFlagClear(unsigned long ulFlags)
```

Parameters:

ulFlags specifies the IPC flag mask for flags being set.

Description:

This function will allow the M3 master system to set the designated IPC flags to send to the C28 control system. The *ulFlags* parameter can be any of the IPC flag values: **IPC_FLAG1** - **IPC_FLAG32**.

Returns:

None.

10.4.2.6 IPCMtoCFlagSet

M3 Sets MtoC IPC Flag

Prototype:

```
void  
IPCMtoCFlagSet(unsigned long ulFlags)
```

Parameters:

ulFlags specifies the IPC flag mask for flags being set.

Description:

This function will allow the M3 master system to set the designated IPC flags to send to the C28 control system. The *ulFlags* parameter can be any of the IPC flag values: **IPC_FLAG1** - **IPC_FLAG32**.

Returns:

None.

10.4.2.7 IPCMtoCSharedRamConvert

Converts a M3 shared RAM address to its C28 equivalent address

Prototype:

```
unsigned long  
IPCMtoCSharedRamConvert(unsigned long ulShareAddress)
```

Parameters:

ulShareAddress specifies the M3 address in shared RAM (Sx RAM or MSG RAM).

Description:

This function will convert a shared RAM address location on the M3 master system to its equivalent C28 control system address (applies to MSG RAM's and Sx shared RAM blocks).

Returns:

C28 equivalent address.

10.5 C28 IPC API Drivers

Functions

- void **IPCCInitialize** (volatile **tlpcController** *psController, Uint16 usCipcInterrupt, Uint16 usMipcInterrupt)
- Uint16 **IPCCtoMBlockRead** (volatile **tlpcController** *psController, Uint32 ulAddress, Uint32 ulShareAddress, Uint16 usLength, Uint16 bBlock, Uint32 ulResponseFlag)
- Uint16 **IPCCtoMBlockWrite** (volatile **tlpcController** *psController, Uint32 ulAddress, Uint32 ulShareAddress, Uint16 usLength, Uint16 usWordLength, Uint16 bBlock)
- Uint16 **IPCCtoMBlockWrite_Protected** (volatile **tlpcController** *psController, Uint32 ulAddress, Uint32 ulShareAddress, Uint16 usLength, Uint16 usWordLength, Uint16 bBlock)
- Uint16 **IPCCtoMClearBits** (volatile **tlpcController** *psController, Uint32 ulAddress, Uint32 ulMask, Uint16 usLength, Uint16 bBlock)
- Uint16 **IPCCtoMClearBits_Protected** (volatile **tlpcController** *psController, Uint32 ulAddress, Uint32 ulMask, Uint16 usLength, Uint16 bBlock)
- Uint16 **IPCCtoMDataRead** (volatile **tlpcController** *psController, Uint32 ulAddress, void *pvData, Uint16 usLength, Uint16 bBlock, Uint32 ulResponseFlag)
- Uint16 **IPCCtoMDataRead_Protected** (volatile **tlpcController** *psController, Uint32 ulAddress, void *pvData, Uint16 usLength, Uint16 bBlock, Uint32 ulResponseFlag)
- Uint16 **IPCCtoMDataWrite** (volatile **tlpcController** *psController, Uint32 ulAddress, Uint32 ulData, Uint16 usLength, Uint16 bBlock, Uint32 ulResponseFlag)
- Uint16 **IPCCtoMDataWrite_Protected** (volatile **tlpcController** *psController, Uint32 ulAddress, Uint32 ulData, Uint16 usLength, Uint16 bBlock, Uint32 ulResponseFlag)
- Uint16 **IPCCtoMFunctionCall** (volatile **tlpcController** *psController, Uint32 ulAddress, Uint32 ulParam, Uint16 bBlock)
- Uint16 **IPCCtoMReqMemAccess** (volatile **tlpcController** *psController, Uint32 ulMask, Uint16 usMaster, Uint16 bBlock)
- Uint16 **IPCCtoMSendMessage** (volatile **tlpcController** *psController, Uint32 ulCommand, Uint32 ulAddress, Uint32 ulDataW1, Uint32 ulDataW2, Uint16 bBlock)

- Uint16 [IPCCtoMSetBits](#) (volatile [tlpcController](#) *psController, Uint32 ulAddress, Uint32 ulMask, Uint16 usLength, Uint16 bBlock)
- Uint16 [IPCCtoMSetBits_Protected](#) (volatile [tlpcController](#) *psController, Uint32 ulAddress, Uint32 ulMask, Uint16 usLength, Uint16 bBlock)
- Uint16 [lpcGet](#) (volatile [tlpcController](#) *psController, [tlpcMessage](#) *psMessage, Uint16 bBlock)
- void [IPCMtoCBlockRead](#) ([tlpcMessage](#) *psMessage)
- void [IPCMtoCBlockWrite](#) ([tlpcMessage](#) *psMessage)
- void [IPCMtoCBlockWrite_Protected](#) ([tlpcMessage](#) *psMessage)
- void [IPCMtoCClearBits](#) ([tlpcMessage](#) *psMessage)
- void [IPCMtoCClearBits_Protected](#) ([tlpcMessage](#) *psMessage)
- void [IPCMtoCDataRead](#) (volatile [tlpcController](#) *psController, [tlpcMessage](#) *psMessage, Uint16 bBlock)
- void [IPCMtoCDataRead_Protected](#) (volatile [tlpcController](#) *psController, [tlpcMessage](#) *psMessage, Uint16 bBlock)
- void [IPCMtoCDataWrite](#) ([tlpcMessage](#) *psMessage)
- void [IPCMtoCDataWrite_Protected](#) ([tlpcMessage](#) *psMessage)
- void [IPCMtoCFunctionCall](#) ([tlpcMessage](#) *psMessage)
- void [IPCMtoCSetBits](#) ([tlpcMessage](#) *psMessage)
- void [IPCMtoCSetBits_Protected](#) ([tlpcMessage](#) *psMessage)
- Uint16 [lpcPut](#) (volatile [tlpcController](#) *psController, [tlpcMessage](#) *psMessage, Uint16 bBlock)

10.5.1 Detailed Description

The main IPC drivers utilize circular buffers (1 Put buffer and 1 Get buffer for each IPC interrupt used) stored in the CtoM and MtoC message RAM's to send messages between processors. [tlpcController](#) data structures store the data for a single Put and Get buffer pair.

Note:

Although the main IPC drivers have the benefit of allowing queued commands for sequential processing, they require additional memory (message RAM's), code changes (command linker file changes and data structure variable definitions), and setup to begin using. For a simpler IPC driver solution which can be used instantly without any additional code changes, see the **IPC-Lite** API drivers.

The figure below shows how the IPC drivers are implemented in the dual core system. Note that buffers and indexes in purple store the MtoC M3 Put buffer/ C28 Get buffer and read/write indexes, while buffers and indexes in yellow store the CtoM C28 Put buffer/ M3 Get buffer and read/write indexes. Additionally, each MtoC circular buffer is tied to an MtoC IPC interrupt, and each CtoM circular buffer is tied to a CtoM IPC interrupt.

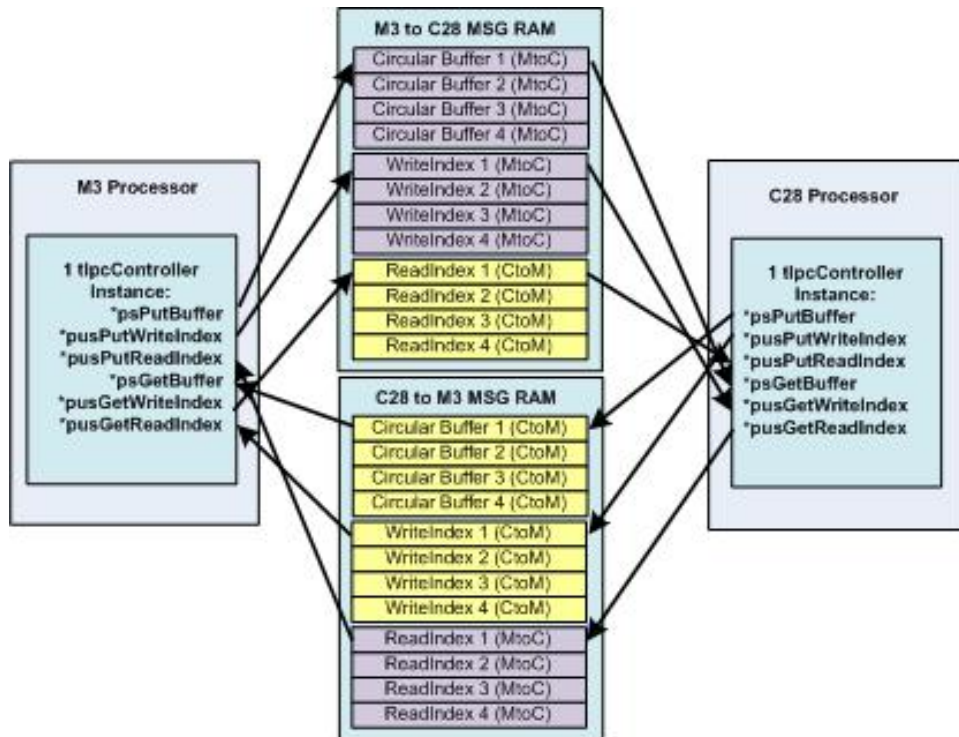


Figure 10.3: Main IPC Drivers Infrastructure Implementation

When one processor (X) wishes to perform an operation on the other processor (Y):

1. Processor X writes messages into its Put Buffer (equivalent to Processor Y's Get Buffer) , increments the PutWriteIndex, and sets Processor Y's XtoY IPC interrupt flag.
2. Processor Y sees IPC interrupt flag and subsequently reads the messages from the Get Buffer and increments the GetReadIndex.
3. Processor Y then proceeds to process the commands according to the message contents and then acknowledges XtoY IPC interrupt flag.
4. Processor X can also read and process messages that Processor Y writes to its Put Buffer (which is Processor X's Get Buffer).

Messages are written to and read from each circular buffer in a FIFO fashion as shown below. N is the maximum number of messages that can be stored in a circular buffer and must be an even number. By default, N= 4 in the main IPC drivers. There can be N-1 Puts into the circular buffer until the receiving processor's application code must perform a Get.

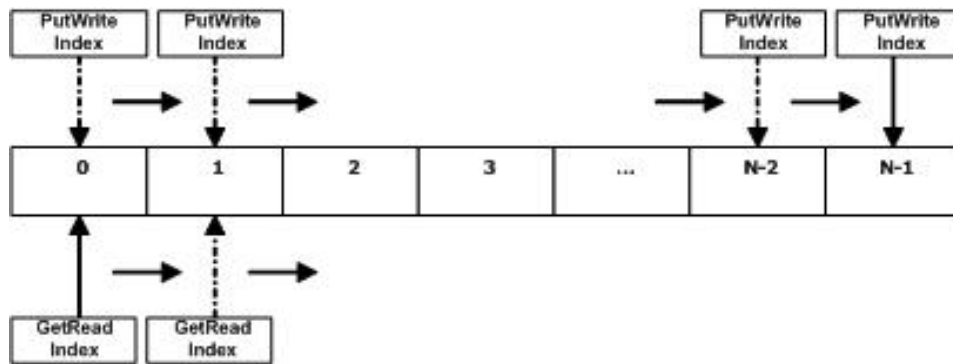


Figure 10.4: Writing to and Reading from Circular Buffer via Indexes

10.5.2 Usage and Examples

The IPC API driver functions will Put commands into the PutBuffer and Get commands from the GetBuffer in a manner that is transparent to the application software. The application code is responsible for initializing the IPC controllers, calling the API driver functions for those commands it wishes to execute, decoding received commands, and acknowledging received IPC interrupt flags.

There are a number of steps which must be taken in the application project in order to use the main IPC API driver functions.

1. Add the `lpc.c` and `lpc_util.c` files which include the `lpc.h` file to the application project. For M3 projects, make sure `MWare/driverlib/lpc.c` and `/driverlib/lpc_util.c` are added as source files in your project, both of which include `/driverlib/lpc.h`. For C28 projects, make sure `/F28M35x_common/source/F28M35x_lpc.c` and `F28M35x_common/source/F28M35x_lpc_util.c` are added as source files in your project, both of which include `/F28M35x_common/include/F28M35x_lpc.h`. The `lpc_util.c` file includes utility functions that can be used by application code and that are used by both the main IPC driver functions and the IPC-lite driver functions.
2. Add IPC buffer and index section definitions to application project's `.cmd` command linker file. Memory regions for the CtoM and MtoC message RAM's should be defined. Additionally, there should be 1 section reserved for the PUTBUFFER, PUTWRITEIDX, and GETREADIDX, and 1 section reserved for the GETBUFFER, GETWRIDEIDX, PUTREADIDX which are mapped to the beginning of each message RAM block by grouping in the `.cmd` file.

The code below is a sample of what a C28 project's `.CMD` linker file should look like when using the main IPC API drivers:

```
MEMORY
{
    PAGE 0 :
        CTOMRAM      : origin = 0x03F800,  length = 0x000400
        MTOCRAM      : origin = 0x03FC00,  length = 0x000400
}

SECTIONS
{
```

```

GROUP : > CTOMRAM, PAGE = 1
{
    PUTBUFFER
    PUTWRITEIDX
    GETREADIDX
}

GROUP : > MTOCRAM, PAGE = 1
{
    GETBUFFER : TYPE = DSECT
    GETWRITEIDX : TYPE = DSECT
    PUTREADIDX : TYPE = DSECT
}
}

```

The code below is a sample of what an M3 project's .CMD linker file should look like when using the main IPC API drivers:

```

MEMORY
{
    CTOMRAM (RX) : origin = 0x2007F000, length = 0x00000800
    MTOCRAM (RWX) : origin = 0x2007F800, length = 0x00000800
}

SECTIONS
{
    GROUP : > MTOCRAM
    {
        PUTBUFFER
        PUTWRITEIDX
        GETREADIDX
    }

    GROUP : > CTOMRAM
    {
        GETBUFFER : TYPE = DSECT
        GETWRITEIDX : TYPE = DSECT
        PUTREADIDX : TYPE = DSECT
    }
}

```

1. Application source code must define and initialize at least 1 volatile global [tlpcController](#) instance on both the M3 and the C28.

The IPC controller data structure ([tlpcController](#)) locally stores the pointers to the Put/Get buffers and their read/write indexes. Application code must define at least 1 instance of the [tlpcController](#) data structure on the M3 and a corresponding data structure on the C28 for a single M3 IPC interrupt and C28 IPC interrupt communication pair (i.e. M3 will send messages to C28 and inform C28 via a C28 IPC interrupt flag, and C28 will respond with a message sent to an M3 IPC interrupt flag). A code example of the definition and initialization of the [tlpcController](#) instance for M3 is shown below (C28 calls [IPCCInitialize\(\)](#) instead of [IPCMIInitialize\(\)](#)):

```
// *****
```

```

// At least 1 volatile global tIpcController instance is required
// when using IPC API Drivers.
//*****
volatile tIpcController g_sIpcController1;
volatile tIpcController g_sIpcController2;

void
main(void)
{
// Initialize IPC Controllers
IPCMIInitialize (&g_sIpcController1, IPC_INT1, IPC_INT1);
IPCMIInitialize (&g_sIpcController2, IPC_INT2, IPC_INT2);
}

```

After these initialization steps, the M3 and C28 application code can set up an IPC interrupt service routine (ISR) corresponding to each [tIpcController](#) instance that was defined, and then proceed to call the IPC command functions using the respective controllers. Generally when an IPC command function is called for the sending processor, the responding function for the receiving processor has the same name. For instance, the M3 application code might call the [IPCMTtoCDataWrite\(\)](#) function to write data to memory accessible only by the C28. The C28 will decode the command received in the Get Buffer message, and upon seeing an IPC_DATA_WRITE command, it will call its own [IPCMTtoCDataWrite\(\)](#) function to process the command.

For IPC commands that must occur sequentially, the application code must use a single [tIpcController](#) instance when calling the sequential IPC functions. This ensures that a single IPC interrupt processes the messages in the order they were received by the Get Buffer. If there is another set of IPC commands that occurs independently from the first set of IPC commands, these can be called using a different [tIpcController](#) instance.

Note:

CTOM IPC INT1 and IPC Flag 32 are used by the C28 boot ROM to report system status errors to the M3 master system during C28 boot time. If the M3 application code uses CTOM IPC INT1, the application must either handle the CTOM Boot ROM status commands in the ISR (Recommended to ensure C28 has booted properly), or ignore the commands by setting the CTOMIPACK bits for IPC_FLAG1 and IPC_FLAG32.

For examples demonstrating many of the basic IPC functions, see the IPC examples in the control-SUITE device_support package for your device.

10.5.3 Customization

There are a number of settings which can be customized for different application needs when using the main IPC API drivers in the headerfile (ipc.h/ (Device)_ipc.h). Any change to the C28 header file must also be made to the M3 header file and vice versa:

1. **IPC_BUFFER_SIZE:** This is the definition for the maximum number of [tIpcMessage](#) messages in the circular buffer. By default this value is 4, but it can be any even number as long as there is enough memory in the message RAM block for the buffer.
2. **NUM_IPC_INTERRUPTS:** This is the number of IPC interrupts for which an IPC driver API circular buffer is reserved (the number must be the same on both the C28 and M3, because a single IPC controller needs information for 1 M3 IPC interrupt and 1 C28 IPC interrupt). This

number is set to 4 by default. The value can be decreased to save message RAM memory if, for instance, one of the other interrupt channels is used for IPC-Lite function processing instead.

3. **tlpcMessage**: This is the data structure of the messages transmitted by the IPC API driver functions. Although the data structure is defined to expect a command, address, and two 32-bit data words for additional information as defined by the IPC API driver functions, the structure can also be used merely to transmit 4 generic 32-bit data words between processors, if desired. Application code will then have to process the Get accordingly in the IPC ISR.

10.5.4 Function Documentation

10.5.4.1 IPCCInitialize

Initializes C28 control system IPC driver controller

Prototype:

```
void  
IPCCInitialize(volatile tIpcController *psController,  
               Uint16 usCIpcInterrupt,  
               Uint16 usMIpcInterrupt)
```

Parameters:

psController specifies the address of a *tIpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

usCIpcInterrupt specifies the C28 IPC interrupt number used by psController.

usMIpcInterrupt specifies the M3 IPC interrupt number used by psController.

Description:

This function initializes the IPC driver controller with circular buffer and index addresses for an IPC interrupt pair (C28-M3). The *usCIpcInterrupt* and *usMIpcInterrupt* parameters can be 1 of the following values: **IPC_INT1**, **IPC_INT2**, **IPC_INT3**, **IPC_INT4**.

Note:

If an interrupt is currently in use by an *tIpcController* instance, that particular interrupt should not be tied to a second *tIpcController* instance.

For a particular usMIpcInterrupt - usCIpcInterrupt pair, there must be an instance of *tIpcController* defined and initialized on both the M3 and the C28 systems.

Returns:

None.

10.5.4.2 IPCCtoMBlockRead

Sends the command to read a block of data from an M3 master system address

Prototype:

```
Uint16  
IPCCtoMBlockRead(volatile tIpcController *psController,  
                 Uint32 ulAddress,
```

```

    Uint32 ulShareAddress,
    Uint16 usLength,
    Uint16 bBlock,
    Uint32 ulResponseFlag)

```

Parameters:

psController specifies the address of a *tlpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the M3 memory block starting address to read from.

ulShareAddress specifies the C28 shared memory address the read block will read to.

usLength designates the block size in 16-bit words.

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

ulResponseFlag indicates the C to M Flag number mask used to report when the read block data is available starting at /e ulShareAddress. (*ulResponseFlag* MUST use IPC flags 17-32, and not 1-16)

Description:

This function will allow the C28 control system to send a read block command to the M3 master system and set a ResponseFlag to track the status of the read. The M3 master system will process the read and place the data in shared memory at the location specified in the *ulShareAddress* parameter and then clear the ResponseFlag, indicating that the block is ready. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**. The *ulResponseFlag* parameter can be any single one of the flags **IPC_FLAG17 - IPC_FLAG32** or **NO_FLAG**.

Returns:

status of command (**STATUS_PASS** = success, **STATUS_FAIL** = error because PutBuffer was full, command was not sent)

10.5.4.3 IPCCtoMBlockWrite

Writes a block of data to an M3 master system address

Prototype:

```

    Uint16
    IPCCtoMBlockWrite(volatile tlpcController *psController,
                      Uint32 ulAddress,
                      Uint32 ulShareAddress,
                      Uint16 usLength,
                      Uint16 usWordLength,
                      Uint16 bBlock)

```

Parameters:

psController specifies the address of a *tlpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the M3 memory block starting address to write to.

ulShareAddress specifies the C28 shared memory address where data to write from resides.

usLength designates the block size in 16- or 32-bit words (depends on *usWordLength*).

usWordLength designates the word size (16-bits = 1 or 32-bits = 2).

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the C28 control system to write a block of data to the M3 master system starting from the location specified by the *ulAddress* parameter. Prior to calling this function, the C28 control system code should place the data to write in shared memory starting at /e *ulShareAddress*. The *usWordLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**. The *ulResponseFlag* parameter can be any single one of the flags **IPC_FLAG17** - **IPC_FLAG32** or **NO_FLAG**.

Note:

If the Sx SARAM blocks are used to pass the RAM block between the processors, the [IPCCtoMReqMemAccess\(\)](#) function must be called prior to the C28 writing the block to /e *ulShareAddress* in order to give the C28 write- access to the Sx memory block(s) used for passing data.

Returns:

status of command (**STATUS_PASS** =success, **STATUS_FAIL** = error because PutBuffer was full, command was not sent)

10.5.4.4 IPCCtoMBlockWrite_Protected

Returns:

status of command (**STATUS_PASS** =success, **STATUS_FAIL** = error because PutBuffer was full, command was not sent)

10.5.4.5 Uint16 IPCCtoMClearBits (volatile [tIpcController](#) * *psController*, Uint32 *ulAddress*, Uint32 *ulMask*, Uint16 *usLength*, Uint16 *bBlock*)

Prototype:

```
Uint16  
IPCCtoMBlockWrite_Protected(volatile tIpcController *psController,  
                             Uint32 ulAddress,  
                             Uint32 ulShareAddress,  
                             Uint16 usLength,  
                             Uint16 usWordLength,  
                             Uint16 bBlock)
```

Description:

Clears the designated bits in a 16-bit data word at an M3 master system address

Parameters:

psController specifies the address of a [tIpcController](#) instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the M3 address to write to

ulMask specifies the 16/32-bit mask for bits which should be cleared at *ulAddress*. 16-bit masks should fill the lower 16-bits (upper 16-bits will be all 0x0000).

usLength specifies the length of the bit mask (1=16-bits, 2=32-bits)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

This function will allow the C28 control system to clear bits specified by the *ulMask* variable in a 16/32-bit word on the M3 master system. The *usLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

status of command (**STATUS_PASS** =success, **STATUS_FAIL** = error because PutBuffer was full, command was not sent)

10.5.4.6 IPCCtoMClearBits_Protected

Clears the designated bits in a 16-bit write-protected data word at an M3 master system address

Prototype:

```
Uint16  
IPCCtoMClearBits_Protected(volatile tlpcController *psController,  
                           Uint32 ulAddress,  
                           Uint32 ulMask,  
                           Uint16 usLength,  
                           Uint16 bBlock)
```

Parameters:

psController specifies the address of a *tlpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the M3 address to write to

ulMask specifies the 16/32-bit mask for bits which should be cleared at *ulAddress*. 16-bit masks should fill the lower 16-bits (upper 16-bits will be all 0x0000).

usLength specifies the length of the bit mask (1=16-bits, 2=32-bits)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the C28 control system to set bits specified by the *ulMask* variable in a write-protected 16/32-bit word on the M3 master system. The *usLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

status of command (**STATUS_PASS** =success, **STATUS_FAIL** = error because PutBuffer was full, command was not sent)

10.5.4.7 IPCCtoMDataRead

Sends the command to read either a 16- or 32-bit data word from an M3 master system address

Prototype:

```
UInt16  
IPCCtoMDataRead(volatile tIpcController *psController,  
                UInt32 ulAddress,  
                void *pvData,  
                UInt16 usLength,  
                UInt16 bBlock,  
                UInt32 ulResponseFlag)
```

Parameters:

psController specifies the address of a *tIpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the M3 address to read from

pvData is a pointer to the 16/32-bit variable where read data will be stored.

usLength designates 16- or 32-bit read (1 = 16-bit, 2 = 32-bit)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

ulResponseFlag indicates the C to M Flag number mask used to report when the read data is available at pvData. (*ulResponseFlag* MUST use IPC flags 17-32, and not 1-16)

Description:

This function will allow the C28 control system to send a 16/32-bit data read command to the M3 master system and set a ResponseFlag to track the status of the read. The M3 master system will respond with a DataWrite command which will place the data in the C28 address pointed to by *pvData*. When the C28 receives the DataWrite command and writes the read data into **pvData* location, it will clear the ResponseFlag, indicating to the rest of the system that the data is ready. The *usLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**. The *ulResponseFlag* parameter can be any single one of the flags **IPC_FLAG17** - **IPC_FLAG32** or **NO_FLAG**.

Returns:

status of command (**STATUS_PASS** =success, **STATUS_FAIL** = error because PutBuffer was full, command was not sent)

10.5.4.8 IPCCtoMDataRead_Protected

Sends the command to read either a 16- or 32-bit data word from an M3 master system address to a write-protected C28 address.

Prototype:

```
UInt16  
IPCCtoMDataRead_Protected(volatile tIpcController *psController,  
                           UInt32 ulAddress,  
                           void *pvData,  
                           UInt16 usLength,  
                           UInt16 bBlock,  
                           UInt32 ulResponseFlag)
```

Parameters:

psController specifies the address of a *tIpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the M3 address to read from

pvData is a pointer to the 16/32-bit variable where read data will be stored.

usLength designates 16- or 32-bit read (1 = 16-bit, 2 = 32-bit)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

ulResponseFlag indicates the C to M Flag number mask used to report when the read data is available at pvData. (*ulResponseFlag* MUST use IPC flags 17-32, and not 1-16)

Description:

This function will allow the C28 control system to send a 16/32-bit data read command to the M3 master system and set a ResponseFlag to track the status of the read. The M3 master system will respond with a DataWrite command which will place the data in the C28 address pointed to by *pvData*. When the C28 receives the DataWrite command and writes the read data into **pvData* location, it will clear the ResponseFlag, indicating to the rest of the system that the data is ready. The *usLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**. The *ulResponseFlag* parameter can be any single one of the flags **IPC_FLAG17** - **IPC_FLAG32** or **NO_FLAG**.

Returns:

status of command (**STATUS_PASS** = success, **STATUS_FAIL** = error because PutBuffer was full, command was not sent)

10.5.4.9 IPCCtoMDataWrite

Writes a 16/32-bit data word to an M3 master system address

Prototype:

```

Uint16
IPCCtoMDataWrite(volatile tIpcController *psController,
                  Uint32 ulAddress,
                  Uint32 ulData,
                  Uint16 usLength,
                  Uint16 bBlock,
                  Uint32 ulResponseFlag)

```

Parameters:

psController specifies the address of a *tIpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the M3 address to write to

ulData specifies the 16/32-bit word which will be written. For 16-bit words, only the lower 16-bits of ulData will be considered by the master system.

usLength is the length of the word to write (1 = 16-bits, 2 = 32-bits)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

ulResponseFlag is used to pass the *ulResponseFlag* back to the M3 only when this function is called in response to *IPCMtoCDataRead()*. Otherwise, set to 0.

Description:

This function will allow the C28 control system to write a 16/32-bit word via the *ulData* variable to an address on the M3 master system. The *usLength* parameter can be one of the

following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**. The *ulResponseFlag* parameter can be any single one of the flags **IPC_FLAG17** - **IPC_FLAG32** or **NO_FLAG**.

Returns:

status of command (**STATUS_PASS** =success, **STATUS_FAIL** = error because PutBuffer was full, command was not sent)

10.5.4.10 IPCCtoMDataWrite_Protected

Writes a 16/32-bit data word to a write-protected M3 master system address

Prototype:

```
UInt16  
IPCCtoMDataWrite_Protected(volatile tIpcController *psController,  
                           UInt32 ulAddress,  
                           UInt32 ulData,  
                           UInt16 usLength,  
                           UInt16 bBlock,  
                           UInt32 ulResponseFlag)
```

Parameters:

psController specifies the address of a *tIpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the write-protected M3 address to write to

ulData specifies the 16/32-bit word which will be written. For 16-bit words, only the lower 16-bits of ulData will be considered by the master system.

usLength is the length of the word to write (1 = 16-bits, 2 = 32-bits)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

ulResponseFlag is used to pass the *ulResponseFlag* back to the M3 only when this function is called in response to *IPCMtoCDataRead()*. Otherwise, set to 0.

Description:

This function will allow the C28 control system to write a 16/32-bit word via the *ulData* variable to a write-protected address on the M3 master system. The *usLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**. The *ulResponseFlag* parameter can be any single one of the flags **IPC_FLAG17** - **IPC_FLAG32** or **NO_FLAG**.

Returns:

status of command (**STATUS_PASS** =success, **STATUS_FAIL** = error because PutBuffer was full, command was not sent)

10.5.4.11 IPCCtoMFunctionCall

Calls an M3 function with 1 optional parameter .

Prototype:

```

Uint16
IPCCtoMFunctionCall (volatile tIpcController *psController,
                    Uint32 ulAddress,
                    Uint32 ulParam,
                    Uint16 bBlock)

```

Parameters:

psController specifies the address of a [tIpcController](#) instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the M3 function address

ulParam specifies the 32-bit optional parameter value. If not used, this can be a dummy value.

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the C28 control system to call a function on the M3. The *ulParam* variable is a single optional 32-bit parameter to pass to the function. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

status of command (**STATUS_PASS** =success, **STATUS_FAIL** = error because PutBuffer was full, command was not sent)

10.5.4.12 IPCCtoMReqMemAccess

C28 Configures master R/W/Exe Access to Shared SARAM.

Prototype:

```

Uint16
IPCCtoMReqMemAccess (volatile tIpcController *psController,
                    Uint32 ulMask,
                    Uint16 usMaster,
                    Uint16 bBlock)

```

Parameters:

psController specifies the address of a [tIpcController](#) instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulMask specifies the 32-bit mask for the SxMSEL RAM control register to indicate which Sx SARAM blocks the C28 is requesting master access to.

usMaster specifies whether the C28 or M3 are given master access to the Sx blocks.

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the C28 control system to configure master R/W/Exe access to the Sx SARAM blocks specified by the *ulMask* parameter. The function calls the [IPCCtoMSetBits_Protected\(\)](#) or [IPCCtoMClearBits_Protected\(\)](#) functions, and therefore in the M3 application code, the corresponding functions should be called. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**. The *usMaster* parameter can be either: **IPC_SX_C28MASTER** or **IPC_SX_M3MASTER**. The *ulMask* parameter can be any of the options: **S0_ACCESS** - **S7_ACCESS**.

Returns:

status of command (**STATUS_PASS** =success, **STATUS_FAIL** = error because PutBuffer was full, command was not sent)

10.5.4.13 IPCctoMSendMessage

Sends generic message to M3 master system

Prototype:

```
Uint16  
IPCctoMSendMessage(volatile tlpcController *psController,  
                   Uint32 ulCommand,  
                   Uint32 ulAddress,  
                   Uint32 ulDataW1,  
                   Uint32 ulDataW2,  
                   Uint16 bBlock)
```

Parameters:

psController specifies the address of a *tlpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulCommand specifies 32-bit command word to insert into message.

ulAddress specifies 32-bit address word to insert into message.

ulDataW1 specifies 1st 32-bit data word to insert into message.

ulDataW2 specifies 2nd 32-bit data word to insert into message.

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with **STATUS_FAIL** if no slot).

Description:

This function will allow the C28 control system to send a generic message to the M3 master system. Note that the user should create a corresponding function on the M3 side to interpret/use the message or fill parameters in such a way that the existing IPC drivers can recognize the command and properly process the message. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

status of command (**STATUS_PASS** =success, **STATUS_FAIL** = error because PutBuffer was full, command was not sent)

10.5.4.14 IPCctoMSetBits

Sets the designated bits in a 16-bit data word at an M3 master system address

Prototype:

```
Uint16  
IPCctoMSetBits(volatile tlpcController *psController,  
               Uint32 ulAddress,  
               Uint32 ulMask,  
               Uint16 usLength,  
               Uint16 bBlock)
```

Parameters:

psController specifies the address of a *tlpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the M3 address to write to

ulMask specifies the 16/32-bit mask for bits which should be set at *ulAddress*. 16-bit masks should fill the lower 16-bits (upper 16-bits will be all 0x0000).

usLength specifies the length of the bit mask (1=16-bits, 2=32-bits)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the C28 control system to set bits specified by the *ulMask* variable in a 16/32-bit word on the M3 master system. The *usLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

status of command (**STATUS_PASS** =success, **STATUS_FAIL** = error because PutBuffer was full, command was not sent)

10.5.4.15 IPCctoMSetBits_Protected

Sets the designated bits in a 16-bit write-protected data word at an M3 master system address

Prototype:

```

Uint16
IPCctoMSetBits_Protected(volatile tlpcController *psController,
                          Uint32 ulAddress,
                          Uint32 ulMask,
                          Uint16 usLength,
                          Uint16 bBlock)

```

Parameters:

psController specifies the address of a *tlpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

ulAddress specifies the M3 address to write to

ulMask specifies the 16/32-bit mask for bits which should be set at *ulAddress*. 16-bit masks should fill the lower 16-bits (upper 16-bits will be all 0x0000).

usLength specifies the length of the bit mask (1=16-bits, 2=32-bits)

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the C28 control system to set bits specified by the *ulMask* variable in a write-protected 16/32-bit word on the M3 master system. The *usLength* parameter can be one of the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

status of command (**STATUS_PASS** =success, **STATUS_FAIL** = error because PutBuffer was full, command was not sent)

10.5.4.16 IpcGet

Reads a message from the GetBuffer.

Prototype:

```
UInt16  
IpcGet (volatile tIpcController *psController,  
        tIpcMessage *psMessage,  
        UInt16 bBlock)
```

Parameters:

psController specifies the address of a *tIpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

psMessage specifies the address of the *tIpcMessage* instance where the message from GetBuffer should be written to.

bBlock specifies whether to allow function to block until GetBuffer has a message (1= wait until message available, 0 = exit with STATUS_FAIL if no message).

Description:

This function checks if there is a message in the GetBuffer. If so, it gets the message in the GetBuffer pointed to by the ReadIndex and writes it to the address pointed to by *psMessage*. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

STATUS_PASS if GetBuffer is empty. **STATUS_FAIL** if Get occurs successfully.

10.5.4.17 IPCMtoCBlockRead

Reads a block of data from a M3 master system address and stores into shared RAM

Prototype:

```
void  
IPCMtoCBlockRead (tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the M3 master system.

Description:

This function will respond to the M3 master system request to read a block of data from the C28 control system, by reading the data and placing that data into the shared RAM location specified in *psMessage*.

Returns:

None.

10.5.4.18 IPCMtoCBlockWrite

Writes a block of data to a C28 control system address from shared RAM

Prototype:

```
void  
IPCMtoCBlockWrite(tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the M3 master system.

Description:

This function will write a block of data to an address on the C28 control system. The data is first written by the M3 to shared RAM. This function reads the shared RAM location, word size (16- or 32-bit), and block size from *psMessage* and writes the block to the C28 address specified in *psMessage*.

Returns:

None.

10.5.4.19 IPCMtoCBlockWrite_Protected

Writes a block of data to a C28 control system write-protected address from shared RAM

Prototype:

```
void  
IPCMtoCBlockWrite_Protected(tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the M3 master system.

Description:

This function will write a block of data to a write-protected group of addresses on the C28 control system. The data is first written by the M3 to shared RAM. This function reads the shared RAM location, word size (16- or 32-bit), and block size from *psMessage* and writes the block to the C28 address specified in *psMessage*.

Returns:

None.

10.5.4.20 IPCMtoCClearBits

Clears the designated bits in a 32-bit data word at a C28 control system address

Prototype:

```
void  
IPCMtoCClearBits(tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the M3 master system.

Description:

This function will allow the M3 master system to clear the bits in a 16/32-bit word on the C28 control system via a C28 address and mask passed in via the *psMessage*.

Returns:

None.

10.5.4.21 IPCMtoCClearBits_Protected

Clears the designated bits in a write-protected 16/32-bit data word at a C28 control system address

Prototype:

```
void  
IPCMtoCClearBits_Protected(tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the M3 master system.

Description:

This function will allow the M3 master system to clear the bits in a 16/32-bit write-protected word on the C28 control system via a C28 address and mask passed in via the *psMessage*.

Returns:

None.

10.5.4.22 IPCMtoCDataRead

Responds to 16/32-bit data word read command from M3 master system.

Prototype:

```
void  
IPCMtoCDataRead(volatile tIpcController *psController,  
                 tIpcMessage *psMessage,  
                 Uint16 bBlock)
```

Parameters:

psController specifies the address of a *tlpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

psMessage specifies the pointer to the message received from the M3 master system.

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the M3 master system to read a 16/32-bit data word at the C28 address specified in /e *psMessage*, and send a Write command with the read data back to the C28 control system. It will also send the Response Flag used to track the read back to the M3. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

None.

10.5.4.23 IPCMtoCDataRead_Protected

Responds to 16/32-bit data word read command from M3 master system. to read into a write-protected word on the M3 system.

Prototype:

```
void  
IPCMtoCDataRead_Protected(volatile tIpcController *psController,  
                           tIpcMessage *psMessage,  
                           Uint16 bBlock)
```

Parameters:

psController specifies the address of a *tIpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

psMessage specifies the pointer to the message received from the M3 master system.

bBlock specifies whether to allow function to block until PutBuffer has a slot (1= wait until slot free, 0 = exit with STATUS_FAIL if no slot).

Description:

This function will allow the M3 master system to read a 16/32-bit data word at the C28 address specified in /e psMessage, and send a Write Protected command with the read data back to the M3 master system at a write protected address. It will also send the Response Flag used to track the read back to the M3. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

None.

10.5.4.24 IPCMtoCDataWrite

Responds to 16/32-bit data word write command from M3 master system

Prototype:

```
void  
IPCMtoCDataWrite(tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the M3 master system which includes the 16/32-bit data word to write to the C28 address.

Description:

This function will allow the C28 control system to write a 16/32-bit word received from the M3 master system to the address indicated in *psMessage*. In the event that the IPC_DATA_WRITE command was received as a result of an IPC_DATA_READ command, this function will also clear the IPC response flag tracking the read so other threads in the C28 system will be aware that the data is ready.

Returns:

None.

10.5.4.25 IPCMtoCDataWrite_Protected

Responds to 16/32-bit write-protected data word write command from M3 master system

Prototype:

```
void  
IPCMtoCDataWrite_Protected(tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the M3 master system which includes the 16/32-bit data word to write to the C28 address.

Description:

This function will allow the C28 control system to write a 16/32-bit word received from the M3 master system to the write-protected address indicated in **psMessage*. In the event that the IPC_DATA_WRITE_PROTECTED command was received as a result of an IPC_DATA_READ_PROTECTED command, this function will also clear the IPC response flag tracking the read so other threads in the C28 system will be aware that the data is ready.

Returns:

None.

10.5.4.26 IPCMtoCFunctionCall

Calls a C28 function with a single optional parameter.

Prototype:

```
void  
IPCMtoCFunctionCall(tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the C28 control system.

Description:

This function will allow the M3 master system to call a C28 function with a a single optional parameter. There is no return value from the executed function.

Returns:

None.

10.5.4.27 IPCMtoCSetBits

Sets the designated bits in a 16/32-bit data word at a C28 control system address

Prototype:

```
void  
IPCMtoCSetBits(tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the M3 master system.

Description:

This function will allow the M3 master system to set the bits in a 16/32-bit word on the C28 control system via a C28 address and mask passed in via the *psMessage*.

Returns:

None.

10.5.4.28 IPCMtoCSetBits_Protected

Sets the designated bits in a 16/32-bit write-protected data word at a C28 control system address

Prototype:

```
void
IPCMtoCSetBits_Protected(tIpcMessage *psMessage)
```

Parameters:

psMessage specifies the pointer to the message received from the M3 master system.

Description:

This function will allow the M3 master system to set the bits in a write-protected 16/32-bit word on the C28 control system via a C28 address and mask passed in via the *psMessage*.

Returns:

None

10.5.4.29 IpcPut

Writes a message into the PutBuffer.

Prototype:

```
Uint16
IpcPut(volatile tIpcController *psController,
       tIpcMessage *psMessage,
       Uint16 bBlock)
```

Parameters:

psController specifies the address of a *tIpcController* instance used to store information about the "Put" and "Get" circular buffers and their respective indexes.

psMessage specifies the address of the *tIpcMessage* instance to be written to PutBuffer.

bBlock specifies whether to allow function to block until PutBuffer has a free slot (1= wait until free spot available, 0 = exit with STATUS_FAIL if no free slot).

Description:

This function checks if there is a free slot in the PutBuffer. If so, it puts the message pointed to by *psMessage* into the free slot and increments the WriteIndex. Then it sets the appropriate IPC interrupt flag specified by *psController->usPutFlag*. The *bBlock* parameter can be one of the following values: **ENABLE_BLOCKING** or **DISABLE_BLOCKING**.

Returns:

STATUS_FAIL if PutBuffer is full. **STATUS_PASS** if Put occurs successfully.

10.6 C28 IPC-Lite API Drivers

Functions

- Uint16 *IPCLiteCtoMClearBits* (Uint32 ulFlag, Uint32 ulAddress, Uint32 ulMask, Uint16 usLength, Uint32 ulStatusFlag)

- Uint16 [IPCLiteCtoMClearBits_Protected](#) (Uint32 ulFlag, Uint32 ulAddress, Uint32 ulMask, Uint16 usLength, Uint32 ulStatusFlag)
- Uint16 [IPCLiteCtoMDataRead](#) (Uint32 ulFlag, Uint32 ulAddress, Uint16 usLength, Uint32 ulStatusFlag)
- Uint16 [IPCLiteCtoMDataWrite](#) (Uint32 ulFlag, Uint32 ulAddress, Uint32 ulData, Uint16 usLength, Uint32 ulStatusFlag)
- Uint16 [IPCLiteCtoMDataWrite_Protected](#) (Uint32 ulFlag, Uint32 ulAddress, Uint32 ulData, Uint16 usLength, Uint32 ulStatusFlag)
- Uint16 [IPCLiteCtoMFunctionCall](#) (Uint32 ulFlag, Uint32 ulAddress, Uint32 ulParam, Uint32 ulStatusFlag)
- Uint16 [IPCLiteCtoMGetResult](#) (void *pvData, Uint16 usLength, Uint32 ulStatusFlag)
- Uint16 [IPCLiteCtoMReqMemAccess](#) (Uint32 ulFlag, Uint32 ulMask, Uint16 ulMaster, Uint32 ulStatusFlag)
- Uint16 [IPCLiteCtoMSetBits](#) (Uint32 ulFlag, Uint32 ulAddress, Uint32 ulMask, Uint16 usLength, Uint32 ulStatusFlag)
- Uint16 [IPCLiteCtoMSetBits_Protected](#) (Uint32 ulFlag, Uint32 ulAddress, Uint32 ulMask, Uint16 usLength, Uint32 ulStatusFlag)
- void [IPCLiteMtoCClearBits](#) (Uint32 ulFlag, Uint32 ulStatusFlag)
- void [IPCLiteMtoCClearBits_Protected](#) (Uint32 ulFlag, Uint32 ulStatusFlag)
- void [IPCLiteMtoCDataRead](#) (Uint32 ulFlag, Uint32 ulStatusFlag)
- void [IPCLiteMtoCDataWrite](#) (Uint32 ulFlag, Uint32 ulStatusFlag)
- void [IPCLiteMtoCDataWrite_Protected](#) (Uint32 ulFlag, Uint32 ulStatusFlag)
- void [IPCLiteMtoCFunctionCall](#) (Uint32 ulFlag, Uint32 ulStatusFlag)
- void [IPCLiteMtoCSetBits](#) (Uint32 ulFlag, Uint32 ulStatusFlag)
- void [IPCLiteMtoCSetBits_Protected](#) (Uint32 ulFlag, Uint32 ulStatusFlag)

10.6.1 Detailed Description

The IPC-Lite drivers utilize the IPC registers to send command messages one at a time via a single IPC interrupt. No additional shared memory/message RAM communication is required. Only one command can be sent and processed while the IPC interrupt flag is set. Once the IPC interrupt flag is acknowledged (cleared), another command can be sent. Only a single ISR can be used with the IPC-Lite drivers at a time.

10.6.2 Usage and Examples

The IPC-Lite drivers operate as follows in a 2 processor device where X and Y are the arbitrary processor identifiers:

1. Processor X's application code calls IPC API driver function (i.e. `IPCLiteXtoYDataWrite()`). If the IPC interrupt flag is still busy processing another command, this function will return `STATUS_FAIL` and not send any command.
2. Processor X's IPC API Driver Function inserts values into `XTOYIPCCOM`, `XTOYIPCADDR`, `XTOYIPCDATAW`, `XTOYIPCDATAR` registers as needed by IPC command.
3. Processor X's IPC API function then sets XtoY IPC INT flag to trigger an interrupt on processor Y.

4. Processor Y's XtoY IPC INT ISR is triggered. In the ISR, processor Y's application code reads the XTOYIPCCOM register and decodes the command to determine which IPC API driver function to call (i.e. IPCLiteXtoYDataWrite()).
5. The IPC API driver function processes the command, writes any data to return to processor X in the XTOYDATAR register, acknowledges the IPC interrupt flag, and the interrupt ISR is exited.
6. When processor X's application code needs the return data from processor Y for that function, it calls the IPCLiteXtoYGetResult() function in a loop and waits until the function returns STATUS_PASS. At this point, the value sent from processor Y can be read by processor X.
7. Only now can Processor X send another command to Processor Y.

There is only one step the application project must take in order to use the main IPC API driver functions.

- Add the `lpc_lite.c` and `lpc_util.c` files which include the `lpc.h` file to the application project. For M3 projects, make sure `/Mware/driverlib/lpc_lite.c` and `/driverlib/lpc_util.c` are added as source files in your project, both of which include `/driverlib/lpc.h`. For C28 projects, make sure `/F28M35x_common/source/F28M35x_lpc_lite.c` and `/F28M35x_common/source/F28M35x_lpc_util.c` are added as source files in your project, both of which include `/F28M35x_common/include/F28M35x_lpc.h`. The `lpc_util.c` file includes utility functions that can be used by application code and that are used by both the main IPC driver functions and the IPC-lite driver functions.

The M3 and C28 application code can immediately start calling the IPC command functions. Generally when an IPC command function is called for the sending processor, the responding function for the receiving processor has the same name. For instance, the M3 application code might call the `IPCMtoCDataRead()` function to write data to memory accessible only by the C28. The C28 will decode the command in the MTOCIPCCOM register when the interrupt is flagged, and upon seeing an `IPC_DATA_READ` command, it will call its own `IPCMtoCDataRead()` function to process the command.

For examples demonstrating many of the basic IPC-Lite functions, see the IPC-Lite examples in the controlSUITE device_support package for your device.

10.6.3 Function Documentation

10.6.3.1 IPCLiteCtoMClearBits

Sets the designated bits in a 16/32-bit data word at an M3 master system address

Prototype:

```

Uint16
IPCLiteCtoMClearBits (Uint32 ulFlag,
                      Uint32 ulAddress,
                      Uint32 ulMask,
                      Uint16 usLength,
                      Uint32 ulStatusFlag)

```

Parameters:

- ulFlag*** specifies C to M IPC Flag number mask used to indicate a command is being sent.
- ulAddress*** specifies the M3 address to write to.

ulMask specifies the 16/32-bit mask for bits which should be set at M3 ulAddress. (For 16-bit mask, only the lower 16-bits of ulMask are considered.

usLength specifies the length of the **ulMask** (1 = 16-bit, 2 = 32-bit).

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the Master system.

Description:

This function will allow the C28 control system to set bits specified by the **usMask** variable in a 16/32-bit word on the M3 master system. The data word at /e ulAddress after the set bits command is then read into the CTOMIPCDATAR register. After calling this function, a call to [IPCLiteCtoMGetResult\(\)](#) will read the data value in the CTOMIPCDATAR register into a 16/32-bit variable in the C28 application. The **usLength** parameter accepts the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The **ulStatusFlag** parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.6.3.2 IPCLiteCtoMClearBits_Protected

Clears the designated bits in a 16/32-bit write-protected data word at an M3 master system address

Prototype:

```
UInt16  
IPCLiteCtoMClearBits_Protected(UInt32 ulFlag,  
                                UInt32 ulAddress,  
                                UInt32 ulMask,  
                                UInt16 usLength,  
                                UInt32 ulStatusFlag)
```

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulAddress specifies the M3 write-protected address to write to.

ulMask specifies the 16/32-bit mask for bits which should be cleared at M3 ulAddress. (For 16-bit mask, only the lower 16-bits of ulMask are considered.

usLength specifies the length of the **ulMask** (1 = 16-bit, 2 = 32-bit).

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the Master system.

Description:

This function will allow the C28 control system to clear bits specified by the **usMask** variable in a write-protected 16/32-bit word on the M3 master system. The data word at /e ulAddress after the clear bits command is then read into the CTOMIPCDATAR register. After calling this function, a call to [IPCLiteCtoMGetResult\(\)](#) will read the data value in the CTOMIPCDATAR register into a 16/32-bit variable in the C28 application. The **usLength** parameter accepts the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The **ulStatusFlag** parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.6.3.3 IPCLiteCtoMDataRead

Reads either a 16- or 32-bit data word from an M3 Master System address

Prototype:

```
Uint16
IPCLiteCtoMDataRead(Uint32 ulFlag,
                    Uint32 ulAddress,
                    Uint16 usLength,
                    Uint32 ulStatusFlag)
```

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulAddress specifies the M3 address to read from

usLength designates 16- or 32-bit read (1 = 16-bit, 2 = 32-bit)

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the Master system.

Description:

This function will allow the C28 Control System to read 16/32-bit data from the M3 Master System into the CTOMIPCDATAR register. After calling this function, a call to [IPCLiteCtoMGetResult\(\)](#) will read the data value in the CTOMIPCDATAR register into a 16- or 32-bit variable in the C28 application. The *usLength* parameter accepts the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *ulStatusFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.6.3.4 IPCLiteCtoMDataWrite

Writes a 16/32-bit data word to an M3 Master System address

Prototype:

```
Uint16
IPCLiteCtoMDataWrite(Uint32 ulFlag,
                    Uint32 ulAddress,
                    Uint32 ulData,
                    Uint16 usLength,
                    Uint32 ulStatusFlag)
```

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulAddress specifies the M3 address to write to

ulData specifies the 16/32-bit word which will be written. For 16-bit words, only the lower 16-bits of ulData will be considered by the master system.

usLength is the length of the word to write (0 = 16-bits, 1 = 32-bits)

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the Master system.

Description:

This function will allow the C28 Control System to write a 16/32-bit word via the *ulData* variable to an address on the M3 Master System. The *usLength* parameter accepts the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *ulStatusFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.6.3.5 IPCLiteCtoMDataWrite_Protected

Writes a 16/32-bit data word to a protected M3 Master System address

Prototype:

```
UInt16  
IPCLiteCtoMDataWrite_Protected(UInt32 ulFlag,  
                                UInt32 ulAddress,  
                                UInt32 ulData,  
                                UInt16 usLength,  
                                UInt32 ulStatusFlag)
```

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulAddress specifies the M3 address to write to

ulData specifies the 16/32-bit word which will be written. For 16-bit words, only the lower 16-bits of ulData will be considered by the master system.

usLength is the length of the word to write (0 = 16-bits, 1 = 32-bits)

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the Master system.

Description:

This function will allow the C28 Control System to write a 16/32-bit word via the *ulData* variable to a write-protected address on the M3 Master System. The *usLength* parameter accepts the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *ulStatusFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.6.3.6 IPLiteCtoMFunctionCall

Calls a M3 function with 1 optional parameter and an optional return value.

Prototype:

```

  Uint16
  IPLiteCtoMFunctionCall (Uint32 ulFlag,
                          Uint32 ulAddress,
                          Uint32 ulParam,
                          Uint32 ulStatusFlag)

```

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulAddress specifies the M3 function address

ulParam specifies the 32-bit optional parameter value

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the control system.

Description:

This function will allow the C28 control system to call a function on the M3. The *ulParam* variable is a single optional 32-bit parameter to pass to the function. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**. The *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.6.3.7 IPLiteCtoMGetResult

Reads single word data result of CtoM IPC command

Prototype:

```

  Uint16
  IPLiteCtoMGetResult (void *pvData,
                       Uint16 usLength,
                       Uint32 ulStatusFlag)

```

Parameters:

pvData is a pointer to the 16/32-bit variable where the result data will be stored.

usLength designates 16- or 32-bit read.

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the Master system. If a status flag was not used with the command call, set this parameter to 0.

Description:

Allows the caller to read the 16/32-bit data result of non-blocking IPC functions from the CTOMIPCDATAR register if the status flag is cleared indicating the IPC command was successfully interpreted. If the status flag is not cleared, the command was not recognized, and the function will return **STATUS_FAIL**. To determine what data is read from a call to this function, see the descriptions of the non-blocking IPC functions. The *usLength* parameter accepts

the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *ulStatusFlag* parameter accepts any of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** or **STATUS_FAIL**.

Returns:

status of command (0=success, 1=error)

10.6.3.8 IPCLiteCtoMReqMemAccess

C28 Requests Master R/W/Exe Access to Shared SARAM.

Prototype:

```
Uint16  
IPCLiteCtoMReqMemAccess(Uint32 ulFlag,  
                        Uint32 ulMask,  
                        Uint16 ulMaster,  
                        Uint32 ulStatusFlag)
```

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulMask specifies the 32-bit mask for the SxMEMSEL RAM control register to indicate which Sx SARAM blocks the C28 is requesting master access to.

ulMaster specifies whether C28 or M3 should be the master of the Sx RAM.

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the Master system.

Description:

This function will allow the C28 Control System to request C28 or M3 mastership of any of the Sx Shared SARAM blocks. The *ulMaster* parameter accepts the following values: **IPC_SX_C28MASTER** or **IPC_SX_M3MASTER**. The *ulStatusFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Note:

This function calls the [*IPCLiteCtoMSetBits_Protected\(\)*](#) or the [*IPCLiteCtoMClearBits_Protected*](#) function, and therefore in order to process this function, the above 2 functions should be ready to be called on the M3 system to process this command.

Returns:

status of command (0=success, 1=error)

10.6.3.9 IPCLiteCtoMSetBits

Sets the designated bits in a 16/32-bit data word at an M3 master system address

Prototype:

```
Uint16  
IPCLiteCtoMSetBits(Uint32 ulFlag,  
                  Uint32 ulAddress,
```

```

    Uint32 ulMask,
    Uint16 usLength,
    Uint32 ulStatusFlag)

```

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulAddress specifies the M3 address to write to.

ulMask specifies the 16/32-bit mask for bits which should be set at M3 ulAddress. (For 16-bit mask, only the lower 16-bits of ulMask are considered.

usLength specifies the length of the *ulMask* (1 = 16-bit, 2 = 32-bit).

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the Master system.

Description:

This function will allow the C28 control system to set bits specified by the *usMask* variable in a 16/32-bit word on the M3 master system. The data word at /e ulAddress after the set bits command is then read into the CTOMIPCDATAR register. After calling this function, a call to [IPCLiteCtoMGetResult\(\)](#) will read the data value in the CTOMIPCDATAR register into a 16/32-bit variable in the C28 application. The *usLength* parameter accepts the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *ulStatusFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.6.3.10 IPCLiteCtoMSetBits_Protected

Sets the designated bits in a 16/32-bit write-protected data word at an M3 master system address

Prototype:

```

    Uint16
    IPCLiteCtoMSetBits_Protected(Uint32 ulFlag,
                                Uint32 ulAddress,
                                Uint32 ulMask,
                                Uint16 usLength,
                                Uint32 ulStatusFlag)

```

Parameters:

ulFlag specifies C to M IPC Flag number mask used to indicate a command is being sent.

ulAddress specifies the M3 write-protected address to write to.

ulMask specifies the 16/32-bit mask for bits which should be set at M3 ulAddress. (For 16-bit mask, only the lower 16-bits of ulMask are considered.

usLength specifies the length of the *ulMask* (1 = 16-bit, 2 = 32-bit).

ulStatusFlag indicates the C to M Flag number mask used to report the status of the command sent back from the Master system.

Description:

This function will allow the C28 control system to set bits specified by the *usMask* variable in a write-protected 16/32-bit word on the M3 master system. The data word at /e ulAddress

after the set bits command is then read into the CTOMIPCDATAR register. After calling this function, a call to [IPCLiteCtoMGetResult\(\)](#) will read the data value in the CTOMIPCDATAR register into a 16/32-bit variable in the C28 application. The *usLength* parameter accepts the following values: **IPC_LENGTH_16_BITS** or **IPC_LENGTH_32_BITS**. The *ulStatusFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**. The function returns **STATUS_PASS** if the command is successful or **STATUS_FAIL** if the request or status flags are unavailable.

Returns:

status of command (0=success, 1=error)

10.6.3.11 IPCLiteMtoCClearBits

Clears the designated bits in a 16/32-bit data word at an C28 control system address

Prototype:

```
void  
IPCLiteMtoCClearBits (Uint32 ulFlag,  
                      Uint32 ulStatusFlag)
```

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

Description:

This function will allow the M3 master system to clear bits specified by a mask variable in a 16/32-bit word on the C28 control system, and then read back the word into the MTOCIPCDATAR register. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**.

10.6.3.12 void IPCLiteMtoCClearBits_Protected (Uint32 ulFlag, Uint32 ulStatusFlag)

Clears the designated bits in a 16/32-bit data word at an C28 control system write-protected address

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

This function will allow the M3 master system to clear bits specified by a mask variable in a 16/32-bit word on the C28 control system, and then read back the word into the MTOCIPCDATAR register. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**.

10.6.3.13 IPCLiteMtoCDataRead

Reads either a 16- or 32-bit data word from an C28 control system address

Prototype:

```
void  
IPCLiteMtoCDataRead(Uint32 ulFlag,  
                    Uint32 ulStatusFlag)
```

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

Description:

This function will allow the M3 master system to read 16/32-bit data from the C28 control system. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**.

10.6.3.14 void IPCLiteMtoCDataWrite (Uint32 ulFlag, Uint32 ulStatusFlag)

Writes a 16/32-bit data word to an C28 control system address

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

This function will allow the M3 master system to write a 16/32-bit word to an address on the C28 control system. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1** - **IPC_FLAG32**, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**.

10.6.3.15 IPCLiteMtoCDataWrite_Protected

Writes a 16/32-bit data word to a write-protected C28 control system address

Prototype:

```
void  
IPCLiteMtoCDataWrite_Protected(Uint32 ulFlag,  
                               Uint32 ulStatusFlag)
```

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

Description:

This function will allow the M3 master system to write a 16/32-bit word to an address on the C28 control system. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1**

- **IPC_FLAG32**, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1 - IPC_FLAG32** and **NO_FLAG**.

10.6.3.16 void IPCLiteMtoCFunctionCall (Uint32 *ulFlag*, Uint32 *ulStatusFlag*)

Calls a C28 function with a single optional parameter and return value.

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

This function will allow the M3 master system to call a C28 function with a single optional parameter and places an optional return value in the MTOCIPCDATAR register. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1 - IPC_FLAG32**, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1 - IPC_FLAG32** and **NO_FLAG**.

10.6.3.17 IPCLiteMtoCSetBits

Sets the designated bits in a 16/32-bit data word at an C28 control system address

Prototype:

```
void
IPCLiteMtoCSetBits(Uint32 ulFlag,
                   Uint32 ulStatusFlag)
```

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

Description:

This function will allow the M3 master system to set bits specified by a mask variable in a 16/32-bit word on the C28 control system, and then read back the word into the MTOCIPCDATAR register. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1 - IPC_FLAG32**, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1 - IPC_FLAG32** and **NO_FLAG**.

10.6.3.18 void IPCLiteMtoCSetBits_Protected (Uint32 *ulFlag*, Uint32 *ulStatusFlag*)

Sets the designated bits in a 16-bit data word at an C28 control system write-protected address

Parameters:

ulFlag specifies M to C IPC Flag number mask used to indicate a command is being sent.

ulStatusFlag indicates the M to C Flag number mask used to report the status of the command sent back from the control system.

This function will allow the M3 master system to set bits specified by a mask variable in a write-protected 16/32-bit word on the C28 control system, and then read back the word into the MTOCIPCDATAR register. The *ulFlag* parameter accepts any one of the flag values **IPC_FLAG1**

- **IPC_FLAG32**, and the *ulStatusFlag* parameter accepts any other one of the flag values **IPC_FLAG1** - **IPC_FLAG32** and **NO_FLAG**.

10.7 C28 IPC Utility Drivers

Functions

- Uint16 [IPCCtoMFlagBusy](#) (Uint32 ulFlags)
- void [IPCCtoMFlagClear](#) (Uint32 ulFlags)
- void [IPCCtoMFlagSet](#) (Uint32 ulFlags)
- Uint32 [IPCCtoMSharedRamConvert](#) (Uint32 ulShareAddress)
- void [IPCMtoCFlagAcknowledge](#) (Uint32 ulFlags)
- Uint16 [IPCMtoCFlagBusy](#) (Uint32 ulFlags)

10.7.1 Detailed Description

The IPC Utility driver functions provide convenient functions to set/clear/check the status of IPC flags, convert between M3 and C28 MSG/shared RAM addresses, and a function to allow the M3 to run the C28 peripheral loaders while the C28 is in boot mode. These functions can be used in conjunction with either the IPC-Lite or main IPC drivers as long as the *ipc_util.c* (M3) or *F28M35x_ipc_Util.c* (C28) file is added to the project.

10.7.2 Function Documentation

10.7.2.1 IPCCtoMFlagBusy

Determines whether the given IPC flags are busy or not.

Prototype:

```
Uint16  
IPCCtoMFlagBusy(Uint32 ulFlags)
```

Parameters:

ulFlags specifies C to M IPC Flag number masks to check the status of.

Description:

Allows the caller to determine whether the designated IPC flags are available for further control to master system communication. If **0** is returned, then all designated tasks have completed and are available. The *ulFlags* parameter can be any of the IPC flag values: **IPC_FLAG1** - **IPC_FLAG32**.

Returns:

Returns **1** if the IPC flags are busy or **0** if designated IPC flags are free.

10.7.2.2 IPCCtoMFlagClear

C28 Clears CtoM IPC Flag

Prototype:

```
void  
IPCCtoMFlagClear (Uint32 ulFlags)
```

Parameters:

ulFlags specifies the IPC flag mask for flags being set.

Description:

This function will allow the C28 control system to set the designated IPC flags to send to the M3 master system. The *ulFlags* parameter can be any of the IPC flag values: **IPC_FLAG1** - **IPC_FLAG32**.

Returns:

None.

10.7.2.3 IPCCtoMFlagSet

C28 Sets CtoM IPC Flag

Prototype:

```
void  
IPCCtoMFlagSet (Uint32 ulFlags)
```

Parameters:

ulFlags specifies the IPC flag mask for flags being set.

Description:

This function will allow the C28 control system to set the designated IPC flags to send to the M3 master system. The *ulFlags* parameter can be any of the IPC flag values: **IPC_FLAG1** - **IPC_FLAG32**.

Returns:

None.

10.7.2.4 IPCCtoMSharedRamConvert

Converts a C28 shared RAM address to its M3 equivalent address

Prototype:

```
Uint32  
IPCCtoMSharedRamConvert (Uint32 ulShareAddress)
```

Parameters:

ulShareAddress specifies the C28 address in shared RAM (Sx RAM or MSG RAM).

Description:

This function will convert a shared RAM address location on the C28 control system to its equivalent M3 master system address (applies to MSG RAM's and Sx shared RAM blocks).

Returns:

status of command (0=success, 1=error - PutBuffer was full, command was not sent)

10.7.2.5 IPCMtoCFlagAcknowledge

C28 Acknowledges MtoC IPC Flag.

Prototype:

```
void  
IPCMtoCFlagAcknowledge(Uint32 ulFlags)
```

Parameters:

ulFlags specifies the IPC flag mask for flags being acknowledged.

Description:

This function will allow the C28 control system to acknowledge/clear the IPC flag set by the M3 master system. The *ulFlags* parameter can be any of the IPC flag values: **IPC_FLAG1** - **IPC_FLAG32**.

Returns:

None.

10.7.2.6 IPCMtoCFlagBusy

Determines whether the given MtoC IPC flags are busy or not.

Prototype:

```
Uint16  
IPCMtoCFlagBusy(Uint32 ulFlags)
```

Parameters:

ulFlags specifies MtoC IPC Flag number masks to check the status of.

Description:

Allows the caller to determine whether the designated IPC flags are pending. The *ulFlags* parameter can be any of the IPC flag values: **IPC_FLAG1** - **IPC_FLAG32**.

Returns:

Returns **1** if the IPC flags are busy or **0** if designated IPC flags are free.

11 Interrupt Controller (NVIC)

Introduction	197
API Functions	198
Programming Example	205

11.1 Introduction

The interrupt controller API provides a set of functions for dealing with the Nested Vectored Interrupt Controller (NVIC). Functions are provided to enable and disable interrupts, register interrupt handlers, and set the priority of interrupts.

One of the parameters used in these functions is `uinterrupt`. The macros for this parameter can be found in the file `hw_ints.h`. This file lists macros that define the interrupt assignments that can be used when using the interrupt drivers. This file is located in `MWare/inc/hw_ints.h`.

The NVIC provides global interrupt masking, prioritization, and handler dispatching. This version of the Concerto family supports thirty-two interrupt sources and eight priority levels. Individual interrupt sources can be masked, and the processor interrupt can be globally masked as well (without affecting the individual source masks).

The NVIC is tightly coupled with the Cortex-M3 microprocessor. When the processor responds to an interrupt, NVIC will supply the address of the function to handle the interrupt directly to the processor. This eliminates the need for a global interrupt handler that queries the interrupt controller to determine the cause of the interrupt and branch to the appropriate handler, reducing interrupt response time.

The interrupt prioritization in the NVIC allows higher priority interrupts to be handled before lower priority interrupts, as well as allowing preemption of lower priority interrupt handlers by higher priority interrupts. Again, this helps reduce interrupt response time (for example, a 1 ms system control interrupt is not held off by the execution of a lower priority 1 second housekeeping interrupt handler).

Sub-prioritization is also possible; instead of having N bits of preemptable prioritization, NVIC can be configured (via software) for N - M bits of preemptable prioritization and M bits of subpriority. In this scheme, two interrupts with the same preemptable prioritization but different subpriorities will not cause a preemption; tail chaining will instead be used to process the two interrupts back-to-back.

If two interrupts with the same priority (and subpriority if so configured) are asserted at the same time, the one with the lower interrupt number will be processed first. NVIC keeps track of the nesting of interrupt handlers, allowing the processor to return from interrupt context only once all nested and pending interrupts have been handled.

Interrupt handlers can be configured in one of two ways; statically at compile time or dynamically at run time. Static configuration of interrupt handlers is accomplished by editing the interrupt handler table in the application's startup code. When statically configured, the interrupts must be explicitly enabled in NVIC via `IntEnable()` before the processor will respond to the interrupt (in addition to any interrupt enabling required within the peripheral itself). Statically configuring the interrupt table provides the fastest interrupt response time since the stacking operation (a write to SRAM) can be performed in parallel with the interrupt handler table fetch (a read from Flash), as well as the prefetch of the interrupt handler itself (assuming it is also in Flash).

Alternatively, interrupts can be configured at run-time using `IntRegister()` (or the analog in each

individual driver). When using [IntRegister\(\)](#), the interrupt must also be enabled as before; when using the analogue in each individual driver, [IntEnable\(\)](#) is called by the driver and does not need to be call by the application. Run-time configuration of interrupts will add a small latency to the interrupt response time since the stacking operation (a write to SRAM) and the interrupt handler table fetch (a read from SRAM) must be performed sequentially.

Run-time configuration of interrupt handlers requires that the interrupt handler table be placed on a 1 kB boundary in SRAM (typically this would be at the beginning of SRAM). Failure to do so will result in an incorrect vector address being fetched in response to an interrupt. The vector table is in a section called “vtable” and should be placed appropriately with a linker script.

This driver is contained in `driverlib/interrupt.c`, with `driverlib/interrupt.h` containing the API definitions for use by applications.

11.2 API Functions

Functions

- void [IntDisable](#) (unsigned long ulInterrupt)
- void [IntEnable](#) (unsigned long ulInterrupt)
- void [IntFlashVTable](#) (void)
- tBoolean [IntMasterDisable](#) (void)
- tBoolean [IntMasterEnable](#) (void)
- void [IntPendClear](#) (unsigned long ulInterrupt)
- void [IntPendSet](#) (unsigned long ulInterrupt)
- long [IntPriorityGet](#) (unsigned long ulInterrupt)
- unsigned long [IntPriorityGroupingGet](#) (void)
- void [IntPriorityGroupingSet](#) (unsigned long ulBits)
- unsigned long [IntPriorityMaskGet](#) (void)
- void [IntPriorityMaskSet](#) (unsigned long ulPriorityMask)
- void [IntPrioritySet](#) (unsigned long ulInterrupt, unsigned char ucPriority)
- void [IntRAMVTable](#) (void)
- void [IntRegister](#) (unsigned long ulInterrupt, void (*pfnHandler)(void))
- void [IntUnregister](#) (unsigned long ulInterrupt)

11.2.1 Detailed Description

The primary function of the interrupt controller API is to manage the interrupt vector table used by the NVIC to dispatch interrupt requests. Registering an interrupt handler is a simple matter of inserting the handler address into the table. By default, the table is filled with pointers to an internal handler that loops forever; it is an error for an interrupt to occur when there is no interrupt handler registered to process it. Therefore, interrupt sources should not be enabled before a handler has been registered, and interrupt sources should be disabled before a handler is unregistered. Interrupt handlers are managed with [IntRegister\(\)](#) and [IntUnregister\(\)](#).

Each interrupt source can be individually enabled and disabled via [IntEnable\(\)](#) and [IntDisable\(\)](#). The processor interrupt can be enabled and disabled via [IntMasterEnable\(\)](#) and [IntMasterDisable\(\)](#);

this does not affect the individual interrupt enable states. Masking of the processor interrupt can be utilized as a simple critical section (only NMI will interrupt the processor while the processor interrupt is disabled), though this will have adverse effects on the interrupt response time.

The priority of each interrupt source can be set and examined via [IntPrioritySet\(\)](#) and [IntPriorityGet\(\)](#). The priority assignments are defined by the hardware; the upper N bits of the 8-bit priority are examined to determine the priority of an interrupt (for the Concerto family, N is 3). This allows priorities to be defined without a real need to know the exact number of supported priorities; moving to a device with more or fewer priority bits will continue to treat the interrupt source with a similar level of priority. Smaller priority numbers correspond to higher interrupt priority, so 0 is the highest priority.

11.2.2 Function Documentation

11.2.2.1 IntDisable

Disables an interrupt.

Prototype:

```
void  
IntDisable(unsigned long ulInterrupt)
```

Parameters:

ulInterrupt specifies the interrupt to be disabled.

Description:

The specified interrupt is disabled in the interrupt controller. Other enables for the interrupt (such as at the peripheral level) are unaffected by this function.

Returns:

None.

11.2.2.2 IntEnable

Enables an interrupt.

Prototype:

```
void  
IntEnable(unsigned long ulInterrupt)
```

Parameters:

ulInterrupt specifies the interrupt to be enabled.

Description:

The specified interrupt is enabled in the interrupt controller. Other enables for the interrupt (such as at the peripheral level) are unaffected by this function.

Returns:

None.

11.2.2.3 IntFlashVTable

Points NVIC to Vector Table in Flash

Prototype:

```
void  
IntFlashVTable(void)
```

Returns:

None.

11.2.2.4 tBoolean IntMasterDisable (void)

Disables the processor interrupt.

Prevents the processor from receiving interrupts. This does not affect the set of interrupts enabled in the interrupt controller; it just gates the single interrupt from the controller to the processor.

Note:

Previously, this function had no return value. As such, it was possible to include `interrupt.h` and call this function without having included `hw_types.h`. Now that the return is a `tBoolean`, a compiler error will occur in this case. The solution is to include `hw_types.h` before including `interrupt.h`.

Returns:

Returns **true** if interrupts were already disabled when the function was called or **false** if they were initially enabled.

11.2.2.5 tBoolean IntMasterEnable (void)

Enables the processor interrupt.

Allows the processor to respond to interrupts. This does not affect the set of interrupts enabled in the interrupt controller; it just gates the single interrupt from the controller to the processor.

Note:

Previously, this function had no return value. As such, it was possible to include `interrupt.h` and call this function without having included `hw_types.h`. Now that the return is a `tBoolean`, a compiler error will occur in this case. The solution is to include `hw_types.h` before including `interrupt.h`.

Returns:

Returns **true** if interrupts were disabled when the function was called or **false** if they were initially enabled.

11.2.2.6 void IntPendClear (unsigned long *ulInterrupt*)

Unpends an interrupt.

Parameters:

ulInterrupt specifies the interrupt to be unpended.

Description:

The specified interrupt is unpended in the interrupt controller. This will cause any previously generated interrupts that have not been handled yet (due to higher priority interrupts or the interrupt not having been enabled yet) to be discarded.

Returns:

None.

11.2.2.7 IntPendSet

Pends an interrupt.

Prototype:

```
void  
IntPendSet(unsigned long ulInterrupt)
```

Parameters:

ulInterrupt specifies the interrupt to be pended.

Description:

The specified interrupt is pended in the interrupt controller. This will cause the interrupt controller to execute the corresponding interrupt handler at the next available time, based on the current interrupt state priorities. For example, if called by a higher priority interrupt handler, the specified interrupt handler will not be called until after the current interrupt handler has completed execution. The interrupt must have been enabled for it to be called.

Returns:

None.

11.2.2.8 IntPriorityGet

Gets the priority of an interrupt.

Prototype:

```
long  
IntPriorityGet(unsigned long ulInterrupt)
```

Parameters:

ulInterrupt specifies the interrupt in question.

Description:

This function gets the priority of an interrupt. See [IntPrioritySet\(\)](#) for a definition of the priority value.

Returns:

Returns the interrupt priority, or -1 if an invalid interrupt was specified.

11.2.2.9 IntPriorityGroupingGet

Gets the priority grouping of the interrupt controller.

Prototype:

```
unsigned long  
IntPriorityGroupingGet(void)
```

Description:

This function returns the split between preemptable priority levels and subpriority levels in the interrupt priority specification.

Returns:

The number of bits of preemptable priority.

11.2.2.10 IntPriorityGroupingSet

Sets the priority grouping of the interrupt controller.

Prototype:

```
void  
IntPriorityGroupingSet(unsigned long ulBits)
```

Parameters:

ulBits specifies the number of bits of preemptable priority.

Description:

This function specifies the split between preemptable priority levels and subpriority levels in the interrupt priority specification. The range of the grouping values are dependent upon the hardware implementation; on the Concerto family, three bits are available for hardware interrupt prioritization and therefore priority grouping values of three through seven have the same effect.

Returns:

None.

11.2.2.11 IntPriorityMaskGet

Gets the priority masking level

Prototype:

```
unsigned long  
IntPriorityMaskGet(void)
```

Description:

This function gets the current setting of the interrupt priority masking level. The value returned is the priority level such that all interrupts of that and lesser priority are masked. A value of 0 means that priority masking is disabled.

Smaller numbers correspond to higher interrupt priorities. So for example a priority level mask of 4 will allow interrupts of priority level 0-3, and interrupts with a numerical priority of 4 and greater will be blocked.

The hardware priority mechanism will only look at the upper N bits of the priority level (where N is 3 for the Concerto family), so any prioritization must be performed in those bits.

Returns:

Returns the value of the interrupt priority level mask.

11.2.2.12 IntPriorityMaskSet

Sets the priority masking level

Prototype:

```
void  
IntPriorityMaskSet(unsigned long ulPriorityMask)
```

Parameters:

ulPriorityMask is the priority level that will be masked.

Description:

This function sets the interrupt priority masking level so that all interrupts at the specified or lesser priority level is masked. This can be used to globally disable a set of interrupts with priority below a predetermined threshold. A value of 0 disables priority masking.

Smaller numbers correspond to higher interrupt priorities. So for example a priority level mask of 4 will allow interrupts of priority level 0-3, and interrupts with a numerical priority of 4 and greater will be blocked.

The hardware priority mechanism will only look at the upper N bits of the priority level (where N is 3 for the Concerto family), so any prioritization must be performed in those bits.

Returns:

None.

11.2.2.13 IntPrioritySet

Sets the priority of an interrupt.

Prototype:

```
void  
IntPrioritySet(unsigned long ulInterrupt,  
              unsigned char ucPriority)
```

Parameters:

ulInterrupt specifies the interrupt in question.

ucPriority specifies the priority of the interrupt.

Description:

This function is used to set the priority of an interrupt. When multiple interrupts are asserted simultaneously, the ones with the highest priority are processed before the lower priority interrupts. Smaller numbers correspond to higher interrupt priorities; priority 0 is the highest interrupt priority.

The hardware priority mechanism will only look at the upper N bits of the priority level (where N is 3 for the Concerto family), so any prioritization must be performed in those bits. The remaining bits can be used to sub-prioritize the interrupt sources, and may be used by the hardware priority mechanism on a future part. This arrangement allows priorities to migrate to different NVIC implementations without changing the gross prioritization of the interrupts.

Returns:

None.

11.2.2.14 IntRAMVTable

Unregisters the function to be called when an interrupt occurs.

Prototype:

```
void  
IntRAMVTable(void)
```

Parameters:

ulInterrupt specifies the interrupt in question.

Description:

This function is used to indicate that no handler should be called when the given interrupt is asserted to the processor. The interrupt source will be automatically disabled (via [IntDisable\(\)](#)) if necessary.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

11.2.2.15 IntRegister

Registers a function to be called when an interrupt occurs.

Prototype:

```
void  
IntRegister(unsigned long ulInterrupt,  
             void (*pfnHandler)(void))
```

Parameters:

ulInterrupt specifies the interrupt in question.

pfnHandler is a pointer to the function to be called.

Description:

This function is used to specify the handler function to be called when the given interrupt is asserted to the processor. When the interrupt occurs, if it is enabled (via [IntEnable\(\)](#)), the handler function will be called in interrupt context. Since the handler function can preempt other code, care must be taken to protect memory or peripherals that are accessed by the handler and other non-handler code.

Note:

The use of this function (directly or indirectly via a peripheral driver interrupt register function) moves the interrupt vector table from flash to SRAM. Therefore, care must be taken when linking the application to ensure that the SRAM vector table is located at the beginning of SRAM; otherwise NVIC will not look in the correct portion of memory for the vector table (it requires the vector table be on a 1 kB memory alignment). Normally, the SRAM vector table is so placed via the use of linker scripts; some tool chains, such as the evaluation version of RV-MDK, do not support linker scripts and therefore will not produce a valid executable. See the discussion of compile-time versus run-time interrupt handler registration in the introduction to this chapter.

Returns:

None.

11.2.2.16 IntUnregister

Unregisters the function to be called when an interrupt occurs.

Prototype:

```
void  
IntUnregister(unsigned long ulInterrupt)
```

Parameters:

ulInterrupt specifies the interrupt in question.

Description:

This function is used to indicate that no handler should be called when the given interrupt is asserted to the processor. The interrupt source will be automatically disabled (via [IntDisable\(\)](#)) if necessary.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

11.3 Programming Example

The following example shows how to use the Interrupt Controller API to register an interrupt handler and enable the interrupt.

```
// The interrupt handler function.  
extern void IntHandler(void);  
  
// Register the interrupt handler function for interrupt 5.  
IntRegister(5, IntHandler);  
  
// Enable interrupt 5.  
IntEnable(5);
```

```
// Enable interrupt 5.  
IntMasterEnable();
```

12 Memory Protection Unit (MPU)

Introduction	207
API Functions	207
Programming Example	214

12.1 Introduction

The Memory Protection Unit (MPU) API provides functions to configure the MPU. The MPU is tightly coupled to the Cortex-M3 processor core and provides a means to establish access permissions on regions of memory.

Up to eight memory regions can be defined. Each region has a base address and a size. The size is specified as a power of 2 between 32 bytes and 4 GB, inclusive. The region's base address must be aligned to the size of the region. Each region also has access permissions. Code execution can be allowed or disallowed for a region. A region can be set for read-only access, read/write access, or no access for both privileged and user modes. This can be used to set up an environment where only kernel or system code can access certain hardware registers or sections of code.

The MPU creates 8 sub-regions within each region. Any sub-region or combination of sub-regions can be disabled, allowing creation of “holes” or complex overlaying regions with different permissions. The sub-regions can also be used to create an unaligned beginning or ending of a region by disabling one or more of the leading or trailing sub-regions.

Once the regions are defined and the MPU is enabled, any access violation of a region will cause a memory management fault, and the fault handler will be activated.

This driver is contained in `driverlib/mpu.c`, with `driverlib/mpu.h` containing the API definitions for use by applications.

12.2 API Functions

Functions

- void [MPUDisable](#) (void)
- void [MPUEnable](#) (unsigned long ulMPUConfig)
- void [MPUIntRegister](#) (void (*pfnHandler)(void))
- void [MPUIntUnregister](#) (void)
- unsigned long [MPURegionCountGet](#) (void)
- void [MPURegionDisable](#) (unsigned long ulRegion)
- void [MPURegionEnable](#) (unsigned long ulRegion)
- void [MPURegionGet](#) (unsigned long ulRegion, unsigned long *pulAddr, unsigned long *pulFlags)
- void [MPURegionSet](#) (unsigned long ulRegion, unsigned long ulAddr, unsigned long ulFlags)

12.2.1 Detailed Description

The MPU APIs provide a means to enable and configure the MPU and memory protection regions.

Generally, the memory protection regions should be defined before enabling the MPU. The regions can be configured by calling [MPURegionSet\(\)](#) once for each region to be configured.

A region that is defined by [MPURegionSet\(\)](#) can be initially enabled or disabled. If the region is not initially enabled, it can be enabled later by calling [MPURegionEnable\(\)](#). An enabled region can be disabled by calling [MPURegionDisable\(\)](#). When a region is disabled, its configuration is preserved as long as it is not overwritten. In this case it can be enabled again with [MPURegionEnable\(\)](#) without the need to reconfigure the region.

Care must be taken when setting up a protection region using [MPURegionSet\(\)](#). The function will write to multiple registers and is not protected from interrupts. Therefore, it is possible that an interrupt which accesses a region may occur while that region is in the process of being changed. The safest way to protect against this is to make sure that a region is always disabled before making any changes. Otherwise, it is up to the caller to ensure that [MPURegionSet\(\)](#) is always called from within code that cannot be interrupted, or from code that will not be affected if an interrupt occurs while the region attributes are being changed.

The attributes of a region that has already been programmed can be retrieved and saved using the [MPURegionGet\(\)](#) function. This function is intended to save the attributes in a format that can be used later to reload the region using the [MPURegionSet\(\)](#) function. Note that the enable state of the region is saved with the attributes and will take effect when the region is reloaded.

When one or more regions are defined, the MPU can be enabled by calling [MPUEnable\(\)](#). This turns on the MPU and also defines the behavior in privileged mode and in the Hard Fault and NMI fault handlers. The MPU can be configured so that when in privileged mode and no regions are enabled, a default memory map is applied. If this feature is not enabled, then a memory management fault is generated if the MPU is enabled and no regions are configured and enabled. The MPU can also be set to use a default memory map when in the Hard Fault or NMI handlers, instead of using the configured regions. All of these features are selected when calling [MPUEnable\(\)](#). When the MPU is enabled, it can be disabled by calling [MPUDisable\(\)](#).

Finally, if the application is using run-time interrupt registration (see [IntRegister\(\)](#)), then the function [MPUIntRegister\(\)](#) can be used to install the fault handler which will be called whenever a memory protection violation occurs. This function will also enable the fault handler. If compile-time interrupt registration is used, then the [IntEnable\(\)](#) function with the parameter **FAULT_MPU** must be used to enable the memory management fault handler. When the memory management fault handler has been installed with [MPUIntRegister\(\)](#), it can be removed by calling [MPUIntUnregister\(\)](#).

12.2.2 Function Documentation

12.2.2.1 MPUDisable

Disables the MPU for use.

Prototype:

```
void  
MPUDisable(void)
```

Description:

This function disables the Cortex-M3 memory protection unit. When the MPU is disabled, the

default memory map is used and memory management faults are not generated.

Returns:

None.

12.2.2.2 MPUEnable

Enables and configures the MPU for use.

Prototype:

```
void  
MPUEnable(unsigned long ulMPUConfig)
```

Parameters:

ulMPUConfig is the logical OR of the possible configurations.

Description:

This function enables the Cortex-M3 memory protection unit. It also configures the default behavior when in privileged mode and while handling a hard fault or NMI. Prior to enabling the MPU, at least one region must be set by calling [MPURegionSet\(\)](#) or else by enabling the default region for privileged mode by passing the **MPU_CONFIG_PRIV_DEFAULT** flag to [MPUEnable\(\)](#). Once the MPU is enabled, a memory management fault will be generated for any memory access violations.

The *ulMPUConfig* parameter should be the logical OR of any of the following:

- **MPU_CONFIG_PRIV_DEFAULT** enables the default memory map when in privileged mode and when no other regions are defined. If this option is not enabled, then there must be at least one valid region already defined when the MPU is enabled.
- **MPU_CONFIG_HARDFLT_NMI** enables the MPU while in a hard fault or NMI exception handler. If this option is not enabled, then the MPU is disabled while in one of these exception handlers and the default memory map is applied.
- **MPU_CONFIG_NONE** chooses none of the above options. In this case, no default memory map is provided in privileged mode, and the MPU will not be enabled in the fault handlers.

Returns:

None.

12.2.2.3 MPUIntRegister

Registers an interrupt handler for the memory management fault.

Prototype:

```
void  
MPUIntRegister(void (*pfnHandler)(void))
```

Parameters:

pfnHandler is a pointer to the function to be called when the memory management fault occurs.

Description:

This sets and enables the handler to be called when the MPU generates a memory management fault due to a protection region access violation.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

12.2.2.4 MPUIntUnregister

Unregisters an interrupt handler for the memory management fault.

Prototype:

```
void  
MPUIntUnregister(void)
```

Description:

This function will disable and clear the handler to be called when a memory management fault occurs.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

12.2.2.5 MPURegionCountGet

Gets the count of regions supported by the MPU.

Prototype:

```
unsigned long  
MPURegionCountGet(void)
```

Description:

This function is used to get the number of regions that are supported by the MPU. This is the total number that are supported, including regions that are already programmed.

Returns:

The number of memory protection regions that are available for programming using [MPURegionSet\(\)](#).

12.2.2.6 MPURegionDisable

Disables a specific region.

Prototype:

```
void  
MPURegionDisable(unsigned long ulRegion)
```

Parameters:

ulRegion is the region number to disable.

Description:

This function is used to disable a previously enabled memory protection region. The region will remain configured if it is not overwritten with another call to [MPURegionSet\(\)](#), and can be enabled again by calling [MPURegionEnable\(\)](#).

Returns:

None.

12.2.2.7 MPURegionEnable

Enables a specific region.

Prototype:

```
void  
MPURegionEnable(unsigned long ulRegion)
```

Parameters:

ulRegion is the region number to enable.

Description:

This function is used to enable a memory protection region. The region should already be set up with the [MPURegionSet\(\)](#) function. Once enabled, the memory protection rules of the region will be applied and access violations will cause a memory management fault.

Returns:

None.

12.2.2.8 MPURegionGet

Gets the current settings for a specific region.

Prototype:

```
void  
MPURegionGet(unsigned long ulRegion,  
              unsigned long *pulAddr,  
              unsigned long *pulFlags)
```

Parameters:

ulRegion is the region number to get.

pulAddr points to storage for the base address of the region.

pulFlags points to the attribute flags for the region.

Description:

This function retrieves the configuration of a specific region. The meanings and format of the parameters is the same as that of the [MPURegionSet\(\)](#) function.

This function can be used to save the configuration of a region for later use with the [MPURegionSet\(\)](#) function. The region's enable state will be preserved in the attributes that are saved.

Returns:

None.

12.2.2.9 MPURegionSet

Sets up the access rules for a specific region.

Prototype:

```
void  
MPURegionSet(unsigned long ulRegion,  
              unsigned long ulAddr,  
              unsigned long ulFlags)
```

Parameters:

ulRegion is the region number to set up.

ulAddr is the base address of the region. It must be aligned according to the size of the region specified in ulFlags.

ulFlags is a set of flags to define the attributes of the region.

Description:

This function sets up the protection rules for a region. The region has a base address and a set of attributes including the size, which must be a power of 2. The base address parameter, *ulAddr*, must be aligned according to the size.

The *ulFlags* parameter is the logical OR of all of the attributes of the region. It is a combination of choices for region size, execute permission, read/write permissions, disabled sub-regions, and a flag to determine if the region is enabled.

The size flag determines the size of a region, and must be one of the following:

- MPU_RGN_SIZE_32B
- MPU_RGN_SIZE_64B
- MPU_RGN_SIZE_128B
- MPU_RGN_SIZE_256B
- MPU_RGN_SIZE_512B
- MPU_RGN_SIZE_1K
- MPU_RGN_SIZE_2K
- MPU_RGN_SIZE_4K
- MPU_RGN_SIZE_8K
- MPU_RGN_SIZE_16K
- MPU_RGN_SIZE_32K
- MPU_RGN_SIZE_64K
- MPU_RGN_SIZE_128K
- MPU_RGN_SIZE_256K

- **MPU_RGN_SIZE_512K**
- **MPU_RGN_SIZE_1M**
- **MPU_RGN_SIZE_2M**
- **MPU_RGN_SIZE_4M**
- **MPU_RGN_SIZE_8M**
- **MPU_RGN_SIZE_16M**
- **MPU_RGN_SIZE_32M**
- **MPU_RGN_SIZE_64M**
- **MPU_RGN_SIZE_128M**
- **MPU_RGN_SIZE_256M**
- **MPU_RGN_SIZE_512M**
- **MPU_RGN_SIZE_1G**
- **MPU_RGN_SIZE_2G**
- **MPU_RGN_SIZE_4G**

The execute permission flag must be one of the following:

- **MPU_RGN_PERM_EXEC** enables the region for execution of code
- **MPU_RGN_PERM_NOEXEC** disables the region for execution of code

The read/write access permissions are applied separately for the privileged and user modes. The read/write access flags must be one of the following:

- **MPU_RGN_PERM_PRV_NO_USR_NO** - no access in privileged or user mode
- **MPU_RGN_PERM_PRV_RW_USR_NO** - privileged read/write, user no access
- **MPU_RGN_PERM_PRV_RW_USR_RO** - privileged read/write, user read-only
- **MPU_RGN_PERM_PRV_RW_USR_RW** - privileged read/write, user read/write
- **MPU_RGN_PERM_PRV_RO_USR_NO** - privileged read-only, user no access
- **MPU_RGN_PERM_PRV_RO_USR_RO** - privileged read-only, user read-only

The region is automatically divided into 8 equally-sized sub-regions by the MPU. Sub-regions can only be used in regions of size 256 bytes or larger. Any of these 8 sub-regions can be disabled. This allows for creation of “holes” in a region which can be left open, or overlaid by another region with different attributes. Any of the 8 sub-regions can be disabled with a logical OR of any of the following flags:

- **MPU_SUB_RGN_DISABLE_0**
- **MPU_SUB_RGN_DISABLE_1**
- **MPU_SUB_RGN_DISABLE_2**
- **MPU_SUB_RGN_DISABLE_3**
- **MPU_SUB_RGN_DISABLE_4**
- **MPU_SUB_RGN_DISABLE_5**
- **MPU_SUB_RGN_DISABLE_6**
- **MPU_SUB_RGN_DISABLE_7**

Finally, the region can be initially enabled or disabled with one of the following flags:

- **MPU_RGN_ENABLE**
- **MPU_RGN_DISABLE**

As an example, to set a region with the following attributes: size of 32 KB, execution enabled, read-only for both privileged and user, one sub-region disabled, and initially enabled; the *ulFlags* parameter would have the following value:

```
(MPU_RG_SIZE_32K | MPU_RGN_PERM_EXEC | MPU_RGN_PERM_PRV_RO_USR_RO |  
MPU_SUB_RGN_DISABLE_2 | MPU_RGN_ENABLE)
```

Note:

This function will write to multiple registers and is not protected from interrupts. It is possible that an interrupt which accesses a region may occur while that region is in the process of being changed. The safest way to handle this is to disable a region before changing it. Refer to the discussion of this in the API Detailed Description section.

Returns:

None.

12.3 Programming Example

The following example sets up a basic set of protection regions to provide the following:

- a 28 KB region in flash for read-only code execution
- 32 KB of RAM for read-write access in privileged and user modes
- an additional 8 KB of RAM for use only in privileged mode
- 1 MB of peripheral space for access only in privileged mode, except for a 128 KB hole that is not accessible at all, and another 128 KB region within that is accessible from user mode

```
// Define a 28 KB region of flash from 0x00000000 to 0x00007000. The  
// region will be executable, and read-only for both privileged and user  
// modes. To set up the region, a 32 KB region (#0) will be defined  
// starting at address 0, and then a 4 KB hole removed at the end by  
// disabling the last sub-region. The region will be initially enabled.  
MPURegionSet(0, 0,  
    MPU_RGN_SIZE_32K |  
    MPU_RGN_PERM_EXEC |  
    MPU_RGN_PERM_PRV_RO_USR_RO |  
    MPU_SUB_RGN_DISABLE_7 |  
    MPU_RGN_ENABLE);  
  
// Define a 32 KB region (#1) of RAM from 0x20000000 to 0x20008000.  
// The region will not be executable, and will be read/write access  
// for privileged and user modes.  
MPURegionSet(1, 0x20000000,  
    MPU_RGN_SIZE_32K |  
    MPU_RGN_PERM_NOEXEC |  
    MPU_RGN_PERM_PRV_RW_USR_RW |  
    MPU_RGN_ENABLE);  
  
// Define an additional 8 KB region (#2) in RAM from 0x20008000 to  
// 0x2000A000, which will be read/write accessible only from privileged  
// mode. This region will be initially disabled, to be enabled later.
```

```

MPURegionSet(2, 0x20008000,
             MPU_RGN_SIZE_8K |
             MPU_RGN_PERM_NOEXEC |
             MPU_RGN_PERM_PRV_RW_USR_NO |
             MPU_RGN_DISABLE);

// Define a region (#3) in peripheral space from 0x40000000 to
// 0x40100000 (1 MB). This region is accessible only in privileged
// mode. There is an area from 0x40020000 to 0x40040000 that has no
// peripherals and is not accessible at all. This is created by
// disabling the second sub-region (1) and creating a hole. Further,
// there is an area from 0x40080000 to 0x400A0000 that should be
// accessible from user mode as well. This is created by disabling
// the fifth sub-region (4), and overlaying an additional region (#4)
// in that space with the appropriate permissions.
MPURegionSet(3, 0x40000000,
             MPU_RGN_SIZE_1M |
             MPU_RGN_PERM_NOEXEC |
             MPU_RGN_PERM_PRV_RW_USR_NO |
             MPU_SUB_RGN_DISABLE_1 | MPU_SUB_RGN_DISABLE_4 |
             MPU_RGN_ENABLE);
MPURegionSet(4, 0x40080000,
             MPU_RGN_SIZE_128K |
             MPU_RGN_PERM_NOEXEC |
             MPU_RGN_PERM_PRV_RW_USR_RW |
             MPU_RGN_ENABLE);

// In this example, compile-time registration of interrupts is used,
// so the handler does not need to be registered. However, it does
// need to be enabled.
IntEnable(FAULT_MPU);

// When setting up the regions, region 2 was initially disabled for
// some reason. At some point it needs to be enabled.
MPURegionEnable(2);

// Now the MPU will be enabled. It will be configured so that a
// default map is available in privileged mode if no regions are
// defined. The MPU will not be enabled for the hard fault and NMI
// handlers, which means a default map will be used whenever these
// handlers are active, effectively giving the fault handlers access
// to all of memory without any protection.
MPUEnable(MPU_CONFIG_PRIV_DEFAULT);

// At this point the MPU is configured and enabled and if any code
// causes an access violation, the memory management fault will occur.

```

The following example shows how to save and restore region configurations.

```

// The following arrays provide space for saving the address and
// attributes for 4 region configurations.
unsigned long ulRegionAddr[4];

```

```
unsigned long ulRegionAttr[4];

...

// At some point in the system code, we want to save the state of
// 4 regions(0-3).
for(uIdx = 0; uIdx < 4; uIdx++)
{
    MPURegionGet(uIdx, &ulRegionAddr[uIdx], &ulRegionAttr[uIdx]);
}

...

// At some other point, the previously saved regions should
// be restored.
for(uIdx = 0; uIdx < 4; uIdx++)
{
    MPURegionSet(uIdx, ulRegionAddr[uIdx], ulRegionAttr[uIdx]);
}
```


13 Synchronous Serial Interface (SSI)

Introduction	217
API Functions	217
Programming Example	226

13.1 Introduction

The Synchronous Serial Interface (SSI) module provides the functionality for synchronous serial communications with peripheral devices, and can be configured to use either the Motorola® SPI™, National Semiconductor® Microwire, or the Texas Instruments® synchronous serial interface frame formats. The size of the data frame is also configurable, and can be set to be between 4 and 16 bits, inclusive.

The SSI module performs serial-to-parallel data conversion on data received from a peripheral device, and parallel-to-serial conversion on data transmitted to a peripheral device. The TX and RX paths are buffered with internal FIFOs allowing up to eight 16-bit values to be stored independently.

The SSI module can be configured as either a master or a slave device. As a slave device, the SSI module can also be configured to disable its output, which allows a master device to be coupled with multiple slave devices.

The SSI module also includes a programmable bit rate clock divider and prescaler to generate the output serial clock derived from the SSI module's input clock. Bit rates are generated based on the input clock and the maximum bit rate supported by the connected peripheral.

For parts that include a DMA controller, the SSI module also provides a DMA interface to facilitate data transfer via DMA.

This driver is contained in `driverlib/ssi.c`, with `driverlib/ssi.h` containing the API definitions for use by applications.

13.2 API Functions

Functions

- tBoolean [SSIBusy](#) (unsigned long ulBase)
- void [SSIConfigSetExpCk](#) (unsigned long ulBase, unsigned long ulSSIClk, unsigned long ulProtocol, unsigned long ulMode, unsigned long ulBitRate, unsigned long ulDataWidth)
- void [SSIDataGet](#) (unsigned long ulBase, unsigned long *pulData)
- long [SSIDataGetNonBlocking](#) (unsigned long ulBase, unsigned long *pulData)
- void [SSIDataPut](#) (unsigned long ulBase, unsigned long ulData)
- long [SSIDataPutNonBlocking](#) (unsigned long ulBase, unsigned long ulData)
- void [SSIDisable](#) (unsigned long ulBase)
- void [SSIDMADisable](#) (unsigned long ulBase, unsigned long ulDMAFlags)
- void [SSIDMAEnable](#) (unsigned long ulBase, unsigned long ulDMAFlags)
- void [SSIEnable](#) (unsigned long ulBase)
- void [SSIIntClear](#) (unsigned long ulBase, unsigned long ulIntFlags)

- void [SSIntDisable](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [SSIntEnable](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [SSIntRegister](#) (unsigned long ulBase, void (*pfnHandler)(void))
- unsigned long [SSIntStatus](#) (unsigned long ulBase, tBoolean bMasked)
- void [SSIntUnregister](#) (unsigned long ulBase)

13.2.1 Detailed Description

The SSI API is broken into 3 groups of functions: those that deal with configuration and state, those that handle data, and those that manage interrupts.

The configuration of the SSI module is managed by the [SSISetExpClk\(\)](#) function, while state is managed by the [SSIEnable\(\)](#) and [SSIDisable\(\)](#) functions. The DMA interface is enabled or disabled by the [SSIDMAEnable\(\)](#) and [SSIDMADisable\(\)](#) functions.

Data handling is performed by the [SSIDataPut\(\)](#), [SSIDataPutNonBlocking\(\)](#), [SSIDataGet\(\)](#), and [SSIDataGetNonBlocking\(\)](#) functions.

Interrupts from the SSI module are managed using the [SSIntClear\(\)](#), [SSIntDisable\(\)](#), [SSIntEnable\(\)](#), [SSIntRegister\(\)](#), [SSIntStatus\(\)](#), and [SSIntUnregister\(\)](#) functions.

The [SSISetExpClk\(\)](#), [SSIDataGetNonBlocking\(\)](#), and [SSIDataPutNonBlocking\(\)](#) APIs from previous versions of the peripheral driver library have been replaced by the [SSISetExpClk\(\)](#), [SSIDataGetNonBlocking\(\)](#), and [SSIDataPutNonBlocking\(\)](#) APIs. Macros have been provided in `ssi.h` to map the old APIs to the new APIs, allowing existing applications to link and run with the new APIs. It is recommended that new applications utilize the new APIs in favor of the old ones.

13.2.2 Function Documentation

13.2.2.1 SSIBusy

Determines whether the SSI transmitter is busy or not.

Prototype:

```
tBoolean  
SSIBusy(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the SSI port.

Description:

This function allows the caller to determine whether all transmitted bytes have cleared the transmitter hardware. If **false** is returned, then the transmit FIFO is empty and all bits of the last transmitted word have left the hardware shift register.

Returns:

Returns **true** if the SSI is transmitting or **false** if all transmissions are complete.

13.2.2.2 SSIConfigSetExpClk

Configures the synchronous serial interface.

Prototype:

```
void
SSIConfigSetExpClk(unsigned long ulBase,
                   unsigned long ulSSIClk,
                   unsigned long ulProtocol,
                   unsigned long ulMode,
                   unsigned long ulBitRate,
                   unsigned long ulDataWidth)
```

Parameters:

ulBase specifies the SSI module base address.
ulSSIClk is the rate of the clock supplied to the SSI module.
ulProtocol specifies the data transfer protocol.
ulMode specifies the mode of operation.
ulBitRate specifies the clock rate.
ulDataWidth specifies number of bits transferred per frame.

Description:

This function configures the synchronous serial interface. It sets the SSI protocol, mode of operation, bit rate, and data width.

The *ulProtocol* parameter defines the data frame format. The *ulProtocol* parameter can be one of the following values: **SSI_FRF_MOTO_MODE_0**, **SSI_FRF_MOTO_MODE_1**, **SSI_FRF_MOTO_MODE_2**, **SSI_FRF_MOTO_MODE_3**, **SSI_FRF_TI**, or **SSI_FRF_NMW**. The Motorola frame formats encode the following polarity and phase configurations:

Polarity	Phase	Mode
0	0	SSI_FRF_MOTO_MODE_0
0	1	SSI_FRF_MOTO_MODE_1
1	0	SSI_FRF_MOTO_MODE_2
1	1	SSI_FRF_MOTO_MODE_3

The *ulMode* parameter defines the operating mode of the SSI module. The SSI module can operate as a master or slave; if it is a slave, the SSI can be configured to disable output on its serial output line. The *ulMode* parameter can be one of the following values: **SSI_MODE_MASTER**, **SSI_MODE_SLAVE**, or **SSI_MODE_SLAVE_OD**.

The *ulBitRate* parameter defines the bit rate for the SSI. This bit rate must satisfy the following clock ratio criteria:

- $F_{SSI} \geq 2 * \text{bit rate}$ (master mode); this speed cannot exceed 25 MHz.
- $F_{SSI} \geq 12 * \text{bit rate}$ or $6 * \text{bit rate}$ (slave modes), depending on the capability of the specific microcontroller

where F_{SSI} is the frequency of the clock supplied to the SSI module.

The *ulDataWidth* parameter defines the width of the data transfers and can be a value between 4 and 16, inclusive.

The peripheral clock is the same as the processor clock. This value is returned by [SysCtlClockGet\(\)](#), or it can be explicitly hard coded if it is constant and known (to save the code/execution overhead of a call to [SysCtlClockGet\(\)](#)).

This function replaces the original SSISConfig() API and performs the same actions. A macro is provided in `ssi.h` to map the original API to this API.

Returns:

None.

13.2.2.3 SSIDataGet

Gets a data element from the SSI receive FIFO.

Prototype:

```
void
SSIDataGet(unsigned long ulBase,
            unsigned long *pulData)
```

Parameters:

ulBase specifies the SSI module base address.

pulData is a pointer to a storage location for data that was received over the SSI interface.

Description:

This function gets received data from the receive FIFO of the specified SSI module and places that data into the location specified by the *pulData* parameter.

Note:

Only the lower N bits of the value written to *pulData* contain valid data, where N is the data width as configured by [SSISConfigSetExpClk\(\)](#). For example, if the interface is configured for 8-bit data width, only the lower 8 bits of the value written to *pulData* contain valid data.

Returns:

None.

13.2.2.4 SSIDataGetNonBlocking

Gets a data element from the SSI receive FIFO.

Prototype:

```
long
SSIDataGetNonBlocking(unsigned long ulBase,
                       unsigned long *pulData)
```

Parameters:

ulBase specifies the SSI module base address.

pulData is a pointer to a storage location for data that was received over the SSI interface.

Description:

This function gets received data from the receive FIFO of the specified SSI module and places that data into the location specified by the *ulData* parameter. If there is no data in the FIFO, then this function returns a zero.

This function replaces the original `SSIDataNonBlockingGet()` API and performs the same actions. A macro is provided in `ssi.h` to map the original API to this API.

Note:

Only the lower N bits of the value written to *pulData* contain valid data, where N is the data width as configured by [SSIConfigSetExpClk\(\)](#). For example, if the interface is configured for 8-bit data width, only the lower 8 bits of the value written to *pulData* contain valid data.

Returns:

Returns the number of elements read from the SSI receive FIFO.

13.2.2.5 SSIDataPut

Puts a data element into the SSI transmit FIFO.

Prototype:

```
void  
SSIDataPut(unsigned long ulBase,  
            unsigned long ulData)
```

Parameters:

ulBase specifies the SSI module base address.

ulData is the data to be transmitted over the SSI interface.

Description:

This function places the supplied data into the transmit FIFO of the specified SSI module.

Note:

The upper 32 - N bits of the *ulData* are discarded by the hardware, where N is the data width as configured by [SSIConfigSetExpClk\(\)](#). For example, if the interface is configured for 8-bit data width, the upper 24 bits of *ulData* are discarded.

Returns:

None.

13.2.2.6 SSIDataPutNonBlocking

Puts a data element into the SSI transmit FIFO.

Prototype:

```
long  
SSIDataPutNonBlocking(unsigned long ulBase,  
                       unsigned long ulData)
```

Parameters:

ulBase specifies the SSI module base address.

ulData is the data to be transmitted over the SSI interface.

Description:

This function places the supplied data into the transmit FIFO of the specified SSI module. If there is no space in the FIFO, then this function returns a zero.

This function replaces the original `SSIDataNonBlockingPut()` API and performs the same actions. A macro is provided in `ssi.h` to map the original API to this API.

Note:

The upper 32 - N bits of the *ulData* are discarded by the hardware, where N is the data width as configured by [SSIConfigSetExpClk\(\)](#). For example, if the interface is configured for 8-bit data width, the upper 24 bits of *ulData* are discarded.

Returns:

Returns the number of elements written to the SSI transmit FIFO.

13.2.2.7 SSIDisable

Disables the synchronous serial interface.

Prototype:

```
void  
SSIDisable(unsigned long ulBase)
```

Parameters:

ulBase specifies the SSI module base address.

Description:

This function disables operation of the synchronous serial interface.

Returns:

None.

13.2.2.8 SSIDMADisable

Disable SSI DMA operation.

Prototype:

```
void  
SSIDMADisable(unsigned long ulBase,  
               unsigned long ulDMAFlags)
```

Parameters:

ulBase is the base address of the SSI port.

ulDMAFlags is a bit mask of the DMA features to disable.

Description:

This function is used to disable SSI DMA features that were enabled by [SSIDMAEnable\(\)](#). The specified SSI DMA features are disabled. The *ulDMAFlags* parameter is the logical OR of any of the following values:

- `SSI_DMA_RX` - disable DMA for receive
- `SSI_DMA_TX` - disable DMA for transmit

Returns:

None.

13.2.2.9 SSIDMAEnable

Enable SSI DMA operation.

Prototype:

```
void  
SSIDMAEnable(unsigned long ulBase,  
              unsigned long ulDMAFlags)
```

Parameters:

ulBase is the base address of the SSI port.

ulDMAFlags is a bit mask of the DMA features to enable.

Description:

This function enables the specified SSI DMA features. The SSI can be configured to use DMA for transmit and/or receive data transfers. The *ulDMAFlags* parameter is the logical OR of any of the following values:

- SSI_DMA_RX - enable DMA for receive
- SSI_DMA_TX - enable DMA for transmit

Note:

The uDMA controller must also be set up before DMA can be used with the SSI.

Returns:

None.

13.2.2.10 SSIEnable

Enables the synchronous serial interface.

Prototype:

```
void  
SSIEnable(unsigned long ulBase)
```

Parameters:

ulBase specifies the SSI module base address.

Description:

This function enables operation of the synchronous serial interface. The synchronous serial interface must be configured before it is enabled.

Returns:

None.

13.2.2.11 SSIIntClear

Clears SSI interrupt sources.

Prototype:

```
void  
SSIIIntClear(unsigned long ulBase,  
             unsigned long ulIntFlags)
```

Parameters:

ulBase specifies the SSI module base address.

ullIntFlags is a bit mask of the interrupt sources to be cleared.

Description:

This function clears the specified SSI interrupt sources so that they no longer assert. This function must be called in the interrupt handler to keep the interrupts from being triggered again immediately upon exit. The *ullIntFlags* parameter can consist of either or both the **SSI_RXTO** and **SSI_RXOR** values.

Note:

Because there is a write buffer in the Cortex-M processor, it may take several clock cycles before the interrupt source is actually cleared. Therefore, it is recommended that the interrupt source be cleared early in the interrupt handler (as opposed to the very last action) to avoid returning from the interrupt handler before the interrupt source is actually cleared. Failure to do so may result in the interrupt handler being immediately reentered (because the interrupt controller still sees the interrupt source asserted).

Returns:

None.

13.2.2.12 SSIIIntDisable

Disables individual SSI interrupt sources.

Prototype:

```
void  
SSIIIntDisable(unsigned long ulBase,  
              unsigned long ulIntFlags)
```

Parameters:

ulBase specifies the SSI module base address.

ullIntFlags is a bit mask of the interrupt sources to be disabled.

Description:

This function disables the indicated SSI interrupt sources. The *ullIntFlags* parameter can be any of the **SSI_TXFF**, **SSI_RXFF**, **SSI_RXTO**, or **SSI_RXOR** values.

Returns:

None.

13.2.2.13 SSIIIntEnable

Enables individual SSI interrupt sources.

Prototype:

```
void  
SSIIntEnable(unsigned long ulBase,  
             unsigned long ulIntFlags)
```

Parameters:

ulBase specifies the SSI module base address.

ulIntFlags is a bit mask of the interrupt sources to be enabled.

Description:

This function enables the indicated SSI interrupt sources. Only the sources that are enabled can be reflected to the processor interrupt; disabled sources have no effect on the processor. The *ulIntFlags* parameter can be any of the **SSI_TXFF**, **SSI_RXFF**, **SSI_RXT0**, or **SSI_RXOR** values.

Returns:

None.

13.2.2.14 SSIIntRegister

Registers an interrupt handler for the synchronous serial interface.

Prototype:

```
void  
SSIIntRegister(unsigned long ulBase,  
               void (*pfnHandler)(void))
```

Parameters:

ulBase specifies the SSI module base address.

pfnHandler is a pointer to the function to be called when the synchronous serial interface interrupt occurs.

Description:

This function registers the handler to be called when an SSI interrupt occurs. This function enables the global interrupt in the interrupt controller; specific SSI interrupts must be enabled via [SSIIntEnable\(\)](#). If necessary, it is the interrupt handler's responsibility to clear the interrupt source via [SSIIntClear\(\)](#).

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

13.2.2.15 SSIIntStatus

Gets the current interrupt status.

Prototype:

```
unsigned long  
SSIIntStatus(unsigned long ulBase,  
              tBoolean bMasked)
```

Parameters:

ulBase specifies the SSI module base address.

bMasked is **false** if the raw interrupt status is required or **true** if the masked interrupt status is required.

Description:

This function returns the interrupt status for the SSI module. Either the raw interrupt status or the status of interrupts that are allowed to reflect to the processor can be returned.

Returns:

The current interrupt status, enumerated as a bit field of **SSI_TXFF**, **SSI_RXFF**, **SSI_RXT0**, and **SSI_RXOR**.

13.2.2.16 SSIntUnregister

Unregisters an interrupt handler for the synchronous serial interface.

Prototype:

```
void  
SSIntUnregister(unsigned long ulBase)
```

Parameters:

ulBase specifies the SSI module base address.

Description:

This function clears the handler to be called when an SSI interrupt occurs. This function also masks off the interrupt in the interrupt controller so that the interrupt handler no longer is called.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

13.3 Programming Example

The following example shows how to use the SSI API to configure the SSI module as a master device, and how to do a simple send of data.

```
char *pcChars = "SSI Master send data.";  
long lIdx;  
  
// Configure the SSI.  
SSIconfigSetExpClk(SSI_BASE, SysCtlClockGet(), SSI_FRF_MOTO_MODE0,  
                   SSI_MODE_MASTER, 2000000, 8);  
  
// Enable the SSI module.  
SSIEnable(SSI_BASE);
```

```
// Send some data.
lIdx = 0;
while (pcChars[lIdx])
{
    if (SSIDataPut (SSI_BASE, pcChars[lIdx]))
    {
        lIdx++;
    }
}
```


14 System Control

Introduction	229
API Functions	230
Programming Example	250

14.1 Introduction

System control determines the overall operation of the device. It controls the clocking of the device, the set of peripherals that are enabled, configuration of the device and its resets, and provides information about the device.

The members of the Concerto family have a varying peripheral set and memory sizes. The device has a set of read-only registers that indicate the size of the memories and the peripherals that are present. This information can be used to write adaptive software that will run on more than one member of the Concerto family.

The device must be clocked from an external oscillator, but there are many different clocking configurations internally which can be used to achieve the desired clocking. Concerto devices employ 2 PLL (one for the main system and one for USB), as well as many dividers. The CAN and USB peripherals have additional clocking options which allow the use of a separate external clock if necessary.

Three modes of operation are supported by the Concerto family: run mode, sleep mode, and deep-sleep mode. In run mode, the processor is actively executing code. In sleep mode, the clocking of the device is unchanged but the processor no longer executes code (and is no longer clocked). In deep-sleep mode, the clocking of the device may change (depending upon the run mode clock configuration) and the processor no longer executes code (and is no longer clocked). An interrupt will return the device to run mode from one of the sleep modes; the sleep modes are entered upon request from the code.

There are several system events that, when detected, will cause system control to reset the device. These events are the input voltage dropping too low, an external reset, a software reset request, and a watchdog timeout. The properties of some of these events can be configured, and the reason for a reset can be determined from system control.

Each peripheral in the device can be individually enabled, disabled, or reset. Additionally, the set of peripherals that remain enabled during sleep mode and deep-sleep mode can be configured, allowing custom sleep and deep-sleep modes to be defined. Care must be taken with deep-sleep mode, though, since in this mode the PLL is no longer used and the system is clocked by the input crystal. Peripherals that depend upon a particular input clock rate (such as a UART) will not operate as expected in deep-sleep mode due to the clock rate change; these peripherals must either be reconfigured upon entry to and exit from deep-sleep mode, or simply not enabled in deep-sleep mode.

This driver is contained in `driverlib/sysctl.c`, with `driverlib/sysctl.h` containing the API definitions for use by applications.

14.2 API Functions

Functions

- `__asm (" .sect \".text:SysCtlDelay\" \n\" .clink \n\" .thumbfunc SysCtlDelay \n\" .thumb \n\" .align 4 \n\" .global SysCtlDelay \n\" SysCtlDelay: \n\" subs r0, #1 \n\" bne.n SysCtlDelay \n\" bx lr \n\" )`
- void `SysCtlC28Disable` (void)
- void `SysCtlC28Enable` (void)
- unsigned long `SysCtlClockConfigGet` (void)
- void `SysCtlClockConfigSet` (unsigned long ulConfig)
- void `SysCtlClockDividersSet` (unsigned long SysDivSelReq, unsigned long M3ClkDivReq)
- unsigned long `SysCtlClockGet` (unsigned long ulClockIn)
- tBoolean `SysCtlClockPllConfig` (unsigned long ClkSrcReq, unsigned long PllMult)
- void `SysCtlDeepSleep` (void)
- void `SysCtlEnableAERRNMI` (void)
- unsigned long `SysCtlFlashSizeGet` (void)
- unsigned long `SysCtlGetNmiConfig` (void)
- unsigned long `SysCtlGetNmiFlagStatus` (void)
- tBoolean `SysCtlGetNmiInterruptStatus` (void)
- void `SysCtlGPIOAHBDisable` (unsigned long ulGPIOPeripheral)
- void `SysCtlGPIOAHBEnable` (unsigned long ulGPIOPeripheral)
- void `SysCtlHoldSubSystemInReset` (unsigned long ulResetConf)
- void `SysCtlNmiFlgClr` (unsigned long ulNmis)
- void `SysCtlNmiFlgClrAll` (void)
- tBoolean `SysCtlNmisSet` (unsigned long ulNmis)
- void `SysCtlPeripheralClockGating` (tBoolean bEnable)
- void `SysCtlPeripheralDeepSleepDisable` (unsigned long ulPeripheral)
- void `SysCtlPeripheralDeepSleepEnable` (unsigned long ulPeripheral)
- void `SysCtlPeripheralDisable` (unsigned long ulPeripheral)
- void `SysCtlPeripheralEnable` (unsigned long ulPeripheral)
- tBoolean `SysCtlPeripheralsEnabled` (unsigned long ulPeripheral)
- tBoolean `SysCtlPeripheralPresent` (unsigned long ulPeripheral)
- void `SysCtlPeripheralReset` (unsigned long ulPeripheral)
- void `SysCtlPeripheralSleepDisable` (unsigned long ulPeripheral)
- void `SysCtlPeripheralSleepEnable` (unsigned long ulPeripheral)
- void `SysCtlPowerOffPLL` (void)
- void `SysCtlReleaseSubSystemFromReset` (unsigned long ulResetConf)
- void `SysCtlReset` (void)
- void `SysCtlResetCauseClear` (unsigned long ulCauses)
- unsigned long `SysCtlResetCauseGet` (void)
- void `SysCtlSleep` (void)
- unsigned long `SysCtlSRAMSizeGet` (void)
- void `SysCtlSubSystemReset` (unsigned long ulResetConf)
- unsigned long `SysCtlUSBPLLConfigGet` (void)

- void [SysCtlUSBPLLConfigSet](#) (unsigned long ulConfig)
- void [SysCtlUSBPLLDisable](#) (void)
- void [SysCtlUSBPLLEnable](#) (void)
- void [SysCtlXPIIClockDividerSet](#) (unsigned long XPIIClkDivReq)

14.2.1 Detailed Description

The SysCtl API is broken up into eight groups of functions: those that provide device information, those that deal with device clocking, those that provide peripheral control, those that deal with the SysCtl interrupt, those that deal with the LDO, those that deal with sleep modes, those that deal with reset reasons, those that deal with the brown-out reset, and those that deal with clock verification timers.

Information about the device is provided by [SysCtlSRAMSizeGet\(\)](#), [SysCtlFlashSizeGet\(\)](#), and [SysCtlPeripheralPresent\(\)](#).

Clocking of the device is configured with [SysCtlClockConfigSet\(\)](#). Information about device clocking is provided by [SysCtlClockGet\(\)](#) and [SysCtlClockConfigGet\(\)](#).

Peripheral enabling and reset are controlled with [SysCtlPeripheralReset\(\)](#), [SysCtlPeripheralEnable\(\)](#), [SysCtlPeripheralDisable\(\)](#), [SysCtlPeripheralIsEnabled\(\)](#), [SysCtlPeripheralSleepEnable\(\)](#), [SysCtlPeripheralSleepDisable\(\)](#), [SysCtlPeripheralDeepSleepEnable\(\)](#), [SysCtlPeripheralDeepSleepDisable\(\)](#), and [SysCtlPeripheralClockGating\(\)](#).

The device is put into sleep modes with [SysCtlSleep\(\)](#) and [SysCtlDeepSleep\(\)](#).

The reset reason is managed with [SysCtlResetCauseGet\(\)](#) and [SysCtlResetCauseClear\(\)](#). A software reset is performed with [SysCtlReset\(\)](#).

14.2.2 Function Documentation

14.2.2.1 `__asm`

Provides a small delay.

Prototype:

```
__asm(".sect \".text:SysCtlDelay\"\\n\".clink\\n\".thumbfunc
SysCtlDelay\\n\".thumb\\n\".align 4\\n\".global SysCtlDelay\\n\"SysCtlDelay:\\n\"subs
r0,
    #1\\n\"bne.n SysCtlDelay\\n\"bx lr\\n")
```

Parameters:

ulCount is the number of delay loop iterations to perform.

Description:

This function provides a means of generating a constant length delay. It is written in assembly to keep the delay consistent across tool chains, avoiding the need to tune the delay based on the tool chain in use.

The loop takes 3 cycles/loop when residing in flash and 4 cycles/loop when residing in RAM.

Returns:

None.

14.2.2.2 SysCtlC28Disable

Disable the Control Subsystem clock.

Prototype:

```
void  
SysCtlC28Disable(void)
```

Description:

Disable the clock to the control subsystem.

Returns:

None.

14.2.2.3 SysCtlC28Enable

Enable the Control Subsystem Clock

Prototype:

```
void  
SysCtlC28Enable(void)
```

Description:

Enable the clock to the control subsystem.

Returns:

None.

14.2.2.4 SysCtlClockConfigGet

Gets the clocking value of device.

Prototype:

```
unsigned long  
SysCtlClockConfigGet(void)
```

Description:

Read the current clock configuration and pass it back in the same format as [SysCtlClockConfigSet\(\)](#).

Returns:

Clock configuration in the following format:

bits 21, 20 of return value represent bit 1, 0 of SYSDIVSEL register

bits 19, 18 of return value represent bit 1, 0 of M3SSDIVSEL register

bits 17, 16 of return value represent bit 1, 0 of XPLLCFG register

bits 9-0 of return value represent bit 9-0 of SYSPLLMULT register
 bit 31 of return value represent bit 0 of SYSPLLCTL register

14.2.2.5 SysCtlClockConfigSet

Configure device clocking

Prototype:

```
void
SysCtlClockConfigSet(unsigned long ulConfig)
```

Parameters:

ulConfig Clock configuration as defined below:
 bits 21, 20 of ulConfig param represent bit 1, 0 of SYSDIVSEL register
 bits 19, 18 of ulConfig param represent bit 1, 0 of M3SSDIVSEL register
 bits 17, 16 of ulConfig param represent bit 1, 0 of XPLLCFG register
 bits 9-0 of ulConfig param represent bit 9-0 of SYSPLLMULT register
 bit 31 of ulConfig param represent bit 0 of SYSPLLCTL register

Description:

Sets up device clocking according to the parameter passed in.

Returns:

None.

14.2.2.6 SysCtlClockDividersSet

Configure SYS CLK and M3SS Clock dividers.

Prototype:

```
void
SysCtlClockDividersSet(unsigned long SysDivSelReq,
                        unsigned long M3ClkDivReq)
```

Parameters:

SysDivSelReq SYSCLK divider value, must be one of:

- SYSCTL_SYSDIV_1
- SYSCTL_SYSDIV_2
- SYSCTL_SYSDIV_4
- SYSCTL_SYSDIV_8
- SYSCTL_SYSDIV_NO_CHANGE

M3ClkDivReq M3SSCLK divider value, must be one of:

- SYSCTL_M3SSDIV_1
- SYSCTL_M3SSDIV_2
- SYSCTL_M3SSDIV_4
- SYSCTL_M3SSDIV_NO_CHANGE

Description:

Sets up device clocking according to the parameter passed in.

Returns:

None.

14.2.2.7 SysCtlClockGet

Returns the current M3 Clock frequency (in Hertz)

Prototype:

```
unsigned long  
SysCtlClockGet(unsigned long ulClockIn)
```

Parameters:

ulClockIn Frequency of the external clock

Returns:

M3 System clock frequency (in Hertz)

14.2.2.8 tBoolean SysCtlClockPllConfig (unsigned long *ClkSrcReq*, unsigned long *PllMult*)

Configure PLL

Parameters:

ClkSrcReq Must be one of:

- SYSCTL_USE_PLL
- SYSCTL_USE_OSC

PllMult bits 0-6 must be PLLIMULT and bit 8:9 must be PLLFMULT; bits 7, 31:10 must be 0 (reserved)

Description:

Sets up PLL and associated multipliers according to the parameter passed in.

Note:

Refer to the device's errata for more information about the process performed in this function.

Returns:

true if the PLL is configured as requested and **false** if it is not.

14.2.2.9 SysCtlDeepSleep

Puts the processor into deep-sleep mode.

Prototype:

```
void  
SysCtlDeepSleep(void)
```

Description:

This function places the processor into deep-sleep mode; it will not return until the processor returns to run mode. The peripherals that are enabled via [SysCtlPeripheralDeepSleepEnable\(\)](#) continue to operate and can wake up the processor (if automatic clock gating is enabled with [SysCtlPeripheralClockGating\(\)](#), otherwise all peripherals continue to operate).

Returns:

None.

14.2.2.10 SysCtlEnableAERRNMI

Enable the AERR NMI.

Prototype:

```
void  
SysCtlEnableAERRNMI(void)
```

Description:

Enable the AERR NMI in MNMICFG register.

Returns:

None.

14.2.2.11 SysCtlFlashSizeGet

Gets the size of the flash.

Prototype:

```
unsigned long  
SysCtlFlashSizeGet(void)
```

Description:

This function determines the size of the flash on the Concerto device.

Returns:

The total number of bytes of flash.

14.2.2.12 SysCtlGetNmiConfig

Read the current value of NMI Config register.

Prototype:

```
unsigned long  
SysCtlGetNmiConfig(void)
```

Description:

Read the current value of NMI Config register.

Returns:

- Value of MNMICFG register as defined in the user guide

14.2.2.13 SysCtlGetNmiFlagStatus

Read NMI Flags.

Prototype:

```
unsigned long  
SysCtlGetNmiFlagStatus(void)
```

Description:

Read the current state of individual NMI interrupts

Returns:

Value of MNMIFLG register.

14.2.2.14 SysCtlGetNmiInterruptStatus

Read NMI interrupts.

Prototype:

```
tBoolean  
SysCtlGetNmiInterruptStatus(void)
```

Description:

Read the current state of NMI interrupt.

Returns:

TRUE if NMI interrupt is triggered, **FALSE** if not.

14.2.2.15 SysCtlGPIOAHBDisable

Disable GPIO port access from AHB memory space

Prototype:

```
void  
SysCtlGPIOAHBDisable(unsigned long ulGPIOPeripheral)
```

Parameters:

ulGPIOPeripheral GPIO port to remove from the AHB bus, can be SYSCTL_PERIPH_GPIOA to SYSCTL_PERIPH_GPIOJ

Description:

Switch accesses to a GPIO address space from the AHB bus to the APB bus.

Returns:

None.

14.2.2.16 SysCtlGPIOAHBEnable

Enable GPIO port to be addresses from AHB memory space

Prototype:

```
void  
SysCtlGPIOAHBEnable(unsigned long ulGPIOPeripheral)
```

Parameters:

ulGPIOPeripheral GPIO port to put on the AHB bus, can be SYSCTL_PERIPH_GPIOA to SYSCTL_PERIPH_GPIOJ.

Description:

Switch accesses to a GPIO address space from the APB bus to the AHB bus.

Returns:

None.

14.2.2.17 SysCtlHoldSubSystemInReset

Hold Control system and/or Analog subsystem in reset

Prototype:

```
void  
SysCtlHoldSubSystemInReset(unsigned long ulResetConf)
```

Parameters:

ulResetConf System to be held in reset
bit 0 - Hold Control system in reset
bit 1 - Hold Analog System in reset

Description:

Hold the selected subsystem(s) in reset indefinitely.

Returns:

None.

14.2.2.18 SysCtlNmiFlgClr

Function to clear individual NMI interrupts

Prototype:

```
void  
SysCtlNmiFlgClr(unsigned long ulNmis)
```

Parameters:

ulNmis Bit mask of the NMI interrupts that user wants to clear. The bit format of this parameter is same as of the MNMIFLG register.

Description:

Clear NMI interrupt flags that correspond with the passed in bit mask.

Returns:

None.

14.2.2.19 SysCtlNmiFlgClrAll

Clear all the NMI that are set.

Prototype:

```
void  
SysCtlNmiFlgClrAll(void)
```

Description:

Clear every NMI flag that is currently set.

Returns:

None.

14.2.2.20 SysCtlNmIsSet

Check if the individual NMI interrupts are set

Prototype:

```
tBoolean  
SysCtlNmIsSet(unsigned long ulNmIs)
```

Parameters:

ulNmIs Bit mask of the NMI interrupts that user wants to check if set. The bit format of this parameter is same as of the MNMIFLG register.

Description:

Check if interrupt flags corresponding to the passed in bit mask are asserted.

Returns:

TRUE if any of the NMI asked for in the parameter bit mask is set. **FALSE** if none of the NMI requested in the parameter bit mask are set.

14.2.2.21 SysCtlPeripheralClockGating

Controls peripheral clock gating in sleep and deep-sleep mode.

Prototype:

```
void  
SysCtlPeripheralClockGating(tBoolean bEnable)
```

Parameters:

bEnable is a boolean that is **true** if the sleep and deep-sleep peripheral configuration should be used and **false** if not.

Description:

This function controls how peripherals are clocked when the processor goes into sleep or deep-sleep mode. By default, the peripherals are clocked the same as in run mode; if peripheral clock gating is enabled they are clocked according to the configuration set by [SysCtlPeripheralSleepEnable\(\)](#), [SysCtlPeripheralSleepDisable\(\)](#), [SysCtlPeripheralDeepSleepEnable\(\)](#), and [SysCtlPeripheralDeepSleepDisable\(\)](#).

Returns:

None.

14.2.2.22 SysCtlPeripheralDeepSleepDisable

Disables a peripheral in deep-sleep mode.

Prototype:

```
void
SysCtlPeripheralDeepSleepDisable(unsigned long ulPeripheral)
```

Parameters:

ulPeripheral is the peripheral to disable in deep-sleep mode.

Description:

This function causes a peripheral to stop operating when the processor goes into deep-sleep mode. Disabling peripherals while in deep-sleep mode helps to lower the current draw of the device, and can keep peripherals that require a particular clock frequency from operating when the clock changes as a result of entering deep-sleep mode. If enabled (via [SysCtlPeripheralEnable\(\)](#)), the peripheral will automatically resume operation when the processor leaves deep-sleep mode, maintaining its entire state from before deep-sleep mode was entered.

Deep-sleep mode clocking of peripherals must be enabled via [SysCtlPeripheralClockGating\(\)](#); if disabled, the peripheral deep-sleep mode configuration is maintained but has no effect when deep-sleep mode is entered.

The *ulPeripheral* parameter must be only one of the following values:

SYSCTL_PERIPH_CAN0,	SYSCTL_PERIPH_CAN1,	SYSCTL_PERIPH_EP10,
SYSCTL_PERIPH_ETH,	SYSCTL_PERIPH_GPIOA,	SYSCTL_PERIPH_GPIOB,
SYSCTL_PERIPH_GPIOC,	SYSCTL_PERIPH_GPIOD,	SYSCTL_PERIPH_GPIOE,
SYSCTL_PERIPH_GPIOF,	SYSCTL_PERIPH_GPIOG,	SYSCTL_PERIPH_GPIOH,
SYSCTL_PERIPH_GPIOJ,	SYSCTL_PERIPH_I2C0,	SYSCTL_PERIPH_I2C1,
SYSCTL_PERIPH_IEEE1588,	SYSCTL_PERIPH_SSI0,	SYSCTL_PERIPH_SSI1,
SYSCTL_PERIPH_SSI2,	SYSCTL_PERIPH_SSI3,	SYSCTL_PERIPH_TIMER0,
SYSCTL_PERIPH_TIMER1,	SYSCTL_PERIPH_TIMER2,	SYSCTL_PERIPH_TIMER3,
SYSCTL_PERIPH_UART0,	SYSCTL_PERIPH_UART1,	SYSCTL_PERIPH_UART2,
SYSCTL_PERIPH_UART3,	SYSCTL_PERIPH_UART4,	SYSCTL_PERIPH_UDMA,
SYSCTL_PERIPH_USB0,	SYSCTL_PERIPH_WDOG0,	SYSCTL_PERIPH_WDOG1,
SYSCTL_PERIPH_GPIOK,	SYSCTL_PERIPH_GPIOL,	SYSCTL_PERIPH_GPIOM,
SYSCTL_PERIPH_GPION,	SYSCTL_PERIPH_GPIOP,	SYSCTL_PERIPH_GPIQ,
SYSCTL_PERIPH_GPIOR,	SYSCTL_PERIPH_GPIOS,	

Returns:

None.

14.2.2.23 SysCtlPeripheralDeepSleepEnable

Enables a peripheral in deep-sleep mode.

Prototype:

```
void  
SysCtlPeripheralDeepSleepEnable(unsigned long ulPeripheral)
```

Parameters:

ulPeripheral is the peripheral to enable in deep-sleep mode.

Description:

This function allows a peripheral to continue operating when the processor goes into deep-sleep mode. Since the clocking configuration of the device may change, not all peripherals can safely continue operating while the processor is in sleep mode. Those that must run at a particular frequency (such as a UART) will not work as expected if the clock changes. It is the responsibility of the caller to make sensible choices.

Deep-sleep mode clocking of peripherals must be enabled via [SysCtlPeripheralClockGating\(\)](#); if disabled, the peripheral deep-sleep mode configuration is maintained but has no effect when deep-sleep mode is entered.

The *ulPeripheral* parameter must be only one of the following values:

SYSCTL_PERIPH_CAN0,	SYSCTL_PERIPH_CAN1,	SYSCTL_PERIPH_EPI0,
SYSCTL_PERIPH_ETH,	SYSCTL_PERIPH_GPIOA,	SYSCTL_PERIPH_GPIOB,
SYSCTL_PERIPH_GPIOC,	SYSCTL_PERIPH_GPIOD,	SYSCTL_PERIPH_GPIOE,
SYSCTL_PERIPH_GPIOF,	SYSCTL_PERIPH_GPIOG,	SYSCTL_PERIPH_GPIOH,
SYSCTL_PERIPH_GPIOJ,	SYSCTL_PERIPH_I2C0,	SYSCTL_PERIPH_I2C1,
SYSCTL_PERIPH_IEEE1588,	SYSCTL_PERIPH_SSI0,	SYSCTL_PERIPH_SSI1,
SYSCTL_PERIPH_SSI2,	SYSCTL_PERIPH_SSI3,	SYSCTL_PERIPH_TIMER0,
SYSCTL_PERIPH_TIMER1,	SYSCTL_PERIPH_TIMER2,	SYSCTL_PERIPH_TIMER3,
SYSCTL_PERIPH_UART0,	SYSCTL_PERIPH_UART1,	SYSCTL_PERIPH_UART2,
SYSCTL_PERIPH_UART3,	SYSCTL_PERIPH_UART4,	SYSCTL_PERIPH_UDMA,
SYSCTL_PERIPH_USB0,	SYSCTL_PERIPH_WDOG0,	SYSCTL_PERIPH_WDOG1,
SYSCTL_PERIPH_GPIOK,	SYSCTL_PERIPH_GPIOL,	SYSCTL_PERIPH_GPIOM,
SYSCTL_PERIPH_GPION,	SYSCTL_PERIPH_GPIOP,	SYSCTL_PERIPH_GPIOQ,
SYSCTL_PERIPH_GPIOR,	or SYSCTL_PERIPH_GPIOS.	

Returns:

None.

14.2.2.24 SysCtlPeripheralDisable

Disables a peripheral.

Prototype:

```
void  
SysCtlPeripheralDisable(unsigned long ulPeripheral)
```

Parameters:

ulPeripheral is the peripheral to disable.

Description:

Peripherals are disabled with this function. Once disabled, they will not operate or respond to register reads/writes.

The *ulPeripheral* parameter must be only one of the following values:

SYSCTL_PERIPH_CAN0,	SYSCTL_PERIPH_CAN1,	SYSCTL_PERIPH_EPIO,
SYSCTL_PERIPH_ETH,	SYSCTL_PERIPH_GPIOA,	SYSCTL_PERIPH_GPIOB,
SYSCTL_PERIPH_GPIOC,	SYSCTL_PERIPH_GPIOD,	SYSCTL_PERIPH_GPIOE,
SYSCTL_PERIPH_GPIOF,	SYSCTL_PERIPH_GPIOG,	SYSCTL_PERIPH_GPIOH,
SYSCTL_PERIPH_GPIOJ,	SYSCTL_PERIPH_I2C0,	SYSCTL_PERIPH_I2C1,
SYSCTL_PERIPH_IEEE1588,	SYSCTL_PERIPH_SSI0,	SYSCTL_PERIPH_SSI1,
SYSCTL_PERIPH_SSI2,	SYSCTL_PERIPH_SSI3,	SYSCTL_PERIPH_TIMER0,
SYSCTL_PERIPH_TIMER1,	SYSCTL_PERIPH_TIMER2,	SYSCTL_PERIPH_TIMER3,
SYSCTL_PERIPH_UART0,	SYSCTL_PERIPH_UART1,	SYSCTL_PERIPH_UART2,
SYSCTL_PERIPH_UART3,	SYSCTL_PERIPH_UART4,	SYSCTL_PERIPH_UDMA,
SYSCTL_PERIPH_USB0,	SYSCTL_PERIPH_WDOG0,	SYSCTL_PERIPH_WDOG1,
SYSCTL_PERIPH_GPIOK,	SYSCTL_PERIPH_GPIOL,	SYSCTL_PERIPH_GPIOM,
SYSCTL_PERIPH_GPION,	SYSCTL_PERIPH_GPIOP,	SYSCTL_PERIPH_GPIOQ,
SYSCTL_PERIPH_GPIOR, or SYSCTL_PERIPH_GPIOS.		

Returns:

None.

14.2.2.25 SysCtlPeripheralEnable

Enables a peripheral.

Prototype:

```
void
SysCtlPeripheralEnable(unsigned long ulPeripheral)
```

Parameters:

ulPeripheral is the peripheral to enable.

Description:

Peripherals are enabled with this function. At power-up, all peripherals are disabled; they must be enabled in order to operate or respond to register reads/writes.

The *ulPeripheral* parameter must be only one of the following values:

SYSCTL_PERIPH_CAN0,	SYSCTL_PERIPH_CAN1,	SYSCTL_PERIPH_EPIO,
SYSCTL_PERIPH_ETH,	SYSCTL_PERIPH_GPIOA,	SYSCTL_PERIPH_GPIOB,
SYSCTL_PERIPH_GPIOC,	SYSCTL_PERIPH_GPIOD,	SYSCTL_PERIPH_GPIOE,
SYSCTL_PERIPH_GPIOF,	SYSCTL_PERIPH_GPIOG,	SYSCTL_PERIPH_GPIOH,
SYSCTL_PERIPH_GPIOJ,	SYSCTL_PERIPH_I2C0,	SYSCTL_PERIPH_I2C1,
SYSCTL_PERIPH_IEEE1588,	SYSCTL_PERIPH_SSI0,	SYSCTL_PERIPH_SSI1,
SYSCTL_PERIPH_SSI2,	SYSCTL_PERIPH_SSI3,	SYSCTL_PERIPH_TIMER0,
SYSCTL_PERIPH_TIMER1,	SYSCTL_PERIPH_TIMER2,	SYSCTL_PERIPH_TIMER3,
SYSCTL_PERIPH_UART0,	SYSCTL_PERIPH_UART1,	SYSCTL_PERIPH_UART2,
SYSCTL_PERIPH_UART3,	SYSCTL_PERIPH_UART4,	SYSCTL_PERIPH_UDMA,
SYSCTL_PERIPH_USB0,	SYSCTL_PERIPH_WDOG0,	SYSCTL_PERIPH_WDOG1,
SYSCTL_PERIPH_GPIOK,	SYSCTL_PERIPH_GPIOL,	SYSCTL_PERIPH_GPIOM,
SYSCTL_PERIPH_GPION,	SYSCTL_PERIPH_GPIOP,	SYSCTL_PERIPH_GPIOQ,
SYSCTL_PERIPH_GPIOR, or SYSCTL_PERIPH_GPIOS.		

Note:

It takes five clock cycles after the write to enable a peripheral before the the peripheral is actually enabled. During this time, attempts to access the peripheral will result in a bus fault. Care should be taken to ensure that the peripheral is not accessed during this brief time period.

Returns:

None.

14.2.2.26 SysCtlPeripheralIsEnabled

Checks if a peripheral is enabled.

Prototype:

```
tBoolean  
SysCtlPeripheralIsEnabled(unsigned long ulPeripheral)
```

Parameters:

ulPeripheral is the peripheral to check on.

Description:

Peripheral clock gating can be checked during runtime with this function.

The *ulPeripheral* parameter must be only one of the following values:

SYSCTL_PERIPH_CAN0,	SYSCTL_PERIPH_CAN1,	SYSCTL_PERIPH_EPI0,
SYSCTL_PERIPH_ETH,	SYSCTL_PERIPH_GPIOA,	SYSCTL_PERIPH_GPIOB,
SYSCTL_PERIPH_GPIOC,	SYSCTL_PERIPH_GPIOD,	SYSCTL_PERIPH_GPIOE,
SYSCTL_PERIPH_GPIOF,	SYSCTL_PERIPH_GPIOG,	SYSCTL_PERIPH_GPIOH,
SYSCTL_PERIPH_GPIOJ,	SYSCTL_PERIPH_I2C0,	SYSCTL_PERIPH_I2C1,
SYSCTL_PERIPH_IEEE1588,	SYSCTL_PERIPH_SSI0,	SYSCTL_PERIPH_SSI1,
SYSCTL_PERIPH_SSI2,	SYSCTL_PERIPH_SSI3,	SYSCTL_PERIPH_TIMER0,
SYSCTL_PERIPH_TIMER1,	SYSCTL_PERIPH_TIMER2,	SYSCTL_PERIPH_TIMER3,
SYSCTL_PERIPH_UART0,	SYSCTL_PERIPH_UART1,	SYSCTL_PERIPH_UART2,
SYSCTL_PERIPH_UART3,	SYSCTL_PERIPH_UART4,	SYSCTL_PERIPH_UDMA,
SYSCTL_PERIPH_USB0,	SYSCTL_PERIPH_WDOG0,	SYSCTL_PERIPH_WDOG1,
SYSCTL_PERIPH_GPIOK,	SYSCTL_PERIPH_GPIOL,	SYSCTL_PERIPH_GPIOM,
SYSCTL_PERIPH_GPION,	SYSCTL_PERIPH_GPIOP,	SYSCTL_PERIPH_GPIOQ,
SYSCTL_PERIPH_GPIOR, or SYSCTL_PERIPH_GPIOS.		

Returns:

True if the peripheral is enabled, False if disabled.

14.2.2.27 SysCtlPeripheralPresent

Determines if a peripheral is present.

Prototype:

```
tBoolean  
SysCtlPeripheralPresent(unsigned long ulPeripheral)
```

Parameters:

ulPeripheral is the peripheral in question.

Description:

Determines if a particular peripheral is present in the device. Each member of the Concerto family has a different peripheral set; this will determine which are present on this device.

The *ulPeripheral* parameter must be only one of the following values:

SYSCTL_PERIPH_CAN0,	SYSCTL_PERIPH_CAN1,	SYSCTL_PERIPH_EPIO,
SYSCTL_PERIPH_ETH,	SYSCTL_PERIPH_GPIOA,	SYSCTL_PERIPH_GPIOB,
SYSCTL_PERIPH_GPIOC,	SYSCTL_PERIPH_GPIOD,	SYSCTL_PERIPH_GPIOE,
SYSCTL_PERIPH_GPIOF,	SYSCTL_PERIPH_GPIOG,	SYSCTL_PERIPH_GPIOH,
SYSCTL_PERIPH_GPIOJ,	SYSCTL_PERIPH_I2C0,	SYSCTL_PERIPH_I2C1,
SYSCTL_PERIPH_IEEE1588,	SYSCTL_PERIPH_SSI0,	SYSCTL_PERIPH_SSI1,
SYSCTL_PERIPH_SSI2,	SYSCTL_PERIPH_SSI3,	SYSCTL_PERIPH_TIMER0,
SYSCTL_PERIPH_TIMER1,	SYSCTL_PERIPH_TIMER2,	SYSCTL_PERIPH_TIMER3,
SYSCTL_PERIPH_UART0,	SYSCTL_PERIPH_UART1,	SYSCTL_PERIPH_UART2,
SYSCTL_PERIPH_UART3,	SYSCTL_PERIPH_UART4,	SYSCTL_PERIPH_UDMA,
SYSCTL_PERIPH_USB0,	SYSCTL_PERIPH_WDOG0,	SYSCTL_PERIPH_WDOG1,
SYSCTL_PERIPH_GPIOK,	SYSCTL_PERIPH_GPIOL,	SYSCTL_PERIPH_GPIOM,
SYSCTL_PERIPH_GPION,	SYSCTL_PERIPH_GPIOP,	SYSCTL_PERIPH_GPIOQ,
SYSCTL_PERIPH_GPIOR, or SYSCTL_PERIPH_GPIOS.		

Returns:

Returns **true** if the specified peripheral is present and **false** if it is not.

14.2.2.28 SysCtlPeripheralReset

Performs a software reset of a peripheral.

Prototype:

```
void
SysCtlPeripheralReset(unsigned long ulPeripheral)
```

Parameters:

ulPeripheral is the peripheral to reset.

Description:

This function performs a software reset of the specified peripheral. An individual peripheral reset signal is asserted for a brief period and then deasserted, leaving the peripheral in a operating state but in its reset condition.

The *ulPeripheral* parameter must be only one of the following values:

SYSCTL_PERIPH_CAN0,	SYSCTL_PERIPH_CAN1,	SYSCTL_PERIPH_EPIO,
SYSCTL_PERIPH_ETH,	SYSCTL_PERIPH_GPIOA,	SYSCTL_PERIPH_GPIOB,
SYSCTL_PERIPH_GPIOC,	SYSCTL_PERIPH_GPIOD,	SYSCTL_PERIPH_GPIOE,
SYSCTL_PERIPH_GPIOF,	SYSCTL_PERIPH_GPIOG,	SYSCTL_PERIPH_GPIOH,
SYSCTL_PERIPH_GPIOJ,	SYSCTL_PERIPH_I2C0,	SYSCTL_PERIPH_I2C1,
SYSCTL_PERIPH_IEEE1588,	SYSCTL_PERIPH_SSI0,	SYSCTL_PERIPH_SSI1,
SYSCTL_PERIPH_SSI2,	SYSCTL_PERIPH_SSI3,	SYSCTL_PERIPH_TIMER0,
SYSCTL_PERIPH_TIMER1,	SYSCTL_PERIPH_TIMER2,	SYSCTL_PERIPH_TIMER3,
SYSCTL_PERIPH_UART0,	SYSCTL_PERIPH_UART1,	SYSCTL_PERIPH_UART2,
SYSCTL_PERIPH_UART3,	SYSCTL_PERIPH_UART4,	SYSCTL_PERIPH_UDMA,
SYSCTL_PERIPH_USB0,	SYSCTL_PERIPH_WDOG0,	SYSCTL_PERIPH_WDOG1,
SYSCTL_PERIPH_GPIOK,	SYSCTL_PERIPH_GPIOL,	SYSCTL_PERIPH_GPIOM,

`SYSCTL_PERIPH_GPION`, `SYSCTL_PERIPH_GPIOP`, `SYSCTL_PERIPH_GPIOQ`,
`SYSCTL_PERIPH_GPIOR`, or `SYSCTL_PERIPH_GPIOS`.

Returns:

None.

14.2.2.29 SysCtlPeripheralSleepDisable

Disables a peripheral in sleep mode.

Prototype:

```
void  
SysCtlPeripheralSleepDisable(unsigned long ulPeripheral)
```

Parameters:

ulPeripheral is the peripheral to disable in sleep mode.

Description:

This function causes a peripheral to stop operating when the processor goes into sleep mode. Disabling peripherals while in sleep mode helps to lower the current draw of the device. If enabled (via [SysCtlPeripheralEnable\(\)](#)), the peripheral will automatically resume operation when the processor leaves sleep mode, maintaining its entire state from before sleep mode was entered.

Sleep mode clocking of peripherals must be enabled via [SysCtlPeripheralClockGating\(\)](#); if disabled, the peripheral sleep mode configuration is maintained but has no effect when sleep mode is entered.

The *ulPeripheral* parameter must be only one of the following values:

<code>SYSCTL_PERIPH_CAN0</code> ,	<code>SYSCTL_PERIPH_CAN1</code> ,	<code>SYSCTL_PERIPH_EPI0</code> ,
<code>SYSCTL_PERIPH_ETH</code> ,	<code>SYSCTL_PERIPH_GPIOA</code> ,	<code>SYSCTL_PERIPH_GPIOB</code> ,
<code>SYSCTL_PERIPH_GPIOC</code> ,	<code>SYSCTL_PERIPH_GPIOD</code> ,	<code>SYSCTL_PERIPH_GPIOE</code> ,
<code>SYSCTL_PERIPH_GPIOF</code> ,	<code>SYSCTL_PERIPH_GPIOG</code> ,	<code>SYSCTL_PERIPH_GPIOH</code> ,
<code>SYSCTL_PERIPH_GPIOJ</code> ,	<code>SYSCTL_PERIPH_I2C0</code> ,	<code>SYSCTL_PERIPH_I2C1</code> ,
<code>SYSCTL_PERIPH_IEEE1588</code> ,	<code>SYSCTL_PERIPH_SSI0</code> ,	<code>SYSCTL_PERIPH_SSI1</code> ,
<code>SYSCTL_PERIPH_SSI2</code> ,	<code>SYSCTL_PERIPH_SSI3</code> ,	<code>SYSCTL_PERIPH_TIMER0</code> ,
<code>SYSCTL_PERIPH_TIMER1</code> ,	<code>SYSCTL_PERIPH_TIMER2</code> ,	<code>SYSCTL_PERIPH_TIMER3</code> ,
<code>SYSCTL_PERIPH_UART0</code> ,	<code>SYSCTL_PERIPH_UART1</code> ,	<code>SYSCTL_PERIPH_UART2</code> ,
<code>SYSCTL_PERIPH_UART3</code> ,	<code>SYSCTL_PERIPH_UART4</code> ,	<code>SYSCTL_PERIPH_UDMA</code> ,
<code>SYSCTL_PERIPH_USB0</code> ,	<code>SYSCTL_PERIPH_WDOG0</code> ,	<code>SYSCTL_PERIPH_WDOG1</code> ,
<code>SYSCTL_PERIPH_GPIOK</code> ,	<code>SYSCTL_PERIPH_GPIOL</code> ,	<code>SYSCTL_PERIPH_GPIOM</code> ,
<code>SYSCTL_PERIPH_GPION</code> ,	<code>SYSCTL_PERIPH_GPIOP</code> ,	<code>SYSCTL_PERIPH_GPIOQ</code> ,
<code>SYSCTL_PERIPH_GPIOR</code> , or <code>SYSCTL_PERIPH_GPIOS</code> .		

Returns:

None.

14.2.2.30 SysCtlPeripheralSleepEnable

Enables a peripheral in sleep mode.

Prototype:

```
void
SysCtlPeripheralSleepEnable(unsigned long ulPeripheral)
```

Parameters:

ulPeripheral is the peripheral to enable in sleep mode.

Description:

This function allows a peripheral to continue operating when the processor goes into sleep mode. Since the clocking configuration of the device does not change, any peripheral can safely continue operating while the processor is in sleep mode, and can therefore wake the processor from sleep mode.

Sleep mode clocking of peripherals must be enabled via [SysCtlPeripheralClockGating\(\)](#); if disabled, the peripheral sleep mode configuration is maintained but has no effect when sleep mode is entered.

The *ulPeripheral* parameter must be only one of the following values:

SYSCTL_PERIPH_CAN0,	SYSCTL_PERIPH_CAN1,	SYSCTL_PERIPH_EPI0,
SYSCTL_PERIPH_ETH,	SYSCTL_PERIPH_GPIOA,	SYSCTL_PERIPH_GPIOB,
SYSCTL_PERIPH_GPIOC,	SYSCTL_PERIPH_GPIOD,	SYSCTL_PERIPH_GPIOE,
SYSCTL_PERIPH_GPIOF,	SYSCTL_PERIPH_GPIOG,	SYSCTL_PERIPH_GPIOH,
SYSCTL_PERIPH_GPIOJ,	SYSCTL_PERIPH_I2C0,	SYSCTL_PERIPH_I2C1,
SYSCTL_PERIPH_IEEE1588,	SYSCTL_PERIPH_SSI0,	SYSCTL_PERIPH_SSI1,
SYSCTL_PERIPH_SSI2,	SYSCTL_PERIPH_SSI3,	SYSCTL_PERIPH_TIMER0,
SYSCTL_PERIPH_TIMER1,	SYSCTL_PERIPH_TIMER2,	SYSCTL_PERIPH_TIMER3,
SYSCTL_PERIPH_UART0,	SYSCTL_PERIPH_UART1,	SYSCTL_PERIPH_UART2,
SYSCTL_PERIPH_UART3,	SYSCTL_PERIPH_UART4,	SYSCTL_PERIPH_UDMA,
SYSCTL_PERIPH_USB0,	SYSCTL_PERIPH_WDOG0,	SYSCTL_PERIPH_WDOG1,
SYSCTL_PERIPH_GPIOK,	SYSCTL_PERIPH_GPIOL,	SYSCTL_PERIPH_GPIOM,
SYSCTL_PERIPH_GPION,	SYSCTL_PERIPH_GPIOP,	SYSCTL_PERIPH_GPIOQ,
SYSCTL_PERIPH_GPIOR, or SYSCTL_PERIPH_GPIOS.		

Returns:

None.

14.2.2.31 SysCtlPowerOffPLL

Power OFF PLL

Prototype:

```
void
SysCtlPowerOffPLL(void)
```

Description:

This function turns OFF PLL completely

Returns:

None.

14.2.2.32 SysCtlReleaseSubSystemFromReset

Release Control system and/or Analog subsystem from reset

Prototype:

```
void  
SysCtlReleaseSubSystemFromReset(unsigned long ulResetConf)
```

Parameters:

ulResetConf System to be released from reset
bit 0 - Release Control system from reset
bit 1 - Release Analog System from reset

Description:

Release the selected subsystem(s) from reset immediately.

Returns:

- None

14.2.2.33 SysCtlReset

Resets the device.

Prototype:

```
void  
SysCtlReset(void)
```

Description:

This function will perform a software reset of the entire device. The processor and all peripherals will be reset and all device registers will return to their default values (with the exception of the reset cause register, which will maintain its current value but have the software reset bit set as well).

Returns:

This function does not return.

14.2.2.34 SysCtlResetCauseClear

Clears reset reasons.

Prototype:

```
void  
SysCtlResetCauseClear(unsigned long ulCauses)
```

Parameters:

ulCauses are the reset causes to be cleared; must be a logical OR of **SYSCTL_CAUSE_LDO**, **SYSCTL_CAUSE_SW**, **SYSCTL_CAUSE_WDOG**, **SYSCTL_CAUSE_POR**, and/or **SYSCTL_CAUSE_EXT**.

Description:

This function clears the specified sticky reset reasons. Once cleared, another reset for the same reason can be detected, and a reset for a different reason can be distinguished (instead of having two reset causes set). If the reset reason is used by an application, all reset causes should be cleared after they are retrieved with [SysCtlResetCauseGet\(\)](#).

Returns:

None.

14.2.2.35 SysCtlResetCauseGet

Gets the reason for a reset.

Prototype:

```
unsigned long  
SysCtlResetCauseGet (void)
```

Description:

This function will return the reason(s) for a reset. Since the reset reasons are sticky until either cleared by software or an external reset, multiple reset reasons may be returned if multiple resets have occurred. The reset reason will be a logical OR of **SYSCTL_CAUSE_LDO**, **SYSCTL_CAUSE_SW**, **SYSCTL_CAUSE_WDOG**, **SYSCTL_CAUSE_POR**, and/or **SYSCTL_CAUSE_EXT**.

Returns:

Returns the reason(s) for a reset.

14.2.2.36 SysCtlSleep

Puts the processor into sleep mode.

Prototype:

```
void  
SysCtlSleep (void)
```

Description:

This function places the processor into sleep mode; it will not return until the processor returns to run mode. The peripherals that are enabled via [SysCtlPeripheralSleepEnable\(\)](#) continue to operate and can wake up the processor (if automatic clock gating is enabled with [SysCtlPeripheralClockGating\(\)](#), otherwise all peripherals continue to operate).

Returns:

None.

14.2.2.37 SysCtlSRAMSizeGet

Gets the size of the shared RAM

Prototype:

```
unsigned long  
SysCtlSRAMSizeGet (void)
```

Description:

This function determines the size of the shared RAM on the Concerto devices.

Returns:

The total number of bytes of shared RAM.

14.2.2.38 SysCtlSubSystemReset

Reset Control and/or analog system

Prototype:

```
void  
SysCtlSubSystemReset(unsigned long ulResetConf)
```

Parameters:

ulResetConf System to be reset
bit 0 - Control system reset
bit 1 - Analog system reset

Description:

Reset the selected subsystem(s).

Returns:

- None

14.2.2.39 SysCtlUSBPLLConfigGet

Get the configuration of the USBPLL.

Prototype:

```
unsigned long  
SysCtlUSBPLLConfigGet(void)
```

Description:

Read the configuration of the USBPLL.

Returns:

Configuration of the USBPLL in the following format:
bits 7-0 of return value represent bits 7-0 of UPLLMULT register
bits 9-8 of return value represent bits 9-8 of UPLLMULT register
bit 31 of return value represents bit 0 of UPLLCTL register
bit 30 of return value represents bit 1 of UPLLCTL register

14.2.2.40 SysCtlUSBPLLConfigSet

Set the configuration of the USBPLL.

Prototype:

```
void  
SysCtlUSBPLLConfigSet(unsigned long ulConfig)
```

Description:

Configure the USBPLL for operation with passed in configuration.

Parameters:

ulConfig unsigned long integer value formatted as below:
bits 7-0 of *ulConfig* represent bits 7-0 of UPLLMULT register
bits 9-8 of *ulConfig* represent bits 9-8 of UPLLMULT register
bit 31 of *ulConfig* represents bit 0 of UPLLCTL register
bit 30 of *ulConfig* represents bit 1 of UPLLCTL register

Note:

Refer to the device's errata for more information about the process performed in this function.

Returns:

None.

14.2.2.41 SysCtlUSBPLLDisable

Power down the USB PLL.

Prototype:

```
void  
SysCtlUSBPLLDisable(void)
```

Description:

Disable the USB controller's PLL. The USB registers are still accessible, but the physical layer will no longer function.

Returns:

None.

14.2.2.42 SysCtlUSBPLLEnable

Power up the USB PLL.

Prototype:

```
void  
SysCtlUSBPLLEnable(void)
```

Description:

Enable the USB controller's PLL. This call is necessary before connecting to any external devices.

Returns:

None.

14.2.2.43 SysCtlXPLLCLKDividerSet

Configure XPLL CLK OUT divider.

Prototype:

```
void  
SysCtlXPLLCLKDividerSet(unsigned long XPLLClkDivReq)
```

Parameters:

XPLLClkDivReq XPLL CLK OUT divider value, must be one of:

- SYSCTL_XCLKDIV_1
- SYSCTL_XCLKDIV_2
- SYSCTL_XCLKDIV_4
- SYSCTL_XCLKDIV_OFF

Description:

Sets up device clocking according to the parameter passed in.

Returns:

None.

14.3 Programming Example

The following example shows how to use the SysCtl API to configure the device for normal operation.

```
// Configure the device to run at 20 MHz from the PLL using
// a 4 MHz crystal as the input.
SysCtlClockConfigSet(SYSCTL_SYSDIV_1 | SYSCTL_M3SSDIV_2 |
                     SYSCTL_USE_PLL | (SYSCTL_SPLLIMULT_M & 0x0F));

// Enable the GPIO blocks and the SSI.
SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOA);
SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOB);
SysCtlPeripheralEnable(SYSCTL_PERIPH_SSI);

// Enable the GPIO blocks and the SSI in sleep mode.
SysCtlPeripheralsSleepEnable(SYSCTL_PERIPH_GPIOA);
SysCtlPeripheralsSleepEnable(SYSCTL_PERIPH_GPIOB);
SysCtlPeripheralsSleepEnable(SYSCTL_PERIPH_SSI);

// Enable peripheral clock gating.
SysCtlPeripheralClockGating(true);
```

15 System Tick (SysTick)

Introduction	251
API Functions	251
Programming Example	255

15.1 Introduction

SysTick is a simple timer that is part of the NVIC controller in the Cortex-M3 microprocessor. Its intended purpose is to provide a periodic interrupt for a RTOS, but it can be used for other simple timing purposes.

The SysTick interrupt handler does not need to clear the SysTick interrupt source. This will be done automatically by NVIC when the SysTick interrupt handler is called.

This driver is contained in `driverlib/systick.c`, with `driverlib/systick.h` containing the API definitions for use by applications.

15.2 API Functions

Functions

- void [SysTickDisable](#) (void)
- void [SysTickEnable](#) (void)
- void [SysTickIntDisable](#) (void)
- void [SysTickIntEnable](#) (void)
- void [SysTickIntRegister](#) (void (*pfnHandler)(void))
- void [SysTickIntUnregister](#) (void)
- unsigned long [SysTickPeriodGet](#) (void)
- void [SysTickPeriodSet](#) (unsigned long ulPeriod)
- unsigned long [SysTickValueGet](#) (void)

15.2.1 Detailed Description

The SysTick API is fairly simple, like SysTick itself. There are functions for configuring and enabling SysTick ([SysTickEnable\(\)](#), [SysTickDisable\(\)](#), [SysTickPeriodSet\(\)](#), [SysTickPeriodGet\(\)](#), and [SysTickValueGet\(\)](#)) and functions for dealing with an interrupt handler for SysTick ([SysTickIntRegister\(\)](#), [SysTickIntUnregister\(\)](#), [SysTickIntEnable\(\)](#), and [SysTickIntDisable\(\)](#)).

15.2.2 Function Documentation

15.2.2.1 SysTickDisable

Disables the SysTick counter.

Prototype:

```
void  
SysTickDisable(void)
```

Description:

This will stop the SysTick counter. If an interrupt handler has been registered, it will no longer be called until SysTick is restarted.

Returns:

None.

15.2.2.2 SysTickEnable

Enables the SysTick counter.

Prototype:

```
void  
SysTickEnable(void)
```

Description:

This will start the SysTick counter. If an interrupt handler has been registered, it will be called when the SysTick counter rolls over.

Note:

Calling this function will cause the SysTick counter to (re)commence counting from its current value. The counter is not automatically reloaded with the period as specified in a previous call to [SysTickPeriodSet\(\)](#). If an immediate reload is required, the **NVIC_ST_CURRENT** register must be written to force this. Any write to this register clears the SysTick counter to 0 and will cause a reload with the supplied period on the next clock.

Returns:

None.

15.2.2.3 SysTickIntDisable

Disables the SysTick interrupt.

Prototype:

```
void  
SysTickIntDisable(void)
```

Description:

This function will disable the SysTick interrupt, preventing it from being reflected to the processor.

Returns:

None.

15.2.2.4 SysTickIntEnable

Enables the SysTick interrupt.

Prototype:

```
void  
SysTickIntEnable(void)
```

Description:

This function will enable the SysTick interrupt, allowing it to be reflected to the processor.

Note:

The SysTick interrupt handler does not need to clear the SysTick interrupt source as this is done automatically by NVIC when the interrupt handler is called.

Returns:

None.

15.2.2.5 SysTickIntRegister

Registers an interrupt handler for the SysTick interrupt.

Prototype:

```
void  
SysTickIntRegister(void (*pfnHandler) (void))
```

Parameters:

pfnHandler is a pointer to the function to be called when the SysTick interrupt occurs.

Description:

This sets the handler to be called when a SysTick interrupt occurs.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

15.2.2.6 SysTickIntUnregister

Unregisters the interrupt handler for the SysTick interrupt.

Prototype:

```
void  
SysTickIntUnregister(void)
```

Description:

This function will clear the handler to be called when a SysTick interrupt occurs.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

15.2.2.7 SysTickPeriodGet

Gets the period of the SysTick counter.

Prototype:

```
unsigned long  
SysTickPeriodGet(void)
```

Description:

This function returns the rate at which the SysTick counter wraps; this equates to the number of processor clocks between interrupts.

Returns:

Returns the period of the SysTick counter.

15.2.2.8 SysTickPeriodSet

Sets the period of the SysTick counter.

Prototype:

```
void  
SysTickPeriodSet(unsigned long ulPeriod)
```

Parameters:

ulPeriod is the number of clock ticks in each period of the SysTick counter; must be between 1 and 16,777,216, inclusive.

Description:

This function sets the rate at which the SysTick counter wraps; this equates to the number of processor clocks between interrupts.

Note:

Calling this function does not cause the SysTick counter to reload immediately. If an immediate reload is required, the **NVIC_ST_CURRENT** register must be written. Any write to this register clears the SysTick counter to 0 and will cause a reload with the *ulPeriod* supplied here on the next clock after the SysTick is enabled.

Returns:

None.

15.2.2.9 SysTickValueGet

Gets the current value of the SysTick counter.

Prototype:

```
unsigned long  
SysTickValueGet(void)
```

Description:

This function returns the current value of the SysTick counter; this will be a value between the period - 1 and zero, inclusive.

Returns:

Returns the current value of the SysTick counter.

15.3 Programming Example

The following example shows how to use the SysTick API to configure the SysTick counter and read its value.

```
unsigned long ulValue;

// Configure and enable the SysTick counter.
SysTickPeriodSet(1000);
SysTickEnable();

// Delay for some time...

// Read the current SysTick value.
ulValue = SysTickValueGet();
```


16 Timer

Introduction	257
API Functions	257
Programming Example	270

16.1 Introduction

The timer API provides a set of functions for dealing with the timer module. Functions are provided to configure and control the timer, along with functions to modify timer/counter values, and to manage interrupt handling for the timer.

The timer module provides two 16-bit timer/counters that can be configured to operate independently as timers or event counters, or they can be configured to operate as one 32-bit timer or one 32-bit Real Time Clock (RTC). For the purpose of this API, the two timers provided by the timer are referred to as TimerA and TimerB.

When configured as either a 32-bit or 16-bit timer, a timer can be set up to run as a one-shot timer or a continuous timer. If configured as a one-shot timer, when it reaches zero the timer will cease counting. If configured as a continuous timer, when it reaches zero the timer will continue counting from a reloaded value. When configured as a 32-bit timer, the timer can also be configured to operate as an RTC. In that case, the timer expects to be driven by a 32.768 KHz external clock, which is divided down to produce 1 second clock ticks.

When in 16-bit mode, the timer can also be configured for event capture or as a Pulse Width Modulation (PWM) generator. When configured for event capture, the timer acts as a counter. It can be configured to either count the time between events, or it can count the events themselves. The type of event being counted can be configured as a positive edge, a negative edge, or both edges. When a timer is configured as a PWM generator, the input line used to capture events becomes an output line, and the timer is used to drive an edge-aligned pulse onto that line.

The timer module also provides the ability to control other functional parameters, such as output inversion, output triggers, and timer behavior during stalls.

Control is also provided over interrupt sources and events. Interrupts can be generated to indicate that an event has been captured, or that a certain number of events have been captured. Interrupts can also be generated when the timer has counted down to zero, or when the RTC matches a certain value.

This driver is contained in `driverlib/timer.c`, with `driverlib/timer.h` containing the API definitions for use by applications.

16.2 API Functions

Functions

- void [TimerConfigure](#) (unsigned long ulBase, unsigned long ulConfig)
- void [TimerControlEvent](#) (unsigned long ulBase, unsigned long ulTimer, unsigned long ulEvent)
- void [TimerControlLevel](#) (unsigned long ulBase, unsigned long ulTimer, tBoolean blInvert)

- void [TimerControlStall](#) (unsigned long ulBase, unsigned long ulTimer, tBoolean bStall)
- void [TimerControlTrigger](#) (unsigned long ulBase, unsigned long ulTimer, tBoolean bEnable)
- void [TimerControlWaitOnTrigger](#) (unsigned long ulBase, unsigned long ulTimer, tBoolean bWait)
- void [TimerDisable](#) (unsigned long ulBase, unsigned long ulTimer)
- void [TimerEnable](#) (unsigned long ulBase, unsigned long ulTimer)
- void [TimerIntClear](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [TimerIntDisable](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [TimerIntEnable](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [TimerIntRegister](#) (unsigned long ulBase, unsigned long ulTimer, void (*pfnHandler)(void))
- unsigned long [TimerIntStatus](#) (unsigned long ulBase, tBoolean bMasked)
- void [TimerIntUnregister](#) (unsigned long ulBase, unsigned long ulTimer)
- unsigned long [TimerLoadGet](#) (unsigned long ulBase, unsigned long ulTimer)
- void [TimerLoadSet](#) (unsigned long ulBase, unsigned long ulTimer, unsigned long ulValue)
- unsigned long [TimerMatchGet](#) (unsigned long ulBase, unsigned long ulTimer)
- void [TimerMatchSet](#) (unsigned long ulBase, unsigned long ulTimer, unsigned long ulValue)
- unsigned long [TimerPrescaleGet](#) (unsigned long ulBase, unsigned long ulTimer)
- unsigned long [TimerPrescaleMatchGet](#) (unsigned long ulBase, unsigned long ulTimer)
- void [TimerPrescaleMatchSet](#) (unsigned long ulBase, unsigned long ulTimer, unsigned long ulValue)
- void [TimerPrescaleSet](#) (unsigned long ulBase, unsigned long ulTimer, unsigned long ulValue)
- void [TimerRTCDisable](#) (unsigned long ulBase)
- void [TimerRTCEnable](#) (unsigned long ulBase)
- unsigned long [TimerValueGet](#) (unsigned long ulBase, unsigned long ulTimer)

16.2.1 Detailed Description

The timer API is broken into three groups of functions: those that deal with timer configuration and control, those that deal with timer contents, and those that deal with interrupt handling.

Timer configuration is handled by [TimerConfigure\(\)](#), which performs the high level setup of the timer module; that is, it is used to set up 32- or 16-bit modes, and to select between PWM, capture, and timer operations. Timer control is performed by [TimerEnable\(\)](#), [TimerDisable\(\)](#), [TimerControlLevel\(\)](#), [TimerControlTrigger\(\)](#), [TimerControlEvent\(\)](#), [TimerControlStall\(\)](#), [TimerRTCEnable\(\)](#), and [TimerRTCDisable\(\)](#).

Timer content is managed with [TimerLoadSet\(\)](#), [TimerLoadGet\(\)](#), [TimerPrescaleSet\(\)](#), [TimerPrescaleGet\(\)](#), [TimerMatchSet\(\)](#), [TimerMatchGet\(\)](#), [TimerPrescaleMatchSet\(\)](#), [TimerPrescaleMatchGet\(\)](#), and [TimerValueGet\(\)](#).

The interrupt handler for the Timer interrupt is managed with [TimerIntRegister\(\)](#) and [TimerIntUnregister\(\)](#). The individual interrupt sources within the timer module are managed with [TimerIntEnable\(\)](#), [TimerIntDisable\(\)](#), [TimerIntStatus\(\)](#), and [TimerIntClear\(\)](#).

The [TimerQuiesce\(\)](#) API from previous versions of the peripheral driver library has been deprecated. [SysCtlPeripheralReset\(\)](#) should be used instead to return the timer to its reset state.

16.2.2 Function Documentation

16.2.2.1 TimerConfigure

Configures the timer(s).

Prototype:

```
void  
TimerConfigure(unsigned long ulBase,  
               unsigned long ulConfig)
```

Parameters:

ulBase is the base address of the timer module.

ulConfig is the configuration for the timer.

Description:

This function configures the operating mode of the timer(s). The timer module is disabled before being configured, and is left in the disabled state. The configuration is specified in *ulConfig* as one of the following values:

- **TIMER_CFG_32_BIT_OS** - 32-bit one-shot timer
- **TIMER_CFG_32_BIT_OS_UP** - 32-bit one-shot timer that counts up instead of down (not available on all parts)
- **TIMER_CFG_32_BIT_PER** - 32-bit periodic timer
- **TIMER_CFG_32_BIT_PER_UP** - 32-bit periodic timer that counts up instead of down (not available on all parts)
- **TIMER_CFG_32_RTC** - 32-bit real time clock timer
- **TIMER_CFG_16_BIT_PAIR** - Two 16-bit timers

When configured for a pair of 16-bit timers, each timer is separately configured. The first timer is configured by setting *ulConfig* to the result of a logical OR operation between one of the following values and *ulConfig*:

- **TIMER_CFG_A_ONE_SHOT** - 16-bit one-shot timer
- **TIMER_CFG_A_ONE_SHOT_UP** - 16-bit one-shot timer that counts up instead of down (not available on all parts)
- **TIMER_CFG_A_PERIODIC** - 16-bit periodic timer
- **TIMER_CFG_A_PERIODIC_UP** - 16-bit periodic timer that counts up instead of down (not available on all parts)
- **TIMER_CFG_A_CAP_COUNT** - 16-bit edge count capture
- **TIMER_CFG_A_CAP_TIME** - 16-bit edge time capture
- **TIMER_CFG_A_PWM** - 16-bit PWM output

Similarly, the second timer is configured by setting *ulConfig* to the result of a logical OR operation between one of the corresponding **TIMER_CFG_B_*** values and *ulConfig*.

Returns:

None.

16.2.2.2 TimerControlEvent

Controls the event type.

Prototype:

```
void  
TimerControlEvent(unsigned long ulBase,  
                  unsigned long ulTimer,  
                  unsigned long ulEvent)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer(s) to be adjusted; must be one of **TIMER_A**, **TIMER_B**, or **TIMER_BOTH**.

ulEvent specifies the type of event; must be one of **TIMER_EVENT_POS_EDGE**, **TIMER_EVENT_NEG_EDGE**, or **TIMER_EVENT_BOTH_EDGES**.

Description:

This function sets the signal edge(s) that triggers the timer when in capture mode.

Returns:

None.

16.2.2.3 TimerControlLevel

Controls the output level.

Prototype:

```
void  
TimerControlLevel(unsigned long ulBase,  
                  unsigned long ulTimer,  
                  tBoolean bInvert)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer(s) to adjust; must be one of **TIMER_A**, **TIMER_B**, or **TIMER_BOTH**.

bInvert specifies the output level.

Description:

This function sets the PWM output level for the specified timer. If the *bInvert* parameter is **true**, then the timer's output is made active low; otherwise, it is made active high.

Returns:

None.

16.2.2.4 TimerControlStall

Controls the stall handling.

Prototype:

```
void  
TimerControlStall(unsigned long ulBase,  
                  unsigned long ulTimer,  
                  tBoolean bStall)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer(s) to be adjusted; must be one of **TIMER_A**, **TIMER_B**, or **TIMER_BOTH**.

bStall specifies the response to a stall signal.

Description:

This function controls the stall response for the specified timer. If the ***bStall*** parameter is **true**, then the timer stops counting if the processor enters debug mode; otherwise the timer keeps running while in debug mode.

Returns:

None.

16.2.2.5 TimerControlTrigger

Enables or disables the trigger output.

Prototype:

```
void  
TimerControlTrigger(unsigned long ulBase,  
                    unsigned long ulTimer,  
                    tBoolean bEnable)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer to adjust; must be one of **TIMER_A**, **TIMER_B**, or **TIMER_BOTH**.

bEnable specifies the desired trigger state.

Description:

This function controls the trigger output for the specified timer. If the ***bEnable*** parameter is **true**, then the timer's output trigger is enabled; otherwise it is disabled.

Returns:

None.

16.2.2.6 TimerControlWaitOnTrigger

Controls the wait on trigger handling.

Prototype:

```
void  
TimerControlWaitOnTrigger(unsigned long ulBase,  
                           unsigned long ulTimer,  
                           tBoolean bWait)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer(s) to be adjusted; must be one of **TIMER_A**, **TIMER_B**, or **TIMER_BOTH**.

bWait specifies if the timer should wait for a trigger input.

Description:

This function controls whether or not a timer waits for a trigger input to start counting. When enabled, the previous timer in the trigger chain must count to its timeout in order for this timer to start counting. Refer to the part's data sheet for a description of the trigger chain.

Note:

This functionality is not available on all parts.

Returns:

None.

16.2.2.7 TimerDisable

Disables the timer(s).

Prototype:

```
void  
TimerDisable(unsigned long ulBase,  
              unsigned long ulTimer)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer(s) to disable; must be one of **TIMER_A**, **TIMER_B**, or **TIMER_BOTH**.

Description:

This function disables operation of the timer module.

Returns:

None.

16.2.2.8 TimerEnable

Enables the timer(s).

Prototype:

```
void  
TimerEnable(unsigned long ulBase,  
            unsigned long ulTimer)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer(s) to enable; must be one of **TIMER_A**, **TIMER_B**, or **TIMER_BOTH**.

Description:

This function enables operation of the timer module. The timer must be configured before it is enabled.

Returns:

None.

16.2.2.9 TimerIntClear

Clears timer interrupt sources.

Prototype:

```
void  
TimerIntClear(unsigned long ulBase,  
              unsigned long ulIntFlags)
```

Parameters:

ulBase is the base address of the timer module.

ulIntFlags is a bit mask of the interrupt sources to be cleared.

Description:

The specified timer interrupt sources are cleared, so that they no longer assert. This function must be called in the interrupt handler to keep the interrupt from being triggered again immediately upon exit.

The *ulIntFlags* parameter has the same definition as the *ulIntFlags* parameter to [TimerIntEnable\(\)](#).

Note:

Because there is a write buffer in the Cortex-M3 processor, it may take several clock cycles before the interrupt source is actually cleared. Therefore, it is recommended that the interrupt source be cleared early in the interrupt handler (as opposed to the very last action) to avoid returning from the interrupt handler before the interrupt source is actually cleared. Failure to do so may result in the interrupt handler being immediately reentered (because the interrupt controller still sees the interrupt source asserted).

Returns:

None.

16.2.2.10 TimerIntDisable

Disables individual timer interrupt sources.

Prototype:

```
void  
TimerIntDisable(unsigned long ulBase,  
                unsigned long ulIntFlags)
```

Parameters:

ulBase is the base address of the timer module.

ulIntFlags is the bit mask of the interrupt sources to be disabled.

Description:

Disables the indicated timer interrupt sources. Only the sources that are enabled can be reflected to the processor interrupt; disabled sources have no effect on the processor.

The *ulIntFlags* parameter has the same definition as the *ulIntFlags* parameter to [TimerIntEnable\(\)](#).

Returns:

None.

16.2.2.11 TimerIntEnable

Enables individual timer interrupt sources.

Prototype:

```
void  
TimerIntEnable(unsigned long ulBase,  
               unsigned long ulIntFlags)
```

Parameters:

ulBase is the base address of the timer module.

ulIntFlags is the bit mask of the interrupt sources to be enabled.

Description:

Enables the indicated timer interrupt sources. Only the sources that are enabled can be reflected to the processor interrupt; disabled sources have no effect on the processor.

The *ulIntFlags* parameter must be the logical OR of any combination of the following:

- **TIMER_CAPB_EVENT** - Capture B event interrupt
- **TIMER_CAPB_MATCH** - Capture B match interrupt
- **TIMER_TIMB_TIMEOUT** - Timer B timeout interrupt
- **TIMER_RTC_MATCH** - RTC interrupt mask
- **TIMER_CAPA_EVENT** - Capture A event interrupt
- **TIMER_CAPA_MATCH** - Capture A match interrupt
- **TIMER_TIMA_TIMEOUT** - Timer A timeout interrupt

Returns:

None.

16.2.2.12 TimerIntRegister

Registers an interrupt handler for the timer interrupt.

Prototype:

```
void  
TimerIntRegister(unsigned long ulBase,  
                unsigned long ulTimer,  
                void (*pfnHandler)(void))
```


Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer(s); must be one of **TIMER_A**, **TIMER_B**, or **TIMER_BOTH**.

pfnHandler is a pointer to the function to be called when the timer interrupt occurs.

Description:

This function sets the handler to be called when a timer interrupt occurs. In addition, this function enables the global interrupt in the interrupt controller; specific timer interrupts must be enabled via [TimerIntEnable\(\)](#). It is the interrupt handler's responsibility to clear the interrupt source via [TimerIntClear\(\)](#).

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

16.2.2.13 TimerIntStatus

Gets the current interrupt status.

Prototype:

```
unsigned long  
TimerIntStatus(unsigned long ulBase,  
                tBoolean bMasked)
```

Parameters:

ulBase is the base address of the timer module.

bMasked is false if the raw interrupt status is required and true if the masked interrupt status is required.

Description:

This function returns the interrupt status for the timer module. Either the raw interrupt status or the status of interrupts that are allowed to reflect to the processor can be returned.

Returns:

The current interrupt status, enumerated as a bit field of values described in [TimerIntEnable\(\)](#).

16.2.2.14 TimerIntUnregister

Unregisters an interrupt handler for the timer interrupt.

Prototype:

```
void  
TimerIntUnregister(unsigned long ulBase,  
                   unsigned long ulTimer)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer(s); must be one of **TIMER_A**, **TIMER_B**, or **TIMER_BOTH**.

Description:

This function clears the handler to be called when a timer interrupt occurs. This function also masks off the interrupt in the interrupt controller so that the interrupt handler no longer is called.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

16.2.2.15 TimerLoadGet

Gets the timer load value.

Prototype:

```
unsigned long
TimerLoadGet(unsigned long ulBase,
              unsigned long ulTimer)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer; must be one of **TIMER_A** or **TIMER_B**. Only **TIMER_A** should be used when the timer is configured for 32-bit operation.

Description:

This function gets the currently programmed interval load value for the specified timer.

Returns:

Returns the load value for the timer.

16.2.2.16 TimerLoadSet

Sets the timer load value.

Prototype:

```
void
TimerLoadSet(unsigned long ulBase,
              unsigned long ulTimer,
              unsigned long ulValue)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer(s) to adjust; must be one of **TIMER_A**, **TIMER_B**, or **TIMER_BOTH**. Only **TIMER_A** should be used when the timer is configured for 32-bit operation.

ulValue is the load value.

Description:

This function sets the timer load value; if the timer is running then the value will be immediately loaded into the timer.

Returns:

None.

16.2.2.17 TimerMatchGet

Gets the timer match value.

Prototype:

```
unsigned long  
TimerMatchGet (unsigned long ulBase,  
               unsigned long ulTimer)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer; must be one of **TIMER_A** or **TIMER_B**. Only **TIMER_A** should be used when the timer is configured for 32-bit operation.

Description:

This function gets the match value for the specified timer.

Returns:

Returns the match value for the timer.

16.2.2.18 TimerMatchSet

Sets the timer match value.

Prototype:

```
void  
TimerMatchSet (unsigned long ulBase,  
               unsigned long ulTimer,  
               unsigned long ulValue)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer(s) to adjust; must be one of **TIMER_A**, **TIMER_B**, or **TIMER_BOTH**. Only **TIMER_A** should be used when the timer is configured for 32-bit operation.

ulValue is the match value.

Description:

This function sets the match value for a timer. This is used in capture count mode to determine when to interrupt the processor and in PWM mode to determine the duty cycle of the output signal.

Returns:

None.

16.2.2.19 TimerPrescaleGet

Get the timer prescale value.

Prototype:

```
unsigned long  
TimerPrescaleGet(unsigned long ulBase,  
                 unsigned long ulTimer)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer; must be one of **TIMER_A** or **TIMER_B**.

Description:

This function gets the value of the input clock prescaler. The prescaler is only operational when in 16-bit mode and is used to extend the range of the 16-bit timer modes.

Returns:

The value of the timer prescaler.

16.2.2.20 TimerPrescaleMatchGet

Get the timer prescale match value.

Prototype:

```
unsigned long  
TimerPrescaleMatchGet(unsigned long ulBase,  
                     unsigned long ulTimer)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer; must be one of **TIMER_A** or **TIMER_B**.

Description:

This function gets the value of the input clock prescaler match value. When in a 16-bit mode that uses the counter match and prescaler, the prescale match effectively extends the range of the counter to 24-bits.

Returns:

The value of the timer prescale match.

16.2.2.21 TimerPrescaleMatchSet

Set the timer prescale match value.

Prototype:

```
void  
TimerPrescaleMatchSet(unsigned long ulBase,  
                     unsigned long ulTimer,  
                     unsigned long ulValue)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer(s) to adjust; must be one of **TIMER_A**, **TIMER_B**, or **TIMER_BOTH**.

ulValue is the timer prescale match value; must be between 0 and 255, inclusive.

Description:

This function sets the value of the input clock prescaler match value. When in a 16-bit mode that uses the counter match and the prescaler, the prescale match effectively extends the range of the counter to 24-bits.

Returns:

None.

16.2.2.22 TimerPrescaleSet

Set the timer prescale value.

Prototype:

```
void  
TimerPrescaleSet(unsigned long ulBase,  
                 unsigned long ulTimer,  
                 unsigned long ulValue)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer(s) to adjust; must be one of **TIMER_A**, **TIMER_B**, or **TIMER_BOTH**.

ulValue is the timer prescale value; must be between 0 and 255, inclusive.

Description:

This function sets the value of the input clock prescaler. The prescaler is only operational when in 16-bit mode and is used to extend the range of the 16-bit timer modes.

Returns:

None.

16.2.2.23 TimerRTCDisable

Disable RTC counting.

Prototype:

```
void  
TimerRTCDisable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the timer module.

Description:

This function causes the timer to stop counting when in RTC mode.

Returns:

None.

16.2.2.24 TimerRTCEnable

Enable RTC counting.

Prototype:

```
void  
TimerRTCEnable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the timer module.

Description:

This function causes the timer to start counting when in RTC mode. If not configured for RTC mode, this function does nothing.

Returns:

None.

16.2.2.25 TimerValueGet

Gets the current timer value.

Prototype:

```
unsigned long  
TimerValueGet(unsigned long ulBase,  
               unsigned long ulTimer)
```

Parameters:

ulBase is the base address of the timer module.

ulTimer specifies the timer; must be one of **TIMER_A** or **TIMER_B**. Only **TIMER_A** should be used when the timer is configured for 32-bit operation.

Description:

This function reads the current value of the specified timer.

Returns:

Returns the current value of the timer.

16.3 Programming Example

The following example shows how to use the timer API to configure the timer as a 16-bit one shot timer and a 16-bit edge capture counter.

```
// Configure TimerA as a 16-bit one shot timer,  
// and TimerB as a 16-bit edge capture counter.  
TimerConfigure(TIMER0_BASE, (TIMER_CFG_16_BIT_PAIR |  
                             TIMER_CFG_A_ONE_SHOT |  
                             TIMER_CFG_B_CAP_COUNT));  
  
// Configure the counter (TimerB) to count both edges.
```

```
TimerControlEvent(TIMER0_BASE, TIMER_B, TIMER_EVENT_BOTH_EDGES);

// Enable the timers.
TimerEnable(TIMER0_BASE, TIMER_BOTH);
```


17 UART

Introduction	273
API Functions	273
Programming Example	293

17.1 Introduction

The Universal Asynchronous Receiver/Transmitter (UART) API provides a set of functions for using the Concerto UART modules. Functions are provided to configure and control the UART modules, to send and receive data, and to manage interrupts for the UART modules.

The Concerto UART performs the functions of parallel-to-serial and serial-to-parallel conversions. It is very similar in functionality to a 16C550 UART, but is not register-compatible.

Some of the features of the Concerto UART are:

- A 16x12 bit receive FIFO and a 16x8 bit transmit FIFO.
- Programmable baud rate generator.
- Automatic generation and stripping of start, stop, and parity bits.
- Line break generation and detection.
- Programmable serial interface
 - 5, 6, 7, or 8 data bits
 - even, odd, stick, or no parity bit generation and detection
 - 1 or 2 stop bit generation
 - baud rate generation, from DC to processor clock/16
- Modem control/flow control
- IrDA serial-IR (SIR) encoder/decoder.
- DMA interface

This driver is contained in `driverlib/uart.c`, with `driverlib/uart.h` containing the API definitions for use by applications.

17.2 API Functions

Functions

- void [UARTBreakCtl](#) (unsigned long ulBase, tBoolean bBreakState)
- tBoolean [UARTBusy](#) (unsigned long ulBase)
- long [UARTCharGet](#) (unsigned long ulBase)
- long [UARTCharGetNonBlocking](#) (unsigned long ulBase)
- void [UARTCharPut](#) (unsigned long ulBase, unsigned char ucData)
- tBoolean [UARTCharPutNonBlocking](#) (unsigned long ulBase, unsigned char ucData)
- tBoolean [UARTCharsAvail](#) (unsigned long ulBase)

- void [UARTConfigGetExpClk](#) (unsigned long ulBase, unsigned long ulUARTClk, unsigned long *pulBaud, unsigned long *pulConfig)
- void [UARTConfigSetExpClk](#) (unsigned long ulBase, unsigned long ulUARTClk, unsigned long ulBaud, unsigned long ulConfig)
- void [UARTDisable](#) (unsigned long ulBase)
- void [UARTDisableSIR](#) (unsigned long ulBase)
- void [UARTDMADisable](#) (unsigned long ulBase, unsigned long ulDMAFlags)
- void [UARTDMAEnable](#) (unsigned long ulBase, unsigned long ulDMAFlags)
- void [UARTEnable](#) (unsigned long ulBase)
- void [UARTEnableSIR](#) (unsigned long ulBase, tBoolean bLowPower)
- void [UARTFIFODisable](#) (unsigned long ulBase)
- void [UARTFIFOEnable](#) (unsigned long ulBase)
- void [UARTFIFOLevelGet](#) (unsigned long ulBase, unsigned long *pulTxLevel, unsigned long *pulRxLevel)
- void [UARTFIFOLevelSet](#) (unsigned long ulBase, unsigned long ulTxLevel, unsigned long ulRxLevel)
- unsigned long [UARTFlowControlGet](#) (unsigned long ulBase)
- void [UARTFlowControlSet](#) (unsigned long ulBase, unsigned long ulMode)
- void [UARTIntClear](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [UARTIntDisable](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [UARTIntEnable](#) (unsigned long ulBase, unsigned long ulIntFlags)
- void [UARTIntRegister](#) (unsigned long ulBase, void (*pfnHandler)(void))
- unsigned long [UARTIntStatus](#) (unsigned long ulBase, tBoolean bMasked)
- void [UARTIntUnregister](#) (unsigned long ulBase)
- void [UARTModemControlClear](#) (unsigned long ulBase, unsigned long ulControl)
- unsigned long [UARTModemControlGet](#) (unsigned long ulBase)
- void [UARTModemControlSet](#) (unsigned long ulBase, unsigned long ulControl)
- unsigned long [UARTModemStatusGet](#) (unsigned long ulBase)
- unsigned long [UARTParityModeGet](#) (unsigned long ulBase)
- void [UARTParityModeSet](#) (unsigned long ulBase, unsigned long ulParity)
- void [UARTRxErrorClear](#) (unsigned long ulBase)
- unsigned long [UARTRxErrorGet](#) (unsigned long ulBase)
- void [UARTSmartCardDisable](#) (unsigned long ulBase)
- void [UARTSmartCardEnable](#) (unsigned long ulBase)
- tBoolean [UARTSpaceAvail](#) (unsigned long ulBase)
- unsigned long [UARTTxIntModeGet](#) (unsigned long ulBase)
- void [UARTTxIntModeSet](#) (unsigned long ulBase, unsigned long ulMode)

17.2.1 Detailed Description

The UART API provides the set of functions required to implement an interrupt driven UART driver. These functions may be used to control any of the available UART ports on a Concerto microcontroller, and can be used with one port without causing conflicts with the other port.

The UART API is broken into three groups of functions: those that deal with configuration and control of the UART modules, those used to send and receive data, and those that deal with interrupt handling.

Configuration and control of the UART are handled by the [UARTConfigGetExpClk\(\)](#), [UARTConfigSetExpClk\(\)](#), [UARTDisable\(\)](#), [UARTEnable\(\)](#), [UARTParityModeGet\(\)](#), and [UARTParityModeSet\(\)](#) functions. The DMA interface can be enabled or disabled by the [UARTDMAEnable\(\)](#) and [UARTDMADisable\(\)](#) functions.

Sending and receiving data via the UART is handled by the [UARTCharGet\(\)](#), [UARTCharGetNonBlocking\(\)](#), [UARTCharPut\(\)](#), [UARTCharPutNonBlocking\(\)](#), [UARTBreakCtl\(\)](#), [UARTCharsAvail\(\)](#), and [UARTSpaceAvail\(\)](#) functions.

Managing the UART interrupts is handled by the [UARTIntClear\(\)](#), [UARTIntDisable\(\)](#), [UARTIntEnable\(\)](#), [UARTIntRegister\(\)](#), [UARTIntStatus\(\)](#), and [UARTIntUnregister\(\)](#) functions.

The [UARTConfigSet\(\)](#), [UARTConfigGet\(\)](#), [UARTCharNonBlockingGet\(\)](#), and [UARTCharNonBlockingPut\(\)](#) APIs from previous versions of the peripheral driver library have been replaced by the [UARTConfigSetExpClk\(\)](#), [UARTConfigGetExpClk\(\)](#), [UARTCharGetNonBlocking\(\)](#), and [UARTCharPutNonBlocking\(\)](#) APIs, respectively. Macros have been provided in `uart.h` to map the old APIs to the new APIs, allowing existing applications to link and run with the new APIs. It is recommended that new applications utilize the new APIs in favor of the old ones.

17.2.2 Function Documentation

17.2.2.1 UARTBreakCtl

Causes a BREAK to be sent.

Prototype:

```
void
UARTBreakCtl(unsigned long ulBase,
              tBoolean bBreakState)
```

Parameters:

ulBase is the base address of the UART port.

bBreakState controls the output level.

Description:

Calling this function with *bBreakState* set to **true** asserts a break condition on the UART. Calling this function with *bBreakState* set to **false** removes the break condition. For proper transmission of a break command, the break must be asserted for at least two complete frames.

Returns:

None.

17.2.2.2 UARTBusy

Determines whether the UART transmitter is busy or not.

Prototype:

```
tBoolean
UARTBusy(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

Allows the caller to determine whether all transmitted bytes have cleared the transmitter hardware. If **false** is returned, the transmit FIFO is empty and all bits of the last transmitted character, including all stop bits, have left the hardware shift register.

Returns:

Returns **true** if the UART is transmitting or **false** if all transmissions are complete.

17.2.2.3 UARTCharGet

Waits for a character from the specified port.

Prototype:

```
long
UARTCharGet(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function gets a character from the receive FIFO for the specified port. If there are no characters available, this function waits until a character is received before returning.

Returns:

Returns the character read from the specified port, cast as a *long*.

17.2.2.4 UARTCharGetNonBlocking

Receives a character from the specified port.

Prototype:

```
long
UARTCharGetNonBlocking(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function gets a character from the receive FIFO for the specified port.

This function replaces the original UARTCharNonBlockingGet() API and performs the same actions. A macro is provided in `uart.h` to map the original API to this API.

Returns:

Returns the character read from the specified port, cast as a *long*. A -1 is returned if there are no characters present in the receive FIFO. The [UARTCharsAvail\(\)](#) function should be called before attempting to call this function.

17.2.2.5 UARTCharPut

Waits to send a character from the specified port.

Prototype:

```
void
UARTCharPut(unsigned long ulBase,
             unsigned char ucData)
```

Parameters:

ulBase is the base address of the UART port.

ucData is the character to be transmitted.

Description:

This function sends the character *ucData* to the transmit FIFO for the specified port. If there is no space available in the transmit FIFO, this function waits until there is space available before returning.

Returns:

None.

17.2.2.6 UARTCharPutNonBlocking

Sends a character to the specified port.

Prototype:

```
tBoolean
UARTCharPutNonBlocking(unsigned long ulBase,
                       unsigned char ucData)
```

Parameters:

ulBase is the base address of the UART port.

ucData is the character to be transmitted.

Description:

This function writes the character *ucData* to the transmit FIFO for the specified port. This function does not block, so if there is no space available, then a **false** is returned, and the application must retry the function later.

This function replaces the original UARTCharNonBlockingPut() API and performs the same actions. A macro is provided in `uart.h` to map the original API to this API.

Returns:

Returns **true** if the character was successfully placed in the transmit FIFO or **false** if there was no space available in the transmit FIFO.

17.2.2.7 UARTCharsAvail

Determines if there are any characters in the receive FIFO.

Prototype:

```
tBoolean
UARTCharsAvail(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function returns a flag indicating whether or not there is data available in the receive FIFO.

Returns:

Returns **true** if there is data in the receive FIFO or **false** if there is no data in the receive FIFO.

17.2.2.8 UARTConfigGetExpClk

Gets the current configuration of a UART.

Prototype:

```
void
UARTConfigGetExpClk(unsigned long ulBase,
                    unsigned long ulUARTClk,
                    unsigned long *pulBaud,
                    unsigned long *pulConfig)
```

Parameters:

ulBase is the base address of the UART port.

ulUARTClk is the rate of the clock supplied to the UART module.

pulBaud is a pointer to storage for the baud rate.

pulConfig is a pointer to storage for the data format.

Description:

The baud rate and data format for the UART is determined, given an explicitly provided peripheral clock (hence the ExpClk suffix). The returned baud rate is the actual baud rate; it may not be the exact baud rate requested or an “official” baud rate. The data format returned in *pulConfig* is enumerated the same as the *ulConfig* parameter of [UARTConfigSetExpClk\(\)](#).

The peripheral clock is the same as the processor clock. The frequency of the system clock is the value returned by [SysCtlClockGet\(\)](#), or it can be explicitly hard coded if it is constant and known (to save the code/execution overhead of a call to [SysCtlClockGet\(\)](#)).

This function replaces the original UARTConfigGet() API and performs the same actions. A macro is provided in `uart.h` to map the original API to this API.

Returns:

None.

17.2.2.9 UARTConfigSetExpClk

Sets the configuration of a UART.

Prototype:

```
void
UARTConfigSetExpClk(unsigned long ulBase,
                    unsigned long ulUARTClk,
                    unsigned long ulBaud,
                    unsigned long ulConfig)
```

Parameters:

ulBase is the base address of the UART port.

ulUARTClk is the rate of the clock supplied to the UART module.

ulBaud is the desired baud rate.

ulConfig is the data format for the port (number of data bits, number of stop bits, and parity).

Description:

This function configures the UART for operation in the specified data format. The baud rate is provided in the *ulBaud* parameter and the data format in the *ulConfig* parameter.

The *ulConfig* parameter is the logical OR of three values: the number of data bits, the number of stop bits, and the parity. **UART_CONFIG_WLEN_8**, **UART_CONFIG_WLEN_7**, **UART_CONFIG_WLEN_6**, and **UART_CONFIG_WLEN_5** select from eight to five data bits per byte (respectively). **UART_CONFIG_STOP_ONE** and **UART_CONFIG_STOP_TWO** select one or two stop bits (respectively). **UART_CONFIG_PAR_NONE**, **UART_CONFIG_PAR_EVEN**, **UART_CONFIG_PAR_ODD**, **UART_CONFIG_PAR_ONE**, and **UART_CONFIG_PAR_ZERO** select the parity mode (no parity bit, even parity bit, odd parity bit, parity bit always one, and parity bit always zero, respectively).

The peripheral clock is the same as the processor clock. The frequency of the system clock is the value returned by [SysCtlClockGet\(\)](#), or it can be explicitly hard coded if it is constant and known (to save the code/execution overhead of a call to [SysCtlClockGet\(\)](#)).

This function replaces the original UARTConfigSet() API and performs the same actions. A macro is provided in `uart.h` to map the original API to this API.

Returns:

None.

17.2.2.10 UARTDisable

Disables transmitting and receiving.

Prototype:

```
void
UARTDisable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function clears the URTEN, TXE, and RXE bits, waits for the end of transmission of the current character, and flushes the transmit FIFO.

Returns:

None.

17.2.2.11 UARTDisableSIR

Disables SIR (IrDA) mode on the specified UART.

Prototype:

```
void
UARTDisableSIR(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function clears the SIREN (IrDA) and SIRLP (Low Power) bits.

Returns:

None.

17.2.2.12 UARTDMADisable

Disable UART DMA operation.

Prototype:

```
void
UARTDMADisable(unsigned long ulBase,
                unsigned long ulDMAFlags)
```

Parameters:

ulBase is the base address of the UART port.

ulDMAFlags is a bit mask of the DMA features to disable.

Description:

This function is used to disable UART DMA features that were enabled by [UARTDMAEnable\(\)](#). The specified UART DMA features are disabled. The *ulDMAFlags* parameter is the logical OR of any of the following values:

- UART_DMA_RX - disable DMA for receive
- UART_DMA_TX - disable DMA for transmit
- UART_DMA_ERR_RXSTOP - do not disable DMA receive on UART error

Returns:

None.

17.2.2.13 UARTDMAEnable

Enable UART DMA operation.

Prototype:

```
void
UARTDMAEnable(unsigned long ulBase,
               unsigned long ulDMAFlags)
```


Parameters:

ulBase is the base address of the UART port.

ulDMAFlags is a bit mask of the DMA features to enable.

Description:

The specified UART DMA features are enabled. The UART can be configured to use DMA for transmit or receive, and to disable receive if an error occurs. The *ulDMAFlags* parameter is the logical OR of any of the following values:

- UART_DMA_RX - enable DMA for receive
- UART_DMA_TX - enable DMA for transmit
- UART_DMA_ERR_RXSTOP - disable DMA receive on UART error

Note:

The uDMA controller must also be set up before DMA can be used with the UART.

Returns:

None.

17.2.2.14 UARTEnable

Enables transmitting and receiving.

Prototype:

```
void  
UARTEnable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function sets the UARTEN, TXE, and RXE bits, and enables the transmit and receive FIFOs.

Returns:

None.

17.2.2.15 UARTEnableSIR

Enables SIR (IrDA) mode on the specified UART.

Prototype:

```
void  
UARTEnableSIR(unsigned long ulBase,  
               tBoolean bLowPower)
```

Parameters:

ulBase is the base address of the UART port.

bLowPower indicates if SIR Low Power Mode is to be used.

Description:

This function enables the SIREN control bit for IrDA mode on the UART. If the *bLowPower* flag is set, then SIRLP bit will also be set.

Returns:

None.

17.2.2.16 UARTFIFODisable

Disables the transmit and receive FIFOs.

Prototype:

```
void  
UARTFIFODisable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This functions disables the transmit and receive FIFOs in the UART.

Returns:

None.

17.2.2.17 UARTFIFOEnable

Enables the transmit and receive FIFOs.

Prototype:

```
void  
UARTFIFOEnable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This functions enables the transmit and receive FIFOs in the UART.

Returns:

None.

17.2.2.18 UARTFIFOLevelGet

Gets the FIFO level at which interrupts are generated.

Prototype:

```
void  
UARTFIFOLevelGet(unsigned long ulBase,  
                  unsigned long *pulTxLevel,  
                  unsigned long *pulRxLevel)
```

Parameters:

ulBase is the base address of the UART port.

pulTxLevel is a pointer to storage for the transmit FIFO level, returned as one of **UART_FIFO_TX1_8**, **UART_FIFO_TX2_8**, **UART_FIFO_TX4_8**, **UART_FIFO_TX6_8**, or **UART_FIFO_TX7_8**.

pulRxLevel is a pointer to storage for the receive FIFO level, returned as one of **UART_FIFO_RX1_8**, **UART_FIFO_RX2_8**, **UART_FIFO_RX4_8**, **UART_FIFO_RX6_8**, or **UART_FIFO_RX7_8**.

Description:

This function gets the FIFO level at which transmit and receive interrupts are generated.

Returns:

None.

17.2.2.19 UARTFIFOLevelSet

Sets the FIFO level at which interrupts are generated.

Prototype:

```
void
UARTFIFOLevelSet(unsigned long ulBase,
                  unsigned long ulTxLevel,
                  unsigned long ulRxLevel)
```

Parameters:

ulBase is the base address of the UART port.

ulTxLevel is the transmit FIFO interrupt level, specified as one of **UART_FIFO_TX1_8**, **UART_FIFO_TX2_8**, **UART_FIFO_TX4_8**, **UART_FIFO_TX6_8**, or **UART_FIFO_TX7_8**.

ulRxLevel is the receive FIFO interrupt level, specified as one of **UART_FIFO_RX1_8**, **UART_FIFO_RX2_8**, **UART_FIFO_RX4_8**, **UART_FIFO_RX6_8**, or **UART_FIFO_RX7_8**.

Description:

This function sets the FIFO level at which transmit and receive interrupts are generated.

Returns:

None.

17.2.2.20 UARTFlowControlGet

Returns the UART hardware flow control mode currently in use.

Prototype:

```
unsigned long
UARTFlowControlGet(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function returns the current hardware flow control mode.

Note:

The availability of hardware flow control varies with the UART in use. Please consult the datasheet for the part you are using to determine whether this support is available.

Returns:

Returns the current flow control mode in use. This is a logical OR combination of values **UART_FLOWCONTROL_TX** if transmit (CTS) flow control is enabled and **UART_FLOWCONTROL_RX** if receive (RTS) flow control is in use. If hardware flow control is disabled, **UART_FLOWCONTROL_NONE** is returned.

17.2.2.21 UARTFlowControlSet

Sets the UART hardware flow control mode to be used.

Prototype:

```
void
UARTFlowControlSet(unsigned long ulBase,
                   unsigned long ulMode)
```

Parameters:

ulBase is the base address of the UART port.

ulMode indicates the flow control modes to be used. This parameter is a logical OR combination of values **UART_FLOWCONTROL_TX** and **UART_FLOWCONTROL_RX** to enable hardware transmit (CTS) and receive (RTS) flow control or **UART_FLOWCONTROL_NONE** to disable hardware flow control.

Description:

This function sets the required hardware flow control modes. If *ulMode* contains flag **UART_FLOWCONTROL_TX**, data is only transmitted if the incoming CTS signal is asserted. If *ulMode* contains flag **UART_FLOWCONTROL_RX**, the RTS output is controlled by the hardware and is asserted only when there is space available in the receive FIFO. If no hardware flow control is required, **UART_FLOWCONTROL_NONE** should be passed.

Note:

The availability of hardware flow control varies with the UART in use. Please consult the datasheet for the part you are using to determine whether this support is available.

Returns:

None.

17.2.2.22 UARTIntClear

Clears UART interrupt sources.

Prototype:

```
void
UARTIntClear(unsigned long ulBase,
             unsigned long ulIntFlags)
```

Parameters:

ulBase is the base address of the UART port.

ullIntFlags is a bit mask of the interrupt sources to be cleared.

Description:

The specified UART interrupt sources are cleared, so that they no longer assert. This function must be called in the interrupt handler to keep the interrupt from being recognized again immediately upon exit.

The *ullIntFlags* parameter has the same definition as the *ullIntFlags* parameter to [UARTIntEnable\(\)](#).

Note:

Because there is a write buffer in the Cortex-M3 processor, it may take several clock cycles before the interrupt source is actually cleared. Therefore, it is recommended that the interrupt source be cleared early in the interrupt handler (as opposed to the very last action) to avoid returning from the interrupt handler before the interrupt source is actually cleared. Failure to do so may result in the interrupt handler being immediately reentered (because the interrupt controller still sees the interrupt source asserted).

Returns:

None.

17.2.2.23 UARTIntDisable

Disables individual UART interrupt sources.

Prototype:

```
void
UARTIntDisable(unsigned long ulBase,
               unsigned long ulIntFlags)
```

Parameters:

ulBase is the base address of the UART port.

ullIntFlags is the bit mask of the interrupt sources to be disabled.

Description:

This function disables the indicated UART interrupt sources. Only the sources that are enabled can be reflected to the processor interrupt; disabled sources have no effect on the processor.

The *ullIntFlags* parameter has the same definition as the *ullIntFlags* parameter to [UARTIntEnable\(\)](#).

Returns:

None.

17.2.2.24 UARTIntEnable

Enables individual UART interrupt sources.

Prototype:

```
void
UARTIntEnable(unsigned long ulBase,
              unsigned long ulIntFlags)
```

Parameters:

ulBase is the base address of the UART port.

ullIntFlags is the bit mask of the interrupt sources to be enabled.

Description:

This function enables the indicated UART interrupt sources. Only the sources that are enabled can be reflected to the processor interrupt; disabled sources have no effect on the processor.

The *ullIntFlags* parameter is the logical OR of any of the following:

- **UART_INT_OE** - Overrun Error interrupt
- **UART_INT_BE** - Break Error interrupt
- **UART_INT_PE** - Parity Error interrupt
- **UART_INT_FE** - Framing Error interrupt
- **UART_INT_RT** - Receive Timeout interrupt
- **UART_INT_TX** - Transmit interrupt
- **UART_INT_RX** - Receive interrupt
- **UART_INT_DSR** - DSR interrupt
- **UART_INT_DCD** - DCD interrupt
- **UART_INT_CTS** - CTS interrupt
- **UART_INT_RI** - RI interrupt

Returns:

None.

17.2.2.25 UARTIntRegister

Registers an interrupt handler for a UART interrupt.

Prototype:

```
void
UARTIntRegister(unsigned long ulBase,
                void (*pfnHandler)(void))
```

Parameters:

ulBase is the base address of the UART port.

pfnHandler is a pointer to the function to be called when the UART interrupt occurs.

Description:

This function does the actual registering of the interrupt handler. This function enables the global interrupt in the interrupt controller; specific UART interrupts must be enabled via [UARTIntEnable\(\)](#). It is the interrupt handler's responsibility to clear the interrupt source.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

17.2.2.26 UARTIntStatus

Gets the current interrupt status.

Prototype:

```
unsigned long
UARTIntStatus(unsigned long ulBase,
               tBoolean bMasked)
```

Parameters:

ulBase is the base address of the UART port.

bMasked is **false** if the raw interrupt status is required and **true** if the masked interrupt status is required.

Description:

This function returns the interrupt status for the specified UART. Either the raw interrupt status or the status of interrupts that are allowed to reflect to the processor can be returned.

Returns:

Returns the current interrupt status, enumerated as a bit field of values described in [UARTIntEnable\(\)](#).

17.2.2.27 UARTIntUnregister

Unregisters an interrupt handler for a UART interrupt.

Prototype:

```
void
UARTIntUnregister(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function does the actual unregistering of the interrupt handler. It clears the handler to be called when a UART interrupt occurs. This function also masks off the interrupt in the interrupt controller so that the interrupt handler no longer is called.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

17.2.2.28 UARTModemControlClear

Clears the states of the DTR and/or RTS modem control signals.

Prototype:

```
void
UARTModemControlClear(unsigned long ulBase,
                      unsigned long ulControl)
```

Parameters:

ulBase is the base address of the UART port.

ulControl is a bit-mapped flag indicating which modem control bits should be set.

Description:

This function clears the states of the DTR or RTS modem handshake outputs from the UART.

The *ulControl* parameter is the logical OR of any of the following:

- **UART_OUTPUT_DTR** - The Modem Control DTR signal
- **UART_OUTPUT_RTS** - The Modem Control RTS signal

Note:

The availability of hardware modem handshake signals varies with the UART in use. Please consult the datasheet for the part you are using to determine whether this support is available.

Returns:

None.

17.2.2.29 UARTModemControlGet

Gets the states of the DTR and RTS modem control signals.

Prototype:

```
unsigned long
UARTModemControlGet(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function returns the current states of each of the two UART modem control signals, DTR and RTS.

Note:

The availability of hardware modem handshake signals varies with the UART in use. Please consult the datasheet for the part you are using to determine whether this support is available.

Returns:

Returns the states of the handshake output signals. This will be a logical OR combination of values **UART_OUTPUT_RTS** and **UART_OUTPUT_DTR** where the presence of each flag indicates that the associated signal is asserted.

17.2.2.30 UARTModemControlSet

Sets the states of the DTR and/or RTS modem control signals.

Prototype:

```
void
UARTModemControlSet(unsigned long ulBase,
                    unsigned long ulControl)
```


Parameters:

ulBase is the base address of the UART port.

ulControl is a bit-mapped flag indicating which modem control bits should be set.

Description:

This function sets the states of the DTR or RTS modem handshake outputs from the UART.

The *ulControl* parameter is the logical OR of any of the following:

- **UART_OUTPUT_DTR** - The Modem Control DTR signal
- **UART_OUTPUT_RTS** - The Modem Control RTS signal

Note:

The availability of hardware modem handshake signals varies with the UART in use. Please consult the datasheet for the part you are using to determine whether this support is available.

Returns:

None.

17.2.2.31 UARTModemStatusGet

Gets the states of the RI, DCD, DSR and CTS modem status signals.

Prototype:

```
unsigned long
UARTModemStatusGet(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function returns the current states of each of the four UART modem status signals, RI, DCD, DSR and CTS.

Note:

The availability of hardware modem handshake signals varies with the UART in use. Please consult the datasheet for the part you are using to determine whether this support is available.

Returns:

Returns the states of the handshake output signals. This value will be a logical OR combination of values **UART_INPUT_RI**, **UART_INPUT_DCD**, **UART_INPUT_CTS** and **UART_INPUT_DSR** where the presence of each flag indicates that the associated signal is asserted.

17.2.2.32 UARTParityModeGet

Gets the type of parity currently being used.

Prototype:

```
unsigned long
UARTParityModeGet(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function gets the type of parity used for transmitting data and expected when receiving data.

Returns:

Returns the current parity settings, specified as one of **UART_CONFIG_PAR_NONE**, **UART_CONFIG_PAR_EVEN**, **UART_CONFIG_PAR_ODD**, **UART_CONFIG_PAR_ONE**, or **UART_CONFIG_PAR_ZERO**.

17.2.2.33 UARTParityModeSet

Sets the type of parity.

Prototype:

```
void
UARTParityModeSet(unsigned long ulBase,
                  unsigned long ulParity)
```

Parameters:

ulBase is the base address of the UART port.

ulParity specifies the type of parity to use.

Description:

This function sets the type of parity to use for transmitting and expect when receiving. The *ulParity* parameter must be one of **UART_CONFIG_PAR_NONE**, **UART_CONFIG_PAR_EVEN**, **UART_CONFIG_PAR_ODD**, **UART_CONFIG_PAR_ONE**, or **UART_CONFIG_PAR_ZERO**. The last two allow direct control of the parity bit; it is always either one or zero based on the mode.

Returns:

None.

17.2.2.34 UARTRxErrorClear

Clears all reported receiver errors.

Prototype:

```
void
UARTRxErrorClear(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function is used to clear all receiver error conditions reported via [UARTRxErrorGet\(\)](#). If using the overrun, framing error, parity error or break interrupts, this function must be called after clearing the interrupt to ensure that later errors of the same type trigger another interrupt.

Returns:

None.

17.2.2.35 UARTRxErrorGet

Gets current receiver errors.

Prototype:

```
unsigned long  
UARTRxErrorGet(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function returns the current state of each of the 4 receiver error sources. The returned errors are equivalent to the four error bits returned via the previous call to [UARTCharGet\(\)](#) or [UARTCharGetNonBlocking\(\)](#) with the exception that the overrun error is set immediately the overrun occurs rather than when a character is next read.

Returns:

Returns a logical OR combination of the receiver error flags, **UART_RXERROR_FRAMING**, **UART_RXERROR_PARITY**, **UART_RXERROR_BREAK** and **UART_RXERROR_OVERRUN**.

17.2.2.36 UARTSmartCardDisable

Disables ISO7816 smart card mode on the specified UART.

Prototype:

```
void  
UARTSmartCardDisable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function clears the SMART (ISO7816 smart card) bits in the UART control register.

Returns:

None.

17.2.2.37 UARTSmartCardEnable

Enables ISO7816 smart card mode on the specified UART.

Prototype:

```
void  
UARTSmartCardEnable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function enables the SMART control bit for ISO7816 smart card mode on the UART. This call also sets 8 bit word length and even parity as required by ISO7816.

Returns:

None.

17.2.2.38 UARTSpaceAvail

Determines if there is any space in the transmit FIFO.

Prototype:

```
tBoolean
UARTSpaceAvail(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function returns a flag indicating whether or not there is space available in the transmit FIFO.

Returns:

Returns **true** if there is space available in the transmit FIFO or **false** if there is no space available in the transmit FIFO.

17.2.2.39 UARTTxIntModeGet

Returns the current operating mode for the UART transmit interrupt.

Prototype:

```
unsigned long
UARTTxIntModeGet(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the UART port.

Description:

This function returns the current operating mode for the UART transmit interrupt. The return value is **UART_TXINT_MODE_EOT** if the transmit interrupt is currently set to be asserted once the transmitter is completely idle - the transmit FIFO is empty and all bits, including any stop bits, have cleared the transmitter. The return value is **UART_TXINT_MODE_FIFO** if the interrupt is set to be asserted based upon the level of the transmit FIFO.

Returns:

Returns **UART_TXINT_MODE_FIFO** or **UART_TXINT_MODE_EOT**.

17.2.2.40 UARTTxIntModeSet

Sets the operating mode for the UART transmit interrupt.

Prototype:

```
void
UARTTxIntModeSet(unsigned long ulBase,
                  unsigned long ulMode)
```

Parameters:

ulBase is the base address of the UART port.

ulMode is the operating mode for the transmit interrupt. It may be **UART_TXINT_MODE_EOT** to trigger interrupts when the transmitter is idle or **UART_TXINT_MODE_FIFO** to trigger based on the current transmit FIFO level.

Description:

This function allows the mode of the UART transmit interrupt to be set. By default, the transmit interrupt is asserted when the FIFO level falls past a threshold set via a call to [UARTFIFOLevelSet\(\)](#). Alternatively, if this function is called with **ulMode** set to **UART_TXINT_MODE_EOT**, the transmit interrupt is asserted once the transmitter is completely idle - the transmit FIFO is empty and all bits, including any stop bits, have cleared the transmitter.

Returns:

None.

17.3 Programming Example

The following example shows how to use the UART API to initialize the UART, transmit characters, and receive characters.

```
// Initialize the UART. Set the baud rate, number of data bits,
// turn off parity, number of stop bits, and stick mode.
UARTConfigSetExpClk(UART0_BASE, SysCtlClockGet(), 38400,
                    (UART_CONFIG_WLEN_8 | UART_CONFIG_STOP_ONE |
                     UART_CONFIG_PAR_NONE));

// Enable the UART.
UARTEnable(UART0_BASE);

// Check for characters. This will spin here until a character
// is placed into the receive FIFO.
while(!UARTCharsAvail(UART0_BASE))
{
}

// Get the character(s) in the receive FIFO.
while(UARTCharGetNonBlocking(UART0_BASE))
{
}

// Put a character in the output buffer.
UARTCharPut(UART0_BASE, 'c');

// Disable the UART.
UARTDisable(UART0_BASE);
```


18 uCRC Peripheral

Introduction	295
API Functions	295
Programming Example	297

18.1 Introduction

The microCRC (uCRC) API provides functions to configure the Concerto uCRC (Cyclic Redundancy Check) peripheral. The uCRC controller is designed to work with the the ARM Cortex-M3 processor and provides an efficient and low-overhead means of verifying large blocks of data in the system.

The uCRC controller has the following features:

- Single cycle per byte CRC's
- Multiple CRC widths and polynomials
- Very easy to use

The uCRC controller supports several different CRC sizes and polynomials:

- **CRC8** Polynomial = 0x03 - Basic byte wide CRC. One cycle per byte.
- **CRC16-1** Polynomial = 0x8005 - Half word CRC. Two cycles per half word.
- **CRC16-2** Polynomial = 0x1021 - Half word CRC. Two cycles per half word.
- **CRC32** Polynomial = 0x04C11DB7 - Full word CRC. Four cycles per word.

The microCRC works by spying on the Concerto devices' address and data busses. Anytime memory is accessed from a specially mapped region (OR 0x04000000 to any RAM or flash address), a CRC is performed on that byte. It should be noted that the microCRC peripheral can only perform CRCs a byte at a time.

The naming convention for the microCRC controller is to use the Greek letter "mu" to represent "micro". For the purposes of this document, and in the software library function names, a lower case "u" will be used in place of "mu" when the controller is referred to as "uCRC".

This driver is contained in `driverlib/ucrc.c`, with `driverlib/ucrc.h` containing the API definitions for use by applications.

18.2 API Functions

Functions

- unsigned long [UCRCCalculation](#) (unsigned long ulBase, unsigned long ulType, unsigned char *pucBuffer, unsigned long ulNumBytes)
- void [UCRCClear](#) (unsigned long ulBase)
- void [UCRCConfig](#) (unsigned long ulBase, unsigned long ulType)

18.2.1 Detailed Description

The uCRC API functions provide a means to enable and configure the Concerto uCRC controller to perform CRC calculations. These functions are not optimized for speed, so if additional performance is required the customer must implement this functionality to suite their application.

Only a single function call to [UCRCCalculation\(\)](#) is all that is needed in most cases to perform a CRC calculation.

18.2.2 Function Documentation

18.2.2.1 UCRCCalculation

Perform a CRC calculation

Prototype:

```
unsigned long
UCRCCalculation(unsigned long ulBase,
                 unsigned long ulType,
                 unsigned char *pucBuffer,
                 unsigned long ulNumBytes)
```

Parameters:

ulBase is the base address of the uCRC peripheral.

ulType is the type of CRC to perform.

pucBuffer is the buffer to perform CRC calculation on.

ulNumBytes is the number of bytes to perform CRC on.

Description:

Configures the uCRC module and performs the requested CRC operation.

Returns:

Requested CRC value for the memory given.

18.2.2.2 UCRCClear

Clear the uCRC result register.

Prototype:

```
void
UCRCClear(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the uCRC peripheral.

Description:

Resets the result register back to initial state

18.2.2.3 void UCRCConfig (unsigned long *ulBase*, unsigned long *ulType*)

Configure the uCRC module for a particular type of CRC.

Parameters:

ulBase is the base address of the uCRC peripheral.

ulType is the type of CRC to perform.

Configures the UCRC peripheral for a given type of CRC. The *ucType* parameter may be any of the following:

- **UCRC_CONFIG_CRC8** - perform CRC8 (Polynomial = 0x03)
- **UCRC_CONFIG_CRC16_1** - perform CRC16 (Polynomial = 0x8005)
- **UCRC_CONFIG_CRC16_2** - perform CRC16 (Polynomial = 0x1021)
- **UCRC_CONFIG_CRC32** - perform CRC32 (Polynomial = 0x04C11DB7)

18.3 Programming Example

The following example sets up the uCRC controller to perform a CRC32 calculation on 128 bytes of RAM:

```
unsigned long ulMyCRC;

// Calculate a CRC32 on 128 bytes of RAM at address 0x20000000
// and store the result in ulMyCRC
ulMyCRC = UCRCCalculation(UCRC_BASE, UCRC_CONFIG_CRC32,
                          (unsigned char *) 0x20000000, 0x80)
```


19 uDMA Controller

Introduction	299
API Functions	300
Programming Example	322

19.1 Introduction

The microDMA (uDMA) API provides functions to configure the Concerto uDMA (Direct Memory Access) controller. The uDMA controller is designed to work with the the ARM Cortex-M3 processor and provides an efficient and low-overhead means of transferring blocks of data in the system.

The uDMA controller has the following features:

- dedicated channels for supported peripherals
- one channel each for receive and transmit for devices with receive and transmit paths
- dedicated channel for software initiated data transfers
- channels can be independently configured and operated
- an arbitration scheme that is configurable per channel
- two levels of priority
- subordinate to Cortex-M3 processor bus usage
- data sizes of 8, 16, or 32 bits
- address increment of byte, half-word, word, or none
- maskable device requests
- optional software initiated transfers on any channel
- interrupt on transfer completion

The uDMA controller supports several different transfer modes, allowing for complex transfer schemes. The following transfer modes are provided:

- **Basic** mode performs a simple transfer when request is asserted by a device. This is appropriate to use with peripherals where the peripheral asserts the request line whenever data should be transferred. The transfer will pause if request is de-asserted, even if the transfer is not complete.
- **Auto-request** mode performs a simple transfer that is started by a request, but will always complete the entire transfer, even if request is de-asserted. This is appropriate to use with software initiated transfers.
- **Ping-Pong** mode is used to transfer data to or from two buffers, switching from one buffer to the other as each buffer fills. This mode is appropriate to use with peripherals as a way to ensure a continuous flow of data to or from the peripheral. However, it is more complex to set up and requires code to manage the ping-pong buffers in the interrupt handler.
- **Memory scatter/gather** mode is a complex mode that provides a way to set up a list of transfer “tasks” for the uDMA controller. Blocks of data can be transferred to and from arbitrary locations in memory.

- **Peripheral scatter/gather** mode is similar to memory scatter/gather mode except that it is controlled by a peripheral request.

Detailed explanation of the various transfer modes is beyond the scope of this document. Please refer to the device data sheet for more information on the operation of the uDMA controller.

The naming convention for the microDMA controller is to use the Greek letter “mu” to represent “micro”. For the purposes of this document, and in the software library function names, a lower case “u” will be used in place of “mu” when the controller is referred to as “uDMA”.

This driver is contained in `driverlib/udma.c`, with `driverlib/udma.h` containing the API definitions for use by applications.

19.2 API Functions

Defines

- `uDMATaskStructEntry`(ulTransferCount, ullItemSize, ulSrcIncrement, pvSrcAddr, ulDstIncrement, pvDstAddr, ulArbSize, ulMode)

Functions

- void `uDMAChannel0_7SelectAltMapping` (unsigned long ulAltPeriphs)
- void `uDMAChannel0_7SelectDefault` (unsigned long ulDefPeriphs)
- void `uDMAChannel16_23SelectAltMapping` (unsigned long ulAltPeriphs)
- void `uDMAChannel16_23SelectDefault` (unsigned long ulDefPeriphs)
- void `uDMAChannel24_31SelectAltMapping` (unsigned long ulAltPeriphs)
- void `uDMAChannel24_31SelectDefault` (unsigned long ulDefPeriphs)
- void `uDMAChannel8_15SelectAltMapping` (unsigned long ulAltPeriphs)
- void `uDMAChannel8_15SelectDefault` (unsigned long ulDefPeriphs)
- void `uDMAChannelAttributeDisable` (unsigned long ulChannelNum, unsigned long ulAttr)
- void `uDMAChannelAttributeEnable` (unsigned long ulChannelNum, unsigned long ulAttr)
- unsigned long `uDMAChannelAttributeGet` (unsigned long ulChannelNum)
- void `uDMAChannelControlSet` (unsigned long ulChannelStructIndex, unsigned long ulControl)
- void `uDMAChannelDisable` (unsigned long ulChannelNum)
- void `uDMAChannelEnable` (unsigned long ulChannelNum)
- tBoolean `uDMAChannelsEnabled` (unsigned long ulChannelNum)
- unsigned long `uDMAChannelModeGet` (unsigned long ulChannelStructIndex)
- void `uDMAChannelRequest` (unsigned long ulChannelNum)
- void `uDMAChannelScatterGatherSet` (unsigned long ulChannelNum, unsigned ulTaskCount, void *pvTaskList, unsigned long ullsPeriphSG)
- unsigned long `uDMAChannelSizeGet` (unsigned long ulChannelStructIndex)
- void `uDMAChannelTransferSet` (unsigned long ulChannelStructIndex, unsigned long ulMode, void *pvSrcAddr, void *pvDstAddr, unsigned long ulTransferSize)
- void * `uDMAControlAlternateBaseGet` (void)
- void * `uDMAControlBaseGet` (void)

- void [uDMAControlBaseSet](#) (void *pControlTable)
- void [uDMADisable](#) (void)
- void [uDMAEnable](#) (void)
- void [uDMAErrorStatusClear](#) (void)
- unsigned long [uDMAErrorStatusGet](#) (void)
- void [uDMAIntRegister](#) (unsigned long ulIntChannel, void (*pfnHandler)(void))
- void [uDMAIntUnregister](#) (unsigned long ulIntChannel)

19.2.1 Detailed Description

The uDMA API functions provide a means to enable and configure the Concerto microDMA controller to perform DMA transfers.

The general order of function calls to set up and perform a uDMA transfer is the following:

- [uDMAEnable\(\)](#) is called once to enable the controller.
- [uDMAControlBaseSet\(\)](#) is called once to set the channel control table.
- [uDMAChannelAttributeEnable\(\)](#) is called once or infrequently to configure the behavior of the channel.
- [uDMAChannelControlSet\(\)](#) is used to set up characteristics of the data transfer. It only needs to be called once if the nature of the data transfer does not change.
- [uDMAChannelTransferSet\(\)](#) is used to set the buffer pointers and size for a transfer. It is called before each new transfer.
- [uDMAChannelEnable\(\)](#) enables a channel to perform data transfers.
- [uDMAChannelRequest\(\)](#) is used to initiate a software based transfer. This is normally not used for peripheral based transfers.

In order to use the uDMA controller, you must first enable it by calling [uDMAEnable\(\)](#). You can later disable it, if no longer needed, by calling [uDMADisable\(\)](#).

Once the uDMA controller is enabled, you must tell it where to find the channel control structures in system memory. This is done by using the function [uDMAControlBaseSet\(\)](#) and passing a pointer to the base of the channel control structure. The control structure must be allocated by the application. One way to do this is to declare an array of data type `char` or `unsigned char`. In order to support all channels and transfer modes, the control table array should be 1024 bytes, but it can be fewer depending on transfer modes used and number of channels actually used.

Note:

The control table must be aligned on a 1024 byte boundary.

The uDMA controller supports multiple channels. Each channel has a set of attribute flags to control certain uDMA features and channel behavior. The attribute flags are set with the function [uDMAChannelAttributeEnable\(\)](#) and cleared with [uDMAChannelAttributeDisable\(\)](#). The setting of the channel attribute flags can be queried by using the function [uDMAChannelAttributeGet\(\)](#).

Next, the control parameters of the DMA transfer must be set. These parameters control the size and address increment of the data items to be transferred. The function [uDMAChannelControlSet\(\)](#) is used to set up these control parameters.

All of the functions mentioned so far are used only once or infrequently to set up the uDMA channel and transfer. In order to set the transfer addresses, transfer size, and transfer mode, use the

function `uDMAChannelTransferSet()`. This function must be called for each new transfer. Once everything is set up, the channel is enabled by calling `uDMAChannelEnable()`, which must be done before each new transfer. The uDMA controller will automatically disable the channel at the completion of a transfer. A channel can be manually disabled by using `uDMAChannelDisable()`.

There are additional functions that can be used to query the status of a channel, either from an interrupt handler or in polling fashion. The function `uDMAChannelSizeGet()` is used to find the amount of data remaining to transfer on a channel. This will be zero when a transfer is complete. The function `uDMAChannelModeGet()` can be used to find the transfer mode of a uDMA channel. This is usually used to see if the mode indicates stopped which means that a transfer has completed on a channel that was previously running. The function `uDMAChannelsEnabled()` can be used to determine if a particular channel is enabled.

If the application is using run-time interrupt registration (see `IntRegister()`), then the function `uDMAIntRegister()` can be used to install an interrupt handler for the uDMA controller. This function will also enable the interrupt on the system interrupt controller. If compile-time interrupt registration is used, then call the function `IntEnable()` to enable uDMA interrupts. When an interrupt handler has been installed with `uDMAIntRegister()`, it can be removed by calling `uDMAIntUnregister()`.

This interrupt handler is only for software initiated transfers or errors. uDMA interrupts for a peripheral occur on the peripheral's dedicated interrupt channel, and should be handled by the peripheral interrupt handler. It is not necessary to acknowledge or clear uDMA interrupt sources. They are cleared automatically when they are serviced.

The uDMA interrupt handler should use the function `uDMAErrorStatusGet()` to test if a uDMA error occurred. If so, the interrupt must be cleared by calling `uDMAErrorStatusClear()`.

Note:

Many of the API functions take a channel parameter that includes the logical OR of one of the values `UDMA_PRI_SELECT` or `UDMA_ALT_SELECT` to choose the primary or alternate control structure. For Basic and Auto transfer modes, only the primary control structure is needed. The alternate control structure is only needed for complex transfer modes of Ping-pong or Scatter/gather. Refer to the device data sheet for detailed information about transfer modes.

Special considerations for using scatter-gather operations

In order to use the scatter-gather modes of the uDMA controller, you must prepare a "task" list in memory that describes the scatter-gather operations. There is a helper macro, `uDMATaskStructEntry` provided to help create the initialization values for the task list structure. Please see the documentation for this macro which includes a code snippet showing how it is to be used.

Once the task list is prepared, the appropriate uDMA channel must be configured for a scatter-gather operation. The best way to do this is to use the function `uDMAChannelScatterGatherSet()`. Alternatively, the functions `uDMAChannelControlSet()` followed by `uDMAChannelTransferSet()` can also be used.

Note:

The scatter-gather task list must be resident in SRAM. The uDMA controller cannot read from flash memory.

About uDMA Channel Function Parameters

Many of the uDMA API functions require a channel number as a parameter. There are two different uses of the channel number. In some cases, it is the number of the uDMA channel and is used to read or write registers within the uDMA controller. In this case it is simply the channel number with no additional qualifier.

However, in other cases the channel number that is supplied as a parameter is really an index into the uDMA channel control structure. Because every uDMA channel has a primary and an alternate channel control structure, this must also be specified as part of the channel number. This is done by passing a value for the channel parameter that is the logical OR of the actual channel number and one of **UDMA_PRI_SELECT** or **UDMA_ALT_SELECT**. The default is the same as **UDMA_PRI_SELECT** so if you do not specify then the primary channel control structure is used, which is the right thing in most cases.

Note:

When **UDMA_ALT_SELECT** is specified, what is really happening is that channel index 32-63 is being used. This is because the alternate channel control structures for channels 0-31 are located at index locations 32-63 in the channel control table.

Here is an example of the first case. In this example a uDMA channel is to be enabled, and only the channel number is used because this is programming a register in the uDMA controller.

```
uDMAChannelEnable (UDMA_CHANNEL_UART0RX);
```

Here is an example of the second case. In this example the channel control structure is to be modified to configure some transfer parameters. Therefore in addition to specifying the channel index, the primary or alternate control structure must also be selected.

```
uDMAChannelControlSet (UDMA_CHANNEL_UART0RX | UDMA_PRI_SELECT, ...);
```

In order to help make it clear when one or the other form is to be used, the parameters are named differently in the API description. For functions that require just the channel number, the name of the parameter is *ulChannelNum*. For functions that require the channel index of the channel control structure, the name of the parameter is *ulChannelStructIdx*.

Selecting uDMA Channels

The uDMA controller has 32 channels, and therefore most of the API functions take a channel number with a value from 0-31 or a channel index with a value from 0-63 (the 32-63 is specified with the logical OR of the channel number with **UDMA_ALT_SELECT**). In order to avoid the need for hardcoded channel numbers in code, macros are provided that map channel names to channel numbers.

To use the default channel mapping, you may use one of the following choices whenever a channel number or index is needed. This list is all the possible channels that are defined by the API. However not all channels are available on all parts, depending on which peripherals are available on the part and which of those support uDMA. Please consult the data sheet for your specific part to see which uDMA channels are supported.

- **UDMA_CHANNEL_USBEP1RX** for USB endpoint 1 receive
- **UDMA_CHANNEL_USBEP1TX** for USB endpoint 1 transmit
- **UDMA_CHANNEL_USBEP2RX** for USB endpoint 2 receive
- **UDMA_CHANNEL_USBEP2TX** for USB endpoint 2 transmit
- **UDMA_CHANNEL_USBEP3RX** for USB endpoint 3 receive
- **UDMA_CHANNEL_USBEP3TX** for USB endpoint 3 transmit
- **UDMA_CHANNEL_ETH0RX** for ethernet receive
- **UDMA_CHANNEL_ETH0TX** for ethernet transmit

- **UDMA_CHANNEL_UART0RX** for UART 0 receive channel
- **UDMA_CHANNEL_UART0TX** for UART 0 transmit channel
- **UDMA_CHANNEL_UART1RX** for UART 1 receive channel
- **UDMA_CHANNEL_UART1TX** for UART 1 transmit channel
- **UDMA_CHANNEL_SSI0RX** for SSI 0 receive channel
- **UDMA_CHANNEL_SSI0TX** for SSI 0 transmit channel
- **UDMA_CHANNEL_SSI1RX** for SSI 1 receive channel
- **UDMA_CHANNEL_SSI1TX** for SSI 1 transmit channel
- **UDMA_CHANNEL_ADC0** for ADC0 sequencer 0
- **UDMA_CHANNEL_ADC1** for ADC0 sequencer 1
- **UDMA_CHANNEL_ADC2** for ADC0 sequencer 2
- **UDMA_CHANNEL_ADC3** for ADC0 sequencer 3
- **UDMA_CHANNEL_TMR0A** for Timer 0A
- **UDMA_CHANNEL_TMR0B** for Timer 0B
- **UDMA_CHANNEL_TMR1A** for Timer 1A
- **UDMA_CHANNEL_TMR1B** for Timer 1B
- **UDMA_CHANNEL_I2S0RX** for I2S receive
- **UDMA_CHANNEL_I2S0TX** for I2S transmit
- **UDMA_CHANNEL_SW** for the software dedicated uDMA channel

Some Concerto parts also provide a secondary channel mapping. For those parts, each channel has a secondary peripheral mapping. This is to allow more choices in channel mapping and to allow some additional peripherals to use uDMA that are not available in the default mapping.

In order to select the default or secondary channel mapping, use the functions `uDMAChannelSelectDefault()` or `uDMAChannelSelectSecondary()`. Each channel can be configured individually to use the default or secondary mapping. This provides a lot of flexibility for channel mapping.

For example, the default for channel 0 is `USBEP1RX`. However this channel also has a secondary mapping to `UART2RX`. If an application requires use of uDMA with `UART2` and does not use `USB`, then this channel could be remapped to `UART2RX` with the following function call:

```
uDMAChannelSelectSecondary(UDMA_DEF_USBEP1RX_SEC_UART2RX);
```

For channels that have been configured to use the secondary mapping, there is a set of macros to use for specifying the channel. Here is the list of channels when secondary mapping is used. As before, this is the full list, the actual channels available depend on which specific Concerto part is used.

- **UDMA_SEC_CHANNEL_UART2RX_0** for UART2 receive using uDMA channel 0
- **UDMA_SEC_CHANNEL_UART2TX_1** for UART2 transmit using uDMA channel 1
- **UDMA_SEC_CHANNEL_TMR3A** for Timer 3A
- **UDMA_SEC_CHANNEL_TMR3B** for Timer 3B
- **UDMA_SEC_CHANNEL_TMR2A_4** for Timer 2A using uDMA channel 4
- **UDMA_SEC_CHANNEL_TMR2B_5** for Timer 2B using uDMA channel 5
- **UDMA_SEC_CHANNEL_TMR2A_6** for Timer 2A using uDMA channel 6

- **UDMA_SEC_CHANNEL_TMR2B_7** for Timer 2B using uDMA channel 7
- **UDMA_SEC_CHANNEL_UART1RX** for UART1 receive
- **UDMA_SEC_CHANNEL_UART1TX** for UART1 transmit
- **UDMA_SEC_CHANNEL_SSI1RX** for SSI1 receive
- **UDMA_SEC_CHANNEL_SSI1TX** for SSI1 transmit
- **UDMA_SEC_CHANNEL_UART2RX_12** for UART2 receive using uDMA channel 12
- **UDMA_SEC_CHANNEL_UART2TX_13** for UART2 transmit using uDMA channel 13
- **UDMA_SEC_CHANNEL_TMR2A_14** for Timer 2A using uDMA channel 14
- **UDMA_SEC_CHANNEL_TMR2B_15** for Timer 2B using uDMA channel 15
- **UDMA_SEC_CHANNEL_TMR1A** for Timer 1A
- **UDMA_SEC_CHANNEL_TMR1B** for Timer 1B
- **UDMA_SEC_CHANNEL_EPI0RX** for EPI read
- **UDMA_SEC_CHANNEL_EPI0TX** for EPI write
- **UDMA_SEC_CHANNEL_ADC10** for ADC1 sequencer 0
- **UDMA_SEC_CHANNEL_ADC11** for ADC1 sequencer 1
- **UDMA_SEC_CHANNEL_ADC12** for ADC1 sequencer 2
- **UDMA_SEC_CHANNEL_ADC13** for ADC1 sequencer 3
- **UDMA_SEC_CHANNEL_SW** for the software dedicated uDMA channel

19.2.2 Define Documentation

19.2.2.1 uDMATaskStructEntry

A helper macro for building scatter-gather task table entries.

Definition:

```
#define uDMATaskStructEntry (ulTransferCount,
                             ulItemSize,
                             ulSrcIncrement,
                             pvSrcAddr,
                             ulDstIncrement,
                             pvDstAddr,
                             ulArbSize,
                             ulMode)
```

Parameters:

ulTransferCount is the count of items to transfer for this task.
ulItemSize is the bit size of the items to transfer for this task.
ulSrcIncrement is the bit size increment for source data.
pvSrcAddr is the starting address of the data to transfer.
ulDstIncrement is the bit size increment for destination data.
pvDstAddr is the starting address of the destination data.
ulArbSize is the arbitration size to use for the transfer task.
ulMode is the transfer mode for this task.

Description:

This macro is intended to be used to help populate a table of uDMA tasks for a scatter-gather transfer. This macro will calculate the values for the fields of a task structure entry based on the input parameters.

There are specific requirements for the values of each parameter. No checking is done so it is up to the caller to ensure that correct values are used for the parameters.

The *ulTransferCount* parameter is the number of items that will be transferred by this task. It must be in the range 1-1024.

The *ullItemSize* parameter is the bit size of the transfer data. It must be one of **UDMA_SIZE_8**, **UDMA_SIZE_16**, or **UDMA_SIZE_32**.

The *ulSrcIncrement* parameter is the increment size for the source data. It must be one of **UDMA_SRC_INC_8**, **UDMA_SRC_INC_16**, **UDMA_SRC_INC_32**, or **UDMA_SRC_INC_NONE**.

The *pvSrcAddr* parameter is a void pointer to the beginning of the source data.

The *ulDstIncrement* parameter is the increment size for the destination data. It must be one of **UDMA_DST_INC_8**, **UDMA_DST_INC_16**, **UDMA_DST_INC_32**, or **UDMA_DST_INC_NONE**.

The *pvDstAddr* parameter is a void pointer to the beginning of the location where the data will be transferred.

The *ulArbSize* parameter is the arbitration size for the transfer, and must be one of **UDMA_ARB_1**, **UDMA_ARB_2**, **UDMA_ARB_4**, and so on up to **UDMA_ARB_1024**. This is used to select the arbitration size in powers of 2, from 1 to 1024.

The *ulMode* parameter is the mode to use for this transfer task. It must be one of **UDMA_MODE_BASIC**, **UDMA_MODE_AUTO**, **UDMA_MODE_MEM_SCATTER_GATHER**, or **UDMA_MODE_PER_SCATTER_GATHER**. Note that normally all tasks will be one of the scatter-gather modes while the last task in a task list will be AUTO or BASIC.

This macro is intended to be used to initialize individual entries of a structure of **tDMAControlTable** type, like this:

```
tDMAControlTable MyTaskList[] =
{
    uDMATaskStructEntry(Task1Count,  UDMA_SIZE_8,
                        UDMA_SRC_INC_8,  MySourceBuf,
                        UDMA_DST_INC_8,  MyDestBuf,
                        UDMA_ARB_8,   UDMA_MODE_MEM_SCATTER_GATHER),
    uDMATaskStructEntry(Task2Count,  ... ),
}
```

Returns:

Nothing; this is not a function.

19.2.3 Function Documentation

19.2.3.1 uDMAChannel0_7SelectAltMapping

Selects the alternate peripheral for a set of uDMA channels between 0-7.

Prototype:

```
void  
uDMAChannel0_7SelectAltMapping(unsigned long ulAltPeriphs)
```

Parameters:

ulAltPeriphs is the logical or of the uDMA channels for which to use an alternate peripheral, instead of the default peripheral.

Description:

This function is used to select an alternate peripheral assignment for a set of uDMA channels between 0-7. By selecting the alternate peripheral assignment for a channel, the default peripheral assignment is no longer available for that channel.

The parameter *ulAltPeriphs* can be the logical OR of any of the following macros. For a single channel, only 1 of the alternate selections can be included in the logical OR. (i.e. cannot OR both a "_SEC_" peripheral selection and a "_THRD_" peripheral selection for the same DMA channel.)

- **UDMA_CHAN0_SEC_UART2RX**
- **UDMA_CHAN1_SEC_UART2TX**
- **UDMA_CHAN2_SEC_TMR3A**
- **UDMA_CHAN3_SEC_TMR3B**
- **UDMA_CHAN4_SEC_TMR2A**
- **UDMA_CHAN5_SEC_TMR2B**
- **UDMA_CHAN6_SEC_TMR2A**
- **UDMA_CHAN7_SEC_TMR2B**

Returns:

None.

19.2.3.2 uDMAChannel0_7SelectDefault

Selects the default peripheral for a set of uDMA channels between 0-7.

Prototype:

```
void  
uDMAChannel0_7SelectDefault(unsigned long ulDefPeriphs)
```

Parameters:

ulDefPeriphs is the logical or of the uDMA channels for which to use the default peripheral, instead of the secondary peripheral.

Description:

This function is used to select the default peripheral assignment for a set of uDMA channels between 0-7.

The parameter *ulDefPeriphs* can be the logical OR of any of the following macros.

- **UDMA_CHAN0_DEF_USBEP1RX_M**
- **UDMA_CHAN1_DEF_USBEP1TX_M**
- **UDMA_CHAN2_DEF_USBEP2RX_M**
- **UDMA_CHAN3_DEF_USBEP2TX_M**

- UDMA_CHAN4_DEF_USBEP3RX_M
- UDMA_CHAN5_DEF_USBEP3TX_M
- UDMA_CHAN6_DEF_ETH0RX_M
- UDMA_CHAN7_DEF_ETH0TX_M

Returns:

None.

19.2.3.3 uDMAChannel16_23SelectAltMapping

Selects the alternate peripheral for a set of uDMA channels between 16-23.

Prototype:

```
void
uDMAChannel16_23SelectAltMapping(unsigned long ulAltPeriphs)
```

Parameters:

ulAltPeriphs is the logical or of the uDMA channels for which to use an alternate peripheral, instead of the default peripheral.

Description:

This function is used to select an alternate peripheral assignment for a set of uDMA channels between 16-23. By selecting the alternate peripheral assignment for a channel, the default peripheral assignment is no longer available for that channel.

The parameter *ulAltPeriphs* can be the logical OR of any of the following macros. For a single channel, only 1 of the alternate selections can be included in the logical OR. (i.e. cannot OR both a "_SEC_" peripheral selection and a "_THRD_" peripheral selection for the same DMA channel.)

- UDMA_CHAN16_THRD_UART3RX
- UDMA_CHAN17_THRD_UART3TX
- UDMA_CHAN18_SEC_TMR1A
- UDMA_CHAN18_THRD_UART4RX
- UDMA_CHAN19_SEC_TMR1B
- UDMA_CHAN19_THRD_UART4TX
- UDMA_CHAN20_SEC_EPI0RX
- UDMA_CHAN21_SEC_EPI0TX
- UDMA_CHAN22_SEC_SW
- UDMA_CHAN22_THRD_ADCINT1
- UDMA_CHAN23_SEC_SW
- UDMA_CHAN23_THRD_ADCINT2

Returns:

None.

19.2.3.4 uDMAChannel16_23SelectDefault

Selects the default peripheral for a set of uDMA channels between 16-23.

Prototype:

```
void  
uDMAChannel16_23SelectDefault(unsigned long ulDefPeriphs)
```

Parameters:

ulDefPeriphs is the logical or of the uDMA channels for which to use the default peripheral, instead of the secondary peripheral.

Description:

This function is used to select the default peripheral assignment for a set of uDMA channels between 16-23.

The parameter *ulDefPeriphs* can be the logical OR of any of the following macros.

- **UDMA_CHAN16_DEF_RSVD_M**
- **UDMA_CHAN17_DEF_RSVD_M**
- **UDMA_CHAN18_DEF_TMR0A_M**
- **UDMA_CHAN19_DEF_TMR0B_M**
- **UDMA_CHAN20_DEF_TMR1A_M**
- **UDMA_CHAN21_DEF_TMR1B_M**
- **UDMA_CHAN22_DEF_UART1RX_M**
- **UDMA_CHAN23_DEF_UART1TX_M**

Returns:

None.

19.2.3.5 uDMAChannel24_31SelectAltMapping

Selects the alternate peripheral for a set of uDMA channels between 24-31.

Prototype:

```
void  
uDMAChannel24_31SelectAltMapping(unsigned long ulAltPeriphs)
```

Parameters:

ulAltPeriphs is the logical or of the uDMA channels for which to use an alternate peripheral, instead of the default peripheral.

Description:

This function is used to select an alternate peripheral assignment for a set of uDMA channels between 24-31. By selecting the alternate peripheral assignment for a channel, the default peripheral assignment is no longer available for that channel.

The parameter *ulAltPeriphs* can be the logical OR of any of the following macros. For a single channel, only 1 of the alternate selections can be included in the logical OR. (i.e. cannot OR both a "_SEC_" peripheral selection and a "_THRD_" peripheral selection for the same DMA channel.)

- **UDMA_CHAN26_THRD_ADCINT3**
- **UDMA_CHAN27_THRD_ADCINT4**
- **UDMA_CHAN28_SEC_SW**
- **UDMA_CHAN28_THRD_ADCINT5**

- UDMA_CHAN29_SEC_SW
- UDMA_CHAN29_THRD_ADCINT6
- UDMA_CHAN30_SEC_SW
- UDMA_CHAN30_THRD_ADCINT7
- UDMA_CHAN31_THRD_ADCINT8

Returns:
None.

19.2.3.6 uDMAChannel24_31SelectDefault

Selects the default peripheral for a set of uDMA channels between 24-31.

Prototype:

```
void
uDMAChannel24_31SelectDefault(unsigned long ulDefPeriphs)
```

Parameters:

ulDefPeriphs is the logical or of the uDMA channels for which to use the default peripheral, instead of the secondary peripheral.

Description:

This function is used to select the default peripheral assignment for a set of uDMA channels between 24-31.

The parameter *ulDefPeriphs* can be the logical OR of any of the following macros.

- UDMA_CHAN24_DEF_SSI1RX_M
- UDMA_CHAN25_DEF_SSI1TX_M
- UDMA_CHAN26_DEF_SW_M
- UDMA_CHAN27_DEF_SW_M
- UDMA_CHAN28_DEF_RSVD_M
- UDMA_CHAN29_DEF_RSVD_M
- UDMA_CHAN30_DEF_SW_M
- UDMA_CHAN31_DEF_RSVD_M

Returns:
None.

19.2.3.7 uDMAChannel8_15SelectAltMapping

Selects the alternate peripheral for a set of uDMA channels between 8-15.

Prototype:

```
void
uDMAChannel8_15SelectAltMapping(unsigned long ulAltPeriphs)
```

Parameters:

ulAltPeriphs is the logical or of the uDMA channels for which to use an alternate peripheral, instead of the default peripheral.

Description:

This function is used to select an alternate peripheral assignment for a set of uDMA channels between 8-15. By selecting the alternate peripheral assignment for a channel, the default peripheral assignment is no longer available for that channel.

The parameter *ulAltPeriphs* can be the logical OR of any of the following macros. For a single channel, only 1 of the alternate selections can be included in the logical OR. (i.e. cannot OR both a "_SEC_" peripheral selection and a "_THRD_" peripheral selection for the same DMA channel.)

- UDMA_CHAN8_SEC_UART1RX
- UDMA_CHAN9_SEC_UART1TX
- UDMA_CHAN10_SEC_SSI1RX
- UDMA_CHAN11_SEC_SSI1TX
- UDMA_CHAN12_SEC_UART2RX
- UDMA_CHAN12_THRD_SSI2RX
- UDMA_CHAN13_SEC_UART2TX
- UDMA_CHAN13_THRD_SSI2TX
- UDMA_CHAN14_SEC_TMR2A
- UDMA_CHAN14_THRD_SSI3RX
- UDMA_CHAN15_SEC_TMR2B
- UDMA_CHAN15_THRD_SSI3TX

Returns:

None.

19.2.3.8 uDMAChannel8_15SelectDefault

Selects the default peripheral for a set of uDMA channels between 8-15.

Prototype:

```
void  
uDMAChannel8_15SelectDefault(unsigned long ulDefPeriphs)
```

Parameters:

ulDefPeriphs is the logical or of the uDMA channels for which to use the default peripheral, instead of the secondary peripheral.

Description:

This function is used to select the default peripheral assignment for a set of uDMA channels between 8-15.

The parameter *ulDefPeriphs* can be the logical OR of any of the following macros.

- UDMA_CHAN8_DEF_UART0RX_M
- UDMA_CHAN9_DEF_UART0TX_M
- UDMA_CHAN10_DEF_SSI0RX_M
- UDMA_CHAN11_DEF_SSI0TX_M
- UDMA_CHAN12_DEF_SW_M
- UDMA_CHAN13_DEF_SW_M
- UDMA_CHAN14_DEF_RSVD_M

■ UDMA_CHAN15_DEF_RSVD_M**Returns:**

None.

19.2.3.9 uDMAChannelAttributeDisable

Disables attributes of a uDMA channel.

Prototype:

```
void  
uDMAChannelAttributeDisable(unsigned long ulChannelNum,  
                             unsigned long ulAttr)
```

Parameters:***ulChannelNum*** is the channel to configure.***ulAttr*** is a combination of attributes for the channel.**Description:**

This function is used to disable attributes of a uDMA channel.

The *ulAttr* parameter is the logical OR of any of the following:

- **UDMA_ATTR_USEBURST** is used to restrict transfers to use only a burst mode.
- **UDMA_ATTR_ALTSELECT** is used to select the alternate control structure for this channel.
- **UDMA_ATTR_HIGH_PRIORITY** is used to set this channel to high priority.
- **UDMA_ATTR_REQMASK** is used to mask the hardware request signal from the peripheral for this channel.

Returns:

None.

19.2.3.10 uDMAChannelAttributeEnable

Enables attributes of a uDMA channel.

Prototype:

```
void  
uDMAChannelAttributeEnable(unsigned long ulChannelNum,  
                             unsigned long ulAttr)
```

Parameters:***ulChannelNum*** is the channel to configure.***ulAttr*** is a combination of attributes for the channel.**Description:**The *ulAttr* parameter is the logical OR of any of the following:

- **UDMA_ATTR_USEBURST** is used to restrict transfers to use only a burst mode.
- **UDMA_ATTR_ALTSELECT** is used to select the alternate control structure for this channel (it is very unlikely that this flag should be used)

- **UDMA_ATTR_HIGH_PRIORITY** is used to set this channel to high priority
- **UDMA_ATTR_REQMASK** is used to mask the hardware request signal from the peripheral for this channel.

Returns:

None.

19.2.3.11 uDMAChannelAttributeGet

Gets the enabled attributes of a uDMA channel.

Prototype:

```
unsigned long  
uDMAChannelAttributeGet(unsigned long ulChannelNum)
```

Parameters:

ulChannelNum is the channel to configure.

Description:

This function returns a combination of flags representing the attributes of the uDMA channel.

Returns:

Returns the logical OR of the attributes of the uDMA channel, which can be any of the following:

- **UDMA_ATTR_USEBURST** is used to restrict transfers to use only a burst mode.
- **UDMA_ATTR_ALTSELECT** is used to select the alternate control structure for this channel.
- **UDMA_ATTR_HIGH_PRIORITY** is used to set this channel to high priority.
- **UDMA_ATTR_REQMASK** is used to mask the hardware request signal from the peripheral for this channel.

19.2.3.12 uDMAChannelControlSet

Sets the control parameters for a uDMA channel control structure.

Prototype:

```
void  
uDMAChannelControlSet(unsigned long ulChannelStructIndex,  
                      unsigned long ulControl)
```

Parameters:

ulChannelStructIndex is the logical OR of the uDMA channel number with **UDMA_PRI_SELECT** or **UDMA_ALT_SELECT**.

ulControl is logical OR of several control values to set the control parameters for the channel.

Description:

This function is used to set control parameters for a uDMA transfer. These are typically parameters that are not changed often.

The *ulChannelStructIndex* parameter should be the logical OR of the channel number with one of **UDMA_PRI_SELECT** or **UDMA_ALT_SELECT** to choose whether the primary or alternate data structure is used.

The *ulControl* parameter is the logical OR of five values: the data size, the source address increment, the destination address increment, the arbitration size, and the use burst flag. The choices available for each of these values is described below.

Choose the data size from one of **UDMA_SIZE_8**, **UDMA_SIZE_16**, or **UDMA_SIZE_32** to select a data size of 8, 16, or 32 bits.

Choose the source address increment from one of **UDMA_SRC_INC_8**, **UDMA_SRC_INC_16**, **UDMA_SRC_INC_32**, or **UDMA_SRC_INC_NONE** to select an address increment of 8-bit bytes, 16-bit halfwords, 32-bit words, or to select non-incrementing.

Choose the destination address increment from one of **UDMA_DST_INC_8**, **UDMA_DST_INC_16**, **UDMA_DST_INC_32**, or **UDMA_DST_INC_NONE** to select an address increment of 8-bit bytes, 16-bit halfwords, 32-bit words, or to select non-incrementing.

The arbitration size determines how many items are transferred before the uDMA controller re-arbitrates for the bus. Choose the arbitration size from one of **UDMA_ARB_1**, **UDMA_ARB_2**, **UDMA_ARB_4**, **UDMA_ARB_8**, through **UDMA_ARB_1024** to select the arbitration size from 1 to 1024 items, in powers of 2.

The value **UDMA_NEXT_USEBURST** is used to force the channel to only respond to burst requests at the tail end of a scatter-gather transfer.

Note:

The address increment cannot be smaller than the data size.

Returns:

None.

19.2.3.13 uDMAChannelDisable

Disables a uDMA channel for operation.

Prototype:

```
void  
uDMAChannelDisable(unsigned long ulChannelNum)
```

Parameters:

ulChannelNum is the channel number to disable.

Description:

This function disables a specific uDMA channel. Once disabled, a channel will not respond to uDMA transfer requests until re-enabled via [uDMAChannelEnable\(\)](#).

Returns:

None.

19.2.3.14 uDMAChannelEnable

Enables a uDMA channel for operation.

Prototype:

```
void  
uDMAChannelEnable(unsigned long ulChannelNum)
```

Parameters:

ulChannelNum is the channel number to enable.

Description:

This function enables a specific uDMA channel for use. This function must be used to enable a channel before it can be used to perform a uDMA transfer.

When a uDMA transfer is completed, the channel will be automatically disabled by the uDMA controller. Therefore, this function should be called prior to starting up any new transfer.

Returns:

None.

19.2.3.15 uDMAChannelIsEnabled

Checks if a uDMA channel is enabled for operation.

Prototype:

```
tBoolean  
uDMAChannelIsEnabled(unsigned long ulChannelNum)
```

Parameters:

ulChannelNum is the channel number to check.

Description:

This function checks to see if a specific uDMA channel is enabled. This can be used to check the status of a transfer, since the channel will be automatically disabled at the end of a transfer.

Returns:

Returns **true** if the channel is enabled, **false** if disabled.

19.2.3.16 uDMAChannelModeGet

Gets the transfer mode for a uDMA channel control structure.

Prototype:

```
unsigned long  
uDMAChannelModeGet(unsigned long ulChannelStructIndex)
```

Parameters:

ulChannelStructIndex is the logical OR of the uDMA channel number with either **UDMA_PRI_SELECT** or **UDMA_ALT_SELECT**.

Description:

This function is used to get the transfer mode for the uDMA channel. It can be used to query the status of a transfer on a channel. When the transfer is complete the mode will be **UDMA_MODE_STOP**.

Returns:

Returns the transfer mode of the specified channel and control structure, which will be one of the following values: **UDMA_MODE_STOP**, **UDMA_MODE_BASIC**, **UDMA_MODE_AUTO**, **UDMA_MODE_PINGPONG**, **UDMA_MODE_MEM_SCATTER_GATHER**, or **UDMA_MODE_PER_SCATTER_GATHER**.

19.2.3.17 uDMAChannelRequest

Requests a uDMA channel to start a transfer.

Prototype:

```
void  
uDMAChannelRequest(unsigned long ulChannelNum)
```

Parameters:

ulChannelNum is the channel number on which to request a uDMA transfer.

Description:

This function allows software to request a uDMA channel to begin a transfer. This could be used for performing a memory to memory transfer, or if for some reason a transfer needs to be initiated by software instead of the peripheral associated with that channel.

Note:

If the channel is **UDMA_CHANNEL_SW** and interrupts are used, then the completion will be signaled on the uDMA dedicated interrupt. If a peripheral channel is used, then the completion will be signaled on the peripheral's interrupt.

Returns:

None.

19.2.3.18 uDMAChannelScatterGatherSet

Configures a uDMA channel for scatter-gather mode.

Prototype:

```
void  
uDMAChannelScatterGatherSet(unsigned long ulChannelNum,  
                             unsigned ulTaskCount,  
                             void *pvTaskList,  
                             unsigned long ulIsPeriphSG)
```

Parameters:

ulChannelNum is the uDMA channel number.

ulTaskCount is the number of scatter-gather tasks to execute.

pvTaskList is a pointer to the beginning of the scatter-gather task list.

ulIsPeriphSG is a flag to indicate it is a peripheral scatter-gather transfer (else it will be memory scatter-gather transfer)

Description:

This function is used to configure a channel for scatter-gather mode. The caller must have already set up a task list, and pass a pointer to the start of the task list as the *pvTaskList* parameter. The *ulTaskCount* parameter is the count of tasks in the task list, not the size of the task list. The flag *blsPeriphSG* should be used to indicate if the scatter-gather should be configured for a peripheral or memory scatter-gather operation.

See also:

[uDMATaskStructEntry](#)

Returns:
None.

19.2.3.19 uDMAChannelSizeGet

Gets the current transfer size for a uDMA channel control structure.

Prototype:

```
unsigned long  
uDMAChannelSizeGet(unsigned long ulChannelStructIndex)
```

Parameters:
ulChannelStructIndex is the logical OR of the uDMA channel number with either **UDMA_PRI_SELECT** or **UDMA_ALT_SELECT**.

Description:
This function is used to get the uDMA transfer size for a channel. The transfer size is the number of items to transfer, where the size of an item might be 8, 16, or 32 bits. If a partial transfer has already occurred, then the number of remaining items will be returned. If the transfer is complete, then 0 will be returned.

Returns:
Returns the number of items remaining to transfer.

19.2.3.20 uDMAChannelTransferSet

Sets the transfer parameters for a uDMA channel control structure.

Prototype:

```
void  
uDMAChannelTransferSet(unsigned long ulChannelStructIndex,  
                        unsigned long ulMode,  
                        void *pvSrcAddr,  
                        void *pvDstAddr,  
                        unsigned long ulTransferSize)
```

Parameters:
ulChannelStructIndex is the logical OR of the uDMA channel number with either **UDMA_PRI_SELECT** or **UDMA_ALT_SELECT**.
ulMode is the type of uDMA transfer.
pvSrcAddr is the source address for the transfer.
pvDstAddr is the destination address for the transfer.
ulTransferSize is the number of data items to transfer.

Description:
This function is used to set the parameters for a uDMA transfer. These are typically parameters that are changed often. The function [uDMAChannelControlSet\(\)](#) MUST be called at least once for this channel prior to calling this function.

The **ulChannelStructIndex** parameter should be the logical OR of the channel number with one of **UDMA_PRI_SELECT** or **UDMA_ALT_SELECT** to choose whether the primary or alternate data structure is used.

The *ulMode* parameter should be one of the following values:

- **UDMA_MODE_STOP** stops the uDMA transfer. The controller sets the mode to this value at the end of a transfer.
- **UDMA_MODE_BASIC** to perform a basic transfer based on request.
- **UDMA_MODE_AUTO** to perform a transfer that will always complete once started even if request is removed.
- **UDMA_MODE_PINGPONG** to set up a transfer that switches between the primary and alternate control structures for the channel. This allows use of ping-pong buffering for uDMA transfers.
- **UDMA_MODE_MEM_SCATTER_GATHER** to set up a memory scatter-gather transfer.
- **UDMA_MODE_PER_SCATTER_GATHER** to set up a peripheral scatter-gather transfer.

The *pvSrcAddr* and *pvDstAddr* parameters are pointers to the first location of the data to be transferred. These addresses should be aligned according to the item size. The compiler will take care of this if the pointers are pointing to storage of the appropriate data type.

The *ulTransferSize* parameter is the number of data items, not the number of bytes.

The two scatter/gather modes, memory and peripheral, are actually different depending on whether the primary or alternate control structure is selected. This function will look for the **UDMA_PRI_SELECT** and **UDMA_ALT_SELECT** flag along with the channel number and will set the scatter/gather mode as appropriate for the primary or alternate control structure.

The channel must also be enabled using [uDMAChannelEnable\(\)](#) after calling this function. The transfer will not begin until the channel has been set up and enabled. Note that the channel is automatically disabled after the transfer is completed, meaning that [uDMAChannelEnable\(\)](#) must be called again after setting up the next transfer.

Note:

Great care must be taken to not modify a channel control structure that is in use or else the results will be unpredictable, including the possibility of undesired data transfers to or from memory or peripherals. For BASIC and AUTO modes, it is safe to make changes when the channel is disabled, or the [uDMAChannelModeGet\(\)](#) returns **UDMA_MODE_STOP**. For PING-PONG or one of the SCATTER_GATHER modes, it is safe to modify the primary or alternate control structure only when the other is being used. The [uDMAChannelModeGet\(\)](#) function will return **UDMA_MODE_STOP** when a channel control structure is inactive and safe to modify.

Returns:

None.

19.2.3.21 uDMAControlAlternateBaseGet

Gets the base address for the channel control table alternate structures.

Prototype:

```
void *  
uDMAControlAlternateBaseGet (void)
```

Description:

This function gets the base address of the second half of the channel control table that holds the alternate control structures for each channel.

Returns:

Returns a pointer to the base address of the second half of the channel control table.

19.2.3.22 uDMAControlBaseGet

Gets the base address for the channel control table.

Prototype:

```
void *  
uDMAControlBaseGet(void)
```

Description:

This function gets the base address of the channel control table. This table resides in system memory and holds control information for each uDMA channel.

Returns:

Returns a pointer to the base address of the channel control table.

19.2.3.23 uDMAControlBaseSet

Sets the base address for the channel control table.

Prototype:

```
void  
uDMAControlBaseSet(void *pControlTable)
```

Parameters:

pControlTable is a pointer to the 1024 byte aligned base address of the uDMA channel control table.

Description:

This function sets the base address of the channel control table. This table resides in system memory and holds control information for each uDMA channel. The table must be aligned on a 1024 byte boundary. The base address must be set before any of the channel functions can be used.

The size of the channel control table depends on the number of uDMA channels, and which transfer modes are used. Refer to the introductory text and the microcontroller datasheet for more information about the channel control table.

Returns:

None.

19.2.3.24 uDMADisable

Disables the uDMA controller for use.

Prototype:

```
void  
uDMADisable(void)
```

Description:

This function disables the uDMA controller. Once disabled, the uDMA controller will not operate until re-enabled with [uDMAEnable\(\)](#).

Returns:

None.

19.2.3.25 uDMAEnable

Enables the uDMA controller for use.

Prototype:

```
void  
uDMAEnable(void)
```

Description:

This function enables the uDMA controller. The uDMA controller must be enabled before it can be configured and used.

Returns:

None.

19.2.3.26 uDMAErrorStatusClear

Clears the uDMA error interrupt.

Prototype:

```
void  
uDMAErrorStatusClear(void)
```

Description:

This function clears a pending uDMA error interrupt. It should be called from within the uDMA error interrupt handler to clear the interrupt.

Returns:

None.

19.2.3.27 uDMAErrorStatusGet

Gets the uDMA error status.

Prototype:

```
unsigned long  
uDMAErrorStatusGet(void)
```

Description:

This function returns the uDMA error status. It should be called from within the uDMA error interrupt handler to determine if a uDMA error occurred.

Returns:

Returns non-zero if a uDMA error is pending.

19.2.3.28 uDMAIntRegister

Registers an interrupt handler for the uDMA controller.

Prototype:

```
void  
uDMAIntRegister(unsigned long ulIntChannel,  
                void (*pfnHandler)(void))
```

Parameters:

ulIntChannel identifies which uDMA interrupt is to be registered.

pfnHandler is a pointer to the function to be called when the interrupt is activated.

Description:

This sets and enables the handler to be called when the uDMA controller generates an interrupt. The *ulIntChannel* parameter should be one of the following:

- **UDMA_INT_SW** to register an interrupt handler to process interrupts from the uDMA software channel (UDMA_CHANNEL_SW)
- **UDMA_INT_ERR** to register an interrupt handler to process uDMA error interrupts

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Note:

The interrupt handler for uDMA is for transfer completion when the channel UDMA_CHANNEL_SW is used, and for error interrupts. The interrupts for each peripheral channel are handled through the individual peripheral interrupt handlers.

Returns:

None.

19.2.3.29 uDMAIntUnregister

Unregisters an interrupt handler for the uDMA controller.

Prototype:

```
void  
uDMAIntUnregister(unsigned long ulIntChannel)
```

Parameters:

ulIntChannel identifies which uDMA interrupt to unregister.

Description:

This function will disable and clear the handler to be called for the specified uDMA interrupt. The *ulIntChannel* parameter should be one of **UDMA_INT_SW** or **UDMA_INT_ERR** as documented for the function [uDMAIntRegister\(\)](#).

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

19.3 Programming Example

The following example sets up the uDMA controller to perform a software initiated memory-to-memory transfer:

```
// The application must allocate the channel control table.
// This one is a full table for all modes and channels.
// NOTE: This table must be 1024 byte aligned.
unsigned char ucDMAControlTable[1024];

// Source and destination buffers used for the DMA transfer.
unsigned char ucSourceBuffer[256];
unsigned char ucDestBuffer[256];

// Enable the uDMA controller.
uDMAEnable();

// Set the base for the channel control table.
uDMAControlBaseSet(&ucDMAControlTable[0]);

// No attributes need to be set for a software based transfer.
// They will be cleared by default, but are explicitly cleared
// here, in case they were set elsewhere.
uDMAChannelAttributeDisable(UDMA_CHANNEL_SW, UDMA_CONFIG_ALL);

// Now set up the characteristics of the transfer. It will
// be 8 bit data size, with source and destination increments
// in bytes, to perform a byte-wise buffer copy. A bus arbitration
// size of 8 is used.
uDMAChannelControlSet(UDMA_CHANNEL_SW | UDMA_PRI_SELECT,
                      UDMA_SIZE_8 | UDMA_SRC_INC_8 |
                      UDMA_DST_INC_8 | UDMA_ARB_8);

// The transfer buffers and transfer size will now be configured.
// The transfer will use AUTO mode, which means that the
// transfer will automatically run to completion after the first
// request.
uDMAChannelTransferSet(UDMA_CHANNEL_SW | UDMA_PRI_SELECT,
                      UDMA_MODE_AUTO, ucSourceBuffer, ucDestBuffer,
                      sizeof(ucDestBuffer));

// Finally, the channel must be enabled. Since this is a software
// initiated transfer, a request must also be made. This will
// start the transfer running.
uDMAChannelEnable(UDMA_CHANNEL_SW);
uDMAChannelRequest(UDMA_CHANNEL_SW);
```

20 USB Controller

Introduction	323
Using uDMA with USB	323
API Functions	327
Programming Example	364

20.1 Introduction

The USB APIs provide a set of functions that are used to access the Concerto USB device or host controllers. The APIs are split into groups according to the functionality provided by the USB controller present in the microcontroller. Because of this, the driver has to handle microcontrollers that have only a USB device interface, a host and/or device interface, or microcontrollers that have an OTG interface. The groups are the following: USBDev, USBHost, USBOTG, USBEndpoint, and USBFIFO. The APIs in the USBDev group are only used with microcontrollers that have a USB device controller. The APIs in the USBHost group can only be used with microcontrollers that have a USB host controller. The USBOTG APIs are used by microcontrollers with an OTG interface. With USB OTG controllers, once the mode of the USB controller is configured, the device or host APIs should be used. The remainder of the APIs are used for both USB host and USB device controllers. The USBEndpoint APIs are used to configure and access the endpoints while the USBFIFO APIs are used to configure the size and location of the FIFOs.

20.2 Using USB with the uDMA Controller

The USB controller can be used with the uDMA for either sending or receiving data with both host and device controllers. The uDMA controller cannot be used to access endpoint 0, however all other endpoints are capable of using the uDMA controller. The uDMA channel numbers for USB are defined by the following values:

- DMA_CHANNEL_USBEP1RX
- DMA_CHANNEL_USBEP1TX
- DMA_CHANNEL_USBEP2RX
- DMA_CHANNEL_USBEP2TX
- DMA_CHANNEL_USBEP3RX
- DMA_CHANNEL_USBEP3TX

Since the uDMA controller views transfers as either transmit or receive, and the USB controller operates on IN/OUT transactions, some care must be taken to use the correct uDMA channel with the correct endpoint. USB host IN and USB device OUT endpoints both use receive uDMA channels while USB host OUT and USB device IN endpoints will use transmit uDMA channels.

When configuring the endpoint there are additional DMA settings needed. When calling [USBDevEndpointConfigSet\(\)](#) for an endpoint that will use uDMA, extra flags need to be added to the *ulFlags* parameter. These flags are one of **USB_EP_DMA_MODE_0** or **USB_EP_DMA_MODE_1** to control the mode of the DMA transaction, and likely **USB_EP_AUTO_SET** to allow the data to be

transmitted automatically once a packet is ready. **USB_EP_DMA_MODE_0** will generate an interrupt whenever there is more space available in the FIFO. This allows the application code to perform operations between each packet. **USB_EP_DMA_MODE_1** will only interrupt when the DMA transfer complete or there is some type of error condition. This can be used for larger transmissions that require no interaction between packets. **USB_EP_AUTO_SET** should normally be specified when using uDMA to prevent the need for application code to start the actual transfer of data.

Example: Endpoint configuration for a device IN endpoint:

```
// Endpoint 1 is a device mode BULK IN endpoint using DMA.
USBDevEndpointConfigSet(USB0_BASE, USB_EP_1, 64,
                        (USB_EP_MODE_BULK | USB_EP_DEV_IN |
                         USB_EP_DMA_MODE_0 | USB_EP_AUTO_SET));
```

The application must provide the configuration of the actual uDMA controller. First, to clear out any previous settings, the application should call `DMAChannelAttributeClear()`. Then the application should call `DMAChannelAttributeSet()` for the uDMA channel that corresponds to the endpoint, and specify the **DMA_CONFIG_USEBURST** flag.

Note:

All uDMA transfers used by the USB controller must enable burst mode.

The application needs to indicate the size of each DMA transactions, combined with the source and destination increments and the arbitration level for the uDMA controller.

Example: Configure endpoint 1 transmit channel.

```
// Set up the DMA for USB transmit.
DMAChannelAttributeClear(DMA_CHANNEL_USBEP1TX, DMA_CONFIG_ALL);

// Enable uDMA burst mode.
DMAChannelAttributeSet(DMA_CHANNEL_USBEP1TX, DMA_CONFIG_USEBURST);

// Data size is 8 bits and the source has a one byte increment.
// Destination has no increment as it is a FIFO.
DMAChannelControlSet(DMA_CHANNEL_USBEP1TX, DMA_DATA_SIZE_8,
                    DMA_ADDR_INC_8, DMA_ADDR_INC_NONE,
                    DMA_ARB_64, 0);
```

The next step is to actually start the uDMA transfer once the data is ready to be sent. There are the only two calls that the application needs to call to start a new transfer. Normally all of the previous uDMA configuration can stay the same. The first call, `DMAChannelTransferSet()`, resets the source and destination addresses for the DMA transfer and specifies how much data will be sent. The next call, `DMAChannelEnable()` actually allows the DMA controller to begin requesting data.

Example: Start the transfer of data on endpoint 1.

```
// Configure the address and size of the data to transfer.
DMAChannelTransferSet(DMA_CHANNEL_USBEP1TX, DMA_MODE_BASIC, pData,
                    USBFIFOAddr(USB0_BASE, USB_EP_1), 64);

// Start the transfer.
DMAChannelEnable(DMA_CHANNEL_USBEP1TX);
```

Because the uDMA interrupt occurs on the same interrupt vector as any other USB interrupt, the application must perform an extra check to determine what was the actual source of the interrupt. It is important to note that this DMA interrupt does not mean that the USB transfer is complete, but that the data has been transferred to the USB controller's FIFO. There will also be an interrupt indicating that the USB transfer is complete. However, both events need to be handled in the same interrupt routine. This because if other code in the system holds off the USB interrupt routine, both the uDMA complete and transfer complete can occur before the USB interrupt handler is called. The USB has no status bit indicating that the interrupt was due to a DMA complete, which means that the application must remember if a DMA transaction was in progress. The example below shows the `g_ulFlags` global variable being used to remember that a DMA transfer was pending.

Example: Interrupt handling with uDMA.

```
if((g_ulFlags & EP1_DMA_IN_PEND) &&
    (DMAChannelModeGet(DMA_CHANNEL_USBEPI1TX) == DMA_MODE_STOP))
{
    // Handle the DMA complete case.
    ...
}

// Get the interrupt status.
ulStatus = USBIntStatusEndpoint(USB0_BASE);

if(ulStatus & USB_INTEP_DEV_IN_1)
{
    // Handler the transfer complete case.
    ...
}
```

To use the USB device controller with an OUT endpoint, the application must use a receive uDMA channel. When calling `USBDevEndpointConfigSet()` for an endpoint that uses uDMA, the application must set extra flags in the `ulFlags` parameter. The **USB_EP_DMA_MODE_0** and **USB_EP_DMA_MODE_1** control the mode of the transaction, **USB_EP_AUTO_CLEAR** allows the data to be received automatically without needing to manually acknowledge that the data has been read. **USB_EP_DMA_MODE_0** will not generate an interrupt when each packet is sent over USB and will only interrupt when the DMA transfer is complete. **USB_EP_DMA_MODE_1** will interrupt when the DMA transfer complete or a short packet is received. This is useful for BULK endpoints that may not have prior knowledge of how much data is being received. **USB_EP_AUTO_CLEAR** should normally be specified when using uDMA to prevent the need for application code to acknowledge that the data has been read from the FIFO. The example below configures endpoint 1 as a Device mode Bulk OUT endpoint using DMA mode 1 with a max packet size of 64 bytes.

Example: Configure endpoint 1 receive channel:

```
// Endpoint 1 is a device mode BULK OUT endpoint using DMA.
USBDevEndpointConfigSet(USB0_BASE, USB_EP_1, 64,
    (USB_EP_DEV_OUT | USB_EP_MODE_BULK |
     USB_EP_DMA_MODE_1 | USB_EP_AUTO_CLEAR));
```

Next the configuration of the actual uDMA controller is needed. Like the transmit case, the first a call to `DMAChannelAttributeClear()` is made to clear any previous settings. This is followed by a call to `DMAChannelAttributeSet()` with the **DMA_CONFIG_USEBURST** value.

Note:

All uDMA transfers used by the USB controller must use burst mode.

The final call sets the read access size to 8 bits wide, the source address increment to 0, the destination address increment to 8 bits and the uDMA arbitration size to 64 bytes.

Example: Configure endpoint 1 transmit channel.

```
// Clear out any uDMA settings.
DMAChannelAttributeClear(DMA_CHANNEL_USBEPIRX, DMA_CONFIG_ALL);

DMAChannelAttributeSet(DMA_CHANNEL_USBEPIRX, DMA_CONFIG_USEBURST);

DMAChannelControlSet(DMA_CHANNEL_USBEPIRX, DMA_DATA_SIZE_8,
                     DMA_ADDR_INC_NONE, DMA_ADDR_INC_8,
                     DMA_ARB_64, 0);
```

The next step is to actually start the uDMA transfer. Unlike the transfer side, if the application is ready, this can be set up right away to wait for incoming data. Like the transmit case, these are the only calls needed to start a new transfer, normally all of the previous uDMA configuration can remain the same.

Example: Start requesting of data on endpoint 1.

```
// Configure the address and size of the data to transfer.
// The transfer is from the USB FIFO for endpoint 0
// to g_DataBufferIn.
DMAChannelTransferSet(DMA_CHANNEL_USBEPIRX, DMA_MODE_BASIC,
                     USBFIFOAddr(USB0_BASE, USB_EP_1),
                     g_DataBufferIn, 64);

// Enable the uDMA channel and wait for data.
DMAChannelEnable(DMA_CHANNEL_USBEPIRX);
```

The uDMA interrupt occurs on the same interrupt vector as any other USB interrupt, this means that the application needs to check to see what was the actual source of the interrupt. It is possible that the USB interrupt does not indicate that the USB transfer was complete. The interrupt could also have been caused by a short packet, error, or even a transmit complete. This requires that the application check both receive cases to determine if this is related to receiving data on the endpoint. Because the USB has no status bit indicating that the interrupt was due to a DMA complete, the application must remember if a DMA transaction was in progress.

Example: Interrupt handling with uDMA.

```
// Get the current interrupt status.
ulStatus = USBIntStatusEndpoint(USB0_BASE);

if(ulStatus & USB_INTEP_DEV_OUT_1)
{
    // Handle a short packet.
    ...
}
else if((g_ulFlags & EP1_DMA_OUT_PEND) &&
```

```

        (DMAChannelModeGet (DMA_CHANNEL_USBEPIRX) == DMA_MODE_STOP)
    {
        // Handle the DMA complete case.
        ...

        // Restart receive DMA if desired.
        ...
    }

```

20.3 API Functions

Functions

- unsigned long [USBDevAddrGet](#) (unsigned long ulBase)
- void [USBDevAddrSet](#) (unsigned long ulBase, unsigned long ulAddress)
- void [USBDevConnect](#) (unsigned long ulBase)
- void [USBDevDisconnect](#) (unsigned long ulBase)
- void [USBDevEndpointConfigGet](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long *pulMaxPacketSize, unsigned long *pulFlags)
- void [USBDevEndpointConfigSet](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulMaxPacketSize, unsigned long ulFlags)
- void [USBDevEndpointDataAck](#) (unsigned long ulBase, unsigned long ulEndpoint, tBoolean blsLastPacket)
- void [USBDevEndpointStall](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulFlags)
- void [USBDevEndpointStallClear](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulFlags)
- void [USBDevEndpointStatusClear](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulFlags)
- void [USBDevMode](#) (unsigned long ulBase)
- unsigned long [USBEndpointDataAvail](#) (unsigned long ulBase, unsigned long ulEndpoint)
- long [USBEndpointDataGet](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned char *pucData, unsigned long *pulSize)
- long [USBEndpointDataPut](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned char *pucData, unsigned long ulSize)
- long [USBEndpointDataSend](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulTransType)
- void [USBEndpointDataToggleClear](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulFlags)
- void [USBEndpointDMAChannel](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulChannel)
- void [USBEndpointDMADisable](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulFlags)
- void [USBEndpointDMAEnable](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulFlags)
- unsigned long [USBEndpointStatus](#) (unsigned long ulBase, unsigned long ulEndpoint)
- unsigned long [USBFIFOAddrGet](#) (unsigned long ulBase, unsigned long ulEndpoint)

- void [USBFIFOConfigGet](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long *pulFIFOAddress, unsigned long *pulFIFOSize, unsigned long ulFlags)
- void [USBFIFOConfigSet](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulFIFOAddress, unsigned long ulFIFOSize, unsigned long ulFlags)
- void [USBFIFOFlush](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulFlags)
- unsigned long [USBFrameNumberGet](#) (unsigned long ulBase)
- unsigned long [USBHostAddrGet](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulFlags)
- void [USBHostAddrSet](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulAddr, unsigned long ulFlags)
- void [USBHostEndpointConfig](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulMaxPayload, unsigned long ulNAKPollInterval, unsigned long ulTargetEndpoint, unsigned long ulFlags)
- void [USBHostEndpointDataAck](#) (unsigned long ulBase, unsigned long ulEndpoint)
- void [USBHostEndpointDataToggle](#) (unsigned long ulBase, unsigned long ulEndpoint, tBoolean bDataToggle, unsigned long ulFlags)
- void [USBHostEndpointStatusClear](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulFlags)
- unsigned long [USBHostHubAddrGet](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulFlags)
- void [USBHostHubAddrSet](#) (unsigned long ulBase, unsigned long ulEndpoint, unsigned long ulAddr, unsigned long ulFlags)
- void [USBHostMode](#) (unsigned long ulBase)
- void [USBHostPwrConfig](#) (unsigned long ulBase, unsigned long ulFlags)
- void [USBHostPwrDisable](#) (unsigned long ulBase)
- void [USBHostPwrEnable](#) (unsigned long ulBase)
- void [USBHostPwrFaultDisable](#) (unsigned long ulBase)
- void [USBHostPwrFaultEnable](#) (unsigned long ulBase)
- void [USBHostRequestIN](#) (unsigned long ulBase, unsigned long ulEndpoint)
- void [USBHostRequestINClear](#) (unsigned long ulBase, unsigned long ulEndpoint)
- void [USBHostRequestStatus](#) (unsigned long ulBase)
- void [USBHostReset](#) (unsigned long ulBase, tBoolean bStart)
- void [USBHostResume](#) (unsigned long ulBase, tBoolean bStart)
- unsigned long [USBHostSpeedGet](#) (unsigned long ulBase)
- void [USBHostSuspend](#) (unsigned long ulBase)
- void [USBIntDisable](#) (unsigned long ulBase, unsigned long ulFlags)
- void [USBIntDisableControl](#) (unsigned long ulBase, unsigned long ulFlags)
- void [USBIntDisableEndpoint](#) (unsigned long ulBase, unsigned long ulFlags)
- void [USBIntEnable](#) (unsigned long ulBase, unsigned long ulFlags)
- void [USBIntEnableControl](#) (unsigned long ulBase, unsigned long ulFlags)
- void [USBIntEnableEndpoint](#) (unsigned long ulBase, unsigned long ulFlags)
- void [USBIntRegister](#) (unsigned long ulBase, void (*pfnHandler)(void))
- unsigned long [USBIntStatus](#) (unsigned long ulBase)
- unsigned long [USBIntStatusControl](#) (unsigned long ulBase)
- unsigned long [USBIntStatusEndpoint](#) (unsigned long ulBase)
- void [USBIntUnregister](#) (unsigned long ulBase)
- unsigned long [USBModeGet](#) (unsigned long ulBase)

- unsigned long [USBNumEndpointsGet](#) (unsigned long ulBase)
- void [USBOTGMode](#) (unsigned long ulBase)
- void [USBOTGSessionRequest](#) (unsigned long ulBase, tBoolean bStart)
- void [USBPHYPowerOff](#) (unsigned long ulBase)
- void [USBPHYPowerOn](#) (unsigned long ulBase)

20.3.1 Detailed Description

The USB APIs provide all of the functions needed by an application to implement a USB device or USB host stack. The APIs abstract the IN/OUT nature of endpoints based on the type of USB controller that is in use. Any API that uses the IN/OUT terminology will comply with the standard USB interpretation of these terms. For example, an OUT endpoint on a microcontroller that has only a device interface will actually receive data on this endpoint, while a microcontroller that has a host interface will actually transmit data on an OUT endpoint.

Another important fact to understand is that all endpoints in the USB controller, whether host or device, have two "sides" to them. This allows each endpoint to both transmit and receive data. An application can use a single endpoint for both IN and OUT transactions. For example: In device mode, endpoint 1 could be configured to have BULK IN and BULK OUT handled by endpoint 1. It is important to note that the endpoint number used is the endpoint number reported to the host. For microcontrollers with host controllers, the application can use an endpoint to communicate with both IN and OUT endpoints of different types as well. For example: Endpoint 2 could be used to communicate with one device's interrupt IN endpoint and another device's bulk OUT endpoint at the same time. This effectively gives the application one dedicated control endpoint for IN or OUT control transactions on endpoint 0, and three IN endpoints and three OUT endpoints.

The USB controller has configurable FIFOs in devices that have a USB device controller as well as those that have a host controller. The overall size of the FIFO RAM is 4096 bytes. It is important to note that the first 64 bytes of this memory are dedicated to endpoint 0 for control transactions. The remaining 4032 bytes are configurable however the application desires. The FIFO configuration is usually set at the beginning of the application and not modified once the USB controller is in use. The FIFO configuration uses the `USBFIFOConfig()` API to set the starting address and the size of the FIFOs that are dedicated to each endpoint.

Example: FIFO Configuration

0-64 - endpoint 0 IN/OUT (64 bytes).

64-576 - endpoint 1 IN (512 bytes).

576-1088 - endpoint 1 OUT (512 bytes).

1088-1600 - endpoint 2 IN (512 bytes).

```
// FIFO for endpoint 1 IN starts at address 64 and is 512 bytes in size.
USBFIFOConfig(USB0_BASE, USB_EP_1, 64, USB_FIFO_SZ_512, USB_EP_DEV_IN);

// FIFO for endpoint 1 OUT starts at address 576 and is 512 bytes in size.
USBFIFOConfig(USB0_BASE, USB_EP_1, 576, USB_FIFO_SZ_512, USB_EP_DEV_OUT);

// FIFO for endpoint 2 IN starts at address 1088 and is 512 bytes in size.
USBFIFOConfig(USB0_BASE, USB_EP_2, 1088, USB_FIFO_SZ_512, USB_EP_DEV_IN);
```

20.3.2 Function Documentation

20.3.2.1 USBDevAddrGet

Returns the current device address in device mode.

Prototype:

```
unsigned long  
USBDevAddrGet(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function returns the current device address. This address was set by a call to [USBDevAddrSet\(\)](#).

Note:

This function should only be called in device mode.

Returns:

The current device address.

20.3.2.2 USBDevAddrSet

Sets the address in device mode.

Prototype:

```
void  
USBDevAddrSet(unsigned long ulBase,  
               unsigned long ulAddress)
```

Parameters:

ulBase specifies the USB module base address.

ulAddress is the address to use for a device.

Description:

This function configures the device address on the USB bus. This address was likely received via a SET ADDRESS command from the host controller.

Note:

This function should only be called in device mode.

Returns:

None.

20.3.2.3 USBDevConnect

Connects the USB controller to the bus in device mode.

Prototype:

```
void  
USBDevConnect(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function causes the soft connect feature of the USB controller to be enabled. Call [USBDevDisconnect\(\)](#) to remove the USB device from the bus.

Note:

This function should only be called in device mode.

Returns:

None.

20.3.2.4 USBDevDisconnect

Removes the USB controller from the bus in device mode.

Prototype:

```
void  
USBDevDisconnect(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function causes the soft connect feature of the USB controller to remove the device from the USB bus. A call to [USBDevConnect\(\)](#) is needed to reconnect to the bus.

Note:

This function should only be called in device mode.

Returns:

None.

20.3.2.5 USBDevEndpointConfigGet

Gets the current configuration for an endpoint.

Prototype:

```
void  
USBDevEndpointConfigGet(unsigned long ulBase,  
                        unsigned long ulEndpoint,  
                        unsigned long *pulMaxPacketSize,  
                        unsigned long *pulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

pulMaxPacketSize is a pointer which is written with the maximum packet size for this endpoint.

pulFlags is a pointer which is written with the current endpoint settings. On entry to the function, this pointer must contain either **USB_EP_DEV_IN** or **USB_EP_DEV_OUT** to indicate whether the IN or OUT endpoint is to be queried.

Description:

This function returns the basic configuration for an endpoint in device mode. The values returned in **pulMaxPacketSize* and **pulFlags* are equivalent to the *ulMaxPacketSize* and *ulFlags* previously passed to [USBDevEndpointConfigSet\(\)](#) for this endpoint.

Note:

This function should only be called in device mode.

Returns:

None.

20.3.2.6 USBDevEndpointConfigSet

Sets the configuration for an endpoint.

Prototype:

```
void
USBDevEndpointConfigSet(unsigned long ulBase,
                        unsigned long ulEndpoint,
                        unsigned long ulMaxPacketSize,
                        unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

ulMaxPacketSize is the maximum packet size for this endpoint.

ulFlags are used to configure other endpoint settings.

Description:

This function sets the basic configuration for an endpoint in device mode. Endpoint zero does not have a dynamic configuration, so this function should not be called for endpoint zero. The *ulFlags* parameter determines some of the configuration while the other parameters provide the rest.

The **USB_EP_MODE_** flags define what the type is for the given endpoint.

- **USB_EP_MODE_CTRL** is a control endpoint.
- **USB_EP_MODE_ISOC** is an isochronous endpoint.
- **USB_EP_MODE_BULK** is a bulk endpoint.
- **USB_EP_MODE_INT** is an interrupt endpoint.

The **USB_EP_DMA_MODE_** flags determine the type of DMA access to the endpoint data FIFOs. The choice of the DMA mode depends on how the DMA controller is configured and how it is being used. See the “Using USB with the uDMA Controller” section for more information on DMA configuration.

When configuring an IN endpoint, the **USB_EP_AUTO_SET** bit can be specified to cause the automatic transmission of data on the USB bus as soon as *ulMaxPacketSize* bytes of data are written into the FIFO for this endpoint. This option is commonly used with DMA as no interaction is required to start the transmission of data.

When configuring an OUT endpoint, the **USB_EP_AUTO_REQUEST** bit is specified to trigger the request for more data once the FIFO has been drained enough to receive *ulMaxPacketSize* more bytes of data. Also for OUT endpoints, the **USB_EP_AUTO_CLEAR** bit can be used to clear the data packet ready flag automatically once the data has been read from the FIFO. If this option is not used, this flag must be manually cleared via a call to [USBDevEndpointStatusClear\(\)](#). Both of these settings can be used to remove the need for extra calls when using the controller in DMA mode.

Note:

This function should only be called in device mode.

Returns:

None.

20.3.2.7 USBDevEndpointDataAck

Acknowledge that data was read from the given endpoint's FIFO in device mode.

Prototype:

```
void
USBDevEndpointDataAck(unsigned long ulBase,
                      unsigned long ulEndpoint,
                      tBoolean bIsLastPacket)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

bIsLastPacket indicates if this packet is the last one.

Description:

This function acknowledges that the data was read from the endpoint's FIFO. The *bIsLastPacket* parameter is set to a **true** value if this is the last in a series of data packets on endpoint zero. The *bIsLastPacket* parameter is not used for endpoints other than endpoint zero. This call can be used if processing is required between reading the data and acknowledging that the data has been read.

Note:

This function should only be called in device mode.

Returns:

None.

20.3.2.8 USBDevEndpointStall

Stalls the specified endpoint in device mode.

Prototype:

```
void
USBDevEndpointStall(unsigned long ulBase,
                    unsigned long ulEndpoint,
                    unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint specifies the endpoint to stall.

ulFlags specifies whether to stall the IN or OUT endpoint.

Description:

This function causes the endpoint number passed in to go into a stall condition. If the *ulFlags* parameter is **USB_EP_DEV_IN**, then the stall is issued on the IN portion of this endpoint. If the *ulFlags* parameter is **USB_EP_DEV_OUT**, then the stall is issued on the OUT portion of this endpoint.

Note:

This function should only be called in device mode.

Returns:

None.

20.3.2.9 USBDevEndpointStallClear

Clears the stall condition on the specified endpoint in device mode.

Prototype:

```
void
USBDevEndpointStallClear(unsigned long ulBase,
                        unsigned long ulEndpoint,
                        unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint specifies which endpoint to remove the stall condition.

ulFlags specifies whether to remove the stall condition from the IN or the OUT portion of this endpoint.

Description:

This function causes the endpoint number passed in to exit the stall condition. If the *ulFlags* parameter is **USB_EP_DEV_IN**, then the stall is cleared on the IN portion of this endpoint. If the *ulFlags* parameter is **USB_EP_DEV_OUT**, then the stall is cleared on the OUT portion of this endpoint.

Note:

This function should only be called in device mode.

Returns:

None.

20.3.2.10 USBDevEndpointStatusClear

Clears the status bits in this endpoint in device mode.

Prototype:

```
void
USBDevEndpointStatusClear(unsigned long ulBase,
                           unsigned long ulEndpoint,
                           unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

ulFlags are the status bits that should be cleared.

Description:

This function clears the status of any bits that are passed in the *ulFlags* parameter. The *ulFlags* parameter can take the value returned from the [USBEndpointStatus\(\)](#) call.

Note:

This function should only be called in device mode.

Returns:

None.

20.3.2.11 USBDevMode

Change the mode of the USB controller to device.

Prototype:

```
void
USBDevMode(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function changes the mode of the USB controller to device mode.

Note:

This function should only be called on microcontrollers that support OTG operation and have the DEVMODOTG bit in the USBGPCS register.

Returns:

None.

20.3.2.12 USBEndpointDataAvail

Determine the number of bytes of data available in a given endpoint's FIFO.

Prototype:

```
unsigned long
USBEndpointDataAvail(unsigned long ulBase,
                    unsigned long ulEndpoint)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

Description:

This function returns the number of bytes of data currently available in the FIFO for the given receive (OUT) endpoint. It may be used prior to calling [USBEndpointDataGet\(\)](#) to determine the size of buffer required to hold the newly-received packet.

Returns:

This call returns the number of bytes available in a given endpoint FIFO.

20.3.2.13 USBEndpointDataGet

Retrieves data from the given endpoint's FIFO.

Prototype:

```
long
USBEndpointDataGet(unsigned long ulBase,
                  unsigned long ulEndpoint,
                  unsigned char *pucData,
                  unsigned long *pulSize)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

pucData is a pointer to the data area used to return the data from the FIFO.

pulSize is initially the size of the buffer passed into this call via the *pucData* parameter. It is set to the amount of data returned in the buffer.

Description:

This function returns the data from the FIFO for the given endpoint. The *pulSize* parameter should indicate the size of the buffer passed in the *pucData* parameter. The data in the *pulSize* parameter is changed to match the amount of data returned in the *pucData* parameter. If a zero-byte packet is received, this call does not return an error but instead just returns a zero in the *pulSize* parameter. The only error case occurs when there is no data packet available.

Returns:

This call returns 0, or -1 if no packet was received.

20.3.2.14 USBEndpointDataPut

Puts data into the given endpoint's FIFO.

Prototype:

```
long
USBEndpointDataPut(unsigned long ulBase,
                  unsigned long ulEndpoint,
                  unsigned char *pucData,
                  unsigned long ulSize)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

pucData is a pointer to the data area used as the source for the data to put into the FIFO.

ulSize is the amount of data to put into the FIFO.

Description:

This function puts the data from the *pucData* parameter into the FIFO for this endpoint. If a packet is already pending for transmission, then this call does not put any of the data into the FIFO and returns -1. Care should be taken to not write more data than can fit into the FIFO allocated by the call to [USBFIFOConfigSet\(\)](#).

Returns:

This call returns 0 on success, or -1 to indicate that the FIFO is in use and cannot be written.

20.3.2.15 USBEndpointDataSend

Starts the transfer of data from an endpoint's FIFO.

Prototype:

```
long
USBEndpointDataSend(unsigned long ulBase,
                   unsigned long ulEndpoint,
                   unsigned long ulTransType)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

ulTransType is set to indicate what type of data is being sent.

Description:

This function starts the transfer of data from the FIFO for a given endpoint. This function should be called if the **USB_EP_AUTO_SET** bit was not enabled for the endpoint. Setting the *ulTransType* parameter allows the appropriate signaling on the USB bus for the type of transaction being requested. The *ulTransType* parameter should be one of the following:

- **USB_TRANS_OUT** for OUT transaction on any endpoint in host mode.
- **USB_TRANS_IN** for IN transaction on any endpoint in device mode.
- **USB_TRANS_IN_LAST** for the last IN transaction on endpoint zero in a sequence of IN transactions.
- **USB_TRANS_SETUP** for setup transactions on endpoint zero.
- **USB_TRANS_STATUS** for status results on endpoint zero.

Returns:

This call returns 0 on success, or -1 if a transmission is already in progress.

20.3.2.16 USBEndpointDataToggleClear

Sets the Data toggle on an endpoint to zero.

Prototype:

```
void
USBEndpointDataToggleClear(unsigned long ulBase,
                           unsigned long ulEndpoint,
                           unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint specifies the endpoint to reset the data toggle.

ulFlags specifies whether to access the IN or OUT endpoint.

Description:

This function causes the USB controller to clear the data toggle for an endpoint. This call is not valid for endpoint zero and can be made with host or device controllers.

The *ulFlags* parameter should be one of **USB_EP_HOST_OUT**, **USB_EP_HOST_IN**, **USB_EP_DEV_OUT**, or **USB_EP_DEV_IN**.

Returns:

None.

20.3.2.17 USBEndpointDMAChannel

Sets the DMA channel to use for a given endpoint.

Prototype:

```
void
USBEndpointDMAChannel(unsigned long ulBase,
                      unsigned long ulEndpoint,
                      unsigned long ulChannel)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint specifies which endpoint's FIFO address to return.

ulChannel specifies which DMA channel to use for which endpoint.

Description:

This function is used to configure which DMA channel to use with a given endpoint. Receive DMA channels can only be used with receive endpoints and transmit DMA channels can only be used with transmit endpoints. As a result, the 3 receive and 3 transmit DMA channels can be mapped to any endpoint other than 0. The values that should be passed into the *ulChannel* value are the **UDMA_CHANNEL_USBEP*** values defined in **udma.h**.

Note:

This function only has an effect on microcontrollers that have the ability to change the DMA channel for an endpoint. Calling this function on other devices has no effect.

Returns:

None.

20.3.2.18 USBEndpointDMADisable

Disable DMA on a given endpoint.

Prototype:

```
void
USBEndpointDMADisable(unsigned long ulBase,
                      unsigned long ulEndpoint,
                      unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

ulFlags specifies which direction to disable.

Description:

This function disables DMA on a given endpoint to allow non-DMA USB transactions to generate interrupts normally. The *ulFlags* should be **USB_EP_DEV_IN** or **USB_EP_DEV_OUT**; all other bits are ignored.

Returns:

None.

20.3.2.19 USBEndpointDMAEnable

Enable DMA on a given endpoint.

Prototype:

```
void
USBEndpointDMAEnable(unsigned long ulBase,
                    unsigned long ulEndpoint,
                    unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

ulFlags specifies which direction and what mode to use when enabling DMA.

Description:

This function will enable DMA on a given endpoint and set the mode according to the values in the *ulFlags* parameter. The *ulFlags* parameter should have **USB_EP_DEV_IN** or **USB_EP_DEV_OUT** set.

Returns:

None.

20.3.2.20 USBEndpointStatus

Returns the current status of an endpoint.

Prototype:

```
unsigned long
USBEndpointStatus(unsigned long ulBase,
                  unsigned long ulEndpoint)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

Description:

This function returns the status of a given endpoint. If any of these status bits must be cleared, then the [USBDevEndpointStatusClear\(\)](#) or the [USBHostEndpointStatusClear\(\)](#) functions should be called.

The following are the status flags for host mode:

- **USB_HOST_IN_PID_ERROR** - PID error on the given endpoint.
- **USB_HOST_IN_NOT_COMP** - The device failed to respond to an IN request.
- **USB_HOST_IN_STALL** - A stall was received on an IN endpoint.
- **USB_HOST_IN_DATA_ERROR** - There was a CRC or bit-stuff error on an IN endpoint in Isochronous mode.
- **USB_HOST_IN_NAK_TO** - NAKs received on this IN endpoint for more than the specified timeout period.
- **USB_HOST_IN_ERROR** - Failed to communicate with a device using this IN endpoint.
- **USB_HOST_IN_FIFO_FULL** - This IN endpoint's FIFO is full.
- **USB_HOST_IN_PKTRDY** - Data packet ready on this IN endpoint.
- **USB_HOST_OUT_NAK_TO** - NAKs received on this OUT endpoint for more than the specified timeout period.
- **USB_HOST_OUT_NOT_COMP** - The device failed to respond to an OUT request.
- **USB_HOST_OUT_STALL** - A stall was received on this OUT endpoint.
- **USB_HOST_OUT_ERROR** - Failed to communicate with a device using this OUT endpoint.
- **USB_HOST_OUT_FIFO_NE** - This endpoint's OUT FIFO is not empty.
- **USB_HOST_OUT_PKTPEND** - The data transfer on this OUT endpoint has not completed.
- **USB_HOST_EP0_NAK_TO** - NAKs received on endpoint zero for more than the specified timeout period.
- **USB_HOST_EP0_ERROR** - The device failed to respond to a request on endpoint zero.
- **USB_HOST_EP0_IN_STALL** - A stall was received on endpoint zero for an IN transaction.
- **USB_HOST_EP0_IN_PKTRDY** - Data packet ready on endpoint zero for an IN transaction.

The following are the status flags for device mode:

- **USB_DEV_OUT_SENT_STALL** - A stall was sent on this OUT endpoint.
- **USB_DEV_OUT_DATA_ERROR** - There was a CRC or bit-stuff error on an OUT endpoint.
- **USB_DEV_OUT_OVERRUN** - An OUT packet was not loaded due to a full FIFO.
- **USB_DEV_OUT_FIFO_FULL** - The OUT endpoint's FIFO is full.
- **USB_DEV_OUT_PKTRDY** - There is a data packet ready in the OUT endpoint's FIFO.
- **USB_DEV_IN_NOT_COMP** - A larger packet was split up, more data to come.
- **USB_DEV_IN_SENT_STALL** - A stall was sent on this IN endpoint.

- **USB_DEV_IN_UNDERRUN** - Data was requested on the IN endpoint and no data was ready.
- **USB_DEV_IN_FIFO_NE** - The IN endpoint's FIFO is not empty.
- **USB_DEV_IN_PKTPEND** - The data transfer on this IN endpoint has not completed.
- **USB_DEV_EP0_SETUP_END** - A control transaction ended before Data End condition was sent.
- **USB_DEV_EP0_SENT_STALL** - A stall was sent on endpoint zero.
- **USB_DEV_EP0_IN_PKTPEND** - The data transfer on endpoint zero has not completed.
- **USB_DEV_EP0_OUT_PKTRDY** - There is a data packet ready in endpoint zero's OUT FIFO.

Returns:

The current status flags for the endpoint depending on mode.

20.3.2.21 USBFIFOAddrGet

Returns the absolute FIFO address for a given endpoint.

Prototype:

```
unsigned long
USBFIFOAddrGet(unsigned long ulBase,
               unsigned long ulEndpoint)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint specifies which endpoint's FIFO address to return.

Description:

This function returns the actual physical address of the FIFO. This address is needed when the USB is going to be used with the uDMA controller and the source or destination address must be set to the physical FIFO address for a given endpoint.

Returns:

None.

20.3.2.22 USBFIFOConfigGet

Returns the FIFO configuration for an endpoint.

Prototype:

```
void
USBFIFOConfigGet(unsigned long ulBase,
                unsigned long ulEndpoint,
                unsigned long *pulFIFOAddress,
                unsigned long *pulFIFOSize,
                unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

ulFIFOAddress is the starting address for the FIFO.

ulFIFOSize is the size of the FIFO as specified by one of the USB_FIFO_SZ_ values.

ulFlags specifies what information to retrieve from the FIFO configuration.

Description:

This function returns the starting address and size of the FIFO for a given endpoint. Endpoint zero does not have a dynamically configurable FIFO, so this function should not be called for endpoint zero. The *ulFlags* parameter specifies whether the endpoint's OUT or IN FIFO should be read. If in host mode, the *ulFlags* parameter should be **USB_EP_HOST_OUT** or **USB_EP_HOST_IN**, and if in device mode, the *ulFlags* parameter should be either **USB_EP_DEV_OUT** or **USB_EP_DEV_IN**.

Returns:

None.

20.3.2.23 USBFIFOConfigSet

Sets the FIFO configuration for an endpoint.

Prototype:

```
void
USBFIFOConfigSet(unsigned long ulBase,
                  unsigned long ulEndpoint,
                  unsigned long ulFIFOAddress,
                  unsigned long ulFIFOSize,
                  unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

ulFIFOAddress is the starting address for the FIFO.

ulFIFOSize is the size of the FIFO specified by one of the USB_FIFO_SZ_ values.

ulFlags specifies what information to set in the FIFO configuration.

Description:

This function configures the starting FIFO RAM address and size of the FIFO for a given endpoint. Endpoint zero does not have a dynamically configurable FIFO, so this function should not be called for endpoint zero. The *ulFIFOSize* parameter should be one of the values in the **USB_FIFO_SZ_** values. If the endpoint is going to use double buffering, it should use the values with the **_DB** at the end of the value. For example, use **USB_FIFO_SZ_16_DB** to configure an endpoint to have a 16- byte, double-buffered FIFO. If a double-buffered FIFO is used, then the actual size of the FIFO is twice the size indicated by the *ulFIFOSize* parameter. For example, the **USB_FIFO_SZ_16_DB** value uses 32 bytes of the USB controller's FIFO memory.

The *ulFIFOAddress* value should be a multiple of 8 bytes and directly indicates the starting address in the USB controller's FIFO RAM. For example, a value of 64 indicates that the FIFO should start 64 bytes into the USB controller's FIFO memory. The *ulFlags* value specifies whether the endpoint's OUT or IN FIFO should be configured. If in host mode, use **USB_EP_HOST_OUT** or **USB_EP_HOST_IN**, and if in device mode, use **USB_EP_DEV_OUT** or **USB_EP_DEV_IN**.

Returns:

None.

20.3.2.24 USBFIFOFlush

Forces a flush of an endpoint's FIFO.

Prototype:

```
void
USBFIFOFlush(unsigned long ulBase,
              unsigned long ulEndpoint,
              unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

ulFlags specifies if the IN or OUT endpoint should be accessed.

Description:

This function forces the USB controller to flush out the data in the FIFO. The function can be called with either host or device controllers and requires the *ulFlags* parameter be one of **USB_EP_HOST_OUT**, **USB_EP_HOST_IN**, **USB_EP_DEV_OUT**, or **USB_EP_DEV_IN**.

Returns:

None.

20.3.2.25 USBFrameNumberGet

Get the current frame number.

Prototype:

```
unsigned long
USBFrameNumberGet(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function returns the last frame number received.

Returns:

The last frame number received.

20.3.2.26 USBHostAddrGet

Gets the current functional device address for an endpoint.

Prototype:

```
unsigned long
USBHostAddrGet (unsigned long ulBase,
                unsigned long ulEndpoint,
                unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

ulFlags determines if this is an IN or an OUT endpoint.

Description:

This function returns the current functional address that an endpoint is using to communicate with a device. The *ulFlags* parameter determines if the IN or OUT endpoint's device address is returned.

Note:

This function should only be called in host mode.

Returns:

Returns the current function address being used by an endpoint.

20.3.2.27 USBHostAddrSet

Sets the functional address for the device that is connected to an endpoint in host mode.

Prototype:

```
void
USBHostAddrSet (unsigned long ulBase,
                unsigned long ulEndpoint,
                unsigned long ulAddr,
                unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

ulAddr is the functional address for the controller to use for this endpoint.

ulFlags determines if this is an IN or an OUT endpoint.

Description:

This function configures the functional address for a device that is using this endpoint for communication. This *ulAddr* parameter is the address of the target device that this endpoint is communicating with. The *ulFlags* parameter indicates if the IN or OUT endpoint should be set.

Note:

This function should only be called in host mode.

Returns:

None.

20.3.2.28 USBHostEndpointConfig

Sets the base configuration for a host endpoint.

Prototype:

```
void
USBHostEndpointConfig(unsigned long ulBase,
                      unsigned long ulEndpoint,
                      unsigned long ulMaxPayload,
                      unsigned long ulNAKPollInterval,
                      unsigned long ulTargetEndpoint,
                      unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

ulMaxPayload is the maximum payload for this endpoint.

ulNAKPollInterval is the either the NAK timeout limit or the polling interval, depending on the type of endpoint.

ulTargetEndpoint is the endpoint that the host endpoint is targeting.

ulFlags are used to configure other endpoint settings.

Description:

This function sets the basic configuration for the transmit or receive portion of an endpoint in host mode. The *ulFlags* parameter determines some of the configuration while the other parameters provide the rest. The *ulFlags* parameter determines whether this is an IN endpoint (**USB_EP_HOST_IN** or **USB_EP_DEV_IN**) or an OUT endpoint (**USB_EP_HOST_OUT** or **USB_EP_DEV_OUT**), whether this is a Full speed endpoint (**USB_EP_SPEED_FULL**) or a Low speed endpoint (**USB_EP_SPEED_LOW**).

The **USB_EP_MODE_** flags control the type of the endpoint.

- **USB_EP_MODE_CTRL** is a control endpoint.
- **USB_EP_MODE_ISOC** is an isochronous endpoint.
- **USB_EP_MODE_BULK** is a bulk endpoint.
- **USB_EP_MODE_INT** is an interrupt endpoint.

The *ulNAKPollInterval* parameter has different meanings based on the **USB_EP_MODE** value and whether or not this call is being made for endpoint zero or another endpoint. For endpoint zero or any Bulk endpoints, this value always indicates the number of frames to allow a device to NAK before considering it a timeout. If this endpoint is an isochronous or interrupt endpoint, this value is the polling interval for this endpoint.

For interrupt endpoints, the polling interval is simply the number of frames between polling an interrupt endpoint. For isochronous endpoints this value represents a polling interval of $2^{(ulNAKPollInterval - 1)}$ frames. When used as a NAK timeout, the *ulNAKPollInterval* value specifies $2^{(ulNAKPollInterval - 1)}$ frames before issuing a time out. There are two special time out values that can be specified when setting the *ulNAKPollInterval* value. The first is **MAX_NAK_LIMIT**, which is the maximum value that can be passed in this variable. The other is **DISABLE_NAK_LIMIT**, which indicates that there should be no limit on the number of NAKs.

The **USB_EP_DMA_MODE_** flags enable the type of DMA used to access the endpoint's data FIFOs. The choice of the DMA mode depends on how the DMA controller is configured

and how it is being used. See the “Using USB with the uDMA Controller” section for more information on DMA configuration.

When configuring the OUT portion of an endpoint, the **USB_EP_AUTO_SET** bit is specified to cause the transmission of data on the USB bus to start as soon as the number of bytes specified by *ulMaxPayload* has been written into the OUT FIFO for this endpoint.

When configuring the IN portion of an endpoint, the **USB_EP_AUTO_REQUEST** bit can be specified to trigger the request for more data once the FIFO has been drained enough to fit *ulMaxPayload* bytes. The **USB_EP_AUTO_CLEAR** bit can be used to clear the data packet ready flag automatically once the data has been read from the FIFO. If this option is not used, this flag must be manually cleared via a call to [USBDevEndpointStatusClear\(\)](#) or [USBHostEndpointStatusClear\(\)](#).

Note:

This function should only be called in host mode.

Returns:

None.

20.3.2.29 USBHostEndpointDataAck

Acknowledge that data was read from the given endpoint's FIFO in host mode.

Prototype:

```
void
USBHostEndpointDataAck(unsigned long ulBase,
                        unsigned long ulEndpoint)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

Description:

This function acknowledges that the data was read from the endpoint's FIFO. This call is used if processing is required between reading the data and acknowledging that the data has been read.

Note:

This function should only be called in host mode.

Returns:

None.

20.3.2.30 USBHostEndpointDataToggle

Sets the value data toggle on an endpoint in host mode.

Prototype:

```
void
USBHostEndpointDataToggle(unsigned long ulBase,
```

```
unsigned long ulEndpoint,  
tBoolean bDataToggle,  
unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint specifies the endpoint to reset the data toggle.

bDataToggle specifies whether to set the state to DATA0 or DATA1.

ulFlags specifies whether to set the IN or OUT endpoint.

Description:

This function is used to force the state of the data toggle in host mode. If the value passed in the *bDataToggle* parameter is **false**, then the data toggle is set to the DATA0 state, and if it is **true** it is set to the DATA1 state. The *ulFlags* parameter can be **USB_EP_HOST_IN** or **USB_EP_HOST_OUT** to access the desired portion of this endpoint. The *ulFlags* parameter is ignored for endpoint zero.

Note:

This function should only be called in host mode.

Returns:

None.

20.3.2.31 USBHostEndpointStatusClear

Clears the status bits in this endpoint in host mode.

Prototype:

```
void  
USBHostEndpointStatusClear(unsigned long ulBase,  
                           unsigned long ulEndpoint,  
                           unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

ulFlags are the status bits that should be cleared.

Description:

This function clears the status of any bits that are passed in the *ulFlags* parameter. The *ulFlags* parameter can take the value returned from the [USBEndpointStatus\(\)](#) call.

Note:

This function should only be called in host mode.

Returns:

None.

20.3.2.32 USBHostHubAddrGet

Get the current device hub address for this endpoint.

Prototype:

```
unsigned long
USBHostHubAddrGet(unsigned long ulBase,
                  unsigned long ulEndpoint,
                  unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

ulFlags determines if this is an IN or an OUT endpoint.

Description:

This function returns the current hub address that an endpoint is using to communicate with a device. The *ulFlags* parameter determines if the device address for the IN or OUT endpoint is returned.

Note:

This function should only be called in host mode.

Returns:

This function returns the current hub address being used by an endpoint.

20.3.2.33 USBHostHubAddrSet

Set the hub address for the device that is connected to an endpoint.

Prototype:

```
void
USBHostHubAddrSet(unsigned long ulBase,
                  unsigned long ulEndpoint,
                  unsigned long ulAddr,
                  unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

ulAddr is the hub address and port for the device using this endpoint. The hub address must be defined in bits 8 through 15 with the port number in bits 0 through 6.

ulFlags determines if this is an IN or an OUT endpoint.

Description:

This function configures the hub address for a device that is using this endpoint for communication. The *ulFlags* parameter determines if the device address for the IN or the OUT endpoint is configured by this call and sets the speed of the downstream device. Valid values are one of **USB_EP_HOST_OUT** or **USB_EP_HOST_IN** optionally ORed with **USB_EP_SPEED_LOW**.

Note:

This function should only be called in host mode.

Returns:

None.

20.3.2.34 USBHostMode

Change the mode of the USB controller to host.

Prototype:

```
void
USBHostMode(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function changes the mode of the USB controller to host mode.

Note:

This function should only be called on microcontrollers that support OTG operation and have the DEVMODOTG bit in the USBGPCS register.

Returns:

None.

20.3.2.35 USBHostPwrConfig

Sets the configuration for USB power fault.

Prototype:

```
void
USBHostPwrConfig(unsigned long ulBase,
                  unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulFlags specifies the configuration of the power fault.

Description:

This function controls how the USB controller uses its external power control pins (USBnPFLT and USBnEPEN). The flags specify the power fault level sensitivity, the power fault action, and the power enable level and source.

One of the following can be selected as the power fault level sensitivity:

- **USB_HOST_PWRFLT_LOW** - An external power fault is indicated by the pin being driven low.
- **USB_HOST_PWRFLT_HIGH** - An external power fault is indicated by the pin being driven high.

One of the following can be selected as the power fault action:

- **USB_HOST_PWRFLT_EP_NONE** - No automatic action when power fault detected.

- **USB_HOST_PWRFLT_EP_TRI** - Automatically tri-state the USBnEPEN pin on a power fault.
- **USB_HOST_PWRFLT_EP_LOW** - Automatically drive USBnEPEN pin low on a power fault.
- **USB_HOST_PWRFLT_EP_HIGH** - Automatically drive USBnEPEN pin high on a power fault.

One of the following can be selected as the power enable level and source:

- **USB_HOST_PWREN_MAN_LOW** - USBnEPEN is driven low by the USB controller when [USBHostPwrEnable\(\)](#) is called.
- **USB_HOST_PWREN_MAN_HIGH** - USBnEPEN is driven high by the USB controller when [USBHostPwrEnable\(\)](#) is called.
- **USB_HOST_PWREN_AUTOLOW** - USBnEPEN is driven low by the USB controller automatically if [USBOTGSessionRequest\(\)](#) has enabled a session.
- **USB_HOST_PWREN_AUTOHIGH** - USBnEPEN is driven high by the USB controller automatically if [USBOTGSessionRequest\(\)](#) has enabled a session.

On devices that support the VBUS glitch filter, the **USB_HOST_PWREN_FILTER** can be added to ignore small, short drops in VBUS level caused by high power consumption. This feature is mainly used to avoid causing VBUS errors caused by devices with high in-rush current.

Note:

The following values have been deprecated and should no longer be used.

- **USB_HOST_PWREN_LOW** - Automatically drive USBnEPEN low when power is enabled.
- **USB_HOST_PWREN_HIGH** - Automatically drive USBnEPEN high when power is enabled.
- **USB_HOST_PWREN_VBLOW** - Automatically drive USBnEPEN low when power is enabled.
- **USB_HOST_PWREN_VBHIGH** - Automatically drive USBnEPEN high when power is enabled.

This function should only be called on microcontrollers that support host mode or OTG operation.

Returns:

None.

20.3.2.36 USBHostPwrDisable

Disables the external power pin.

Prototype:

```
void
USBHostPwrDisable(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function disables the USBnEPEN signal, which disables an external power supply in host mode operation.

Note:

This function should only be called in host mode.

Returns:

None.

20.3.2.37 USBHostPwrEnable

Enables the external power pin.

Prototype:

```
void  
USBHostPwrEnable(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function enables the USBnEPEN signal, which enables an external power supply in host mode operation.

Note:

This function should only be called in host mode.

Returns:

None.

20.3.2.38 USBHostPwrFaultDisable

Disables power fault detection.

Prototype:

```
void  
USBHostPwrFaultDisable(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function disables power fault detection in the USB controller.

Note:

This function should only be called in host mode.

Returns:

None.

20.3.2.39 USBHostPwrFaultEnable

Enables power fault detection.

Prototype:

```
void  
USBHostPwrFaultEnable(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function enables power fault detection in the USB controller. If the USBnPFLT pin is not in use, this function should not be used.

Note:

This function should only be called in host mode.

Returns:

None.

20.3.2.40 USBHostRequestIN

Schedules a request for an IN transaction on an endpoint in host mode.

Prototype:

```
void  
USBHostRequestIN(unsigned long ulBase,  
                 unsigned long ulEndpoint)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

Description:

This function schedules a request for an IN transaction. When the USB device being communicated with responds with the data, the data can be retrieved by calling [USBEndpointDataGet\(\)](#) or via a DMA transfer.

Note:

This function should only be called in host mode and only for IN endpoints.

Returns:

None.

20.3.2.41 USBHostRequestINClear

Clears a scheduled IN transaction for an endpoint in host mode.

Prototype:

```
void  
USBHostRequestINClear(unsigned long ulBase,  
                      unsigned long ulEndpoint)
```

Parameters:

ulBase specifies the USB module base address.

ulEndpoint is the endpoint to access.

Description:

This function clears a previously scheduled IN transaction if it is still pending. This function should be used to safely disable any scheduled IN transactions if the endpoint specified by *ulEndpoint* is reconfigured for communications with other devices.

Note:

This function should only be called in host mode and only for IN endpoints.

Returns:

None.

20.3.2.42 USBHostRequestStatus

Issues a request for a status IN transaction on endpoint zero.

Prototype:

```
void  
USBHostRequestStatus(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function is used to cause a request for a status IN transaction from a device on endpoint zero. This function can only be used with endpoint zero as that is the only control endpoint that supports this ability. This function is used to complete the last phase of a control transaction to a device and an interrupt is signaled when the status packet has been received.

Returns:

None.

20.3.2.43 USBHostReset

Handles the USB bus reset condition.

Prototype:

```
void  
USBHostReset(unsigned long ulBase,  
              tBoolean bStart)
```

Parameters:

ulBase specifies the USB module base address.

bStart specifies whether to start or stop signaling reset on the USB bus.

Description:

When this function is called with the *bStart* parameter set to **true**, this function causes the start of a reset condition on the USB bus. The caller should then delay at least 20ms before calling this function again with the *bStart* parameter set to **false**.

Note:

This function should only be called in host mode.

Returns:

None.

20.3.2.44 USBHostResume

Handles the USB bus resume condition.

Prototype:

```
void
USBHostResume(unsigned long ulBase,
               tBoolean bStart)
```

Parameters:

ulBase specifies the USB module base address.

bStart specifies if the USB controller is entering or leaving the resume signaling state.

Description:

When in device mode, this function brings the USB controller out of the suspend state. This call should first be made with the *bStart* parameter set to **true** to start resume signaling. The device application should then delay at least 10ms but not more than 15ms before calling this function with the *bStart* parameter set to **false**.

When in host mode, this function signals devices to leave the suspend state. This call should first be made with the *bStart* parameter set to **true** to start resume signaling. The host application should then delay at least 20ms before calling this function with the *bStart* parameter set to **false**. This action causes the controller to complete the resume signaling on the USB bus.

Returns:

None.

20.3.2.45 USBHostSpeedGet

Returns the current speed of the USB device connected.

Prototype:

```
unsigned long
USBHostSpeedGet(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function returns the current speed of the USB bus.

Note:

This function should only be called in host mode.

Returns:

Returns either **USB_LOW_SPEED**, **USB_FULL_SPEED**, or **USB_UNDEF_SPEED**.

20.3.2.46 USBHostSuspend

Puts the USB bus in a suspended state.

Prototype:

```
void  
USBHostSuspend(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

When used in host mode, this function puts the USB bus in the suspended state.

Note:

This function should only be called in host mode.

Returns:

None.

20.3.2.47 USBIntDisable

Disables the sources for USB interrupts.

Prototype:

```
void  
USBIntDisable(unsigned long ulBase,  
               unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulFlags specifies which interrupts to disable.

Description:

This function disables the USB controller from generating the interrupts indicated by the *ulFlags* parameter. There are three groups of interrupt sources, IN Endpoints, OUT Endpoints, and general status changes, specified by **USB_INT_HOST_IN**, **USB_INT_HOST_OUT**, **USB_INT_DEV_IN**, **USB_INT_DEV_OUT**, and **USB_INT_STATUS**. If **USB_INT_ALL** is specified, then all interrupts are disabled.

Note:

WARNING: This API cannot be used on endpoint numbers greater than endpoint 3 so [USBIntDisableControl\(\)](#) or [USBIntDisableEndpoint\(\)](#) should be used instead.

Returns:

None.

20.3.2.48 USBIntDisableControl

Disable control interrupts on a given USB controller.

Prototype:

```
void
USBIntDisableControl(unsigned long ulBase,
                     unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulFlags specifies which control interrupts to disable.

Description:

This function disables the control interrupts for the USB controller specified by the *ulBase* parameter. The *ulFlags* parameter specifies which control interrupts to disable. The flags passed in the *ulFlags* parameters should be the definitions that start with **USB_INTCTRL_*** and not any other **USB_INT** flags.

Returns:

None.

20.3.2.49 USBIntDisableEndpoint

Disable endpoint interrupts on a given USB controller.

Prototype:

```
void
USBIntDisableEndpoint(unsigned long ulBase,
                     unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulFlags specifies which endpoint interrupts to disable.

Description:

This function disables endpoint interrupts for the USB controller specified by the *ulBase* parameter. The *ulFlags* parameter specifies which endpoint interrupts to disable. The flags passed in the *ulFlags* parameters should be the definitions that start with **USB_INTEP_*** and not any other **USB_INT** flags.

Returns:

None.

20.3.2.50 USBIntEnable

Enables the sources for USB interrupts.

Prototype:

```
void
USBIntEnable(unsigned long ulBase,
              unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulFlags specifies which interrupts to enable.

Description:

This function enables the USB controller's ability to generate the interrupts indicated by the *ulFlags* parameter. There are three groups of interrupt sources, IN Endpoints, OUT Endpoints, and general status changes, specified by **USB_INT_HOST_IN**, **USB_INT_HOST_OUT**, **USB_INT_DEV_IN**, **USB_INT_DEV_OUT**, and **USB_STATUS**. If **USB_INT_ALL** is specified then all interrupts are enabled.

Note:

A call must be made to enable the interrupt in the main interrupt controller to receive interrupts. The [USBIntRegister\(\)](#) API performs this controller-level interrupt enable. However if static interrupt handlers are used, then then a call to [IntEnable\(\)](#) must be made in order to allow any USB interrupts to occur.

WARNING: This API cannot be used on endpoint numbers greater than endpoint 3 so [USBIntEnableControl\(\)](#) or [USBIntEnableEndpoint\(\)](#) should be used instead.

Returns:

None.

20.3.2.51 USBIntEnableControl

Enable control interrupts on a given USB controller.

Prototype:

```
void
USBIntEnableControl(unsigned long ulBase,
                    unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulFlags specifies which control interrupts to enable.

Description:

This function enables the control interrupts for the USB controller specified by the *ulBase* parameter. The *ulFlags* parameter specifies which control interrupts to enable. The flags passed in the *ulFlags* parameters should be the definitions that start with **USB_INTCTRL_*** and not any other **USB_INT** flags.

Returns:

None.

20.3.2.52 USBIntEnableEndpoint

Enable endpoint interrupts on a given USB controller.

Prototype:

```
void
USBIntEnableEndpoint(unsigned long ulBase,
                     unsigned long ulFlags)
```

Parameters:

ulBase specifies the USB module base address.

ulFlags specifies which endpoint interrupts to enable.

Description:

This function enables endpoint interrupts for the USB controller specified by the *ulBase* parameter. The *ulFlags* parameter specifies which endpoint interrupts to enable. The flags passed in the *ulFlags* parameters should be the definitions that start with **USB_INTEP_*** and not any other **USB_INT** flags.

Returns:

None.

20.3.2.53 USBIntRegister

Registers an interrupt handler for the USB controller.

Prototype:

```
void
USBIntRegister(unsigned long ulBase,
               void (*pfnHandler)(void))
```

Parameters:

ulBase specifies the USB module base address.

pfnHandler is a pointer to the function to be called when a USB interrupt occurs.

Description:

This function registers the handler to be called when a USB interrupt occurs and enables the global USB interrupt in the interrupt controller. The specific desired USB interrupts must be enabled via a separate call to [USBIntEnable\(\)](#). It is the interrupt handler's responsibility to clear the interrupt sources via calls to [USBIntStatusControl\(\)](#) and [USBIntStatusEndpoint\(\)](#).

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

20.3.2.54 USBIntStatus

Returns the status of the USB interrupts.

Prototype:

```
unsigned long
USBIntStatus(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function reads the source of the interrupt for the USB controller. There are three groups of interrupt sources, IN Endpoints, OUT Endpoints, and general status changes. This call returns the current status for all of these interrupts. The bit values returned should be compared against the **USB_HOST_IN**, **USB_HOST_OUT**, **USB_HOST_EP0**, **USB_DEV_IN**, **USB_DEV_OUT**, and **USB_DEV_EP0** values.

Note:

This call clears the source of all of the general status interrupts.

WARNING: This API cannot be used on endpoint numbers greater than endpoint 3 so [USBIntStatusControl\(\)](#) or [USBIntStatusEndpoint\(\)](#) should be used instead.

Returns:

Returns the status of the sources for the USB controller's interrupt.

20.3.2.55 USBIntStatusControl

Returns the control interrupt status on a given USB controller.

Prototype:

```
unsigned long
USBIntStatusControl(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function reads control interrupt status for a USB controller. This call returns the current status for control interrupts only, the endpoint interrupt status is retrieved by calling [USBIntStatusEndpoint\(\)](#). The bit values returned should be compared against the **USB_INTCTRL_*** values.

The following are the meanings of all **USB_INTCTRL_*** flags and the modes for which they are valid. These values apply to any calls to [USBIntStatusControl\(\)](#), [USBIntEnableControl\(\)](#), and [USBIntDisableControl\(\)](#). Some of these flags are only valid in the following modes as indicated in the parentheses: Host, Device, and OTG.

- **USB_INTCTRL_ALL** - A full mask of all control interrupt sources.
- **USB_INTCTRL_VBUS_ERR** - A VBUS error has occurred (Host Only).
- **USB_INTCTRL_SESSION** - Session Start Detected on A-side of cable (OTG Only).
- **USB_INTCTRL_SESSION_END** - Session End Detected (Device Only)
- **USB_INTCTRL_DISCONNECT** - Device Disconnect Detected (Host Only)
- **USB_INTCTRL_CONNECT** - Device Connect Detected (Host Only)
- **USB_INTCTRL_SOF** - Start of Frame Detected.

- **USB_INTCTRL_BABBLE** - USB controller detected a device signaling past the end of a frame. (Host Only)
- **USB_INTCTRL_RESET** - Reset signaling detected by device. (Device Only)
- **USB_INTCTRL_RESUME** - Resume signaling detected.
- **USB_INTCTRL_SUSPEND** - Suspend signaling detected by device (Device Only)
- **USB_INTCTRL_MODE_DETECT** - OTG cable mode detection has completed (OTG Only)
- **USB_INTCTRL_POWER_FAULT** - Power Fault detected. (Host Only)

Note:

This call clears the source of all of the control status interrupts.

Returns:

Returns the status of the control interrupts for a USB controller.

20.3.2.56 USBIntStatusEndpoint

Returns the endpoint interrupt status on a given USB controller.

Prototype:

```
unsigned long  
USBIntStatusEndpoint(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function reads endpoint interrupt status for a USB controller. This call returns the current status for endpoint interrupts only, the control interrupt status is retrieved by calling [USBIntStatusControl\(\)](#). The bit values returned should be compared against the **USB_INTEP_*** values. These values are grouped into classes for **USB_INTEP_HOST_*** and **USB_INTEP_DEV_*** values to handle both host and device modes with all endpoints.

Note:

This call clears the source of all of the endpoint interrupts.

Returns:

Returns the status of the endpoint interrupts for a USB controller.

20.3.2.57 USBIntUnregister

Unregisters an interrupt handler for the USB controller.

Prototype:

```
void  
USBIntUnregister(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function unregisters the interrupt handler. This function also disables the USB interrupt in the interrupt controller.

See also:

[IntRegister\(\)](#) for important information about registering or unregistering interrupt handlers.

Returns:

None.

20.3.2.58 USBModeGet

Returns the current operating mode of the controller.

Prototype:

```
unsigned long  
USBModeGet(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function returns the current operating mode on USB controllers with OTG or Dual mode functionality.

For OTG controllers:

The function returns one of the following values on OTG controllers: **USB_OTG_MODE_ASIDE_HOST**, **USB_OTG_MODE_ASIDE_DEV**, **USB_OTG_MODE_BSIDE_HOST**, **USB_OTG_MODE_BSIDE_DEV**, **USB_OTG_MODE_NONE**.

USB_OTG_MODE_ASIDE_HOST indicates that the controller is in host mode on the A-side of the cable.

USB_OTG_MODE_ASIDE_DEV indicates that the controller is in device mode on the A-side of the cable.

USB_OTG_MODE_BSIDE_HOST indicates that the controller is in host mode on the B-side of the cable.

USB_OTG_MODE_BSIDE_DEV indicates that the controller is in device mode on the B-side of the cable. If an OTG session request is started with no cable in place, this mode is the default.

USB_OTG_MODE_NONE indicates that the controller is not attempting to determine its role in the system.

For Dual Mode controllers:

The function returns one of the following values: **USB_DUAL_MODE_HOST**, **USB_DUAL_MODE_DEVICE**, or **USB_DUAL_MODE_NONE**.

USB_DUAL_MODE_HOST indicates that the controller is acting as a host.

USB_DUAL_MODE_DEVICE indicates that the controller acting as a device.

USB_DUAL_MODE_NONE indicates that the controller is not active as either a host or device.

Returns:

Returns **USB_OTG_MODE_ASIDE_HOST**, **USB_OTG_MODE_ASIDE_DEV**,
 USB_OTG_MODE_BSIDE_HOST, **USB_OTG_MODE_BSIDE_DEV**,
 USB_OTG_MODE_NONE, **USB_DUAL_MODE_HOST**, **USB_DUAL_MODE_DEVICE**,
or **USB_DUAL_MODE_NONE**.

20.3.2.59 USBNumEndpointsGet

Returns the number of USB endpoint pairs on the device.

Prototype:

```
unsigned long  
USBNumEndpointsGet(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function returns the number of endpoint pairs supported by the USB controller corresponding to the passed base address. The value returned is the number of IN or OUT endpoints available and does not include endpoint 0 (the control endpoint). For example, if 15 is returned, there are 15 IN and 15 OUT endpoints available in addition to endpoint 0.

Returns:

Returns the number of IN or OUT endpoints available.

20.3.2.60 USBOTGMode

Change the mode of the USB controller to OTG.

Prototype:

```
void  
USBOTGMode(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function changes the mode of the USB controller to OTG mode. This function is only valid on microcontrollers that have the OTG capabilities.

Returns:

None.

20.3.2.61 USBOTGSessionRequest

Starts or ends a session.

Prototype:

```
void
USBOTGSessionRequest(unsigned long ulBase,
                     tBoolean bStart)
```

Parameters:

ulBase specifies the USB module base address.
bStart specifies if this call starts or ends a session.

Description:

This function is used in OTG mode to start a session request or end a session. If the *bStart* parameter is set to **true**, then this function starts a session and if it is **false** it ends a session.

Returns:

None.

20.3.2.62 USBPHYPowerOff

Powers off the USB PHY.

Prototype:

```
void
USBPHYPowerOff(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function powers off the USB PHY, reducing the current consumption of the device. While in the powered-off state, the USB controller is unable to operate.

Returns:

None.

20.3.2.63 USBPHYPowerOn

Powers on the USB PHY.

Prototype:

```
void
USBPHYPowerOn(unsigned long ulBase)
```

Parameters:

ulBase specifies the USB module base address.

Description:

This function powers on the USB PHY, enabling it return to normal operation. By default, the PHY is powered on, so this function should only be called if [USBPHYPowerOff\(\)](#) has previously been called.

Returns:

None.

20.4 Programming Example

This example code makes the calls necessary to configure end point 1, in device mode, as a bulk IN end point. The first call configures end point 1 to have a maximum packet size of 64 bytes and makes it a bulk IN end point. The call to `USBFIFOConfig()` sets the starting address to 64 bytes in and 64 bytes long. It specifies **USB_EP_DEV_IN** to indicate that this is a device mode IN endpoint. The next two calls demonstrate how to fill the data FIFO for this endpoint and then have it scheduled for transmission on the USB bus. The `USBEndpointDataPut()` call puts data into the FIFO but does not actually start the data transmission. The `USBEndpointDataSend()` call will schedule the transmission to go out the next time the host controller requests data on this endpoint.

```
// Configure Endpoint 1.
USBDevEndpointConfigSet(USB0_BASE, USB_EP_1, 64, DISABLE_NAK_LIMIT,
                        USB_EP_MODE_BULK | USB_EP_DEV_IN);

// Configure FIFO as a device IN endpoint FIFO starting at address 64
// and is 64 bytes in size.
USBFIFOConfig(USB0_BASE, USB_EP_1, 64, USB_FIFO_SZ_64, USB_EP_DEV_IN);

...

// Put the data in the FIFO.
USBEndpointDataPut(USB0_BASE, USB_EP_1, pucData, 64);

// Start the transmission of data.
USBEndpointDataSend(USB0_BASE, USB_EP_1, USB_TRANS_IN);
```

21 Watchdog Timer

Introduction	365
API Functions	366
Programming Example	374

21.1 Introduction

The Watchdog Timer API provides a set of functions for using the Concerto watchdog timer modules. Functions are provided to deal with the watchdog timer interrupts, and to handle status and configuration of the watchdog timer.

The watchdog timer module's function is to prevent system hangs. The watchdog timer module consists of a 32-bit down counter, a programmable load register, interrupt generation logic, and a locking register. Once the watchdog timer has been configured, the lock register can be written to prevent the timer configuration from being inadvertently altered.

The watchdog timer can be configured to generate an interrupt to the processor upon its first timeout, and to generate a reset signal upon its second timeout. The watchdog timer module generates the first timeout signal when the 32-bit counter reaches the zero state after being enabled; enabling the counter also enables the watchdog timer interrupt. After the first timeout event, the 32-bit counter is reloaded with the value of the watchdog timer load register, and the timer resumes counting down from that value. If the timer counts down to its zero state again before the first timeout interrupt is cleared, and the reset signal has been enabled, the watchdog timer asserts its reset signal to the system. If the interrupt is cleared before the 32-bit counter reaches its second timeout, the 32-bit counter is loaded with the value in the load register, and counting resumes from that value. If the load register is written with a new value while the watchdog timer counter is counting, then the counter is loaded with the new value and continues counting.

The watchdog timer 1 (WDT1) module has an independent clocking domain and its registers must be written with a timing gap between accesses. The software must guarantee that this delay is inserted between all reads and writes. The WRC bit in the Watchdog Control (WDTCTL) register for WDT1 indicates that the required timing gap has elapsed. This bit is cleared on a write operation and set once the write completes, indicating to software that another write/read may be started safely. Software should poll WDTCTL for WRC=1 prior to accessing another register. Note that WDT0 does not have this restriction as it runs off the system clock.

There is a function, `WatchdogWriteReady`, that checks the write compatible bit of the Watchdog Control register. This function returns a boolean indicating whether a write to one of the registers is in progress or not. This function must be called before every read or write to Watchdog Timer 1 module.

This driver is contained in `driverlib/watchdog.c`, with `driverlib/watchdog.h` containing the API definitions for use by applications.

21.2 API Functions

Functions

- void [WatchdogEnable](#) (unsigned long ulBase)
- void [WatchdogIntClear](#) (unsigned long ulBase)
- void [WatchdogIntEnable](#) (unsigned long ulBase)
- void [WatchdogIntRegister](#) (unsigned long ulBase, void (*pfnHandler)(void))
- unsigned long [WatchdogIntStatus](#) (unsigned long ulBase, tBoolean bMasked)
- void [WatchdogIntUnregister](#) (unsigned long ulBase)
- void [WatchdogLock](#) (unsigned long ulBase)
- tBoolean [WatchdogLockState](#) (unsigned long ulBase)
- unsigned long [WatchdogReloadGet](#) (unsigned long ulBase)
- void [WatchdogReloadSet](#) (unsigned long ulBase, unsigned long ulLoadVal)
- void [WatchdogResetDisable](#) (unsigned long ulBase)
- void [WatchdogResetEnable](#) (unsigned long ulBase)
- tBoolean [WatchdogRunning](#) (unsigned long ulBase)
- void [WatchdogStallDisable](#) (unsigned long ulBase)
- void [WatchdogStallEnable](#) (unsigned long ulBase)
- void [WatchdogUnlock](#) (unsigned long ulBase)
- unsigned long [WatchdogValueGet](#) (unsigned long ulBase)
- tBoolean [WatchdogWriteReady](#) (void)

21.2.1 Detailed Description

The Watchdog Timer API is broken into two groups of functions: those that deal with interrupts, and those that handle status and configuration.

The Watchdog Timer interrupts are handled by the [WatchdogIntRegister\(\)](#), [WatchdogIntUnregister\(\)](#), [WatchdogIntEnable\(\)](#), [WatchdogIntClear\(\)](#), and [WatchdogIntStatus\(\)](#) functions.

Status and configuration functions for the Watchdog Timer module are [WatchdogEnable\(\)](#), [WatchdogRunning\(\)](#), [WatchdogLock\(\)](#), [WatchdogUnlock\(\)](#), [WatchdogLockState\(\)](#), [WatchdogReloadSet\(\)](#), [WatchdogReloadGet\(\)](#), [WatchdogValueGet\(\)](#), [WatchdogResetEnable\(\)](#), [WatchdogResetDisable\(\)](#), [WatchdogStallEnable\(\)](#), [WatchdogStallDisable\(\)](#), and [WatchdogWriteStatus\(\)](#).

21.2.2 Function Documentation

21.2.2.1 WatchdogEnable

Enables the watchdog timer.

Prototype:

```
void  
WatchdogEnable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the watchdog timer module.

Description:

This will enable the watchdog timer counter and interrupt.

Note:

This function will have no effect if the watchdog timer has been locked.

See also:

[WatchdogLock\(\)](#), [WatchdogUnlock\(\)](#)

Returns:

None.

21.2.2.2 WatchdogIntClear

Clears the watchdog timer interrupt.

Prototype:

```
void  
WatchdogIntClear(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the watchdog timer module.

Description:

The watchdog timer interrupt source is cleared, so that it no longer asserts.

Note:

Because there is a write buffer in the Cortex-M3 processor, it may take several clock cycles before the interrupt source is actually cleared. Therefore, it is recommended that the interrupt source be cleared early in the interrupt handler (as opposed to the very last action) to avoid returning from the interrupt handler before the interrupt source is actually cleared. Failure to do so may result in the interrupt handler being immediately reentered (because the interrupt controller still sees the interrupt source asserted).

Returns:

None.

21.2.2.3 WatchdogIntEnable

Enables the watchdog timer interrupt.

Prototype:

```
void  
WatchdogIntEnable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the watchdog timer module.

Description:

Enables the watchdog timer interrupt.

Note:

This function will have no effect if the watchdog timer has been locked.

See also:

[WatchdogLock\(\)](#), [WatchdogUnlock\(\)](#), [WatchdogEnable\(\)](#)

Returns:

None.

21.2.2.4 WatchdogIntRegister

Registers an interrupt handler for watchdog timer interrupt.

Prototype:

```
void  
WatchdogIntRegister(unsigned long ulBase,  
                    void (*pfnHandler)(void))
```

Parameters:

ulBase is the base address of the watchdog timer module.

pfnHandler is a pointer to the function to be called when the watchdog timer interrupt occurs.

Description:

This function does the actual registering of the interrupt handler. This will enable the global interrupt in the interrupt controller; the watchdog timer interrupt must be enabled via [WatchdogEnable\(\)](#). It is the interrupt handler's responsibility to clear the interrupt source via [WatchdogIntClear\(\)](#).

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

21.2.2.5 WatchdogIntStatus

Gets the current watchdog timer interrupt status.

Prototype:

```
unsigned long  
WatchdogIntStatus(unsigned long ulBase,  
                  tBoolean bMasked)
```

Parameters:

ulBase is the base address of the watchdog timer module.

bMasked is **false** if the raw interrupt status is required and **true** if the masked interrupt status is required.

Description:

This returns the interrupt status for the watchdog timer module. Either the raw interrupt status or the status of interrupt that is allowed to reflect to the processor can be returned.

Returns:

Returns the current interrupt status, where a 1 indicates that the watchdog interrupt is active, and a 0 indicates that it is not active.

21.2.2.6 WatchdogIntUnregister

Unregisters an interrupt handler for the watchdog timer interrupt.

Prototype:

```
void  
WatchdogIntUnregister(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the watchdog timer module.

Description:

This function does the actual unregistering of the interrupt handler. This function will clear the handler to be called when a watchdog timer interrupt occurs. This will also mask off the interrupt in the interrupt controller so that the interrupt handler no longer is called.

See also:

[IntRegister\(\)](#) for important information about registering interrupt handlers.

Returns:

None.

21.2.2.7 WatchdogLock

Enables the watchdog timer lock mechanism.

Prototype:

```
void  
WatchdogLock(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the watchdog timer module.

Description:

Locks out write access to the watchdog timer configuration registers.

Returns:

None.

21.2.2.8 WatchdogLockState

Gets the state of the watchdog timer lock mechanism.

Prototype:

```
tBoolean  
WatchdogLockState(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the watchdog timer module.

Description:

Returns the lock state of the watchdog timer registers.

Returns:

Returns **true** if the watchdog timer registers are locked, and **false** if they are not locked.

21.2.2.9 WatchdogReloadGet

Gets the watchdog timer reload value.

Prototype:

```
unsigned long  
WatchdogReloadGet(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the watchdog timer module.

Description:

This function gets the value that is loaded into the watchdog timer when the count reaches zero for the first time.

See also:

[WatchdogReloadSet\(\)](#)

Returns:

None.

21.2.2.10 WatchdogReloadSet

Sets the watchdog timer reload value.

Prototype:

```
void  
WatchdogReloadSet(unsigned long ulBase,  
                  unsigned long ulLoadVal)
```

Parameters:

ulBase is the base address of the watchdog timer module.

ulLoadVal is the load value for the watchdog timer.

Description:

This function sets the value to load into the watchdog timer when the count reaches zero for the first time; if the watchdog timer is running when this function is called, then the value will be immediately loaded into the watchdog timer counter. If the *ulLoadVal* parameter is 0, then an interrupt is immediately generated.

Note:

This function will have no effect if the watchdog timer has been locked.

See also:

[WatchdogLock\(\)](#), [WatchdogUnlock\(\)](#), [WatchdogReloadGet\(\)](#)

Returns:

None.

21.2.2.11 WatchdogResetDisable

Disables the watchdog timer reset.

Prototype:

```
void  
WatchdogResetDisable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the watchdog timer module.

Description:

Disables the capability of the watchdog timer to issue a reset to the processor upon a second timeout condition.

Note:

This function will have no effect if the watchdog timer has been locked.

See also:

[WatchdogLock\(\)](#), [WatchdogUnlock\(\)](#)

Returns:

None.

21.2.2.12 WatchdogResetEnable

Enables the watchdog timer reset.

Prototype:

```
void  
WatchdogResetEnable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the watchdog timer module.

Description:

Enables the capability of the watchdog timer to issue a reset to the processor upon a second timeout condition.

Note:

This function will have no effect if the watchdog timer has been locked.

See also:

[WatchdogLock\(\)](#), [WatchdogUnlock\(\)](#)

Returns:

None.

21.2.2.13 WatchdogRunning

Determines if the watchdog timer is enabled.

Prototype:

```
tBoolean  
WatchdogRunning(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the watchdog timer module.

Description:

This will check to see if the watchdog timer is enabled.

Returns:

Returns **true** if the watchdog timer is enabled, and **false** if it is not.

21.2.2.14 WatchdogStallDisable

Disables stalling of the watchdog timer during debug events.

Prototype:

```
void  
WatchdogStallDisable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the watchdog timer module.

Description:

This function disables the debug mode stall of the watchdog timer. By doing so, the watchdog timer continues to count regardless of the processor debug state.

Returns:

None.

21.2.2.15 WatchdogStallEnable

Enables stalling of the watchdog timer during debug events.

Prototype:

```
void  
WatchdogStallEnable(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the watchdog timer module.

Description:

This function allows the watchdog timer to stop counting when the processor is stopped by the debugger. By doing so, the watchdog is prevented from expiring (typically almost immediately from a human time perspective) and resetting the system (if reset is enabled). The watchdog will instead expire after the appropriate number of processor cycles have been executed while debugging (or at the appropriate time after the processor has been restarted).

Returns:

None.

21.2.2.16 WatchdogUnlock

Disables the watchdog timer lock mechanism.

Prototype:

```
void  
WatchdogUnlock(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the watchdog timer module.

Description:

Enables write access to the watchdog timer configuration registers.

Returns:

None.

21.2.2.17 WatchdogValueGet

Gets the current watchdog timer value.

Prototype:

```
unsigned long  
WatchdogValueGet(unsigned long ulBase)
```

Parameters:

ulBase is the base address of the watchdog timer module.

Description:

This function reads the current value of the watchdog timer.

Returns:

Returns the current value of the watchdog timer.

21.2.2.18 WatchdogWriteReady

Checks the write compatible bit of the watchdog control register.

Prototype:

```
tBoolean  
WatchdogWriteReady(void)
```

Description:

Checks the WDTCTL register to see if the required timing gap has elapsed. If WRC = 1, return true which indicates that a write access is not in progress. The WDT1 register can be read or written. If WRC = 0, return false which indicates that a write to one of the WDT1 registers is in progress.

Returns:

Returns **true** if a write access is not in progress, and **false** if a write access is in progress.

21.3 Programming Example

The following example shows how to set up the watchdog timer API to reset the processor after two timeouts.

```
// Check to see if the registers are locked, and if so, unlock them.  
if(WatchdogLockState(WATCHDOG0_BASE) == true)  
{  
    WatchdogUnlock(WATCHDOG0_BASE);  
}  
  
// Initialize the watchdog timer.  
WatchdogReloadSet(WATCHDOG0_BASE, 0xFEEFEE);  
  
// Enable the reset.  
WatchdogResetEnable(WATCHDOG0_BASE);  
  
// Enable the watchdog timer.  
WatchdogEnable(WATCHDOG0_BASE);  
  
// Wait for the reset to occur.  
while(1)  
{  
}
```

IMPORTANT NOTICE

Texas Instruments Incorporated and its subsidiaries (TI) reserve the right to make corrections, modifications, enhancements, improvements, and other changes to its products and services at any time and to discontinue any product or service without notice. Customers should obtain the latest relevant information before placing orders and should verify that such information is current and complete. All products are sold subject to TI's terms and conditions of sale supplied at the time of order acknowledgment.

TI warrants performance of its hardware products to the specifications applicable at the time of sale in accordance with TI's standard warranty. Testing and other quality control techniques are used to the extent TI deems necessary to support this warranty. Except where mandated by government requirements, testing of all parameters of each product is not necessarily performed.

TI assumes no liability for applications assistance or customer product design. Customers are responsible for their products and applications using TI components. To minimize the risks associated with customer products and applications, customers should provide adequate design and operating safeguards.

TI does not warrant or represent that any license, either express or implied, is granted under any TI patent right, copyright, mask work right, or other TI intellectual property right relating to any combination, machine, or process in which TI products or services are used. Information published by TI regarding third-party products or services does not constitute a license from TI to use such products or services or a warranty or endorsement thereof. Use of such information may require a license from a third party under the patents or other intellectual property of the third party, or a license from TI under the patents or other intellectual property of TI.

Reproduction of TI information in TI data books or data sheets is permissible only if reproduction is without alteration and is accompanied by all associated warranties, conditions, limitations, and notices. Reproduction of this information with alteration is an unfair and deceptive business practice. TI is not responsible or liable for such altered documentation. Information of third parties may be subject to additional restrictions.

Resale of TI products or services with statements different from or beyond the parameters stated by TI for that product or service voids all express and any implied warranties for the associated TI product or service and is an unfair and deceptive business practice. TI is not responsible or liable for any such statements.

TI products are not authorized for use in safety-critical applications (such as life support) where a failure of the TI product would reasonably be expected to cause severe personal injury or death, unless officers of the parties have executed an agreement specifically governing such use. Buyers represent that they have all necessary expertise in the safety and regulatory ramifications of their applications, and acknowledge and agree that they are solely responsible for all legal, regulatory and safety-related requirements concerning their products and any use of TI products in such safety-critical applications, notwithstanding any applications-related information or support that may be provided by TI. Further, Buyers must fully indemnify TI and its representatives against any damages arising out of the use of TI products in such safety-critical applications.

TI products are neither designed nor intended for use in military/aerospace applications or environments unless the TI products are specifically designated by TI as military-grade or "enhanced plastic." Only products designated by TI as military-grade meet military specifications. Buyers acknowledge and agree that any such use of TI products which TI has not designated as military-grade is solely at the Buyer's risk, and that they are solely responsible for compliance with all legal and regulatory requirements in connection with such use.

TI products are neither designed nor intended for use in automotive applications or environments unless the specific TI products are designated by TI as compliant with ISO/TS 16949 requirements. Buyers acknowledge and agree that, if they use any non-designated products in automotive applications, TI will not be responsible for any failure to meet such requirements.

Following are URLs where you can obtain information on other Texas Instruments products and application solutions:

Products

Amplifiers	amplifier.ti.com
Data Converters	dataconverter.ti.com
DLP® Products	www.dlp.com
DSP	dsp.ti.com
Clocks and Timers	www.ti.com/clocks
Interface	interface.ti.com
Logic	logic.ti.com
Power Mgmt	power.ti.com
Microcontrollers	microcontroller.ti.com
RFID	www.ti-rfid.com
RF/IF and ZigBee® Solutions	www.ti.com/lprf

Applications

Audio	www.ti.com/audio
Automotive	www.ti.com/automotive
Broadband	www.ti.com/broadband
Digital Control	www.ti.com/digitalcontrol
Medical	www.ti.com/medical
Military	www.ti.com/military
Optical Networking	www.ti.com/opticalnetwork
Security	www.ti.com/security
Telephony	www.ti.com/telephony
Video & Imaging	www.ti.com/video
Wireless	www.ti.com/wireless

Mailing Address: Texas Instruments, Post Office Box 655303, Dallas, Texas 75265
Copyright © 2016, Texas Instruments Incorporated